

MERN

Guía Práctica de Aplicaciones Web

MERN

Guía Práctica de Aplicaciones Web

Jesús Fontecha

Manuel Ángel Serrano

Iván González

Ramón Hervás





MERN. Guía Práctica de Aplicaciones Web

© Jesús Fontecha, Manuel Ángel Serrano, Iván González, Ramón Hervás

© De la edición: Ra-Ma 2020

MARCAS COMERCIALES. Las designaciones utilizadas por las empresas para distinguir sus productos (hardware, software, sistemas operativos, etc.) suelen ser marcas registradas. RA-MA ha intentado a lo largo de este libro distinguir las marcas comerciales de los términos descriptivos, siguiendo el estilo que utiliza el fabricante, sin intención de infringir la marca y solo en beneficio del propietario de la misma. Los datos de los ejemplos y pantallas son ficticios a no ser que se especifique lo contrario.

RA-MA es marca comercial registrada.

Se ha puesto el máximo empeño en ofrecer al lector una información completa y precisa. Sin embargo, RA-MA Editorial no asume ninguna responsabilidad derivada de su uso ni tampoco de cualquier violación de patentes ni otros derechos de terceras partes que pudieran ocurrir. Esta publicación tiene por objeto proporcionar unos conocimientos precisos y acreditados sobre el tema tratado. Su venta no supone para el editor ninguna forma de asistencia legal, administrativa o de ningún otro tipo. En caso de precisarse asesoría legal u otra forma de ayuda experta, deben buscarse los servicios de un profesional competente.

Reservados todos los derechos de publicación en cualquier idioma.

Según lo dispuesto en el Código Penal vigente, ninguna parte de este libro puede ser reproducida, grabada en sistema de almacenamiento o transmitida en forma alguna ni por cualquier procedimiento, ya sea electrónico, mecánico, reprográfico, magnético o cualquier otro sin autorización previa y por escrito de RA-MA; su contenido está protegido por la ley vigente, que establece penas de prisión y/o multas a quienes, intencionadamente, reprodujeran o plagiaran, en todo o en parte, una obra literaria, artística o científica.

Editado por:

RA-MA Editorial

Calle Jarama, 3A, Polígono Industrial Igarsa

28860 PARACUELLOS DE JARAMA, Madrid

Teléfono: 91 658 42 80

Fax: 91 662 81 39

Correo electrónico: editorial@ra-ma.com

Internet: www.ra-ma.es y www.ra-ma.com

ISBN: 978-84-1855-105-5

Depósito legal: M-26658-2020

Maquetación: Antonio García Tomé

Diseño de portada: Antonio García Tomé

Filmación e impresión: Safekat

Impreso en España en noviembre de 2020

*A todas las personas apasionadas del desarrollo web
y las nuevas tecnologías, capaces de contribuir
a la digitalización responsable de la sociedad.
Entre todos podemos hacer un mundo mejor.*

Los autores.

ÍNDICE

ACERCA DE LOS AUTORES	11
PRÓLOGO	13
¿QUIÉN ES CARLOS AZAUSTRE?.....	14
ACERCA DEL LIBRO.....	15
¿QUÉ ES Y QUÉ NO ES ESTE LIBRO?	16
ACERCA DEL CONTENIDO.....	16
EL DESARROLLO WEB FULL-STACK.....	17
PATRONES DE DISEÑO EN EL DESARROLLO WEB.....	19
GENERALIDADES	19
INTRODUCCIÓN A LA PILA MERN	23
OPERACIONES CRUD Y HTTP. EL CORAZÓN DEL BACK-END.....	25
LA ESPECIFICACIÓN JAVASCRIPT	29
CAPÍTULO 1. MONGODB.....	31
1.1 CONSIDERACIONES INICIALES.....	32
1.2 BASES DE DATOS LOCALES VERSUS SERVICIOS DE TERCEROS.....	32
1.2.1 Registro y configuración de MongoDB Atlas	33
1.2.2 Instalación local y uso de la shell de MongoDB.....	38
1.3 HERRAMIENTAS GRÁFICAS PARA LA MANIPULACIÓN DE DATOS.....	41
1.3.1 Instalación y uso de MongoDB Compass	41
1.4 LECTURAS RECOMENDADAS.....	43
CAPÍTULO 2. NODE.JS.....	45
2.1 PRIMEROS PASOS	46
2.2 HOLA MUNDO EN NODEJS	47

2.3	EL PACKAGE.JSON Y EL GESTOR DE PAQUETES NPM.....	49
2.3.1	El gestor npm	51
2.4	ATOM COMO EDITOR PARA EL DESARROLLO DE APLICACIONES WEB.....	53
2.4.1	Instalación de Atom.....	54
2.5	CONTROL DE VERSIONES CON GIT	56
2.5.1	Instalación de Git	57
2.5.2	Introducción a Github.....	58
2.6	LECTURAS RECOMENDADAS.....	62
CAPÍTULO 3. EXPRESS.JS		63
3.1	¿QUÉ ES EXPRESS.JS?	64
3.2	HOLA MUNDO EN EXPRESS.JS	64
3.3	GENERACIÓN DE UNA APLICACIÓN EXPRESS.JS	66
3.3.1	Acerca de la ejecución y depuración.....	68
3.4	INTRODUCCIÓN A LAS APIS REST Y ENRUTADO CON EXPRESS.JS	71
3.4.1	Características generales de REST.....	72
3.4.2	Rutas y manejadores.....	73
3.4.3	CRUD y Enrutado	75
3.4.4	Códigos de estado y formatos de contenido.....	78
3.4.5	Express Router	80
3.4.6	Creación de mi primera API REST	83
3.4.7	Cómo consumir una API REST. Introducción a los clientes REST	88
3.5	MAPEADORES DE DATOS Y ODM	92
3.5.1	Creación de modelos con mongoose en Express.js.....	94
3.5.2	Conexión con MongoDB Atlas	98
3.6	INTRODUCCIÓN A LAS VARIABLES DE ENTORNO	100
3.6.1	Creación y acceso a variables de entorno.....	101
3.7	SERVICIOS REST PARA LA GESTIÓN DE USUARIOS.....	102
3.7.1	Devolución de todos los usuarios.....	103
3.7.2	Devolución de un usuario.....	104
3.7.3	Inserción de un usuario	104
3.7.4	Actualización de un usuario	105
3.7.5	Borrado de un usuario	105
3.7.6	Login de un usuario.....	106
3.7.7	Comprobación de servicios y refinamiento del “app.js”.....	107
3.8	SERVICIOS REST PARA LA GESTIÓN DE POSTS.....	109
3.8.1	Devolución de todos los posts existentes	110
3.8.2	Devolución de los posts de un usuario	111
3.8.3	Inserción de un nuevo post.....	111
3.8.4	Actualización de un post	112
3.8.5	Borrado de un post	113

3.9	PUESTA EN PRODUCCIÓN DE LA API REST	115
3.9.1	Introducción a Heroku.....	116
3.9.2	Instalación y despliegue de la aplicación en Heroku	118
3.9.3	Despliegue desde Github.....	121
3.10	CONCEPTOS AVANZADOS.....	124
3.10.1	Seguridad de la API REST y uso de tokens	125
3.11	LECTURAS RECOMENDADAS.....	127
CAPÍTULO 4. REACT.JS		129
4.1	INTRODUCCIÓN AL FRAMEWORK REACT.JS.....	130
4.1.1	Empaquetadores de módulos.....	131
4.1.2	Hola Mundo en React.js	132
4.1.3	React Developer Tools	140
4.1.4	Introducción a la sintaxis JSX	141
4.1.5	Uso de componentes. Tipos y fases del ciclo de vida	144
4.1.6	Propiedades y estados.....	153
4.1.7	Enrutado con React Router	158
4.2	DEFINIENDO EL ESTILO DE LA INTERFAZ CON REACTSTRAP	164
4.3	ESTRUCTURA DE LA APLICACIÓN REACT.JS	167
4.3.1	Fundamentos y primeros pasos	170
4.4	CONSUMO DE UNA API REST DESDE REACT.JS CON AXIOS.....	174
4.4.1	GET con axios	175
4.4.2	POST con axios	176
4.4.3	DELETE con axios.....	178
4.4.4	Definición de una instancia base	179
4.5	DESARROLLO DEL CLIENTE. PÁGINAS, COMPONENTES Y COMUNICACIÓN CON EL API REST	180
4.5.1	Login de un usuario.....	181
4.5.2	Registro de un nuevo usuario	190
4.5.3	Visualización de todas las entradas	195
4.5.4	Panel de control o Dashboard.....	200
4.5.5	Visualización de las entradas del usuario	206
4.5.6	Añadir y editar entradas	214
4.6	PUESTA EN PRODUCCIÓN DE LA APLICACIÓN REACT.JS.....	223
4.7	CONCEPTOS AVANZADOS.....	224
4.7.1	Reducers y Redux.....	224
4.8	INTRODUCCIÓN A LOS HOOKS EN REACT.JS	226
4.8.1	¿Qué son los hooks?.....	226
4.8.2	Clases como funciones	227
4.8.3	Hooks de estado	229
4.8.4	Hooks de efecto	232
4.8.5	Reglas y hooks personalizados.....	235
4.9	LECTURAS RECOMENDADAS.....	236

APÉNDICE A. CÓDIGO FUENTE.....	239
APÉNDICE B. RECETARIOS.....	241
JAVASCRIPT. EXPRESIONES Y OPERACIONES BÁSICAS.....	242
MONGODB. OPERACIONES CRUD Y FILTROS.....	243
NODE PACKAGE MANAGER. COMANDOS BÁSICOS	244
CÓDIGOS DE ESTADO HTTP	245
EXPRESS.JS. CONFIGURACIONES, SOLICITUDES Y RESPUESTAS	246
GIT Y GITHUB. COMANDOS BÁSICOS.....	246
REACT.JS. FUNDAMENTOS Y COMPONENTES.....	251
REACT.JS. HOOKS	255

ACERCA DE LOS AUTORES



Jesús Fontecha es Doctor en Informática y Máster en Tecnologías Informáticas Avanzadas por la Universidad de Castilla-La Mancha (UCLM). Actualmente es Profesor Ayudante Doctor en la Escuela Superior de Informática de la UCLM, desempeñando labores docentes y de investigación en el grupo MAmI (*Modelling Ambient Intelligence*) y en el Instituto de Tecnologías y Sistemas de Información (ITSI), ambos adscritos a la UCLM. Aunque sus trabajos y líneas de investigación en la universidad están ligados al uso de tecnologías aplicadas a la prevención y detección de patologías en el ámbito de la salud, es también un entusiasta del desarrollo web. De hecho, sus primeras experiencias profesionales fueron como desarrollador PHP y ASP.NET. Desde entonces no ha dejado de tener curiosidad y aprender sobre nuevas tecnologías y herramientas web. Página personal: <https://jesusfontecha.name/>



Manuel Ángel Serrano es Profesor Titular de Universidad en la Escuela Superior de Informática de Ciudad Real. Originalmente, comenzó a investigar sobre la calidad del software y actualmente se centra en líneas de investigación como la calidad de los datos, la inteligencia de negocio, la ciberseguridad o la computación cuántica. Con una producción científica considerable (habiendo publicado más de medio centenar de contribuciones en revistas y conferencias), ha participado en una veintena de proyectos de investigación y ha tenido una activa participación en contratos de transferencia y modelos de utilidad con empresas. En cuanto a su trayectoria profesional, ha participado en dos *spin-offs*, tiene varias certificaciones profesionales y ha trabajado como desarrollador web. En la actualidad, es un ferviente defensor de las metodologías ágiles y un apasionado de las tecnologías de desarrollo web.



Iván González es Doctor en Informática y Máster en Tecnologías Informáticas Avanzadas por la Universidad de Castilla-La Mancha (UCLM). Actualmente es investigador postdoctoral y tecnólogo en el grupo MAmI (*Modelling Ambient Intelligence*) de la Escuela Superior de Informática de la UCLM donde, además, desempeña labores docentes. Como parte de su actividad investigadora, lidera el laboratorio MAmIoT (<https://mamiot.com/>), dirigido a la investigación y desarrollo del Internet de las Cosas (IoT) y la construcción de entornos inteligentes. Entre sus trabajos, destaca el desarrollo de dispositivos *wearables* con aplicaciones en Salud y el desarrollo de soluciones basadas en redes de sensores y sistemas empotrados. En este contexto particular, siente fascinación por las tecnologías Web que permiten la integración y comunicación de estos dispositivos y redes de sensores con el mundo que nos rodea.



Ramón Hervás es Profesor Titular de la Universidad de Castilla – La Mancha (UCLM), en el área de lenguajes y sistemas. Su docencia e investigación se enmarca principalmente en la interacción persona-ordenador, con especial hincapié en la evaluación de sistemas interactivos para la salud, con propuestas para el análisis cuantitativo de la usabilidad y la experiencia de usuario. Es autor de numerosos artículos en revistas impacto internacional e investigador principal de proyectos de I+D+i nacionales. Su experiencia e intereses científicos y su pasión por los videojuegos le ha servido para liderar la línea de investigación sobre juegos serios para la salud del laboratorio MAmI, creando aplicaciones web y videojuegos para la evaluación y mejora de cognitiva. Página personal: <https://blog.uclm.es/ramonhlucas/>



PRÓLOGO

Hoy en día tenemos gran cantidad de recursos disponibles en la red para aprender cualquier cosa que se nos ocurra, y en el campo en el que nos movemos de las tecnologías web, esto no es una excepción.

Disponemos de artículos en *blogs*, pequeñas píldoras en *tweets*, videotutoriales en *Youtube*, directos de gente programando en vivo en *Twitch*, y un largo etcétera. En todos estos sitios y plataformas puedes llegar a aprender lo que necesitas para tu carrera profesional dentro del desarrollo web.

Sin embargo, hay algo que a mi parecer es insustituible y eso es tener un buen libro de referencia, que te guíe paso a paso, capítulo a capítulo, a armar una aplicación web real, y poco a poco te vaya explicando los conceptos necesarios para entender todo lo que se está haciendo.

Así es como yo, personalmente, he ido aprendiendo en mis últimos años. Lamentablemente, en idioma español se encuentran pocos recursos, o bien estos no son de suficiente calidad para formarse adecuadamente.

Por suerte, este libro que tienes entre tus manos ha sido escrito con ese fin. Te enseña la teoría suficiente sin abrumarte ni aburrirte, para que entiendas los conceptos que tienes que aplicar y enseguida se pone manos a la obra, primero con pequeños ejemplos sencillos, para que puedas probar rápidamente y tener una mejor referencia y retroalimentación de lo que estás haciendo, y tras ello, mostrándote paso a paso la construcción de un proyecto web básico pero real, que bien podrías encontrarte en tu día a día dentro del campo del desarrollo web.

Además, ahonda en uno de los “stacks” más utilizados del momento como es MERN. React.js es ya un “estándar” en muchas empresas de producto, y Node.

js (junto con Express.js) cada vez es más utilizado en *back-ends* basados en APIs REST y aplicaciones de tiempo real.

Sin duda creo que será un libro de referencia para toda aquella persona que quiera iniciarse en el desarrollo *full-stack* con tecnologías web actuales y muy demandadas en el mercado laboral.

Mis felicitaciones al equipo de la Universidad de Castilla-La Mancha por esta gran labor de divulgación.

Carlos Azaustre

¿QUIÉN ES CARLOS AZAUSTRE?

Carlos Azaustre es Ingeniero en Telemática por la Universidad Carlos III de Madrid y ha trabajado durante más de 8 años como Desarrollador web en empresas como *Eventbrite*, *IBM Research* o *Google*. Además, ha sido *freelance* y cofundador de su propia *Startup*. Actualmente se dedica a la formación y divulgación de contenido sobre tecnologías web, contando con miles de seguidores en la red. Es reconocido por *Google* como *Developer Expert* (GDE) en Tecnologías Web.



ACERCA DEL LIBRO

El libro que tienes en tus manos nace fruto del trabajo realizado en el ámbito del curso de enseñanzas propias, con el mismo título, impartido en la Escuela Superior de Informática de la Universidad de Castilla-La Mancha. Su publicación está también motivada por la escasez de libros en español que abordan el desarrollo web con MERN, así como la creciente demanda en el uso de este tipo de tecnologías de cara a la profesionalización del propio desarrollo web; ofreciendo al lector el material que le proporcione los conocimientos necesarios para iniciarse y profundizar en todas las tareas clave que puedan ayudarlo a comenzar, aprender, comprender e implementar aplicaciones web completas *full-stack* usando la pila o “stack” de tecnologías MERN.

De este modo, se guiará al lector en el proceso de instalación y configuración de todos los requisitos para poder crear, de forma práctica, una **aplicación web de microblogging** con React.js del lado del cliente (*front-end*), y capaz de consumir una API REST desarrollada con Node.js, Express.js y MongoDB del lado del servidor (*back-end*); con todo lo que rodea a este apasionante y demandado *core* de tecnologías web.

Para el correcto seguimiento del libro es recomendable disponer de mínimos conocimientos sobre el desarrollo web y el lenguaje JavaScript. No obstante, cualquier persona con disposición de aprender podrá seguirlo sin demasiados inconvenientes. Igualmente, si deseas, como lector, profundizar sobre los contenidos presentados, es recomendable echar un vistazo a las secciones de lecturas recomendadas, al final de cada una de las partes desarrolladas.

¿QUÉ ES Y QUÉ NO ES ESTE LIBRO?

Este libro no constituye ningún manual o guía oficial de las tecnologías incluidas en la pila MERN, sino una guía práctica para la creación de aplicaciones web mediante dichas tecnologías. Se ha tenido en cuenta la orientación abierta y multiplataforma de las tecnologías y herramientas utilizadas, pero tanto los ejemplos mostrados como su desarrollo se ha llevado a cabo en un entorno Microsoft Windows.

Este libro puede verse como una receta paso a paso para la creación de una aplicación web *full-stack*, con secciones introductorias de cada una de las tecnologías y partes que constituyen la pila MERN, así como ejemplos y código de cada elemento de la aplicación web implementada.

ACERCA DEL CONTENIDO

El contenido del libro se divide en varias secciones con fundamentos acerca del desarrollo web seguido de cuatro partes bien diferenciadas y que corresponden a las cuatro iniciales de MERN, comenzando con la instalación y configuración de una base de datos MongoDB, la creación de una API REST mediante Node.js y Express.js, que supone el *back-end* de la aplicación, y finalmente el desarrollo de una aplicación *front-end* utilizando el *framework* React.js.

Todo el contenido correspondiente al desarrollo de la aplicación que se presenta en este libro es un trabajo original por parte de los autores. El resto de material complementario utilizado como apoyo para la implementación paso a paso de la aplicación de *microblogging* se encuentra debidamente referenciado y todos los enlaces web incluidos han sido comprobados a fecha de edición del libro. No obstante, si como lector detectas algún dato erróneo, no lo dudes y ponte en contacto con los autores.

Sin más que añadir, te damos las gracias por adquirir este libro. Esperamos que disfrutes de su lectura y sea de utilidad.



EL DESARROLLO WEB FULL-STACK

Hoy en día, el desarrollo de aplicaciones web supone un nicho de mercado realmente atractivo y consolidado, siendo uno de los dominios que más favorecen la empleabilidad de ingenieros, diseñadores, y técnicos informáticos, entre otros.

El desarrollo web comprende un conjunto de habilidades que todo profesional que se dedique a ello debería conocer. Este conjunto de habilidades puede traducirse en diferentes especialidades. Comúnmente, estas especialidades reciben el nombre genérico de: *front-end*, *back-end* y *full-stack*.

Un desarrollador *front-end* trabaja del lado del cliente, es decir, en lo que cualquiera de nosotros, como usuarios, vemos de una aplicación web cuando navegamos por ella. Estos desarrolladores deben conocer los principios de diseño y estructura de las vistas y páginas de la aplicación. Sin embargo, sus conocimientos van más allá, teniendo en cuenta fundamentos y estándares de usabilidad, legibilidad, accesibilidad, etc. pero sin preocuparse demasiado de la lógica (más allá de la requerida por las propias vistas) y las transacciones, ya que estas no suelen ocurrir en el lado del cliente.

A diferencia del desarrollador *front-end*, un desarrollador *back-end* trabaja del lado del servidor. De modo que tratará de desarrollar y coordinar la mayor parte de la lógica y las transacciones existentes en la aplicación web. En este caso, el desarrollador *back-end* tiene que conocer y dominar diferentes lenguajes y *frameworks*, y estar familiarizado con protocolos de comunicación y sistemas de bases de datos.

Es evidente que, cuando se trabaja con personas y/o equipos dedicados, existe una especialización entre el personal que da lugar a estos dos tipos de desarrolladores. Sin embargo, en la mayoría de las ocasiones, es el desarrollador *back-end* el que

adquiere progresivamente responsabilidades sobre el *front-end* de la aplicación. De igual forma, que los desarrolladores *front-end* están cada vez más obligados a tener conocimientos del servidor y de la parte *back-end* de las aplicaciones. Teniendo esto en cuenta, aparece la figura del desarrollador *full-stack*.

Este tipo de desarrollador, popularizado hace unos años por el departamento de ingeniería de Facebook, es responsable de la implementación de la aplicación tanto en su parte cliente como en el servidor. Desde la configuración de este último e implementación de la lógica de la aplicación, hasta la maquetación y diseño de esta de cara a su uso por el usuario final.

En la actualidad es bastante difícil ser un buen desarrollador *full-stack*, ya que es casi inviable dominar las tecnologías de cada lado: cliente y servidor (ver Figura 1).



Figura 1. Algunas tecnologías web y el desarrollador full-stack

Generalmente, los desarrolladores *full-stack* se centran más en alguna de las dos partes, pero teniendo conocimientos suficientes sobre la otra, como para permitir el diseño, desarrollo y despliegue de la aplicación. Estos perfiles de desarrolladores suelen trabajar en pequeñas y medianas empresas, y son ellos los encargados de todo el desarrollo web.

En este libro trataremos de plasmar, de forma sencilla, sistemática y organizada, las tareas realizadas por un desarrollador *full-stack* a la hora de crear una aplicación web completa, considerando tanto el *back-end* como el *front-end*, implementando ambas partes como aplicaciones independientes, para facilitar su posterior mantenimiento.



PATRONES DE DISEÑO EN EL DESARROLLO WEB

GENERALIDADES

Partiendo de que los patrones de diseño son herramientas que nos proporcionan soluciones a problemas típicos y recurrentes cuando desarrollamos una aplicación, en el desarrollo web su aplicación es de vital importancia. Tanto es así, que cualquier *framework* o tecnología escogida para la implementación de la aplicación nos obliga de forma intrínseca a utilizar uno de estos patrones.

El principal objetivo a la hora de desarrollar una aplicación, en nuestro caso una aplicación web, utilizando un patrón de diseño, es hacer que dichas aplicaciones sean más robustas a la vez que fáciles de mantener, controlando la cohesión y el acoplamiento, o favoreciendo la reutilización de código. En la actualidad, existen muchísimos patrones de diseño de elevado nivel de abstracción. A continuación, se presenta alguno de los más extendidos con ciertas variantes surgidas a lo largo del tiempo.

Podríamos decir que el patrón de referencia utilizado por la mayoría de *frameworks* y librerías de desarrollo web, es el denominado MVC (Modelo-Vista-Controlador), el cual ha ido evolucionando con el tiempo dando lugar a diversos patrones derivados de este. La Figura 2 muestra un diagrama con las principales capas de este patrón.

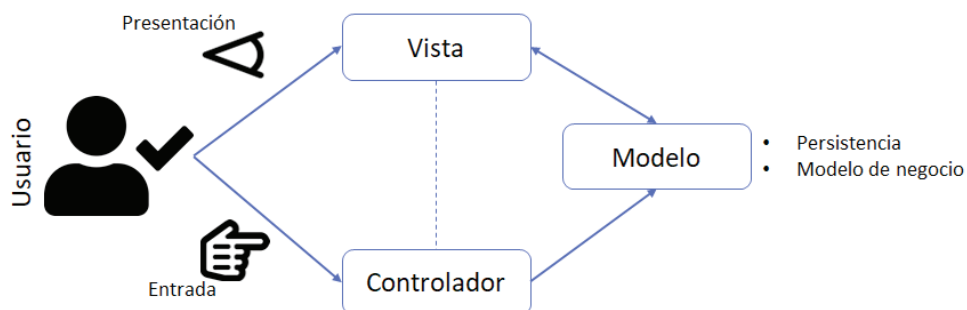


Figura 2. Capas del patrón MVC¹

La capa del modelo es la que trabaja con los datos, de modo que contendrá todos los mecanismos necesarios para el acceso y actualización de la información, conectando directamente con la base de datos, normalmente a través de “mapeadores” de datos, representando estos como instancias de objetos para facilitar su manipulación, algo que veremos cómo se hace utilizando la librería de `mongoose.js`.

En cuanto a la capa de la vista, esta corresponde a todo el código de la aplicación que produce la visualización de la misma a través de interfaces gráficas de usuario, que en última instancia será código HTML. En la vista se trabajará con los datos, a los que no se accede directamente, sino a través de los modelos.

Finalmente, la capa del controlador es la que contiene el código necesario para responder a las acciones que solicita el usuario a través de la vista. Sirve de enlace entre la vista y el modelo, y contiene gran parte de la lógica de la aplicación.

Como dijimos anteriormente, este patrón MVC ha ido evolucionando, y lo que existe actualmente difiere de esta abstracción, pero sigue fundamentándose en ella. Ahora los controladores son el punto de entrada de las aplicaciones, y las vistas adquieren funciones de interacción a través del manejo de eventos a partir de la interfaz de usuario. Por supuesto, todo esto depende del *framework* escogido.

En el caso de React.js, las aplicaciones web suelen tener cierta complejidad interna y todo tiende a estar representado en una única página, lo que se denomina *Single Page App* (SPA). Todas las capas anteriores procedentes del patrón MVC,

¹ Imagen inspirada en: https://cdn-images-1.medium.com/max/1000/0*1VVNmPlwpfeytDMh
[Último acceso: Julio, 2020]

suelen verse embebidas en el navegador del cliente, y los controladores responden con documentos de tipo JSON a peticiones usualmente manejadas mediante el protocolo HTTP a través de una API. La Figura 3 muestra el comportamiento de una aplicación de este tipo fundamentada en el patrón MVC.

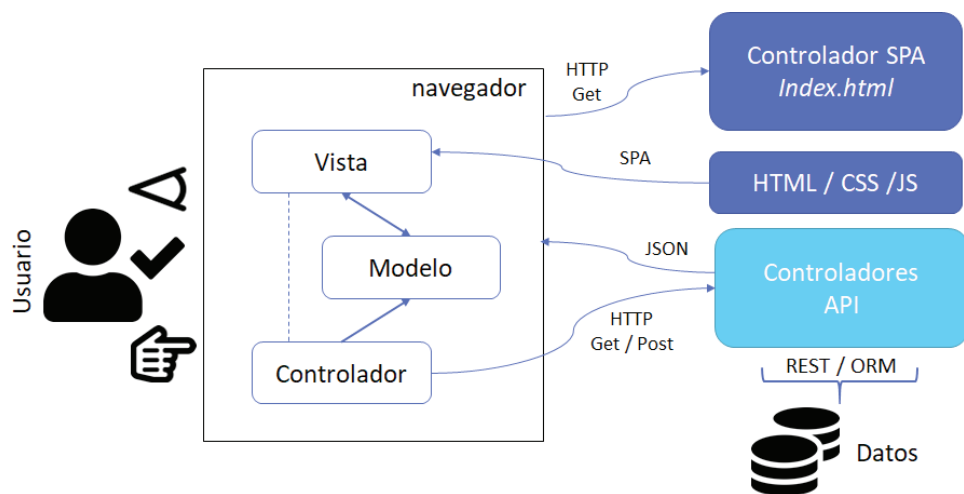


Figura 3. Representación de una web SPA con el patrón MVC²

A la hora de depurar y resolver problemas en este tipo de aplicaciones, el patrón utilizado hace que esto pueda suponer algo tratable o por el contrario nos de algún quebradero de cabeza. Es por ello, que *frameworks* como React.js intentan seguir patrones unidireccionales como Flux³, donde los datos fluyen en una única dirección, favoreciendo la detección y corrección de errores, mejorando el control de cambios en el *front-end* de la aplicación.

Por supuesto, hay múltiples patrones, y muchos de los *frameworks* de desarrollo actuales comparten algunos de estos, aunque con algunas modificaciones. No es objetivo de este libro profundizar sobre las arquitecturas y patrones de diseño existentes, pero se aconseja al lector revisar la literatura, adquiriendo un conocimiento de base sobre el que fundamentar sus decisiones de desarrollo y el uso de determinados *frameworks* y librerías.

2 Imagen inspirada en: https://cdn-images-1.medium.com/max/1600/0*dTPUy_vSyQPHe-kX [Último acceso: Julio, 2020]

3 Fuente original: <https://facebook.github.io/flux/docs/in-depth-overview/> [Último acceso: Julio, 2020]

INTRODUCCIÓN A LA PILA MERN

Dado que en este libro se trata el desarrollo de una aplicación web usando las tecnologías del *stack* o pila MERN, conviene describir cada una de estas, conforme a dicho acrónimo.

MERN es una combinación de las siguientes cuatro tecnologías: **M**ongoDB, **E**xpress.js, **R**eact.js y **N**ode.js, de ahí sus iniciales. La unión de estas tecnologías (junto con otras librerías de soporte) permite desarrollar aplicaciones web *full-stack* usando JavaScript como lenguaje de base, tanto en el cliente como en el servidor.

Este *stack* es similar a otros como MEAN, el cual utiliza el *framework* Angular.js en lugar de React.js para el desarrollo de la parte del *front-end* o del cliente. No obstante, existen otros muchos *stacks* que mezclan diferentes tecnologías, y sobre los cuales animamos a investigar y desarrollar, pero que no son objeto de este libro.

En el caso de MERN, la popularidad del lenguaje JavaScript y librerías como React.js, utilizada y liberada por la propia Facebook, en combinación con la sólida base que presentan MongoDB, Express.js y Node.js, hace que el desarrollo web *full-stack* con MERN sea uno de los más demandados a la vez que atractivos en el ámbito del desarrollo web.

Pero ¿qué podemos decir de cada uno de los elementos de la pila MERN?

MongoDB es una de las bases de datos no relacionales (NoSQL) orientadas a documentos más extendidas y populares que existen. Con un concepto muy diferente al de las bases de datos relacionales, supone una alternativa interesante y con multitud de ventajas que podemos aprovechar (escalabilidad, rapidez, eficiencia con grandes cantidades de datos, etc.). Así, en lugar de guardar los datos en registros y tablas, estos se almacenan en documentos BSON, que es una representación binaria del estándar JSON.

Express.js es un *framework* ligero para Node.js con el que podemos crear aplicaciones web de forma rápida y sencilla, ya que nos provee de toda la infraestructura de enrutamiento, sesiones, cookies y un largo etcétera. Realizar una infraestructura de servicios REST con Express.js es realmente rápido y sencillo.

React.js por su parte, es una librería de código abierto creada y mantenido por Facebook para el desarrollo de aplicaciones *front-end* de forma limpia, organizada y en una sola página, haciendo más efectiva la interacción entre los usuarios y la propia aplicación a través de componentes embebidos. El uso del denominado *Virtual DOM* hace que nuestra aplicación solo renderice y muestre al usuario las partes cambiantes, mejorando su eficiencia.

Por último, **Node.js** es un entorno JavaScript del lado del servidor que utiliza un modelo asíncrono y dirigido por eventos. La facilidad para el manejo de la entrada/salida, su rapidez, el excepcional soporte de los protocolos de comunicación web, así como la escalabilidad en el número de conexiones que es capaz de manejar, lo hacen uno de los entornos preferidos por la comunidad de desarrolladores para el despliegue de aplicaciones de *back-end*. Otra de las características de éxito de Node.js es su gestor de paquetes *npm*, que nos permite acceder a una ingente cantidad de librerías de código abierto desarrolladas por la propia comunidad. De hecho, Express.js y React.js se ofrecen como paquetes disponibles dentro de este ecosistema.

A lo largo del presente libro, haremos uso de estos elementos para el desarrollo de nuestras aplicaciones web y veremos de forma práctica y con más detalle las características que cada tecnología de las anteriores es capaz de ofrecernos. La Figura 4 muestra las principales herramientas y tecnologías heredadas de MERN que utilizaremos para el desarrollo de nuestra aplicación de microblogging, así como el lugar que ocupan en el desarrollo web. Durante las distintas secciones del libro usaremos dicha figura para identificar en qué parte de la pila nos encontramos, y qué tecnologías y conceptos se abordarán.

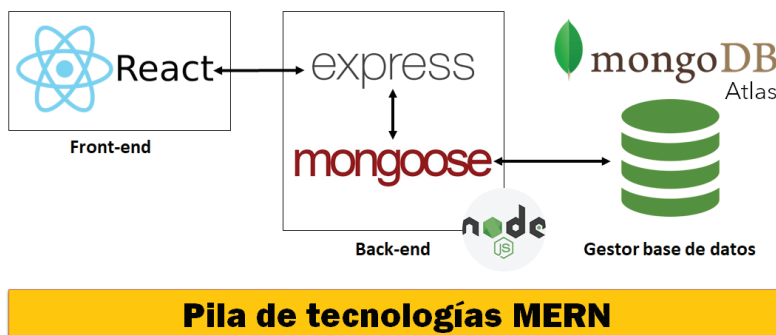


Figura 4. Tecnologías y herramientas de desarrollo a partir de la pila MERN

OPERACIONES CRUD Y HTTP. EL CORAZÓN DEL BACK-END

Antes de abordar el desarrollo web general y, de forma específica, el proceso de implementación bajo la pila o *stack* MERN, es deseable mencionar el rol que juegan las operaciones CRUD en las aplicaciones web (ver Figura 5), sobre todo en la parte del *back-end*. Si aún no las conoces, conviene que busques información al respecto, ya que suponen la base de cualquier desarrollo en el lado del servidor. No obstante, en esta sección explicaremos su propósito de forma general y mencionaremos los tipos existentes, así como su relación con las peticiones HTTP.



Figura 5. Operaciones CRUD

CRUD es el acrónimo de: *Create, Read, Update, Delete*; el cual hace referencia a las maneras en las que se puede operar sobre información almacenada. CRUD usualmente se refiere a operaciones llevadas a cabo en una base de datos o en un almacén de datos, pero también se puede aplicar a funciones de un nivel superior. A bajo nivel, las funciones utilizadas en las operaciones CRUD son dependientes del lenguaje y del sistema gestor de base de datos.

Cuando queremos llamar a operaciones CRUD desde un cliente web o una API, normalmente utilizamos servicios o solicitudes HTTP. Existen diversos tipos de solicitudes HTTP, sin embargo, todas ellas suelen estar asociadas a las diferentes operaciones CRUD, de modo que se establece una similitud entre cada operación

CRUD y cada solicitud HTTP. Por ejemplo, una solicitud POST podrá estar ligada a una operación *Create*, la solicitud GET a *Read*, PUT se asociaría al *Update* y el *Delete* de CRUD correspondería al DELETE de una solicitud HTTP. A continuación, veremos brevemente algunos ejemplos de implementación de estas operaciones CRUD.

Las operaciones *Create* o *Insert* añaden nuevos documentos JSON a una colección. Aquí hablamos de documentos JSON ya que las bases de datos con las que vamos a trabajar atendiendo a la pila de desarrollo MERN, serán no relacionales (NoSQL) basadas en documentos, y más concretamente con MongoDB. En MongoDB se manejan documentos en lugar de registros, y colecciones en lugar de tablas. En el caso de que la colección no exista, las operaciones `insert` crearán dicha colección. Así mismo, se puede insertar un solo documento (con `insertOne`) o muchos de una vez (con `insertMany`) en una colección. La Figura 6 muestra un ejemplo de inserción de un documento `user` en una colección `users` de MongoDB. Las figuras de esta sección están inspiradas en las figuras originales de las operaciones CRUD que aparecen en la web de referencia de MongoDB⁴.

```
db.users.insertOne(  ← colección
  {
    name: "sue",      ← campo: valor
    age: 26,          ← campo: valor
    status: "pending" ← campo: valor } Document (JSON)
  )
```

Figura 6. Ejemplo de inserción de un documento en una colección MongoDB

Por su parte, las operaciones *Read* devuelven uno o más documentos de una colección. Incluso se pueden especificar filtros para devolver únicamente los documentos que cumplan determinados criterios. En este caso, se usa la función `find` tal y como muestra el ejemplo de la Figura 7 para buscar y devolver el nombre y la dirección de cinco `users` mayores de 18 años, según el criterio de consulta proporcionado.

```
db.users.find(  ← colección
  { age: { $gt: 18 } }, ← criterio de consulta
  { name: 1, address: 1 } ← proyección
).limit(5)      ← modificador cursor
```

Figura 7. Ejemplo de búsqueda y devolución de un documento en una colección MongoDB

4 <https://docs.mongodb.com/manual/crud/> [Último acceso: Julio, 2020]

Las operaciones *Update* modifican uno o más documentos de una colección existente. Al igual que ocurría con las operaciones *Create*, en este caso tendremos varias funciones, dependiendo si queremos actualizar uno (`updateOne`), varios (`updateMany`), o bien llevar a cabo un reemplazo (`replaceOne`). En MongoDB, estas operaciones solamente se realizan sobre una colección. De igual forma, se pueden especificar criterios o filtros que identifican los documentos a actualizar, con la misma sintaxis que las operaciones *Read*. En la Figura 8 podemos ver un ejemplo de operación *Update*, la cual actualiza “el `status` de todos los `users` con `age` menor a 18”.

```
db.users.updateMany(           ← colección
  { age: { $lt: 18 } },        ← filtro de actualización
  { $set: { status: "reject" } } ← acción de actualización
)
```

Figura 8. Ejemplo de actualización de un documento en una colección, en base a unos criterios definidos

Finalmente, las operaciones *Delete* son las encargadas de eliminar y borrar documentos de una colección existente. Así, podremos eliminar un único documento (con `deleteOne`) o varios (con `deleteMany`); además de especificar los criterios o el filtro para el borrado de dichos documentos. Estas operaciones utilizan la misma sintaxis que venimos mostrando. Por ejemplo, la Figura 9 muestra un ejemplo de borrado de documentos de la colección `users` “cuyo `status` es `reject`”.

```
db.users.deleteMany(           ← colección
  { status: "reject" }          ← filtro de borrado
)
```

Figura 9. Ejemplo de borrado de documentos de una colección, en base a un criterio específico

Es importante destacar que todas las operaciones anteriores se pueden realizar de forma masiva, es decir, secuencialmente sobre una colección en particular, llevando a cabo varias operaciones bajo una misma función. Puedes ampliar información acerca de estos y otros detalles sobre la manipulación de datos en la web de referencia de MongoDB⁵.

5 <https://docs.mongodb.com/manual/> [Último acceso: Julio, 2020]

En secciones posteriores del libro complementaremos estas explicaciones con la configuración de una base de datos MongoDB, el uso de solicitudes HTTP y su relación con estas operaciones CRUD, y la integración de estas en una API REST desarrollada con Express.js. Así como el uso de *mapeadores* de datos ODM que nos facilitarán las tareas realizadas por las operaciones CRUD, consiguiendo un mayor nivel de abstracción sobre estas.



LA ESPECIFICACIÓN JAVASCRIPT

El lenguaje JavaScript constituye uno de los principales lenguajes en el mundo de Internet. Junto con CSS y HTML conforma la tupla más famosa del desarrollo web de base. Desde sus inicios, JavaScript ha estado pensado para el manejo del comportamiento de los elementos de la interfaz, que en sus inicios fueron documentos HTML estáticos, acompañados del estándar CSS para definir el estilo visual de dichas páginas web. Su orientación a eventos dota a JavaScript de uno de los lenguajes más potentes para manipular el comportamiento de los elementos de cualquier página o aplicación web.

JavaScript ha ido evolucionando año tras año, presentando gran compatibilidad con todos los navegadores actuales. Tras la especificación ES5 llegó la denominada ECMAScript 6 (ES6), la cual supuso un nuevo estándar JavaScript con importantes novedades sobre la versión anterior. Uno de sus principales cambios respecto a versiones anteriores, es la introducción de una nueva sintaxis manteniendo la retrocompatibilidad, además es modular, y esta vez sí que tiene soporte directo para definir clases y herencia. No obstante, como muchos navegadores actuales “no entienden” esta nueva especificación, existen multitud de herramientas que permiten la traducción entre estándares JavaScript y otras particularidades, permitiéndonos trabajar de forma nativa con ES6 (y posteriores) sin preocuparnos demasiado por ese proceso de conversión (que es llevado a cabo por los denominados *transpiladores* de código).

No es objetivo de este libro enseñar JavaScript ni entrar en detalle sobre los cambios que las distintas versiones traen consigo año tras año, pero es muy recomendable conocer las generalidades del lenguaje para entender el desarrollo del proyecto que aquí se presenta, así como el código que se irá mostrando en secciones posteriores. Igualmente, se recomienda encarecidamente indagar sobre la

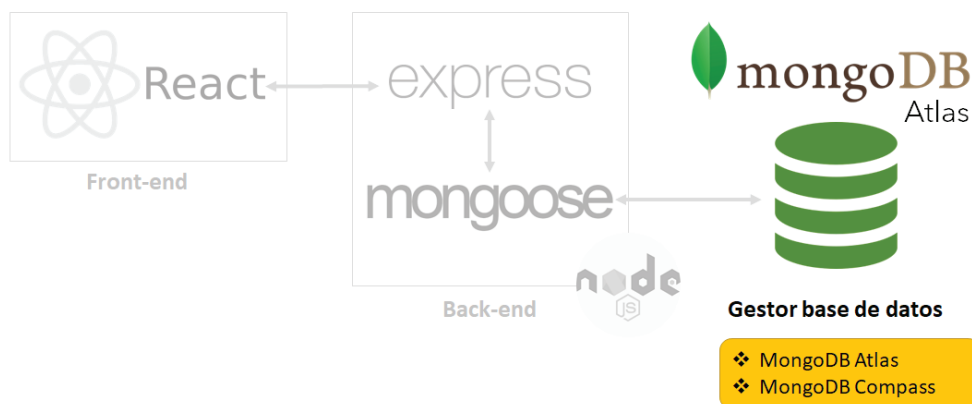
implementación y uso de promesas, relacionado con manejo de datos asíncronos en JavaScript, a partir de su especificación ES6.

Por su parte, los *frameworks* y herramientas que forman parte de la pila MERN están implementados en JavaScript (lo que incluye Node.js, Express.js y React.js), de ahí su importancia. Además, las librerías utilizadas en el desarrollo de nuestra aplicación son igualmente librerías implementadas en JavaScript, donde muchas de ellas se han desarrollado bajo las últimas especificaciones del lenguaje y cuentan con soporte de llamadas a funciones basado en promesas.

Actualmente existe una ingente cantidad de recursos que nos permiten aprender y mejorar nuestros conocimientos de JavaScript de forma sencilla y rápida, con lo que esto no debería representar ningún inconveniente a partir de ahora, así que ¡no nos preocupemos demasiado por ello!

MONGODB

Atendiendo a la zona coloreada de la Figura 10, en esta parte nos centraremos en la creación y configuración de una base de datos MongoDB (en la Nube), como servicio de *MongoDB Atlas* y la posibilidad de manipular los datos con herramientas gráficas como *MongoDB Compass*. Es interesante destacar que la definición del esquema de base de datos se corresponde con los cimientos de nuestra aplicación web.



Pila de tecnologías MERN

Figura 10. Tecnologías y herramientas de desarrollo a partir de la pila MERN. MongoDB

1.1 CONSIDERACIONES INICIALES

El almacenamiento de datos en el desarrollo web con MERN se realiza utilizando MongoDB como motor de base de datos. MongoDB es hoy en día uno de los sistemas de base de datos NoSQL más famosos y utilizados. En MongoDB, los datos no están almacenados en tablas, ni en registros, sino en colecciones de documentos en formato BSON (que constituye una representación binaria del formato JSON).

No es objetivo de este libro detenerse en las ventajas y desventajas de estas bases de datos con respecto a las relacionales SQL. Sin embargo, debido a la esquematización dinámica de estas bases de datos entre otras características y, como consecuencia, su rapidez y facilidad de acceso, muchos sistemas que almacenan grandes cantidades de datos optan por su uso.

1.2 BASES DE DATOS LOCALES VERSUS SERVICIOS DE TERCEROS

Otra de las preguntas que nos podríamos hacer respecto al almacenamiento de los datos de nuestras aplicaciones web, es la posibilidad de tener un servidor propio de base de datos y gestionarlo nosotros mismos, o bien delegar esta tarea a una compañía externa haciendo uso de servicios de terceros, desplegando dicha base de datos en la Nube. La mayoría de las aplicaciones comerciales actuales optan por tener una arquitectura descentralizada mediante esta última opción. De esta forma, no debemos preocuparnos por el mantenimiento del servidor de base de datos, su seguridad, su escalabilidad y todos esos temas que deberíamos tener en consideración si decidimos tener un servidor de bases de datos propio.

Los creadores de MongoDB conocen este problema y nos ofrecen un completo entorno de almacenamiento en la Nube denominado MongoDB Atlas. MongoDB Atlas se trata de un DaS (*Database as a Service*) con el que podemos crear completos clústeres de bases de datos MongoDB en la Nube de forma sencilla y sin preocuparnos por su instalación y administración.

Por supuesto, hoy en día, MongoDB Atlas no es la única opción existente. Pero gracias a que tiene el soporte de la compañía que creó MongoDB, su facilidad de uso, y la posibilidad de crear un clúster de forma gratuita, lo utilizaremos para alojar la base de datos de nuestra aplicación web. En el caso de que la cantidad de datos almacenados crezca, las transacciones con la base de datos aumenten y nuestras necesidades sean mayores, el servicio nos aconsejará pasarnos a un clúster de pago si lo deseamos.

En los siguientes apartados veremos, por un lado, cómo utilizar MongoDB Atlas como servicio de base de datos MongoDB en la Nube (recomendado), y por otro, la opción de realizar una instalación de un servidor MongoDB en local, así como usar la *shell* de MongoDB para conectarse a nuestra base de datos previamente creada en MongoDB Atlas.

1.2.1 Registro y configuración de MongoDB Atlas

Lo primero que haremos será dirigirnos a la web de MongoDB Atlas⁶ y crear una nueva cuenta gratuita como se muestra en la Figura 11. Tengamos en cuenta que las capturas mostradas a continuación podrían variar con respecto a la apariencia actual de la web.

Comience gratis

No se necesita tarjeta de crédito.
La interfaz de usuario de MongoDB Atlas sólo está disponible en inglés.

Dirección de correo electrónico

Nombre Apellidos

Contraseña

- ✓ 8 caracteres como mínimo
- ✓ Un número
- ✓ Una letra
- ✓ Un carácter especial

☐ Acepto términos de servicio.

Comience gratis

Included with your free cloud database:

- ✓ 512 MB of Storage
- ✓ Shared RAM
- ✓ Highly available replica sets, end-to-end encryption, automated patches, REST API

Additionally, get access to the following when you launch a dedicated cluster:

- ✓ 10 GB or more of storage
- ✓ Dedicated RAM
- ✓ Performance optimization tools
- ✓ Backups & point-in-time recovery
- ✓ Enterprise security features including encryption key management, LDAP integration, and granular database auditing

Clústeres globales:

Figura 11. Registro gratuito en MongoDB Atlas

Una vez dado de alta, esperamos que se inicialice y configure nuestro espacio, y procedemos a la creación de un clúster gratuito (ver Figura 12).

6 <https://www.mongodb.com/cloud/atlas> [Último acceso: Julio, 2020]

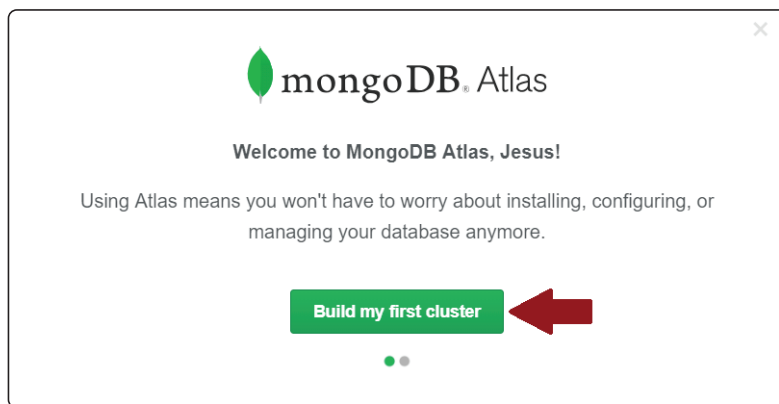


Figura 12. Creación del primer clúster en MongoDB Atlas

Podremos hacerlo respondiendo a algunas preguntas y/o seleccionando las opciones de configuración necesarias. Podemos optar por aceptar las opciones de configuración por defecto que nos propone MongoDB Atlas y crear nuestro clúster (ver ejemplo en la Figura 13).

Figura 13. Configuración general del clúster

Hecho esto, esperamos algunos minutos a que se cree el clúster en un servidor del proveedor seleccionado con las opciones correspondientes, y visualizaremos el panel de control que nos ofrece MongoDB Atlas. Lo siguiente será crear un usuario (en la pestaña *Security / MongoDB Users / Add new user*). Le damos un nombre de usuario, una clave o *password* y unos privilegios (ejemplo: *Read & Write to any database*) tal y como se muestra en la Figura 14.

NOTA

Recuerda que la plataforma web de MongoDB Atlas podría haber sufrido algunas actualizaciones y disponer de las opciones citadas en esta sección en otras partes de la web, aunque el proceso seguirá siendo el mismo que el presentado.

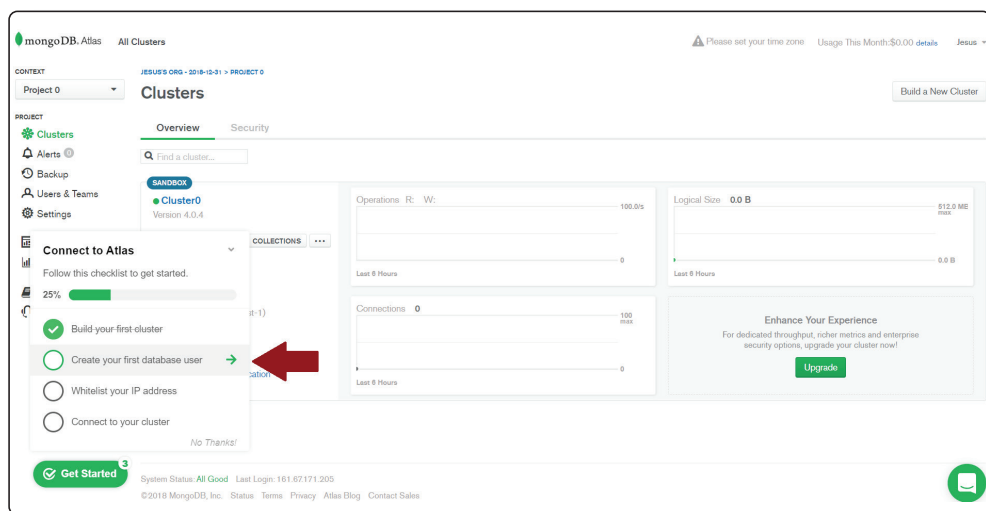


Figura 14. Configuración del usuario del clúster

Lo siguiente será crear una lista blanca de conexiones a nuestro clúster de base de datos o servicios MongoDB Atlas, es decir, configurar qué direcciones IP podrán conectarse con nuestra cuenta de MongoDB Atlas (*Security / IP Whitelist / Add IP Address*). Para nuestro ejemplo, permitiremos acceso desde cualquier sitio (*Allow Access from Anywhere*) tal y como se muestra en la Figura 15.

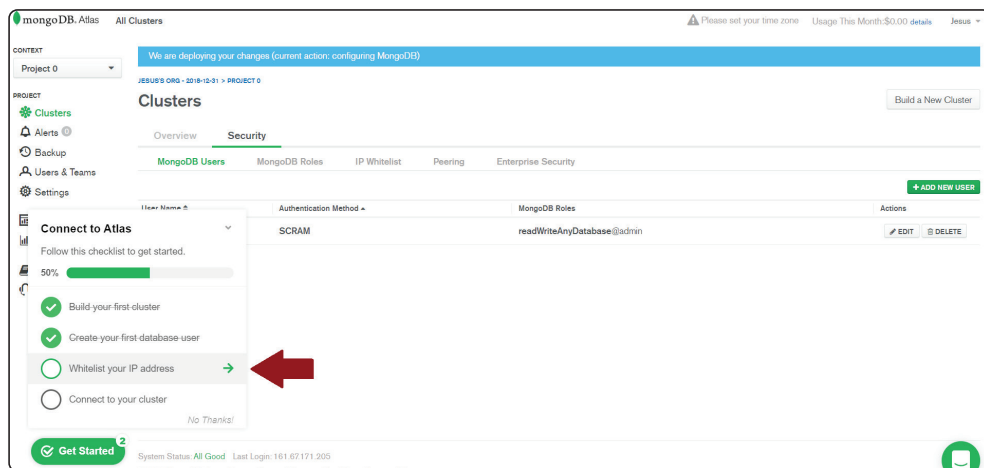


Figura 15. Configuración de la IP whitelist

Desde el propio panel podemos crear una base de datos para trabajar sobre ella a partir de ahora (ver Figura 16).

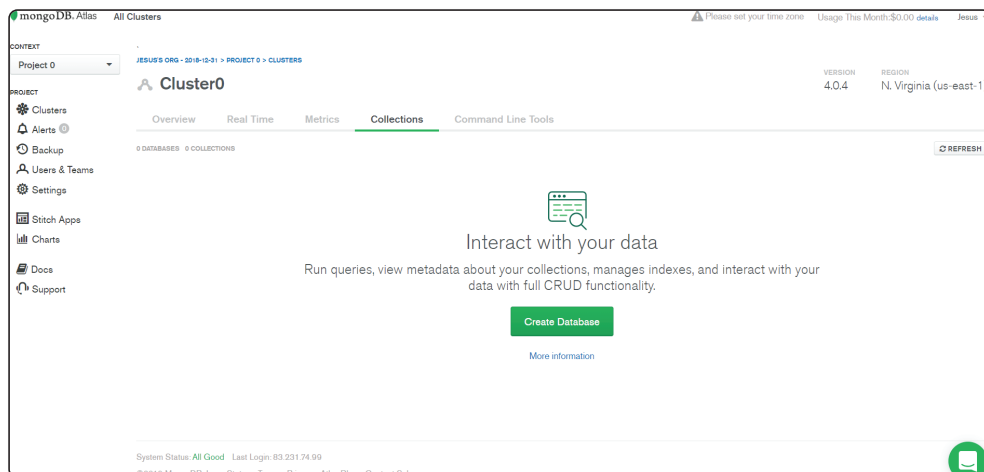
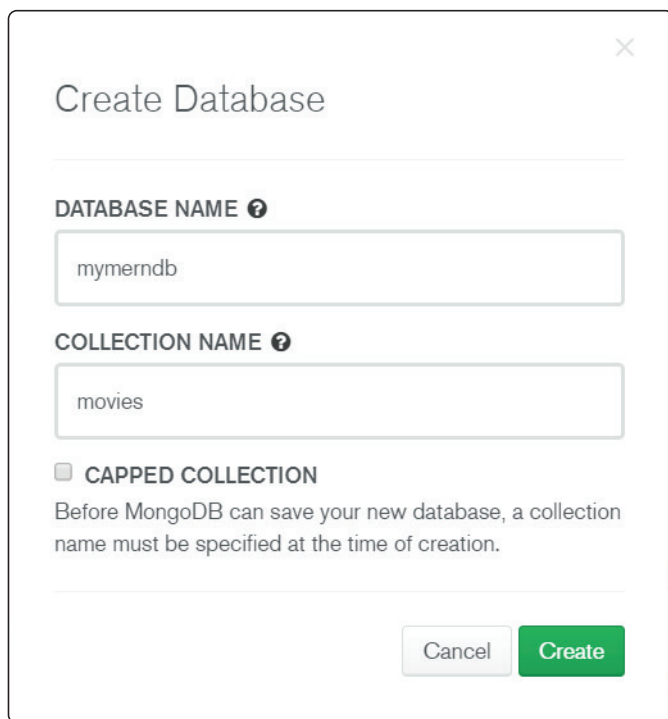


Figura 16. Clúster listo para crear nuestra base de datos

Para ello, pulsamos el botón “collections” del panel, y crearemos nuestra base de datos (llamémosla por ejemplo “mymerndb”) tal y como se presenta en la Figura 17. Es requisito necesario crear la base de datos con una primera colección (por ejemplo: “users”). La opción “capped collection” permite crear colecciones de

un tamaño (en bytes) determinado de forma que cuando dicha colección se llena de documentos, se borran los primeros para poder seguir almacenando los últimos. Queda a tu elección seleccionar o no esta opción.



Create Database

DATABASE NAME ⓘ

mymerndb

COLLECTION NAME ⓘ

movies

☐ CAPPED COLLECTION

Before MongoDB can save your new database, a collection name must be specified at the time of creation.

Cancel Create

Figura 17. Creación de la base de datos en MongoDB Atlas⁷

A partir de ahora, ya podemos conectar con nuestro clúster “desde el exterior”. Tenemos varias opciones: hacerlo mediante la consola o *shell* de MongoDB desde la línea de comandos, a través de nuestra aplicación, o bien desde MongoDB Compass. En estos últimos casos basta con seguir las indicaciones que nos ofrece el panel MongoDB Atlas, tal y como se indica en la Figura 18 y en la Figura 19.

⁷ También la podemos crear desde la Shell de MongoDB conectando el clúster mediante un par de comandos con el terminal de nuestro PC, aunque no entraremos en detalle sobre esto.

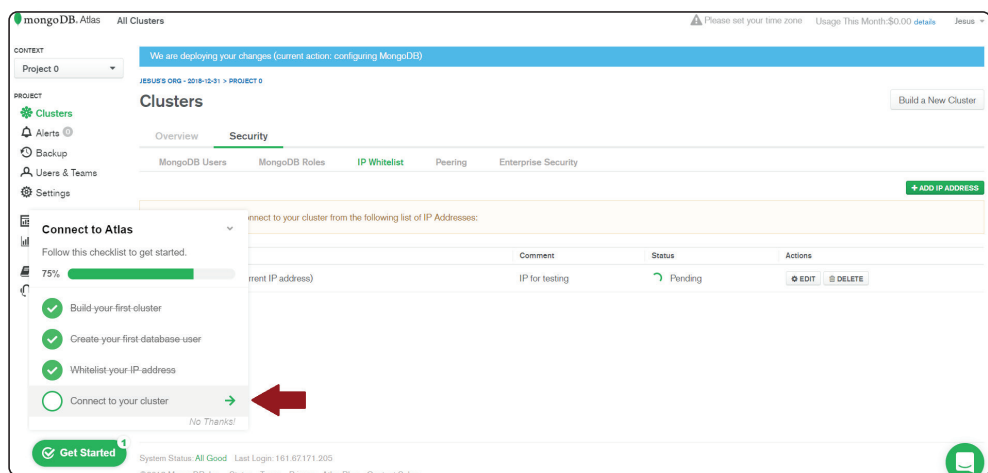


Figura 18. Conexión al clúster de base de datos creado

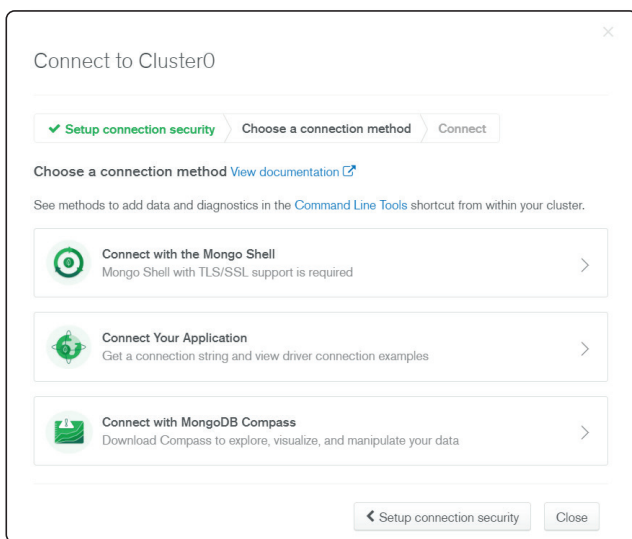


Figura 19. Opciones de conexión al clúster creado

1.2.2 Instalación local y uso de la shell de MongoDB

Si lo que vamos a usar es una base de datos remota a través de servicios en la Nube como los ofrecidos por MongoDB Atlas, no es necesario instalar todo el paquete

MongoDB en el PC, sino que basta únicamente con instalar la `shell` de MongoDB para poder operar con dichos servicios. Sin embargo, en este punto efectuaremos brevemente una instalación completa de un servidor MongoDB, aunque la intención será testear nuestro clúster de base de datos en la Nube con el servicio MongoDB Atlas configurado. Por supuesto, incluso desde la web de MongoDB Atlas podemos descargar múltiples herramientas como puede ser la propia `shell` de MongoDB y la aplicación MongoDB Compass, la cual veremos en las siguientes secciones.

Lo primero será ir a la sección de descargas de la web oficial de MongoDB⁸ y descargar la versión adecuada a nuestro sistema operativo.

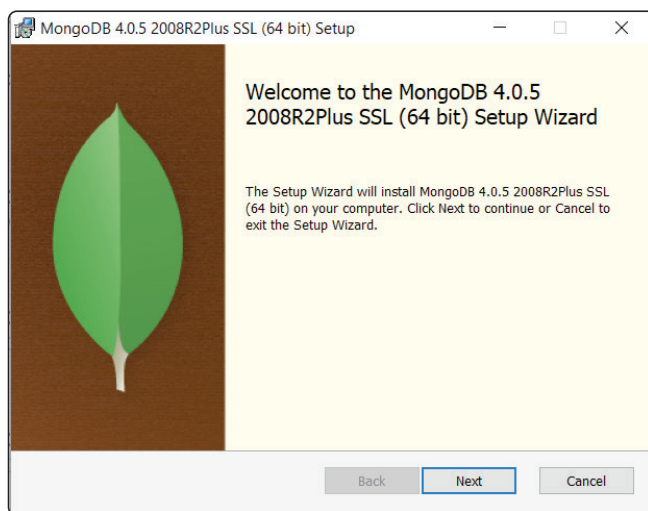


Figura 20. Asistente de instalación de MongoDB

A continuación, iniciamos la instalación del servidor MongoDB en nuestro PC (por ejemplo, la versión “Complete”) usando el asistente de instalación (ver Figura 20), con las opciones por defecto. En el mismo proceso de instalación de MongoDB podemos indicar que se nos instale la herramienta MongoDB Compass, la cual explicaremos en el siguiente apartado. También podemos obtenerla e instalarla desde la propia web de MongoDB Compass e incluso desde la plataforma de MongoDB Atlas.

8 <https://www.mongodb.com/download-center/community> [Último acceso: Julio, 2020]

Comprobamos la instalación correcta de MongoDB en “Archivos de Programa” (recuerda que estamos trabajando con el sistema operativo Windows 10, y las rutas pueden variar si se instala en un sistema operativo diferente). Igualmente podemos abrir un terminal, dirigimos a la ruta de instalación (podría variar de la siguiente) y comprobar que está correctamente instalado tecleando:

```
.....  
C:\Program Files\MongoDB\Server\4.0\bin>mongo --help  
.....
```

También podemos configurar el PATH anterior en el sistema operativo para no tener que acceder a la ruta `/bin` cada vez que deseemos ejecutar un comando, pero no entraremos en detalle sobre esto ya que no forma parte del alcance de esta sección y no se estima necesario.

Si deseamos configurar nuestro PC como un servidor MongoDB para alojar bases de datos Mongo en él, debemos seguir los pasos indicados en la documentación de MongoDB⁹. En nuestro caso bastará con comprobar que podemos ejecutar los comandos de la `shell` de MongoDB. Con ello, podemos comprobar, como ya se mencionó en el apartado anterior, la conexión con nuestro clúster de MongoDB Atlas previamente creado, de esta forma:

```
.....  
C:\Program Files\MongoDB\Server\4.0\bin>mongo "mongodb+srv://cluster0-40okh.  
mongodb.net/test" --username <MI_USUARIO>  
.....
```

Igualmente debemos sustituir esta cadena por la suministrada por MongoDB Atlas, de acuerdo también al nombre de usuario correcto y la ruta donde está alojado nuestro MongoDB.

Una vez conectamos con el clúster, podemos hacer uso de los comandos de la `shell` de MongoDB¹⁰ para operar sobre las bases de datos de nuestro clúster desde el propio terminal del sistema operativo, como si fuera una base de datos local.

9 <https://docs.mongodb.com/manual/administration/install-community/> [Último acceso: Julio, 2020]

10 <https://docs.mongodb.com/manual/reference/mongo-shell/> [Último acceso: Julio, 2020]

1.3 HERRAMIENTAS GRÁFICAS PARA LA MANIPULACIÓN DE DATOS

Aunque no es necesario disponer de herramientas visuales para la gestión de bases de datos, sí que son de gran utilidad puesto que hacen más intuitiva y eficiente la gestión de dichos datos (en el caso de bases de datos NoSQL: manipulación de colecciones, documentos, operaciones de inserción, consultas, actualizaciones, borrados, etc.). Por ello, es altamente recomendable la instalación de alguna de estas herramientas que nos permitan gestionar y visualizar los datos de forma gráfica.

El servicio MongoDB Atlas posee características para la manipulación gráfica de los datos contenidos en la base de datos de nuestro clúster. Sin embargo, esta suele ser una opción de pago a priori.

En el caso de bases de datos MongoDB, una de las herramientas más utilizadas es la conocida como MongoDB Compass, la cual puede instalarse en nuestro PC como parte del servidor MongoDB o bien como herramienta adicional. A continuación, veremos cómo instalar la herramienta, así como los primeros pasos para comenzar a usarla.

1.3.1 Instalación y uso de MongoDB Compass

Vamos a instalar MongoDB Compass en el caso de que no se instalara junto con MongoDB Server (ver apartado **Instalación local**). Como se ha mencionado anteriormente, MongoDB Compass es una herramienta gráfica para trabajar con bases de datos MongoDB sencilla de utilizar, que además nos permite la conexión con bases de datos remotas como las creadas en MongoDB Atlas.

Primero accedemos a su página oficial¹¹, y descargamos la última versión estable del programa para nuestro sistema operativo. Iniciaremos la instalación y mantendremos seleccionadas las opciones por defecto.

Opcionalmente también podemos acceder a la descarga e instalación de MongoDB Compass desde Mongo DB Atlas. Una vez instalado, vamos a crear una nueva conexión a nuestro clúster de MongoDB Atlas, para que se conecte con este desde la herramienta, tal y como se indica en la Figura 21.

11 <https://www.mongodb.com/products/compass> [Último acceso: Julio, 2020]

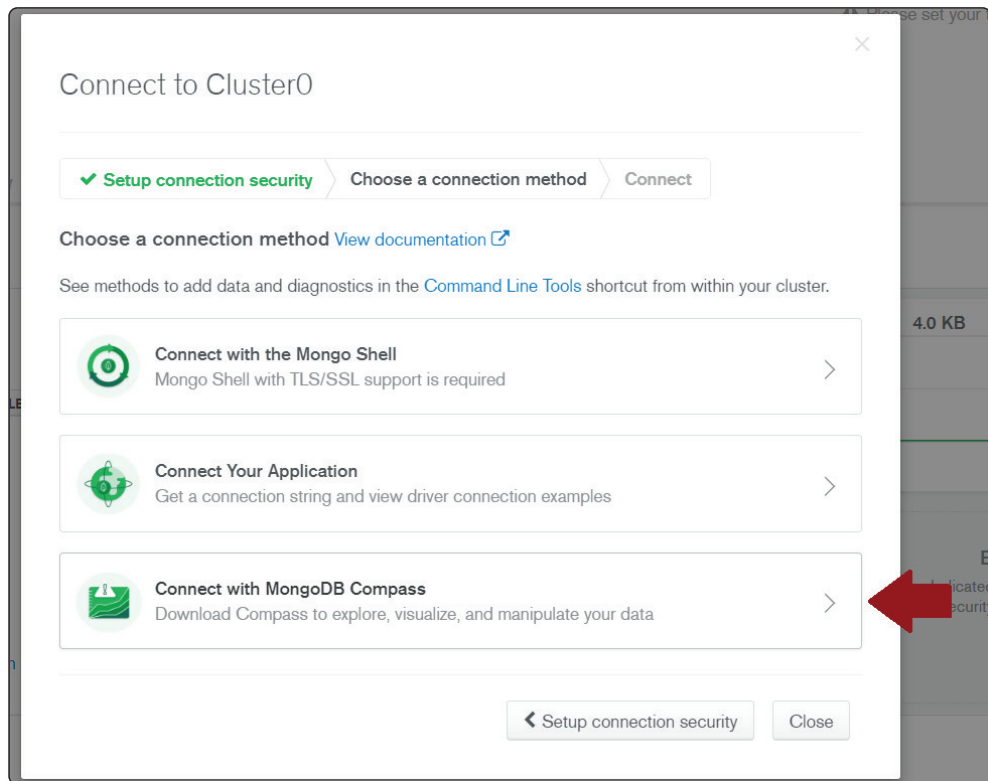


Figura 21. Selección de MongoDB Compass para la conexión del clúster

Copiaremos la cadena de conexión que nos ofrece MongoDB Atlas en el portapapeles para usarla en MongoDB Compass. Así, abrimos la aplicación MongoDB Compass en nuestro PC y se detectará automáticamente dicha cadena, rellenando el formulario, a falta del *password* (que debemos insertar manualmente) para conectar con los servicios de MongoDB Atlas.

A partir de aquí podemos seleccionar la base de datos creada anteriormente, y ver las colecciones y sus documentos. Desde la interfaz de MongoDB Compass podemos añadir nuevos documentos, editarlos, realizar consultas, visualizar estadísticas, etc. La Figura 22 muestra las bases de datos de nuestro clúster de MongoDB Atlas en la herramienta MongoDB Compass.

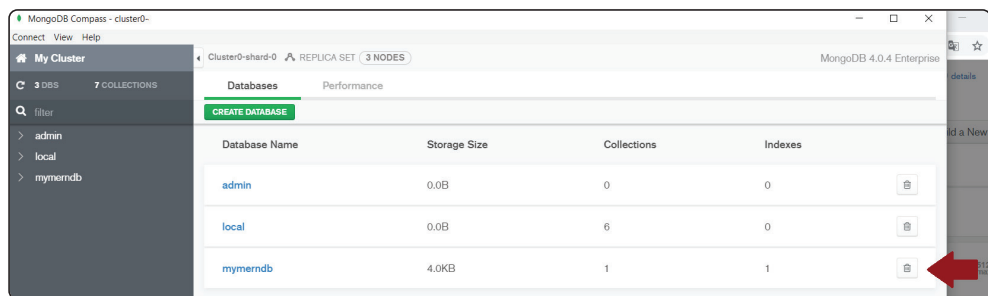


Figura 22. Visualización de las bases de datos del clúster de MongoDB Atlas en MongoDB Compass

1.4 LECTURAS RECOMENDADAS

Conforme a los contenidos desarrollados en esta primera parte, a continuación, se presentan una serie de referencias a tener en cuenta si se desea profundizar sobre las bases de datos NoSQL y MongoDB:

- Membrey, P., Hows, D., Plugge, E. *MongoDB Basics*. Apress, 2014.
- Bradshaw, S., Brazil, E., Chodorow, K. *MongoDb: The Definitive Guide: Powerful and Scalable Data Storage*. O'Reilly UK Ltd. (3ª Edición), 2019.

- Documentación Oficial de MongoDB:

<https://docs.mongodb.com/manual/>, [Último acceso: Julio, 2020].

En el caso de MongoDB Atlas, la mejor y más actual fuente de consulta es la ofrecida por su web de referencia:

- Documentación Oficial de MongoDB Atlas:

<https://docs.atlas.mongodb.com/>, [Último acceso: Julio, 2020].

Por otra parte, aunque multitud de libros sobre MongoDB lo mencionan como parte de su contenido, la mejor y más completa fuente de información sobre MongoDB Compass se encuentra en su web oficial:

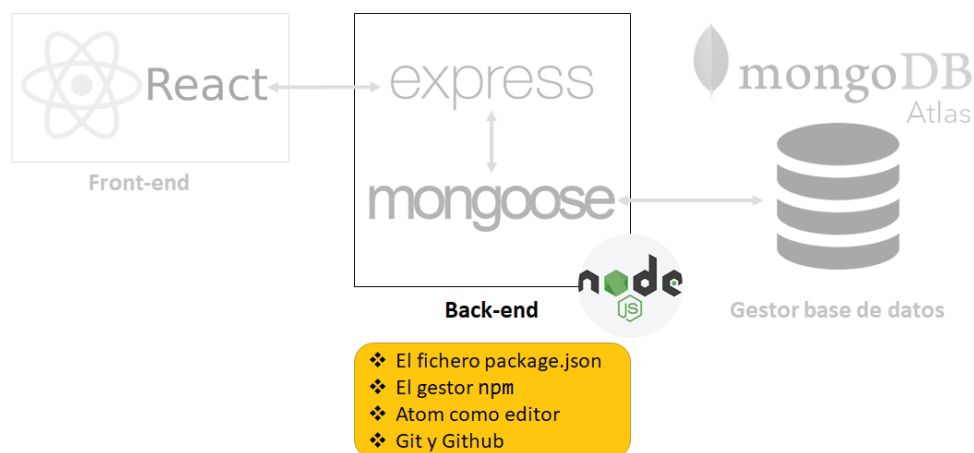
- Documentación Oficial de MongoDB Compass:

<https://docs.mongodb.com/compass/>, [Último acceso: Julio, 2020].

2

NODE.JS

Volviendo a la pila de tecnologías MERN, en esta parte centraremos nuestra atención en el entorno Node.js. Atendiendo al área coloreada de la Figura 23, puede verse en qué lugar de la pila nos encontramos, así como varios conceptos y herramientas imprescindibles que serán tratados a continuación, como el papel que juega el fichero “*package.json*” o el gestor de paquetes npm.



Pila de tecnologías MERN

Figura 23. Tecnologías y herramientas de desarrollo a partir de la pila MERN. Node.js

2.1 PRIMEROS PASOS

Tal y como se mencionó al comienzo del libro, Node.js es un entorno JavaScript que permite el fácil despliegue y configuración de aplicaciones del lado del servidor. Node.js está pensado para facilitar el manejo de las comunicaciones asíncronas, la entrada/salida y los diferentes protocolos de red constituyéndose como un auténtico ecosistema. Además, cuenta con un gran soporte de la comunidad, posibilitando el uso de multitud de librerías y *frameworks* para realizar prácticamente lo que necesitemos. Gracias al gestor de paquetes *npm*, el cual explicaremos en secciones posteriores, podemos llevar a cabo la instalación de nuevas librerías de forma sencilla y eficiente.

Para instalar el entorno Node.js en nuestro PC, nos dirigimos a la web oficial¹² y descargamos la última versión estable del sistema (LTS recomendada) según nuestro sistema operativo. Una vez descargada, comenzamos el proceso de instalación con el asistente (ver Figura 24).

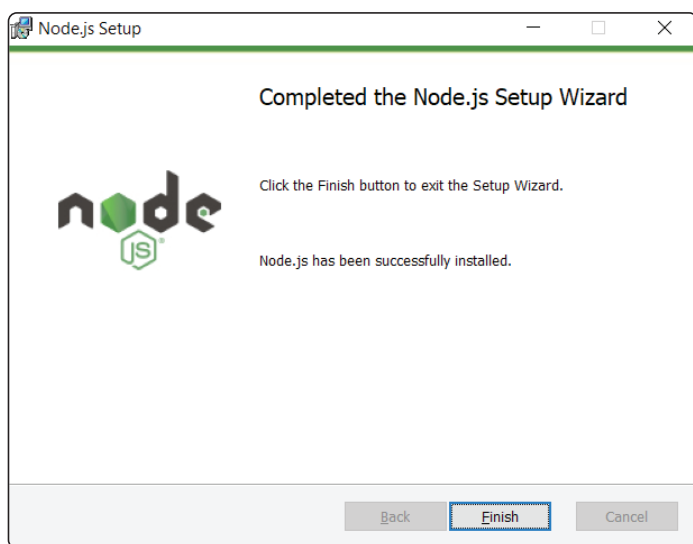


Figura 24. Asistente de instalación de Node.js

Una vez instalado, comprobamos que se encuentra en “Archivos de programa” (en el caso de trabajar con el sistema operativo Windows 10), así como la existencia de una nueva aplicación que nos abre la *shell* de Node.js. No obstante,

12 <https://nodejs.org/es/> [Último acceso: Julio, 2020]

podemos comprobar que todo está correctamente instalado abriendo un terminal o consola en el sistema operativo y tecleando el siguiente comando, que nos mostrará la versión de Node.js instalada:

```
.....  
C:\>node --version  
.....
```

Junto con el entorno Node.js, se nos instala una versión del gestor de paquetes npm (que explicaremos más adelante). Igualmente podemos conocer la versión instalada tecleando:

```
.....  
C:\>npm --version  
.....
```

Si lo queremos instalar en un sistema operativo Linux, podemos seguir las instrucciones de páginas existentes en internet¹³ con los pasos necesarios.

Con esto, ya tenemos Node.js instalado en nuestro sistema.

2.2 HOLA MUNDO EN NODEJS

.....

Para comenzar con un ejemplo sencillo dedicaremos esta sección a la implementación del clásico “Hola Mundo” en Node.js.

Es importante destacar que la arquitectura Node.js está orientada a eventos, es decir, que la ejecución de los programas está determinada por los sucesos que ocurren en el sistema.

Antes de profundizar en el ejemplo, abriremos la consola de Node.js (o ejecutaremos el comando **node** en un terminal), y probaremos a mostrar un mensaje en pantalla con la sentencia:

```
.....  
console.log('Hola Mundo!');  
.....
```

El resultado debería ser algo como lo mostrado en la Figura 25.

13 <https://geekytheory.com/instalacion-de-nodejs/> [Último acceso: Julio, 2020]

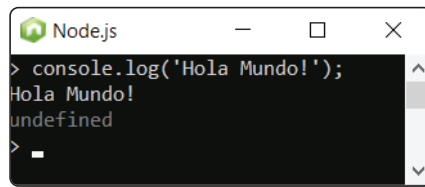


Figura 25. Ejemplo de “Hola Mundo” con Node.js

Para detener la ejecución de Node.js y salir del programa basta con pulsar CTRL+C.

Ya sabemos cómo mostrar mensajes en la consola. Ahora vamos a ver una implementación básica de un servidor Node.js que servirá una respuesta sencilla al usuario. Dado que Node.js se basa en JavaScript, los archivos tendrán una extensión “*.js”. Como ya se ha comentado, este libro no tiene como objetivo explicar JavaScript, con lo que convendría echar un vistazo a las lecturas recomendadas en esta parte para profundizar sobre el lenguaje si aún no lo conoces.

Si por ejemplo creamos un archivo que se llame “server.js”, la ejecución de este programa la realizaremos tecleando el comando:

```
.....  
C:\>node server.js  
.....
```

Como nuestro programa será el típico “hola mundo”, crearemos un fichero “holamundo.js” con un editor cualquiera, como por ejemplo Notepad++¹⁴. Más adelante veremos cómo codificar usando editores más completos.

Para mostrar lo que queremos en el navegador web, tenemos que implementar un servidor básico en nuestro programa “holamundo.js”. Para ello, importaremos el módulo `http` de esta forma:

```
.....  
var http = require('http');  
.....
```

Después creamos el servidor con una función que tiene dos parámetros: la petición (`req`) y la respuesta (`res`). Las siguientes líneas se encargan de escribir en la cabecera de la petición el código de estado 200 (página satisfactoria) y el tipo de contenido ‘text/plain’ (texto plano). Con el `res.end`, enviamos en este caso la cadena ‘Hola Mundo’ al navegador como respuesta. Y la última línea le dice al servidor en qué puerto va a escuchar.

14 <https://notepad-plus-plus.org/> [Último acceso: Julio, 2020]

```
const server = http.createServer(function (req, res) {  
  res.writeHead(200, {'content-type': 'text/plain'});  
  res.end('Hola Mundo');  
});  
server.listen(8000);
```

Con ello, vamos a la ruta dónde tenemos alojado nuestro programa “holamundo.js” y lo ejecutamos con:

```
C:\>node holamundo.js
```

Ahora nos dirigimos al navegador y tecleamos `localhost:8000` en la barra de dirección. A continuación, veremos nuestro “Hola Mundo” en Node.js. En este caso, “holamundo.js” será una aplicación sencilla con las características necesarias para actuar de programa servidor.

Por supuesto existen varias formas de implementarlo, pero la estructura suele ser la misma. Node.js nos servirá como base para inicializar nuestro servidor y servir páginas y/o respuestas a diferentes peticiones. Todo esto lo trataremos con más detalle en la **Parte 3: Express**.

2.3 EL PACKAGE.JSON Y EL GESTOR DE PAQUETES NPM

Cuando se trabaja con proyectos Node.js, es habitual tener un fichero denominado “package.json” que define las características principales y las dependencias de nuestro proyecto con respecto a otras aplicaciones y librerías.

El “package.json” puede considerarse como un manifiesto de nuestro proyecto. Además, sigue el estándar JSON (<clave, valor>), con las especificaciones que esto implica (cada clave debe tener comillas dobles, solo ciertos valores son válidos, etc.). Dentro del archivo “package.json” se definen y manejan características como:

- Nombre del proyecto
- Versión
- Dependencias
- Repositorio
- Autores
- Licencia
- Scripts (útiles por ejemplo para seleccionar entornos de depuración y/o producción)
- Muchas cosas más.

Pero ¿cómo se puede crear dicho archivo? Existen dos formas de hacerlo:

- **Manual:** Simplemente se crea un archivo denominado “package.json” en la raíz del proyecto (es decir, el directorio que contendrá nuestra aplicación) y se incluye como mínimo las características “name, version”. A medida que necesitemos trabajar con otras librerías, o añadir otras características, editaremos nuestro fichero “package.json”.
- **Automática:** A través de comandos como el definido a continuación y usando un gestor de paquetes como npm (ver más abajo) podemos crear de forma automática el fichero “package.json”. Con un asistente básico le daremos información acerca del autor, versión, repositorio, etc. que la propia herramienta se encargará de incluir en nuestro “package.json”. Después, ya tendremos el archivo creado en la raíz del directorio del proyecto.

```
.....  
C:\miproyecto>npm init  
.....
```

A continuación se presenta un ejemplo de archivo “package.json”, en el que se definen las características principales de un proyecto y una serie de dependencias.

```
.....  
{  
  "name": "test-project",  
  "version": "1.0.0",  
  "description": "Another project",  
  "main": "src/main.js",  
  "private": true,  
  "scripts": {  
    "dev": "webpack-dev-server --inline --progress --config build/webpack.dev.  
conf.js",  
    "start": "npm run dev",  
    "build": "node build/build.js"  
  },  
  "dependencies": {  
    "vue": "^2.5.2"  
  },  
  "devDependencies": {  
    "autoprefixer": "^7.1.2",  
    "babel-core": "^6.22.1",  
    "babel-eslint": "^8.2.1",  
    "babel-loader": "^7.1.1",  
    "babel-plugin-dynamic-import-node": "^1.2.0",  
    "eslint": "^4.15.0",  
.....
```

```
    "file-loader": "^1.1.4",
    "friendly-errors-webpack-plugin": "^1.6.1",
    "html-webpack-plugin": "^2.30.1",
    "node-notifier": "^5.1.2",
    "webpack-merge": "^4.1.0"
  },
  "engines": {
    "node": ">= 6.0.0",
    "npm": ">= 3.0.0"
  },
  "browserslist": [
    "> 1%",
    "last 2 versions",
    "not ie <= 8"
  ]
}
```

Puedes encontrar más información acerca del contenido y estructura del archivo “package.json” en la red¹⁵.

Por otra parte, cuando creamos un proyecto, por ejemplo, con una herramienta de *scaffolding*¹⁶ como veremos en la **Parte 3: Express**, se creará un “package.json” por defecto, y usando los comandos del gestor de paquetes *npm*, editaremos sus dependencias y características sin manipular directamente el contenido del archivo.

2.3.1 El gestor npm

npm es un gestor de paquetes para Node.js, con el que los desarrolladores pueden crear, compartir y reutilizar desde módulos o pequeñas librerías, hasta aplicaciones completas de un extenso catálogo gestionado por la comunidad.

Cuando instalamos Node.js, suele incluirse una versión de *npm* (como ocurre al instalarlo en un sistema operativo Windows). En caso contrario, deberíamos instalarlo a posteriori. En sistemas operativos GNU/Linux, el comando sería (siempre a partir de la ruta de nuestro proyecto):

```
>sudo apt-get install npm
```

15 <https://flaviocopes.com/package-json> [Último acceso: Julio, 2020]

16 Generación semiautomática del esqueleto de proyectos y aplicaciones (incluyendo directorios, rutas y código base) con herramientas software y comandos.

Debemos tener en cuenta que, cuando instalamos nuevos paquetes, lo que hace el comando `npm` es instalarlo de manera local en nuestro proyecto dentro de la carpeta `/node_modules`, pero también podemos indicarle que lo instale de forma global si fuera necesario. Además, existen módulos que ya vienen incluidos por defecto en Node.js por lo que no será necesario usar `npm` para instalarlos (módulos nativos).

Pero *¿cómo se instalan módulos con npm?*, por medio del siguiente comando podemos instalar nuevos módulos, que serán incluidos como dependencias en el fichero “package.json”:

```
.....  
C:\miproyecto>npm install [nombre_modulo] --save  
.....
```

Con la opción `--save` indicamos a `npm` que guarde el nombre del módulo en la sección de dependencias del “package.json” aparte de instalarlo. Así, en la jerarquía de directorios de nuestro proyecto tendremos algo como:

```
  /App.js  
  package.json  
  /node_modules  
    nombre_modulo
```

Si quisiéramos instalar un paquete de forma global, para usarlo desde cualquier lugar de nuestro sistema de archivos (y en cualquier proyecto), lo haremos con el comando siguiente (abriremos el terminal como Administrador):

```
.....  
C:\miproyecto>npm install -g [nombre_modulo]  
.....
```

Finalmente, para utilizar los módulos instalados en nuestras aplicaciones Node.js bastará con incluirlos mediante esta sintaxis:

```
.....  
var modulo = require ('modulo');  
.....
```

Existen otras sintaxis de importación fruto del uso de otros *frameworks* y especificaciones JavaScript (por ejemplo, ES6), que usan `import` en lugar de `require`, pero esto es un tema que no nos debería preocupar por el momento.

Como se ha mencionado, los módulos instalados (en los que incluso podemos definir la versión que se instalará del módulo y otras características) se encuentran

como dependencias en el “package.json”. Si nosotros tuviéramos un “package.json” con todos los módulos necesarios en nuestro proyecto y quisiéramos utilizarlos en otro proyecto o una nueva versión del mismo, usaríamos el comando siguiente, que instala todos los módulos del “package.json” que aparecen como dependencias del proyecto, sin necesidad de instalarlos de uno en uno.

```
C:\miproyecto>npm install
```

De esta forma, cuando compartimos nuestra aplicación o la migramos a otro sistema, no hace falta incluir la carpeta `/node_modules` con todos los módulos instalados, simplemente, el fichero “package.json” y el comando anterior harán ese trabajo en el destino.

2.4 ATOM COMO EDITOR PARA EL DESARROLLO DE APLICACIONES WEB

La elección y uso de un entorno de desarrollo o IDE es fundamental si queremos aprovechar las características que estos nos brindan para programar, ya sea aplicaciones de escritorio como web. Actualmente, la mayoría de ellos son capaces de trabajar en multitud de lenguajes.

Por otra parte, existen editores tan completos y con tal cantidad de herramientas que pueden actuar como lo haría un IDE. Hoy en día, uno de los editores más usados para el desarrollo web es Atom¹⁷, un editor multiplataforma de código abierto creado por Github¹⁸ con un gran apoyo de la comunidad, y para el que existen infinidad de *plugins* o paquetes. Entre los lenguajes que soporta Atom tenemos: HTML, CSS, Less, C#, Objective-C, JSON, JavaScript, Python, PHP, PERL, XML, etc. Los paquetes existentes van desde dotar al editor de funcionalidades extra, a la integración con Git, pasando por la personalización del propio entorno.

En un principio, Atom es un editor “ligero”, pero a medida que instalamos *plugins* y hacemos uso de sus funcionalidades (más que necesarias), se puede volver algo lento y pesado. Sin embargo, es uno de los mejores y más populares editores para el desarrollo web hoy en día.

A continuación, se enumeran los pasos a seguir para instalar Atom y algunos de sus *plugins* más interesantes.

17 <https://atom.io/> [Último acceso: Julio, 2020]

18 <https://github.com/> [Último acceso: Julio, 2020]

2.4.1 Instalación de Atom

Primero nos dirigimos a su web oficial y descargamos la última versión estable del editor para nuestro sistema operativo, y una vez descargada la instalamos como una aplicación más del sistema (ver Figura 26).

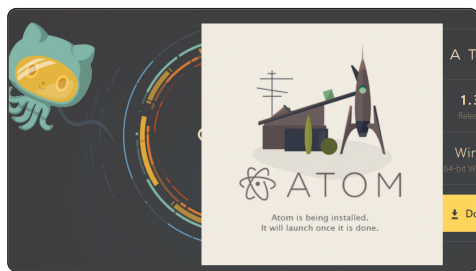


Figura 26. Pantalla de instalación de Atom

Tras la instalación, la aplicación se inicia y nos muestra la pestaña de bienvenida, invitándonos a instalar los primeros paquetes, temas, etc.

Para instalar paquetes nuevos en Atom, desde la pestaña “Welcome Guide” seleccionamos “Install a package” y abrimos el instalador de paquetes. Desde aquí podemos navegar por los paquetes o buscarlos en el buscador, tal y como muestra la Figura 27.

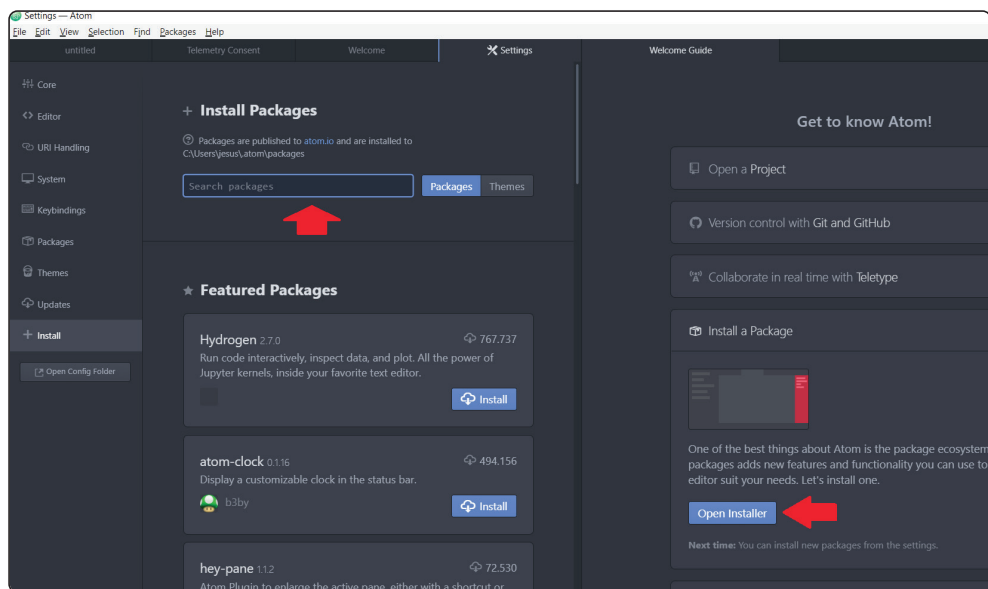


Figura 27. Pantalla inicial de Atom e instalación de paquetes

Es recomendable al menos instalar los siguientes paquetes:

- **file-icons**: Añade iconos y colores a los archivos que tenemos para su fácil identificación.
- **minimap**: Añade una previsualización de los contenidos del archivo en la parte derecha del código. Así resultará muy sencillo ver dónde te encuentras. Muy útil en archivos extensos y con muchas líneas de código.
- **pigments**: Para mostrar los colores que encuentra en los archivos. Dado que estos estarán representados en hexadecimal o RGB, es bastante útil, sobre todo editando hojas de estilo CSS.
- **highlight-selected**: Permite resaltar una palabra del código y todas sus ocurrencias en el archivo, haciendo doble click sobre esta.
- **goto-definition**: Permite ir a la definición de una función, objeto o clase que seleccionamos en el código (con CTRL + click).
- **git-plus**: Permite trabajar con comandos **git** dentro del propio editor, sin necesidad de abrir un terminal para tratar con el control de versiones del proyecto. Además, si quieres ver las diferencias entre archivos controlados por **git**, puedes instalar el plugin **git-time-machine**. Igualmente, para arreglar conflictos entre versiones (sobre todo si se trabaja en desarrollos colaborativos) podemos instalar **merge-conflicts**.
- **todo-show**: Con este *plugin*, encontrarás todas esas partes que dejaste “a medias” o para cambios futuros y que se nombraron con sentencias como TODO, FIXME, etc. Muy útil para repasar tu código antes de llevarlo a producción.
- **linter**: Marca los errores que va detectando en el código. No lo detecta todo, pero ayuda bastante. Además, existen extensiones específicas para la detección y visualización de errores en archivos CSS, JSON, JavaScript, etc.
- **atom-beauty**: Hace que los archivos del proyecto se vean más bonitos y legibles, ya que se encarga de controlar que su indentación sea correcta. Soporta la mayoría de lenguajes.
- **atom-ternjs**: Paquete para facilitar el autocompletado en JavaScript.
- **react**: Añade soporte a la sintaxis JSX, además de indentación, autocompletado, etc., para programación en React.js. Normalmente este *plugin* requiere de la instalación de Git (ver siguiente apartado) en el

sistema operativo. Este paquete es imprescindible, ya que el desarrollo del *front-end* de nuestro proyecto se realizará en React.js, conforme a la pila MERN.

- **atom-ide-ui**¹⁹: Puede decirse que este es un *plugin* de *plugins*. Desarrollado por un equipo de Facebook, permite autocompletado inteligente, previsualización jerárquica de funciones en la parte derecha, formateo del código, herramientas de diagnóstico, etc. Al usar este plugin es posible que se nos recomiende desactivar otros tales como *linter* ya que tiene sus funcionalidades, y además mejoradas.

Hay paquetes que necesitan instalar dependencias, en ese caso, también confirmaremos la instalación de dichas dependencias.

Con esto, queda nuestro entorno Atom configurado y listo para comenzar un nuevo proyecto.

NOTA

Por supuesto, existen numerosas alternativas al uso de Atom para el desarrollo, con lo que siempre puedes utilizar otro editor, como por ejemplo Visual Studio Code²⁰, cuya filosofía basada en *plugins* es similar a la expuesta y cada vez tiene más adeptos.

2.5 CONTROL DE VERSIONES CON GIT

Git es un software de control de versiones (VCS) diseñado por Linus Torvalds. De esta forma, se pueden gestionar los diversos cambios que se realizan sobre algún artefacto o configuración. Si dichos artefactos son archivos con código fuente, podemos gestionar los cambios en dichos archivos a lo largo del tiempo, y por más de una persona o equipo de desarrollo, dado que Git fue creado pensando precisamente en el desarrollo software en equipo.

Git tiene un gran potencial, ya que permite manejar ramas, versiones, temas de almacenamiento, y principios de la gestión eficiente de proyectos, desde el punto de vista del desarrollo. Pero si una cosa caracteriza a Git además del control de versiones en un proyecto, es la posibilidad de interacción con repositorios remotos

19 <https://ide.atom.io/> [Último acceso: Julio, 2020]

20 <https://code.visualstudio.com/> [Último acceso: Julio, 2020]

como Github. Esto nos permitirá tener una copia de nuestro proyecto en la Nube (de forma pública o privada), actualizada y con control de cambios.

No es objetivo de este libro explicar en detalle el control de versiones o todas las características de Git. Sin embargo, a continuación, se presentan algunos de los comandos básicos de Git útiles para cualquier desarrollador.

- **git init**: Iniciar un repositorio vacío en una carpeta específica. Cuando se crea un repositorio nuevo, lo hace con una rama principal denominada “master”.
- **git add [miArchivo]**: Añadir un archivo específico al repositorio. También puede añadir recursivamente todos los archivos de un directorio determinado.
- **git add .**: Añadir todos los archivos del directorio actual al repositorio.
- **git commit -m [mensaje]**: Confirmar los cambios realizados, acompañados de un mensaje.
- **git push origin [miRama]**: Subir la rama “miRama” al servidor remoto.
- **git status**: Mostrar el estado actual de la rama, y los cambios no confirmados con commit.

2.5.1 Instalación de Git

Git es multiplataforma y está disponible para todos los sistemas operativos. Para su instalación en un sistema operativo GNU/Linux, puedes usar la herramienta *apt-get* con el comando

```
>sudo apt-get install git
```

De otra forma, accedemos a la web de referencia²¹ y descargamos la versión correspondiente a nuestro sistema operativo. Instalaremos Git manteniendo las opciones por defecto que nos muestra el instalador.

21 <https://git-scm.com> [Último acceso: Julio, 2020]

Una vez instalado, podemos ejecutar el bash de Git o bien acceder a un terminal. Podemos comprobar que se instaló correctamente mostrando la versión de git con:

```
>git --version
```

Git puede ser complementado con otras herramientas GUI (o de interfaz gráfica de usuario) en el sistema operativo Windows, así como en otros, que podemos descargar igualmente de la web de referencia²².

NOTA

Recordemos que con Git instalado, ya podemos instalar el paquete react en el editor Atom.

2.5.2 Introducción a Github

Existen numerosas definiciones para Github. De forma general, lo podemos definir como una plataforma web centrada en el desarrollo colaborativo que además se integra muy bien con Git.

Así, Github nos permite tener un espacio remoto para almacenar nuestros proyectos en la Nube y sincronizarlos mediante las herramientas de Git, aparte de otras muchas características (facilitando incluso la puesta en producción de los proyectos ahí alojados). En la página oficial de Github²³ reside toda la información al respecto, así como la posibilidad de crear una nueva cuenta, gestionar repositorios y proyectos.

Dicho esto, es el momento de crear una cuenta en Github y con ella un repositorio asociado a nuestro proyecto. De esta forma siempre tendremos nuestro código disponible en la Nube, con posibilidad de control de cambios y numerosas ventajas para su sincronización y despliegue.

Una vez creada la cuenta en Github, o en el caso de que ya tengamos una, podemos ingresar al *dashboard* o panel principal con nuestras credenciales. Una vez

22 <https://git-scm.com/download/gui/windows> [Último acceso: Julio, 2020]

23 <https://github.com/>

dentro, en el menú de la parte superior derecha podemos hacer *click* en la opción correspondiente para crear un nuevo repositorio, tal y como se muestra en la Figura 28.

i NOTA

Como desarrolladores web conviene conocer las oportunidades que nos ofrece el tándem Git-Github.

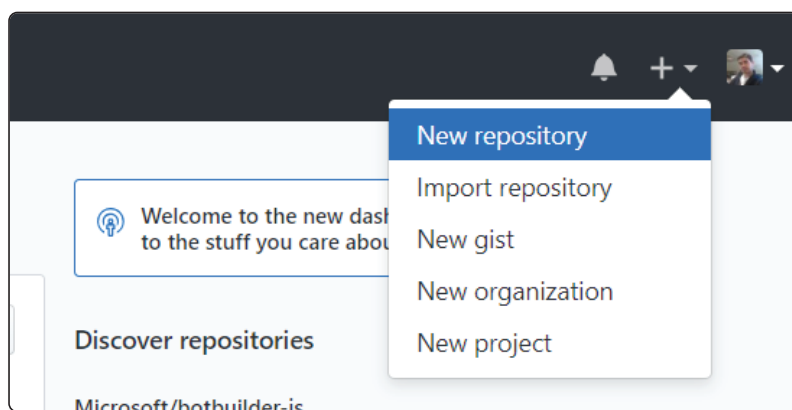


Figura 28. Creación de nuevo repositorio en Github

A la hora de nombrar al repositorio, lo podemos llamar igual que el nombre que le hayamos dado al directorio del proyecto. Como en la **Parte 3: Express** vamos a desarrollar una API REST, lo podemos llamar “microblogging-example-api” si lo deseamos. Acto seguido, daremos una descripción al repositorio y modificaremos o dejaremos por defecto el resto de las opciones. Hecho esto, crearemos nuestro repositorio.

Ahora vamos a enlazar nuestro proyecto, que en principio tan solo se tratará de un directorio vacío con el nombre “microblogging-example-api” a nuestro recién creado repositorio en Github. Usaremos el mismo nombre que el del repositorio, pero podría ser diferente.

Hay veces en las que no conviene sincronizar todos los ficheros o directorios de nuestro proyecto con Github. Por ejemplo, cuando tenemos directorios de módulos o librerías que usamos y que no es necesaria su sincronización, ya que podremos hacer uso de estas con un simple comando antes de ejecutar dicha aplicación en local (de otro modo, en Github estarían ocupando un espacio innecesario). Además, podríamos

tener ficheros con variables sensibles como claves o *passwords*, nombres de usuario, cadenas de conexión, etc., que no nos interesa exponer públicamente al subirlo al repositorio de Github. Por ello, para ignorar la sincronización de determinados ficheros y/o directorios, crearemos el fichero “.gitignore” en el directorio raíz del proyecto. En dicho fichero, incluiremos los nombres de directorios y ficheros que no queremos sincronizar con Github (esto lo veremos más adelante).

A continuación, en la consola (o terminal del sistema operativo) y desde el directorio del proyecto ejecutamos los siguientes comandos de Git:

```
.....
C:\microblogging-example-api>git init
C:\microblogging-example-api>git status
C:\microblogging-example-api>git add .
C:\microblogging-example-api>git status
C:\microblogging-example-api>git commit -m "first commit"
C:\microblogging-example-api>git remote add origin https://github.com/<USUARIO>/
microblogging-example-api.git
C:\microblogging-example-api>git push -u origin master
.....
```

Ya hemos explicado anteriormente lo que hacen dichos comandos. El comando `git status` nos da información de cómo el estado de los ficheros contenidos en el directorio del proyecto cambia cuando los añadimos como archivos a sincronizar. Por su parte, el comando `git remote add origin` conectará nuestro proyecto con el repositorio creado, de forma que podremos sincronizar ambos con los comandos `commit` y `push` indicados. Una vez sincronizado, ya no es necesario ejecutar dicho comando en el caso de que queramos actualizar el repositorio con nuevo código. Con `git commit -m "mensaje del commit"`, notificamos explícitamente que ha habido cambios en el proyecto y describimos estos cambios brevemente. Finalmente, con `git push -u origin master` volcaremos todo el contenido de nuestro proyecto a la rama master del repositorio remoto.

Obviamente, nuestro proyecto está vacío por el momento, no obstante, en la siguiente sección del libro veremos cómo crear el esqueleto y estructura de directorios necesaria para el desarrollo de la API REST correspondiente.

Por otra parte, si ocurre algún error durante la ejecución de los comandos anteriormente listados, podría ser porque aún no nos hayamos identificado en la configuración de Git. Para hacerlo, asignaremos un email y un nombre de la siguiente forma:

```
.....
C:\microblogging-example-api>git config --global user.email "you@example.com"
C:\microblogging-example-api>git config --global user.name "your name"
.....
```


Y volvemos a ejecutar los comandos que fallaron anteriormente. Cuando intentemos hacer el **push**, Github nos preguntará por nuestras credenciales (ver Figura 29).

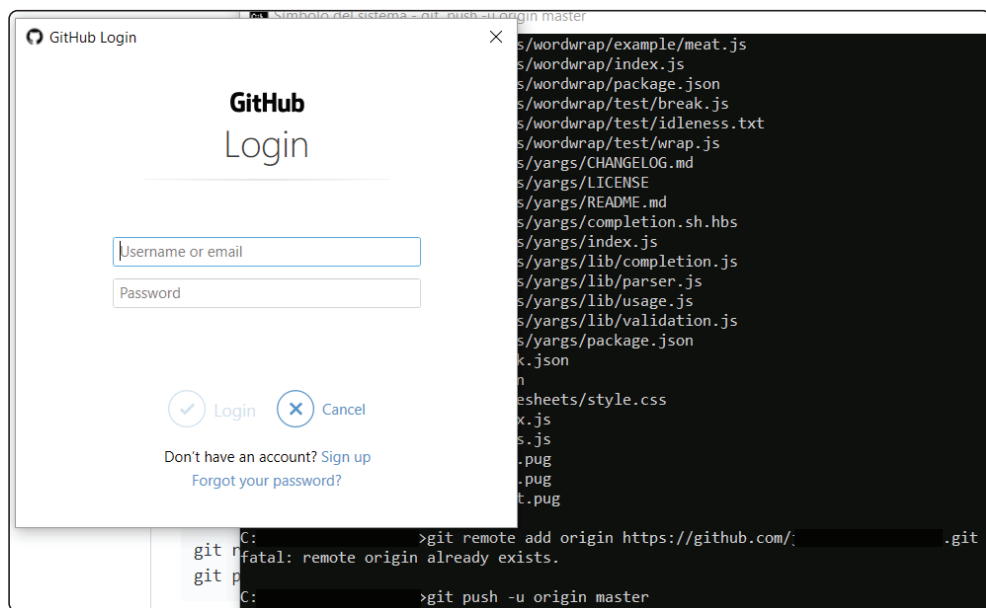


Figura 29. Inserción de credenciales en Github

Una vez confirmadas, nuestro proyecto se subirá y se sincronizará con el repositorio remoto, pudiendo ver en Github todos los directorios y archivos de este.

A partir de ahora, cualquier modificación en el código del proyecto podremos sincronizarla con el repositorio (en la rama **máster**) usando los siguientes comandos:

```
.....
C:\microblogging-example-api>git add .
C:\microblogging-example-api>git commit -m "mensaje del commit"
C:\microblogging-example-api>git push -u origin master
.....
```

Si necesitamos de otras operaciones de Git, podemos consultarlas en la web de referencia de la herramienta o haciendo uso del comando **help**.

Al igual que creamos y subimos nuestros propios proyectos a Github, podemos descargarnos o clonar aquellos que se encuentren en la plataforma y que tengan visibilidad pública. De momento no entraremos en este aspecto.

2.6 LECTURAS RECOMENDADAS

Dado que tanto el entorno Node.js como el resto de *frameworks* y tecnologías usadas están desarrolladas con JavaScript, se hace imprescindible tener conocimientos básicos acerca del lenguaje. A continuación, te presentamos algunas referencias interesantes para ampliar tus competencias tanto en JavaScript como en HTML y CSS:

- Azaustre, C. *Aprendiendo JavaScript*, <https://carlosazaustre.es/> (versión electrónica) 2016.
- Gauchat, J. D. *El gran libro de HTML5, CSS3 y JavaScript*, Marcombo S.A (3ª Edición), 2019.
- Duckett, J. *Web Design with HTML, CSS, JavaScript and jQuery Set*, John Wiley & Sons Inc., 2014.
- Haverbeke, M. *Eloquent JavaScript*, No Starch Press (3ª Edición), 2018.

Sobre Node.js, también tenemos algunas referencias interesantes:

- Hughes-Croucher, T., Wilson, M. *Node: Up and Running: Scalable Server-Side Code with JavaScript*, O'Reilly Media, 2012.
- Basarat, S. *Beginning Node.js*, Apress, 2014.
- Casciaro, M., Mammino, L. *Node.js Design Patterns: Master best practices to build modular and scalable server-side web applications*, Packt Publishing (2ª Edición), 2016.
- Morales, G. *Introducción a Node.js*, Publicación independiente, 2019.

Finalmente, y con el objetivo de profundizar más sobre Git y Github, te recomendamos echar un vistazo a estos recursos:

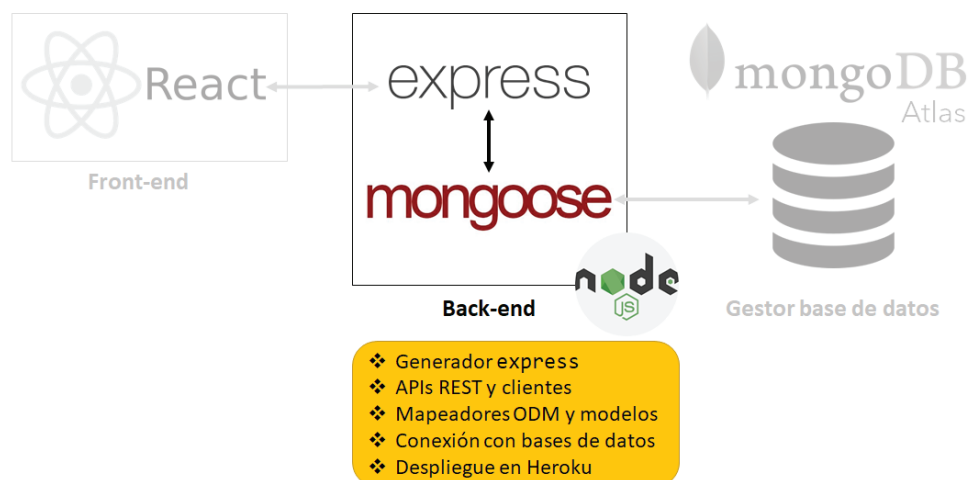
- Sánchez, L. J., *Git y Github. Guía de supervivencia*. Publicación independiente, 2016.
- Chacon, S., Straub, B. *Pro Git*, Apress (2ª Edición), 2014.
- Tsitoara, M. *Beginning Git and GitHub: A Comprehensive Guide to Version Control, Project Management, and Teamwork for the New Developer*. Apress, 2019.

Recuerda que, para ampliar el conocimiento de los conceptos y tecnologías mencionadas a lo largo del libro, es conveniente acudir primero a la fuente original de las mismas, existente en internet, a través de sus webs oficiales.

3

EXPRESS.JS

En esta parte explicaremos paso a paso el desarrollo de la API REST de la aplicación web. Veremos cómo generar una aplicación básica Express.js, la creación de solicitudes HTTP y rutas que las satisfacen, el uso de modelos para la manipulación de los datos de la base de datos, la conexión con el servicio de MongoDB Atlas, y el despliegue de la aplicación utilizando las herramientas de la plataforma Heroku. La zona coloreada de la Figura 30 muestra el lugar en el que se encuentra esta parte dentro de la pila MERN soportada por Node.js.



Pila de tecnologías MERN

Figura 30. Tecnologías y herramientas de desarrollo a partir de la pila MERN. Express.js + mongoose

3.1 ¿QUÉ ES EXPRESS.JS?

Express.js es uno de los *frameworks* de Node.js más interesantes y parte importante en el conjunto de la pila de desarrollo MERN.

El *framework* Express.js destaca por su facilidad y optimización de recursos en el manejo de solicitudes HTTP complejas, y supone una herramienta esencial en la implementación de una API REST, necesaria en la mayoría de las aplicaciones web.

Dado que Express.js es un *framework* de Node.js podremos instalarlo como un paquete en el directorio de nuestro proyecto. Para ello, crearemos el proyecto Node.js con:

```
C:\microblogging-example-api>npm init
```

Una vez se han rellenado los campos que solicita el asistente de la herramienta `npm init`, el “package.json” es generado automáticamente y el proyecto Node.js está creado. El siguiente paso es instalar Express.js como paquete de esta forma:

```
C:\microblogging-example-api>npm install express --save
```

Añadiremos `--save` para salvar esa dependencia en nuestro “package.json” de forma explícita. Lo podemos comprobar accediendo al propio “package.json” y viendo como el módulo `express` se ha añadido como dependencia.

Igualmente podemos añadir `-g` al comando de instalación si lo que queremos es instalar `express` de forma global y tenerlo accesible al resto de proyectos que se creen.

Con esto ya tenemos instalado el *framework* Express.js como parte de nuestro proyecto.

3.2 HOLA MUNDO EN EXPRESS.JS

Con Express.js instalado, en el directorio de nuestro proyecto, crearemos un archivo denominado “app.js” y añadiremos el siguiente código.

```
var express = require('express');  
var app = express();
```

```
app.get('/', function (req, res) {  
  res.send('¡Hola Mundo!');  
});  
  
app.listen(3000, function () {  
  console.log('Aplicación de ejemplo escuchando en el puerto 3000');  
});
```

Lo primero que haremos en el código JavaScript del archivo anterior será importar el módulo **express** instalado y almacenarlo en la variable **app**.

La siguiente línea es la implementación de una solicitud GET cuando llamamos a una ruta específica que, en este caso, será la URL raíz. Como respuesta, enviará al cliente que lo solicitó el mensaje “¡Hola Mundo!”. En caso de usar otra ruta, el servidor nos responderá con un error “404 Not Found”.

En las siguientes 3 últimas líneas, simplemente la aplicación inicia un servidor y escucha las conexiones al puerto 3000.

Para ejecutar y probar nuestra aplicación basta con estar en el directorio del proyecto y ejecutar el comando:

```
C:\microblogging-example-api>node app.js
```

Y acto seguido, cargar la página *http://localhost:3000/* en un navegador para ver la salida de la aplicación.

Si queremos ir más allá y pasar como parámetro a la ruta anterior nuestro nombre, también lo podemos hacer, simplemente cambiando nuestro código de la petición GET a:

```
app.get('/:name', function (req, res) {  
  res.send('¡Hola ' + req.params.name + '!');  
});
```

En este caso, si volvemos a ejecutar la aplicación y cargamos la página con la siguiente URL: *http://localhost:3000/Jesus*, la salida será “¡Hola Jesus!”. Como vemos, las solicitudes pueden ir acompañadas de parámetros.

3.3 GENERACIÓN DE UNA APLICACIÓN EXPRESS.JS

Existe una herramienta que nos facilita la tarea de crear aplicaciones con Express.js, y que al igual que ocurre con el propio *framework* **express**, se instala como si fuera un paquete de nuestro proyecto, aunque en este caso es conveniente instalarlo de forma global (con permisos de administrador), mediante el siguiente comando:

```
C:\>npm install -g express-generator
```

A continuación, prepararemos el directorio `/microblogging-example-api` para alojar en él nuestra aplicación de *back-end*, por lo que borraremos el directorio completo con todo el contenido creado anteriormente, si es que queremos usar el mismo nombre para el nuevo proyecto.

La herramienta **express-generator** nos proporciona el comando **express**, que nos permitirá crear el esqueleto de la aplicación Express.js. Podemos acceder a las opciones de dicho comando con:

```
C:\>express -h
```

Por tanto, si queremos crear una aplicación Express.js con el nombre **microblogging-example-api** lo haremos de la siguiente forma:

```
C:\>express microblogging-example-api
```

Esto creará el directorio del proyecto, así como todos los subdirectorios y ficheros correspondientes.

NOTA

La intención aquí es alojar ambas aplicaciones o proyectos (*back-end* y *front-end*) en directorios diferentes dentro de la unidad raíz “C:”. No obstante, puedes ubicar tus proyectos en la ruta del sistema que prefieras. Recuerda que cada proyecto se compone de un directorio principal y una serie de subdirectorios y ficheros.

Existen opciones adicionales que se pueden incluir al generar la aplicación **express**, de modo que podamos definir el motor de vistas que queramos, etc. Es

importante destacar que, aunque utilizaremos Express.js por sus ventajas para la creación de APIs en base a uso de recursos y enrutado, también es posible crear aplicaciones completas con sus vistas y estilos CSS. Y eso se puede hacer gracias a este generador. No obstante, esto no es nuestro cometido por ahora.

NOTA

El uso de la pila MERN utiliza React.js para la creación de las vistas, con lo que los directorios y fichero de vistas generados por express, no serán necesarios en nuestra aplicación de *back-end* y los podremos eliminar (sin olvidarnos de suprimir las dependencias en los ficheros que las incluyan). Para nosotros la utilidad de express residirá en la potente gestión y acceso a recursos que realiza a la hora de consumir una API REST.

La estructura de ficheros y directorios creada por **express** será algo parecido a lo siguiente:

```
> express microblogging-example-api

create : microblogging-example-api
create : microblogging-example-api/package.json
create : microblogging-example-api/app.js
create : microblogging-example-api/public
create : microblogging-example-api/public/javascripts
create : microblogging-example-api/public/images
create : microblogging-example-api/routes
create : microblogging-example-api/routes/index.js
create : microblogging-example-api/routes/users.js
create : microblogging-example-api/public/stylesheets
create : microblogging-example-api/public/stylesheets/style.css
create : microblogging-example-api/views
create : microblogging-example-api/views/index.jade
create : microblogging-example-api/views/layout.jade
create : microblogging-example-api/views/error.jade
create : microblogging-example-api/bin
create : microblogging-example-api/bin/www
```

No es objetivo de este libro entrar en detalle sobre cada uno de esos ficheros generados, sin embargo, a medida que trabajemos en el desarrollo de la aplicación conoceremos detalles acerca de estos. Se recomienda abrir el editor Atom e importar el directorio de nuestro proyecto en él.

Una vez creada la aplicación, entraremos al directorio de la misma e instalaremos las dependencias, definidas en el “package.json”, con el comando:

```
.....  
C:\microblogging-example-api>npm install  
.....
```

Y ya podemos ejecutar la aplicación generada, que usaremos como base para nuestra propia API REST gracias a esta herramienta de *scaffolding* (recuerda que ya comentamos anteriormente este concepto). Para ello, usaremos el siguiente comando:

```
.....  
C:\microblogging-example-api>npm start  
.....
```

A continuación, cargamos la URL *http://localhost:3000/* en el navegador para acceder a la aplicación. Igualmente, desde la consola podemos ver la realización de las peticiones a las páginas o servicios asociados a las rutas definidas en el código. Gracias a Node.js, se nos ha creado un servidor escuchando en el puerto 3000, que servirá nuestra aplicación Express.js en el navegador. Si deseamos usar otro puerto, lo podemos cambiar en el fichero ‘*bin/www*’ o mediante una variable de entorno (ver sección **Introducción a las variables de entorno**).

Desde el navegador, podemos acceder al recurso de la URL *http://localhost:3000/users* y comprobar el resultado, la salida por consola asociada a la solicitud del recurso, y el código en “app.js” y “routes/users.js”. El comando **express** nos creó una pequeña aplicación totalmente funcional con alguna petición. Prueba a cambiar algunos parámetros del código, por ejemplo, rutas, y sentencias del fichero “app.js”, y comprueba los resultados (puede ser necesario reiniciar el proyecto tras los cambios).

¡Enhorabuena, ya has creado tu primera aplicación con **express**!

3.3.1 Acerca de la ejecución y depuración

Para ejecutar una aplicación Node.js, previamente al uso de **express**, usábamos directamente el comando **node** de esta forma:

```
.....  
C:\microblogging-example-api>node app.js  
.....
```


Sin embargo, al crear nuestra aplicación mediante el generador de aplicaciones **express**, por defecto podemos lanzarla mediante:

```
.....  
C:\microblogging-example-api>npm start  
.....
```

En este caso, **start** hace referencia a un *script* del fichero “package.json”, que puede contener el comando **node** para iniciar la aplicación. Por supuesto, ese *script* y otros los podremos modificar en función de nuestras necesidades.

Cuando queremos depurar nuestra aplicación, no es buena idea imprimir mensajes por consola (por ejemplo, usando **console.log**), ya que estas son funciones asíncronas con bastantes limitaciones que nos pueden suponer bastantes quebraderos de cabeza cuando nuestra aplicación se encuentra en producción.

Por tanto, para depurar de forma correcta en Node.js y por ende en Express.js, se utiliza el módulo **debug**²⁴, que podemos instalar como un paquete con el siguiente comando, aunque el *framework* Express.js ya lo trae por defecto.

```
.....  
C:\microblogging-example-api>npm install debug  
.....
```

El módulo de **debug** es usado para registrar internamente información sobre la coincidencia de rutas que se están usando, modalidad de aplicación y el flujo del ciclo de solicitud/respuesta.

Así, para iniciar nuestra aplicación en modo depuración usaremos el comando **npm start** y fijaremos el entorno **DEBUG** sobre nuestra aplicación, tal y como se indica a continuación:

```
.....  
C:\microblogging-example-api>set DEBUG=microblogging-example-api:* & npm start  
.....
```

Si utilizamos otro sistema operativo como GNU/Linux o MAC, el comando sería similar:

```
.....  
>DEBUG=microblogging-example-api:* npm start  
.....
```

24 <https://github.com/visionmedia/debug> [Último acceso: Julio, 2020]

De esta forma, veremos en consola todos los detalles de la aplicación a nivel de middleware. Si con la aplicación iniciada en modo depuración, realizamos una solicitud a la misma, la consola nos mostrará algo similar a esto:

```
.....
express:router dispatching GET / +4h
  express:router query : / +2ms
  express:router expressInit : / +0ms
  express:router favicon : / +0ms
  express:router logger : / +1ms
  express:router jsonParser : / +0ms
  express:router urlencodedParser : / +1ms
  express:router cookieParser : / +0ms
  express:router stylus : / +0ms
  express:router serveStatic : / +2ms
  express:router router : / +2ms
  express:router dispatching GET / +1ms
  express:view lookup "index.pug" +338ms
  express:view stat "/projects/example/views/index.jade" +0ms
  express:view render "/projects/example/views/index.jade" +1ms
.....
```

Por supuesto, la depuración en Express.js es muy flexible y nos permite incluso crear nuestro propio depurador desde código con una configuración determinada. Por ejemplo, en un *script* “server.js”, representado abajo, podemos utilizar la rutina `debug`, usada para mostrar mensajes de depuración, y utilizarlo de la siguiente forma (mostrándose únicamente cuando iniciemos la aplicación en modo depuración):

```
.....
const app = require('./app');
const server = require('http').Server(app);
const debug = require('debug')('app:server');
const port = process.env.NODE_PORT || 3000;

server.listen(port, () => {
  debug(`Servidor corriendo en el puerto: ${server.address().port}`);
});
.....
```

La mejor forma de utilizar el modo depuración es, en el “package.json”, a través de un *script* `start` seguido de `:dev`, de forma que cuando lancemos nuestra aplicación desde la línea de comandos decidamos si queremos ejecutarla en producción o depuración. A continuación, se muestran las sentencias a insertar en el caso de un sistema operativo Windows (en GNU/Linux sería sin “set” ni “&”):

```
.....  
.  
.  
  "scripts": {  
    "start": "node server.js",  
    "start:dev": "set DEBUG=* & set NODE_ENV=development & node server.js"  
  },  
.  
.  
.....
```

Con esto, podemos iniciar nuestra aplicación en modo depuración (y de paso fijar la variable de entorno `NODE_ENV` a `development`, para saber, por ejemplo, que estamos en depuración) desde la línea de comandos así:

```
.....  
> npm run start:dev  
.....
```

Por supuesto, existen herramientas y extensiones que nos permiten, de forma complementaria, depurar nuestras aplicaciones en el propio navegador. Una de las más conocidas es Google Chrome DevTools²⁵. No es objetivo de este libro entrar en profundidad en este aspecto, si no dar al lector una introducción general a la ejecución y depuración.

3.4 INTRODUCCIÓN A LAS APIS REST Y ENRUTADO CON EXPRESS.JS

.....

REST (REpresentational State Transfer) se define como un protocolo de intercambio y manipulación de datos en los servicios de Internet basado a su vez en HTTP. Los datos son devueltos en formatos específicos y estandarizados, como pueden ser XML y JSON. Existen otros protocolos, pero la simplicidad y funcionalidad de REST lo han llevado a ser uno de los más usados y conocidos de Internet.

Desde el punto de vista del desarrollador, una API REST no es más que una librería de servicios REST (funcionalidades) que satisface diferentes necesidades de intercambio y manipulación de datos a través de Internet.

Pero ¿qué es un servicio?

25 <https://developers.google.com/web/tools/chrome-devtools/?hl=es> [Último acceso: Julio, 2020]

Realmente cuando hablamos de REST, el concepto de servicio puede desvanecerse, ya que es más apropiado usar el término recurso. El acceso a los recursos se realiza a través de URLs que los identifican.

Si por ejemplo tenemos un recurso llamado “colección de usuarios” con la URL “/users” que lo identifica; si invocamos con REST dicha URL, bajo un esquema URI (*Uniform Resource Identifier*), deberíamos obtener una representación de dicho recurso, es decir, el listado de todos los usuarios de la colección.

Si quisieramos obtener los datos de un usuario en concreto, habrá otro recurso (usuario) con un esquema URI asociado. Además, entre los recursos existen relaciones: está claro que un usuario forma parte de la “colección de usuarios”, así que parece lógico que a partir de esa colección pueda acceder a uno de sus elementos con una ruta del tipo “/users/12”, siendo “12” el ID del usuario. Esta es la idea que está detrás de las librerías de servicios REST.

3.4.1 Características generales de REST

Por citar algunas de las características más relevantes y de forma resumida, diremos que:

- Todo servicio REST se apoya en el protocolo de Internet HTTP y en sus métodos básicos (GET, POST, PUT, DELETE), que corresponden a operaciones CRUD (Create, Read, Update, Delete).
- Crea una petición HTTP con toda la información necesaria, es decir, un “request” que envía a un servidor, y solo espera un “response”, una respuesta en concreto a esa solicitud.
- Todos los objetos/recursos se manipulan mediante URIs. Una URI (Uniform Resource Identifier) nos facilita el acceso a la información del recurso para su modificación o borrado, o bien para compartir su ubicación con terceros.
- Nos aporta escalabilidad y diferenciación entre la parte cliente y servidor.
- Es totalmente independiente de la plataforma, pudiendo hacer uso de REST desde cualquier sistema operativo y lenguaje.

En la actualidad muchas de las empresas punteras de Internet y miles de startups usan REST. Todas estas empresas tienen sus servicios disponibles a través de APIs REST con la lógica de negocio necesaria para el acceso a los recursos y, por otro lado, tienen la parte *front-end*, independiente de ese *back-end* de servicios.

Las APIs REST pueden ser públicas o privadas. Si son públicas, pueden consumir sus servicios otros usuarios. Esto puede verse en multitud de librerías que se nos proporcionan como recursos de terceros para cubrir nuestras necesidades de desarrollo. A su vez, determinadas APIs REST pueden requerir de autenticación previa para acceder a sus recursos.

3.4.2 Rutas y manejadores

El *routing* o direccionamiento hace referencia a la definición de puntos finales de aplicación para el acceso a recursos o también llamados *endpoints*; y cómo responden a las solicitudes HTTP del cliente, normalmente asociadas a esas operaciones CRUD, mediante una función de manejador que procesa la petición y construye una respuesta para ser retornada al cliente. Esta parte se desarrollará en detalle en la sección **CRUD y Enrutado**.

Cada ruta puede tener una o varias funciones de manejador, y tienen la siguiente estructura:

```
.....  
app.METHOD(PATH, HANDLER);  
.....
```

Donde:

- **app** es una instancia de **express**
- **METHOD** es un método de solicitud HTTP
- **PATH** es una vía de acceso o ruta en el servidor, que caracteriza al punto final o *endpoint* para el acceso a los recursos.
- **HANDLER** o manejador es la función que se ejecuta cuando se correlaciona la ruta.

Si lo recuerdas, en nuestro ejemplo del “Hola Mundo” en Express.js ya ilustrábamos esta estructura, con una solicitud GET simple:

```
.....  
app.get('/', function (req, res) {  
  res.send('¡Hola Mundo!');  
});  
.....
```

Como podemos observar esa solicitud GET se corresponde con uno de los métodos de ruta derivados de solicitudes HTTP (por ejemplo: *get*, *post*, *update*, *delete*), que a su vez coinciden con las operaciones CRUD (*create*, *read*, *update*, *delete*).

Podemos implementar otros *endpoints* de rutas de la misma manera. En el siguiente ejemplo podemos ver lo propio con la ruta “/about”:

```
.....  
app.get('/about', function (req, res) {  
  res.send('about');  
});  
.....
```

NOTA

Incluso podemos utilizar expresiones regulares para las vías de acceso o rutas definidas.

La parte **HANDLER** de la primera sentencia mostrada proporciona el manejador a través de una función con unos parámetros determinados, que normalmente son *req* (solicitud) y *res* (respuesta). Excepcionalmente, la devolución de llamada de la función del manejador puede invocar *next('route')* para omitir el resto de las devoluciones de llamada de ruta. Este mecanismo se puede utilizar para imponer condiciones previas en una ruta, y a continuación pasar el control a las rutas posteriores si no hay motivo para continuar con la ruta actual.

A continuación podemos ver un ejemplo de función con *next()*:

```
.....  
app.get('/example/b', function (req, res, next) {  
  console.log('la respuesta será enviada por la función next...');  
  next();  
}, function (req, res) {  
  res.send('¡Hola desde B!');  
});  
.....
```

Para entrar más en detalle sobre estas características conviene consultar otras fuentes dedicadas²⁶.

26 <https://expressjs.com/en/guide/writing-middleware.html> [Último acceso: Julio, 2020]

Por su parte, los métodos en el objeto de respuesta “**res**” de la Tabla 1 pueden enviar una respuesta al cliente y finalizar el ciclo de solicitud/respuestas. Si ninguno de estos métodos se invoca desde un manejador de rutas, la solicitud de cliente se quedará colgada.

Método	Descripción
<i>res.download()</i>	Solicita un archivo para descargarlo.
<i>res.end()</i>	Finaliza el proceso de respuesta.
<i>res.json()</i>	Envía una respuesta JSON.
<i>res.jsonp()</i>	Envía una respuesta JSON con soporte JSONP ²⁸ .
<i>res.redirect()</i>	Redirecciona una solicitud.
<i>res.render()</i>	Representa una plantilla de vista.
<i>res.send()</i>	Envía una respuesta de varios tipos.
<i>res.sendFile</i>	Envía un archivo como una secuencia de octetos.
<i>res.sendStatus()</i>	Establece el código de estado de la respuesta y envía su representación de serie como el cuerpo de respuesta.

Tabla 1. Algunos tipos de métodos invocados por el cliente²⁷

En la siguiente sección, se verá cómo se pueden crear manejadores de rutas modulares asociados a operaciones CRUD, para ser utilizados en nuestra API REST.

3.4.3 CRUD y Enrutado

Cuando queremos acceder a recursos en **express**, lo hacemos a través de puntos finales o *endpoints*, como ya habíamos mencionado anteriormente. Estas URLs se asocian a solicitudes del cliente, y dichas solicitudes son transmitidas mediante el protocolo HTTP para el acceso y manipulación de recursos.

En HTTP encontramos distintos tipos de solicitudes, que a su vez suelen corresponderse con operaciones CRUD, base de cualquier *back-end*, tal y como comentamos en una de las secciones iniciales del libro. Una vez realizadas las solicitudes al servidor, este debería devolver una respuesta al cliente. De esta forma tendremos:

²⁷ Información obtenida de: <https://expressjs.com/es/4x/api.html#res> [Último acceso: Julio, 2020]

²⁸ https://www.w3schools.com/js/js_json_jsonp.asp [Último acceso: Julio, 2020]

- GET: Accede a los recursos a través de un *endpoint*, permitiendo leerlos y/o consultarlos (por ejemplo, de una base de datos). Análogo a READ.
- POST: Permite crear un nuevo recurso a partir de la información suministrada por el cliente. Por ejemplo, para almacenarlo en una base de datos. Análogo a CREATE.
- PUT: Permite actualizar un recurso, usando la información suministrada por el cliente. Análogo a UPDATE.
- DELETE: Accede a un recurso gracias a su *endpoint* para eliminarlo. Igual que DELETE en el modelo CRUD.

Aparte, existe otra solicitud denominada PATCH que nos permite editar partes concretas de un recurso. Esta operación es la menos conocida y utilizada.

Las URIs contienen varias partes, entre ellas la propia ruta o URL de acceso, que además de permitir identificar de forma única el recurso, nos permite localizarlo para poder acceder a él o compartir su ubicación.

Una URI se podría estructurar de la siguiente forma:

```
.....  
{protocolo}://{dominio o hostname}[:puerto (opcional)]/{ruta del  
recurso}?{consulta de filtrado}  
.....
```

Existen varias reglas adoptadas para nombrar a la ruta de un recurso, las cuales se presentan a continuación.

Las rutas no deben implicar acciones y deben ser únicas

Por ejemplo, la siguiente ruta sería incorrecta ya que tenemos el verbo editar (la acción) en la misma.

```
/posts/123/editar
```

Para el recurso *post* con el identificador 123, la siguiente ruta sería correcta, independientemente de que vayamos a editarla, borrarla, consultarla o leer solo uno de sus conceptos (estas operaciones vendrán determinadas por la solicitud HTTP implementada en la lógica de negocio).

```
/posts/123
```


Las rutas deben ser independientes de formato

Por ejemplo, la siguiente ruta sería incorrecta, ya que estamos indicando la extensión pdf en la misma:

```
/posts/123.pdf
```

Mientras, la siguiente ruta sí sería correcta, independientemente de que vayamos a consultarla en formato pdf, txt, xml o JSON.

```
/posts/123
```

Las rutas deben mantener una jerarquía lógica

Por ejemplo, la siguiente URI no sería correcta por no seguir una jerarquía lógica.

```
/posts/123/usuario/007
```

Para el recurso *post* con el identificador 123 del usuario 007, la siguiente ruta sería la correcta:

```
/usuarios/007/posts/123
```

Filtrados y otras operaciones no se hacen en la URL

Para filtrar, ordenar, paginar o buscar información en un recurso, podremos hacer una consulta sobre la ruta, utilizando parámetros HTTP en lugar de incluirlos en la misma.

Por ejemplo, la siguiente ruta sería incorrecta ya que el recurso de listado de posts sería el mismo, pero utilizaríamos una ruta distinta para filtrarlo, ordenarlo o paginarlo.

```
/posts/orden/desc/fecha-desde/2007/pagina/2
```

En este caso, la ruta correcta sería:

```
/posts?fecha-desde=2007&orden=DESC&pagina=2
```

Visto todo lo anterior, para un recurso posts tendríamos las solicitudes mostradas en la Tabla 2.

Tipo de solicitud	URL	Descripción
GET	/posts	Nos permite acceder al listado de posts
POST	/posts	Nos permite crear un post nuevo
GET	/posts/123	Nos permite acceder al detalle de un post
PUT	/posts/123	Nos permite editar el post, sustituyendo la totalidad de la información anterior por la nueva
DELETE	/posts/123	Nos permite eliminar el post
PATCH	/posts/123	Nos permite modificar cierta información del post, como la descripción o la fecha del mismo

Tabla 2. Ejemplos de solicitudes HTTP

3.4.4 Códigos de estado y formatos de contenido

Cuando realizamos una solicitud como las anteriores a una API REST, es de vital importancia saber si dicha operación se ha realizado con éxito o en caso contrario, por qué ha fallado.

En la actualidad existe una lista bastante completa de estos códigos de estado que nos permiten conocer cómo ha ido la petición cuando esta es procesada por el servidor²⁹.

Cuando construimos servicios (entendidos como funcionalidades de una API REST), deberíamos devolver el código de estado asociado al resultado de la solicitud. De esta forma tendremos un código más mantenible y eficiente en depuración y después en producción.

En el siguiente ejemplo vemos una respuesta a una solicitud HTTP que nos devuelve el código 200 (petición satisfactoria, “OK”). No obstante, si miramos con detenimiento el contenido de la respuesta, comprobamos que refleja un error, por falta de datos de entrada en la petición del cliente. Así pues, el código devuelto no es el adecuado ni tampoco el contenido y formato de la respuesta emitida.

Se requiere un esfuerzo adicional a la hora de conocer las diferentes respuestas del servidor, fruto de una petición de un cliente a un determinado *endpoint*, así como comprender por qué se transmite una determinada respuesta y cuál es el código de estado HTTP que debe retornarse.

29 <https://developer.mozilla.org/es/docs/Web/HTTP/Status> [Último acceso: Julio, 2020]

```
.....
Petición
=====
PUT /posts/123

Respuesta
=====
Status Code 200
Content:
{
  success: false,
  code:    734,
  error:   "datos insuficientes"
}
.....
```

En el siguiente ejemplo, sin embargo, sí estaría bien construida la respuesta emitida, retornando el código de estado 400 (de petición mal formalizada o “*bad request*”), al faltar un identificador de usuario (dato de entrada) en la petición:

```
.....
Petición
=====
PUT /posts/123

Respuesta
=====
Status Code 400
Content:
{
  message: "se debe especificar un id de usuario para el post"
}
.....
```

Por otra parte, y como vimos en la sección anterior, no era correcto especificar, en la URL, el tipo de formato de un recurso al que queremos acceder o manipular.

HTTP nos permite especificar en qué formato queremos recibir el recurso, pudiendo indicar varios en orden de preferencia, para lo que usaremos el *header* (o cabecera) **Accept** en la petición.

Nuestra API deberá devolver el recurso en el formato correcto y/o disponible de los indicados y, en el caso de no poder mostrarlo, retornará el código de estado HTTP **406** (“*Not acceptable*”).

Tras la negociación del formato de contenido entre el cliente y el servidor, en la respuesta se devolverá el *header Content-Type*, para que el cliente sepa en qué formato recibirá el contenido, por ejemplo:

```
.....
Petición
=====
GET /posts/123
Accept: application/epub+zip, application/pdf, application/json

Respuesta
=====
Status Code 200
Content-Type: application/pdf
.....
```

En este caso, el cliente solicita el post en “epub” comprimido con “zip”, y de no tenerlo, en “pdf” o “json” por orden de preferencia.

En la siguiente sección vamos a ver cómo trabajar con rutas desde **express** aplicando estos conocimientos.

3.4.5 Express Router

En la sección **Rutas y manejadores**, ya vimos como implementar manejadores de rutas con **express** de forma sencilla, aunque no prestábamos especial atención a las operaciones CRUD asociadas. Incluso podemos crear manejadores de rutas encadenados para una ruta determinada. Como esta se especifica en una única ubicación, la creación de rutas modulares es muy útil, favoreciendo la reducción de redundancia y errores tipográficos.

A continuación se muestra un ejemplo de manejadores de rutas encadenados que se definen utilizando `app.route()`. Además, podemos observar cómo se están utilizando las solicitudes HTTP vistas anteriormente; en este caso GET, POST y PUT. La única implementación en el ejemplo es el envío de una respuesta al cliente con `res.send()`.

```
.....
app.route('/post')
  .get(function(req, res) {
    //...
    res.send('Recupera un post cualquiera');
  })
  .post(function(req, res) {
```

```
//...
res.send('Añade un post');
})
.put(function(req, res) {
  //...
  res.send('Actualiza el post');
});
```

En Express.js tenemos la clase `express.Router` que permite crear manejadores de rutas modulares. Una instancia `Router` es un sistema de middleware y direccionamiento completo; por este motivo, a menudo se conoce como una “miniaplicación”. De hecho, la potencia de Express.js radica en su manejo de rutas mediante el `Router`.

A continuación, se muestra un ejemplo donde se crea un direccionador como un módulo, carga una función del middleware en él, define algunas rutas y monta el módulo de direccionador en una vía de acceso en la aplicación principal.

Para ello, tendremos que crear un archivo direccionador, en este caso “posts.js” en el correspondiente directorio de la aplicación (usualmente en `/routes`), con este código:

```
var express = require('express');
var router = express.Router();

// middleware que es específico a este router
router.use(function timeLog(req, res, next) {
  console.log('Fecha actual: ', Date.now());
  next();
});
// define la ruta de la página del home
router.get('/', function(req, res) {
  res.send('Página inicial de los posts');
});
// define la ruta de la página about
router.get('/about', function(req, res) {
  res.send('Acerca de los posts');
});

module.exports = router;
```

Así, para cargar el módulo de direccionador en la aplicación (por ejemplo: en “index.js”), se haría con el siguiente código:

```
var posts = require('./posts');  
//...  
app.use('/posts', posts);
```

La aplicación ahora podrá manejar solicitudes a `/posts` y `/posts/about`. Además, para ambas peticiones se ejecutará la función de middleware (`timeLog`) previo al código de estas.

Por supuesto, a la hora de optimizar nuestro código (a medida que aumenta la complejidad y funcionalidades), también podemos separar la lógica de negocio de forma más eficiente, y no incluir el código de las funciones de las distintas rutas en el mismo archivo. De esta forma podremos tener algo como:

```
var express = require('express');  
var PostController = require('./controllers/post');  
  
var api = express.Router();  
  
// ruta GET para recuperar todos los posts (o entradas de un blog)  
api.get('/posts', PostController.posts);  
  
// ruta POST para guardar nuevos posts (o entradas de un blog)  
api.post('/posts', PostController.savePost);  
  
//...  
  
module.exports = api;
```

Las funciones de manejadores y su código ya no se incluyen en este script (módulo direccionador), sino que lo hacen en un archivo “post.js” en el directorio asociado a los controladores (`/controllers`).

Como se puede observar, el código contenido en las funciones asociadas a las rutas de acceso de los recursos puede suponer desde el envío de un simple mensaje como respuesta, hasta la realización de transacciones con bases de datos, algo que abordaremos más adelante.

3.4.6 Creación de mi primera API REST

Después de explicar los fundamentos que envolverán al desarrollo de una API REST con Express.js, es hora de ponernos manos a la obra, y comenzar a construir la API REST de nuestra aplicación. Para ello, iremos al directorio “/microblogging-example-api” creado con el generador de `express` (ver sección **Generación de una aplicación Express.js**).

Como primer paso, editaremos el fichero “users.js”, construido automáticamente por el generador `express` y alojado inicialmente en “/microblogging-example-api/routes/”, con la intención de implementar en él todos los manejadores CRUD que tienen que ver con las operaciones relativas a los usuarios. A continuación, describiremos las funciones a desarrollar.

Obtener usuarios

Usando la raíz de la URL “/users” en el navegador, tal y como se muestra en el fichero “app.js” generado por `express` y una solicitud GET, devolveremos todos los usuarios al cliente que realizó la petición. Como aún no hemos visto como conectar nuestra API REST a la base de datos construida, devolveremos un documento JSON de usuarios estático para comprobar su funcionamiento.

```
// GET del listado de usuarios
router.get('/', function(req, res) {
  res.json({
    "users": [
      {
        "id": 123,
        "name": "Eladio Guardiola",
        "phones": {
          "home": "800-123-4567",
          "mobile": "877-123-1234"
        },
        "email": [
          "jd@example.com",
          "jd@example.org"
        ],
        "dateOfBirth": "1980-01-02T00:00:00.000Z",
        "registered": true
      },
      {
        "id": 456,
        "name": "Nemesio Tornero",
        "phones": {
          "home": "800-123-3498",
          "mobile": "877-432-1278"
        },

```

```

    "email": [
      "pt@example.com",
      "pt@example.org"],
    "dateOfBirth": "1983-01-09T00:00:00.000Z",
    "registered": false
  }
]
});
});

```

Conforme al código presentado, el cliente recibirá como respuesta el documento JSON con todos los usuarios existentes, en este caso se trata de un array de 2 usuarios y toda la información de estos, convenientemente formateada en JSON.

Para crear documentos JSON bien formados y hacer pruebas con ellos, es aconsejable echar un vistazo a herramientas online como *ObjGen*³⁰.

Obtener un usuario por su Id

Usando la URL `"/users/:id"` y una solicitud GET, devolveremos al usuario cuyo "Id" sea el indicado en parámetro "id" de la URL. En este caso "id" es un número entero, pero podría ser de cualquier otro tipo que se quiera pasar en la petición, por ejemplo, una cadena. También, devolvemos un documento JSON, pero esta vez con la información del usuario cuyo Id es "123" (el cual es un parámetro de la propia URL), en caso de no encontrarlo devolveremos el estado 404 (*Not found*) acompañado de un mensaje. A continuación, se presenta el código asociado.

```

// GET de un usuario por su id
router.get('/:id', function(req, res) {
  if (req.params.id == '123'){
    res.json({
      "id": 123,
      "name": "Eladio Guardiola",
      "phones": {
        "home": "800-123-4567",
        "mobile": "877-123-1234"
      },
      "email": [
        "jd@example.com",
        "jd@example.org"
      ]
    });
  }
});

```

30 <http://www.objgen.com/json> [Último acceso: Julio, 2020]


```
    ],
    "dateOfBirth": "1980-01-02T00:00:00.000Z",
    "registered": true
  });
}else
  res.status(404).send('¡Lo siento, el ítem no se ha encontrado!');
});
```

Nuevamente, el JSON a devolver al cliente por parte del servidor, es un documento estático. El objetivo de estos ejemplos es observar el comportamiento de las funciones conforme a las rutas establecidas. Recuerda que en el caso de nuestra API REST final, realizaremos peticiones al servidor sobre datos almacenados en una base de datos, lo que se verá en las secciones **Conexión con MongoDB Atlas** y **Servicios REST para la gestión de usuarios**.

Agregar un nuevo usuario

Usando la URL raíz `"/users"` y una solicitud POST, podremos agregar un nuevo usuario a la lista de usuarios. En este ejemplo, el nuevo usuario será un documento JSON bien formado como por ejemplo el siguiente (que no necesariamente debe contener todos los campos de los usuarios de anteriores ejemplos):

```
{
  "id": 789,
  "name": "Tomás Rabero",
  "phones": {
    "home": "855-123-8884",
    "mobile": "877-555-1234"
  }
}
```

A la operación anterior podemos añadir la devolución de un mensaje, por parte del servidor, que indique si la operación de agregación tuvo éxito. El código de la función podría ser el siguiente:

```
// POST de un nuevo usuario
router.post('/', function(req, res) {
  var new_user = req.body;
  //ToDo (hacer algo con el nuevo usuario)
  res.status(200).send('Usuario ' + req.body.name + ' ha sido añadido satisfacto-
riamente');
});
```

En este caso, el nuevo usuario a agregar se encuentra en el cuerpo del mensaje enviado por el cliente, de ahí que accedamos a él con la sentencia `req.body`. Una vez recuperado, haremos lo necesario y devolveremos la respuesta de la operación al cliente.

Actualizar un usuario por su Id

Usando la URL `"/users/:id"` y una solicitud PUT, podremos actualizar un usuario existente de la lista de usuarios. Se actualizará el documento JSON del usuario cuyo Id coincida con el dado. En este caso se pasa a la función el id del usuario como parámetro y el JSON del usuario actualizado en el cuerpo de la solicitud. Un ejemplo del nuevo JSON podría ser:

```
{
  "id": 789,
  "name": "Tomás Rabero",
  "phones": {
    "home": "855-123-8884",
    "mobile": "877-555-1234"
  },
  "registered": true
}
```

Y el código de la función el siguiente:

```
// PUT de un usuario por su Id
router.put('/:id', function(req, res) {
  var updated_user = req.body;
  //ToDo (hacer algo con el usuario)
  res.status(200).send('Usuario ' + req.body.name + ' ha sido actualizado satisfactoriamente');
});
```

Igualmente, necesitamos procesar la solicitud (en base al nuevo usuario) para determinar si se realizó correctamente, por ejemplo, al actualizar el documento en la base de datos, pero por ahora lo dejaremos así, enviando un mensaje con el código 200 al cliente.

Eliminar un usuario por su Id

Usando la URL “/users/:id” y una solicitud DELETE, podremos borrar un usuario existente de la lista de usuarios. En este caso se pasa a la función el id del usuario como parámetro, se comprobará la existencia del usuario y se eliminará, pudiendo devolver un mensaje asociado a la operación. De esta forma, si queremos borrar el documento JSON del usuario con id = 123, lo haríamos como muestra el bloque de código siguiente.

Una vez implementada la parte del API dedicada al manejo de recursos “usuarios”, podremos desplegarla y hacer uso de sus funciones a través del navegador (para peticiones GET) o un cliente REST, para probar todas las solicitudes implementadas. Podremos ver el resultado de estas operaciones también en la consola. En la siguiente sección, explicaremos como consumir una API REST.

```
// DELETE de un usuario por su Id
router.delete('/:id', function(req, res) {
  //ToDo (hacer algo con el usuario)
  res.status(200).send('Usuario con id '+ req.params.id + ' ha sido borrado satisfactoriamente');
});
```

Adicionalmente, recuerda que, si inicializaste tu proyecto con Git puedes crear un archivo “.gitignore” en el directorio raíz con el siguiente contenido antes de sincronizarlo con el repositorio correspondiente de Github (ver sección **Introducción a Github**).

```
# dependencies
/node_modules
```

Con esto le indicaremos al control de versiones que no sincronice el directorio “/node_modules”, que contiene todas las dependencias del proyecto (definidas en el “package.json”). Si quisiéramos instalar dichas dependencias lo haríamos con el comando `npm install` (por ejemplo, cuando clonamos un repositorio de Github que no tiene el “/node_modules” pero cuyas dependencias están definidas en el “package.json”).

NOTA

Con el proyecto actualizado con el nuevo código, añadiremos los archivos con `git add`, haremos el `git commit` correspondiente y lo sincronizaremos con el repositorio creado en Github con `git push`.

3.4.7 Cómo consumir una API REST. Introducción a los clientes REST

Para consumir una API REST (por ejemplo, las funciones implementadas en la sección anterior), necesitamos un cliente REST.

El cliente REST se encargará de realizar peticiones y/o solicitudes (en nuestro caso HTTP) a uno o varios servicios, o funciones REST para cubrir sus necesidades. En realidad, el cliente hará llamadas a una librería o API REST y es independiente del lenguaje en el que esté implementado el *back-end* de dicha librería.

De esta forma, es esencial disponer de un cliente que consuma los servicios que necesitamos de un servidor (mediante llamadas a URIs), bien para realizar un cálculo, o una transacción con una base de datos, por ejemplo.

En la actualidad, existen multitud de opciones de clientes REST que nos facilitan probar nuestras propias APIs REST o bien librerías de terceros. Incluso podemos crearnos los clientes, embebidos como parte de la aplicación del *front-end* (esto lo trataremos cuando veamos el *framework* React.js). Independientemente del tipo de cliente, este deberá ser capaz de manejar la respuesta del servidor, la cual de forma estándar estará acompañada de un código de estado (y un contenido), tal y como vimos en secciones anteriores.

Desde el sistema operativo (en nuestro caso Windows), lo podremos hacer usando el comando `curl`, ya que permite la transferencia de datos, y soporta 22 protocolos incluido HTTP. De este modo, podemos hacer peticiones GET, POST, PUT y DELETE usando una URL conocida. En el ejemplo siguiente se muestra una solicitud **GET** con `curl` a una instancia de un servicio que se encuentra accesible a través del puerto y la URL especificada.

```
>curl -v http://localhost:8082/spring-rest/foos/9
```

Gracias a la opción `-v` (*verbose*), se nos mostrará en consola información sobre la solicitud GET realizada, de esta forma:

```
* Trying ::1...
* TCP_NODELAY set
* Connected to localhost (::1) port 8082 (#0)
> GET /spring-rest/foos/9 HTTP/1.1
> Host: localhost:8082
> User-Agent: curl/7.60.0
> Accept: */*
>
```

```
< HTTP/1.1 200
< X-Application-Context: application:8082
< Content-Type: application/json;charset=UTF-8
< Transfer-Encoding: chunked
< Date: Sun, 15 Jul 2018 11:55:26 GMT
<
{
  "id" : 9,
  "name" : "TuwJ"
}* Connection #0 to host localhost left intact
```

Igualmente, podemos realizar un **POST**, esta vez enviado los datos al servidor de esta forma:

```
>curl -d 'id=9&name=baeldung' http://localhost:8082/spring-rest/foos/new
```

El POST puede incluir un fichero a enviar al servidor con el cuerpo de la solicitud:

```
>curl -d @request.json -H "Content-Type: application/json" http://localhost:8082/spring-rest/foos/new
```

O bien crear el documento en la misma línea de comandos. Por defecto, `curl` añade la cabecera de tipo `"Content-Type: application/x-www-form-urlencoded"`, pero podemos modificarla de acuerdo a nuestras necesidades, de esta forma (en el caso de esperarse un documento JSON):

```
>curl -d '{"id":9,"name":"baeldung"}' -H 'Content-Type: application/json' http://localhost:8082/spring-rest/foos/new
```

Por regla general, la consola de Windows no soporta comillas simples con lo que debemos modificar el comando de esta forma (con los caracteres de escape asociados):

```
>curl -d "{ \"id\":9, \"name\":\"baeldung\"}" -H "Content-Type: application/json" http://localhost:8082/spring-rest/foos/new
```

El método **PUT** es muy similar a POST, pero lo usamos cuando queremos enviar al servidor una nueva versión de un recurso existente (usando la opción -X):

```
>curl -d @request.json -H 'Content-Type: application/json' -X PUT http://localhost:8082/spring-rest/foos/9
```

En el caso de **DELETE**, también lo especificaremos con la opción -X:

```
>curl -X DELETE http://localhost:8082/spring-rest/foos/9
```

El comando `curl` también permite trabajar con cabeceras propias y servicios que requieran autenticación, pero no es objetivo de este libro profundizar sobre ello.

Si nos centramos en nuestra “fake” API REST de usuarios implementada anteriormente, podemos hacer uso de este cliente y comprobar su funcionamiento. En primer lugar, debemos desplegar la API REST con el comando:

```
C:\microblogging-example-api>npm start
```

A continuación, vamos a probar a hacer un GET desde línea de comandos, usando el comando `curl`. Así, abrimos un nuevo terminal y probamos a ingresar lo siguiente:

```
>curl -v http://localhost:3000/users
```

Aparte de la información desprendida de la solicitud, se nos devolverá el listado de usuarios en formato JSON.

Para el resto de peticiones el comando funciona de forma similar. Si tenemos que actualizar un recurso a una nueva versión o borrarlo, basta con incluir -X seguido del tipo de solicitud. Vamos a probar con el borrado de un usuario existente de esta forma:

```
>curl -v -X DELETE http://localhost:3000/users/123
```

Una vez ejecutado, podemos observar la salida mostrada en ambos terminales (el de la ejecución del comando `curl` y dónde se encuentra desplegada nuestra API REST).

Por supuesto, existen otras alternativas para probar una API REST. De hecho, se aconseja la descarga de aplicaciones de terceros que nos permiten mayor flexibilidad y funcionalidades extra. Una de estas aplicaciones es Advanced REST Client³¹ (ARC) la cual podemos descargar desde su web oficial.

Una vez descargada la aplicación, esta nos servirá para probar nuestra API REST (o cualquier otra), tal y como hemos hecho con el comando `curl`, pero de una forma más cómoda y como ya se ha mencionado, con mucha más funcionalidad.

En la pantalla principal de la aplicación, podemos ingresar la URL de la petición, seleccionando que es de tipo GET, tal y como se muestra en la Figura 31.

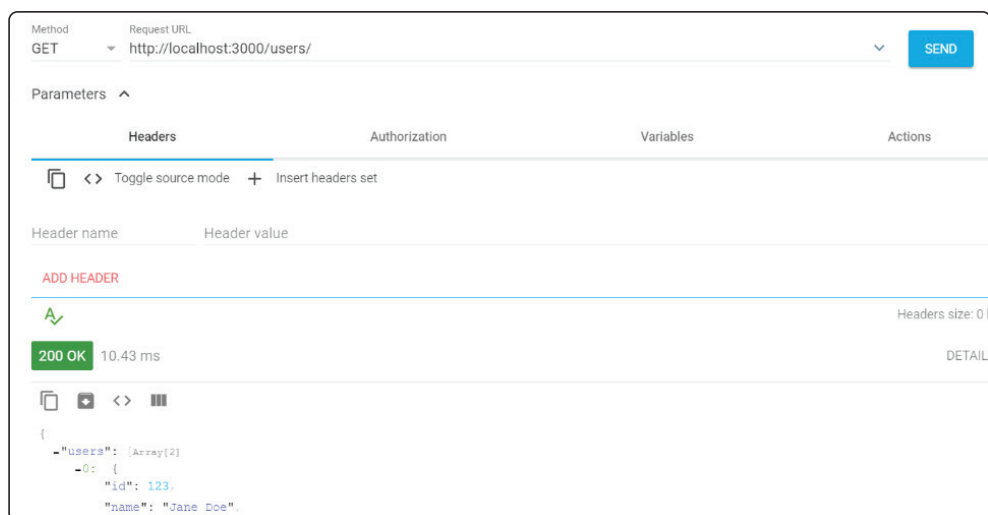


Figura 31. Ejemplo de petición GET usando la aplicación ARC

Enviamos la petición con el botón “Send” y observamos el resultado, el tiempo que tardó en servirnos la respuesta, y otros datos de interés. Además, podemos ver cómo fue la ejecución de la solicitud en la consola en la que está desplegada la API REST.

31 <https://install.advancedrestclient.com/install> [Último acceso: Julio, 2020]

Para hacer un POST con la herramienta, seleccionamos el método POST y en la pestaña “Body”, debemos indicar el tipo de datos y el documento JSON que queremos agregar, como se muestra en la Figura 32.

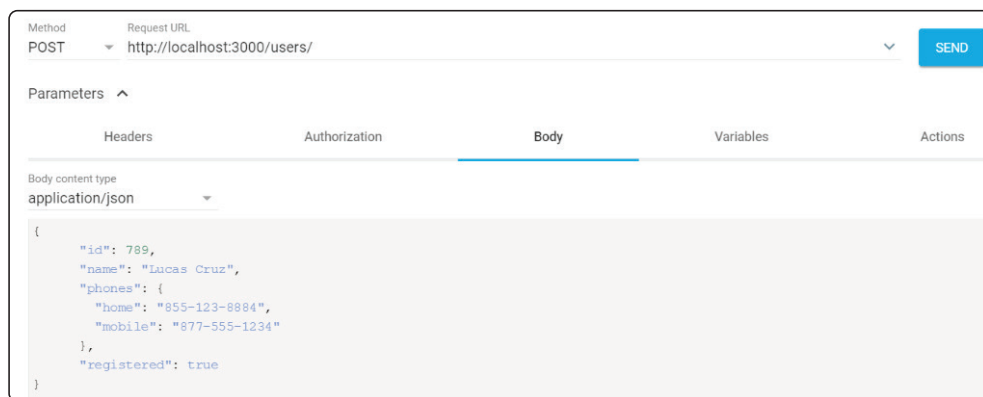


Figura 32. Ejemplo de petición POST usando la aplicación ARC

Nuevamente realizamos la solicitud y observamos el resultado. Podemos probar algunas peticiones más de las implementadas, e incluso testear con rutas y/o datos inválidos, que requieran parámetros, etc., y analizar los resultados.

Aparte de esto, obviamente podemos implementar nuestro propio cliente, como parte de una aplicación web para consumir los recursos ofrecidos por nuestra API REST. Esto se verá más adelante en el libro, ya que antes debemos familiarizarnos con el *framework* React.js, desde el cual realizaremos todas las peticiones a la API REST implementada.

3.5 MAPEADORES DE DATOS Y ODM

Cuando trabajamos con datos almacenados en bases de datos, existen herramientas y mecanismos que nos ayudan a su manipulación considerando un alto nivel de abstracción, sin necesidad de hacerlo a nivel de consultas (por ejemplo: SQL). En este sentido, trabajan los mapeadores ORM (*Object Relational Mapper*) y ODM (*Object Document Mapper*). Mientras que el primero de ellos mapea entre un modelo de objeto y una base de datos relacional, el segundo lo hace entre un modelo de objeto y una base de datos de documentos, como puede ser MongoDB. De esta forma se logra una independencia entre los modelos y la fuente de datos.

Pero ¿Qué es un modelo?

Podría decirse que es una representación de un tipo de contenido mediante el cual podemos, a alto nivel, trabajar con objetos de esa representación, haciendo más sencillo y eficiente la manipulación de los datos y la interacción con las bases de datos.

Los modelos son creados mediante esquemas, siguiendo unos “estándares” para la definición de atributos y sus tipos, aparte de algún comportamiento adicional.

Así, si queremos por ejemplo almacenar y/o recuperar datos de una base de datos, lo haremos a través de objetos de un modelo previamente definido siguiendo un esquema. Obviamente podemos optar por no usar modelos ni mapeadores a la hora de manipular los datos y realizar las transacciones con la base de datos, pero sin duda nos estaremos perdiendo un sinfín de ventajas de los ORM/ODM relacionadas con la mantenibilidad, eficiencia, optimización, validación de datos, o agnosticismo.

A continuación, se muestra un ejemplo de modelo implementado en JavaScript siguiendo un esquema mongoose:

```
.....  
var userSchema = mongoose.Schema({  
  _id: mongoose.Schema.Types.ObjectId,  
  name: {  
    firstName: String,  
    lastName: String  
  },  
  biography: String,  
  twitter: String,  
  facebook: String,  
  linkedin: String,  
  profilePicture: Buffer,  
  created: {  
    type: Date,  
    default: Date.now  
  }  
});  
.....
```

A partir de este esquema podremos derivar el modelo correspondiente con:

```
.....  
var User = mongoose.model('User', userSchema);  
.....
```

Y crear objetos `user` a partir de una serie de datos de entrada. Incluso relacionar modelos (por ejemplo: `user -> posts`), como si se tratase de una relación entre tablas a través de claves ajenas, si buscamos el símil con las bases de datos relacionales.

Usualmente, los modelos suelen almacenarse como archivos “*.js”, en un directorio “/models” de nuestro proyecto por cuestiones organizativas y de mantenimiento.

Cuando trabajamos con MERN o cualquier otro *stack*, los modelos deben ser conocidos por el *back-end* y muchas veces también por el *front-end*, para poder operar sobre los objetos creados a partir de esos modelos de forma apropiada.

Dentro del entorno Node.js, tenemos la librería `mongoose` que se encontraría entre lo que es la base de datos MongoDB y el *framework* Express.js, a nivel del *back-end*. Aunque no es esencial usar esta librería como ODM, y podemos usar otra diferente, se ha optado por esta debido a su potencial e integración con MongoDB y Node.js.

3.5.1 Creación de modelos con mongoose en Express.js

Más concretamente, `mongoose` es una biblioteca JavaScript que implementa un ODM y permite definir esquemas con datos fuertemente tipados. Una vez se define un esquema, `mongoose` puede crear un modelo basado en ese esquema específico. Así, un modelo `mongoose` se asigna a un documento MongoDB a través de la definición del esquema del modelo.

Una vez definidos los esquemas y modelos, `mongoose` contiene muchas funciones para validar, guardar, eliminar y consultar los datos utilizando a su vez las funciones comunes de MongoDB.

Actualmente, esta librería contiene 8 *SchemaTypes* que pueden adquirir los atributos del modelo, a saber:

- String
- Number
- Date
- Buffer
- Boolean
- Mixed
- ObjectId
- Array

Cada tipo de los anteriores puede especificar:

- Un valor determinado
- Una función de validación personalizada
- La obligatoriedad de un campo
- Una función *get* que permite manipular los datos antes de devolverlos como un objeto
- Una función de conjunto que le permite manipular los datos antes de guardarlos en la base de datos.
- Crear índices para permitir que los datos se obtengan más rápido.

Además de estas opciones comunes, ciertos tipos de datos pueden personalizarse aún más si cabe para el almacenamiento y recuperación de los datos de la base de datos.

En el ejemplo siguiente se muestra un esquema con los *SchemaTypes* más comunes junto con su declaración.

```
.....  
var schema = new Schema(  
  {  
    name: String,  
    binary: Buffer,  
    living: Boolean,  
    updated: { type: Date, default: Date.now },  
    age: { type: Number, min: 18, max: 65, required: true },  
    mixed: Schema.Types.Mixed,  
    _someId: Schema.Types.ObjectId,  
    array: [],  
    ofString: [String], //También puedes tener un array de cada uno de los otros  
    tipos.  
    nested: { stuff: { type: String, lowercase: true, trim: true } }  
  });  
.....
```

Visto esto, es hora de usar *mongoose* en nuestro proyecto. Para ello, basta con instalarlo mediante el gestor *npm*, como nueva dependencia en el “*package.json*” de esta forma:

```
.....  
C:\microblogging-example-api>npm install mongoose --save  
.....
```

Ahora supongamos que en nuestra base de datos se almacenará información relativa a una serie de usuarios y un conjunto de posts (o entradas) escritos por dichos usuarios. La forma más adecuada de modelar esa información, para facilitar la manipulación de usuarios y posts, es a través de modelos. En nuestro caso haremos uso de **mongoose**, tanto para definir los esquemas de los modelos como la conexión con la base de datos.

Tras instalar **mongoose** como dependencia del proyecto, crearemos una nueva carpeta en el directorio raíz denominada “/models”, para almacenar ahí nuestros modelos. En primer lugar, crearemos el modelo *Post*, como un fichero JavaScript denominado “Post.js” con el siguiente código.

```
var mongoose = require('mongoose');

var Schema = mongoose.Schema;
var User = require('../models/User.js');

var PostSchema = new Schema({
  user: { type: Schema.ObjectId, ref: 'User' },
  title: String,
  description: String,
  publicationdate: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Post', PostSchema);
```

Como vemos, el modelo *Post* tendrá dos atributos de tipo **String** (“title” y “description”), un atributo de tipo **Date** (“publicationdate”) cuyo valor por defecto será la fecha del sistema en la que se almacenará el post, y una referencia al modelo *User*, ya que un post será creado y/o publicado por un usuario. De modo que entre ambos modelos existirá una relación.

En el caso del modelo *User*, este tendrá el siguiente código, almacenado en un fichero denominado “User.js” también en el directorio “/models”.

```
var mongoose = require('mongoose');

var Schema = mongoose.Schema;
var Post = require('../models/Post.js');

//Para la encriptación del password
var bcrypt = require('bcryptjs');
```

```

var SALT_WORK_FACTOR = 10;

var UserSchema = new Schema({
  username: { type: String, required: true, index:{ unique: true} },
  password: { type: String, required: true },
  fullname: String,
  email: { type: String, required: true},
  creationdate: { type: Date, default: Date.now },
  role: { type: String, enum: ['admin','subscriber'], default: 'subscriber' },
  posts: [{ type: Schema.ObjectId, ref: 'Post', default: null }]
});

```

Aquí vemos los tipos de datos que contendrá un usuario, así como una referencia a los posts que ha creado y/o publicado (en este caso será un array de posts).

Igualmente, los modelos pueden albergar código para procesar la información contenida antes de almacenarla, o incluso contener funciones de validación de los datos. En este caso, utilizaremos algunas de estas funciones, por ejemplo, para encriptar los *passwords* de los nuevos usuarios que se creen (antes de almacenarlos en la base de datos), así como la comparación entre dos *passwords* (útil cuando se realiza el *login* del usuario), usando también la encriptación/desencriptación que proporciona el módulo *bcryptjs*.

Para poder hacer uso de este módulo, tenemos que instalarlo como una dependencia del proyecto de la forma que ya sabemos:

```
C:\microblogging-example-api>npm install --save bcryptjs
```

E insertar el nuevo código que llevará a cabo este proceso en nuestro archivo “User.js”:

```

UserSchema.pre('save', function(next) {
  var user = this;
  // solo aplica una función hash al password si ha sido modificado (o es nuevo)
  if (!user.isModified('password')) return next();
  // genera la salt
  bcrypt.genSalt(SALT_WORK_FACTOR, function(err, salt) {
    if (err) return next(err);
    // aplica una función hash al password usando la nueva salt
    bcrypt.hash(user.password, salt, function(err, hash) {

```

```
        if (err) return next(err);
        // sobrescribe el password escrito con el "hasheado"
        user.password = hash;
        next();
    });
  });
});

UserSchema.methods.comparePassword = function(candidatePassword, cb) {
  bcrypt.compare(candidatePassword, this.password, function(err, isMatch) {
    if (err) return cb(err);
    cb(null, isMatch);
  });
};

module.exports = mongoose.model('User', UserSchema);
```

Por otra parte, no hay que olvidar exportar los modelos creados, con `module.exports`, para poder importarlos y usarlos en otras partes del proyecto.

Con esto, ya están creados nuestros modelos. A partir de ahora, podremos hacer consultas a nuestra base de datos MongoDB para almacenar, editar y/o borrar documentos JSON asociados, como si fueran objetos. En la web de referencia de `mongoose`³² puedes encontrar información más detallada al respecto.

NOTA

Ten en cuenta que cuando instalamos dependencias con el gestor de paquetes `npm` sin explicitar la versión del paquete a instalar, se nos instalará la última versión estable de este, con lo que puede que algunos paquetes hayan sufrido cambios en la forma de usarlos o en algunas de sus funciones internas, afectando incluso a su sintaxis. Se recomienda echar un vistazo a la versión de las dependencias utilizadas para este proyecto en el repositorio del autor (ver **Apéndice A. Código fuente**).

3.5.2 Conexión con MongoDB Atlas

Como se ha mencionado anteriormente, `mongoose` también nos provee de las herramientas necesarias para conectarnos a una base de datos MongoDB, en

32 <https://mongoosejs.com/docs/guide.html> [Último acceso: Julio, 2020]

nuestro caso a la base de datos remota de nuestro clúster en MongoDB Atlas. Si lo recuerdas, el *dashboard* de MongoDB (pulsando sobre el botón “Connect”) nos ofrece diferentes opciones de conexión al clúster (ver sección **Registro y configuración de MongoDB Atlas**). En este caso debemos indicarle que queremos conectar desde nuestra aplicación, seleccionando “*Connect Your Application*”. Acto seguido, la herramienta nos proporcionará una cadena de conexión que debemos insertar como parte del código para la conexión a la base de datos en nuestro proyecto. El bloque de código con la cadena de conexión suministrada será añadido al fichero raíz “app.js” de la siguiente forma:

```
.....  
var mongoose = require('mongoose');  
  
mongoose.connect('mongodb+srv://<USER>:<PASSWORD>@<CLUSTER>.mongodb.net/<DATABASE>  
E?retryWrites=true', { useNewUrlParser: true })  
  .then(() => console.log('mymerndb connection successful'))  
  .catch(err => console.error(err));  
.....
```

Debemos sustituir las cadenas <USER>, <PASSWORD>, <CLUSTER> y <DATABASE> por nuestros datos de conexión. La cadena de conexión anterior puede variar en función de nuestro clúster y la versión de MongoDB (3.4+ o 3.6+).

Además, para evitar un posible “*warning deprecated*” con el `index` del `username` de `UserSchema`, podemos añadir esta línea de código también al fichero “app.js” (antes de `mongoose.connect`):

```
.....  
mongoose.set('useCreateIndex', true);  
.....
```

Adicionalmente, desde MongoDB Atlas podemos crear dos colecciones en nuestra base de datos `mymerndb` si aún no las tenemos creadas: “users” y “posts”, para almacenar los documentos relativos a los usuarios y a los posts. No obstante, al trabajar con modelos, si dichas colecciones no existieran y deseamos almacenar datos en ambas, se crearían de forma automática (a partir del nombre del modelo, en minúsculas y en plural).

Una vez realizados los pasos anteriores, podemos desplegar la aplicación y ver el log mostrado en consola.

```
.....  
C:\microblogging-example-api>npm start  
.....
```

El mensaje “mymerndb connection successful” indica que la conexión con la base de datos se realizó de forma satisfactoria (conforme al código anterior).

3.6 INTRODUCCIÓN A LAS VARIABLES DE ENTORNO

En el mundo del desarrollo, y concretamente del desarrollo web, existe un tipo especial de variables denominadas variables de entorno. Si recuerdas de secciones anteriores, cuando el generador de `express` creó nuestra aplicación `Express.js`, existía una forma de asignar el puerto en el que escucha la aplicación (ver “/bin/www”) tal y como se muestra a continuación:

```
var port = normalizePort(process.env.PORT || '3000');
app.set('port', port);
```

Esto quiere decir que nuestra aplicación de *back-end* escuchará en el puerto definido en la variable de entorno `PORT`, o en su defecto en el puerto 3000. El hecho de que, en el futuro decidamos cambiar el valor del puerto en nuestra aplicación ya desplegada, tendríamos que cambiarlo en nuestro código y volver a desplegar la aplicación, a no ser que el valor de ese puerto lo asigne una variable de entorno.

La mayoría de las plataformas de despliegue como Heroku, la cual veremos más adelante, permiten cambiar esos valores sin tener que modificar nada en el código. Por supuesto, otros valores que podrían ser cambiantes en el tiempo o bien porque son de carácter “sensible”, como por ejemplo cadenas de conexión a bases de datos, nombres de usuarios, nombres de bases de datos, *passwords*, etc. convendría tenerlos asociados a variables de entorno.

Existen varias formas de crear y acceder a las variables de entorno definidas en nuestras aplicaciones. En el siguiente punto se explicará cómo crear un archivo que almacene las variables de entorno con sus valores, y cómo podemos usarlos en la aplicación.

El uso de variables de entorno puede ser más complejo, pudiendo definir diferentes alcances de las variables, por ejemplo, en función de si nos encontramos en entornos de desarrollo, de producción, o bajo unas condiciones específicas; pero esto es algo en lo que no entraremos en este libro, aunque si lo deseas puedes profundizar usando otros medios.

3.6.1 Creación y acceso a variables de entorno

Aunque, como se ha dicho anteriormente, los valores de las variables de entorno pueden definirse en el momento del despliegue de la aplicación, conviene tener en un fichero de nuestro proyecto, todas las variables de entorno con sus valores por defecto.

Para ello, vamos a crear un fichero denominado “.env” en el directorio raíz del proyecto con el siguiente contenido:

```
.....  
NODE_ENV=development  
PORT=5000  
DB_URI='mongodb+srv://<USER>:<PASSWORD>@<CLUSTER_ID>.mongodb.  
net/<BBDD>?retryWrites=true'  
.....
```

En este fichero estamos indicando que tenemos 3 variables de entorno con el par: nombre de la variable y valor. Concretamente definiremos una variable de entorno para conocer opcionalmente si estamos en un entorno de desarrollo (lo que vendrá dado por el valor de `NODE_ENV`). También una variable que defina el puerto por defecto usado por la aplicación. En este caso, y teniendo en cuenta que cuando vayamos a crear la aplicación *front-end* con React.js, se nos asignará esta al puerto 3000, indicaremos que vamos a usar el 5000 en la aplicación del *back-end* para no generar conflicto de puertos, ya que como vimos anteriormente, si no tenemos valor en la variable de entorno `PORT`, le asignará el valor 3000.

Finalmente tenemos la variable con nombre `DB_URI`, la cual contendrá como valor la cadena de conexión a nuestra base de datos en MongoDB Atlas. Obviamente, al tener dicha cadena definida en una variable de entorno, tenemos que sustituirla en “app.js” por el nombre de dicha variable precedido de `process.env`, que nos indica que se trata de una variable de entorno, de esta forma:

```
.....  
mongoose.connect(process.env.DB_URI, { useNewUrlParser: true })  
  .then(() => console.log('mymerndb connection successful'))  
  .catch((err) => console.error(err));  
.....
```

Ahora solamente nos queda indicar a nuestra aplicación que tenemos un fichero “.env” que contiene los valores por defecto de las variables de entorno para que sean usadas a la hora de desplegar la aplicación. Para ello, instalaremos el paquete “dotenv” desde el directorio del proyecto:

```
C:\microblogging-example-api>npm install dotenv --save
```

Y lo importaremos en el fichero “app.js”, añadiendo esta línea de código:

```
require('dotenv').config();
```

A partir de este momento, cuando ejecutemos o realicemos el despliegue de nuestra aplicación sin asignar valores a dichas variables, utilizará los valores por defecto de las mismas contenidos en el fichero “.env”. Esto también favorecerá el mantenimiento de nuestras aplicaciones.

Por último, este fichero “.env” debería ser ignorado cuando sincronicemos nuestro proyecto con el repositorio de Github correspondiente. Para ello, añadiremos al fichero “.gitignore” la siguiente línea (el símbolo “#” representa comentarios en el fichero):

```
# misc
.env
```

Con ello, una vez sincronicemos nuevamente el proyecto con Github, no se enviará el fichero “.env”, preservando los valores de las variables de entorno definidas.

3.7 SERVICIOS REST PARA LA GESTIÓN DE USUARIOS

Una vez implementados los modelos y comprobada la conexión de nuestra aplicación con la base de datos correspondiente, vamos a desarrollar una serie de servicios que podrá consumir un cliente conforme a unas rutas definidas. Además, estos servicios cuyo código base fue implementado en la sección **Creación de mi primera API REST**, se comunicarán con la base de datos gracias a los modelos para recuperar o editar información de los usuarios y/o los posts.

En el archivo “routes/users.js” vamos a implementar servicios para: añadir nuevos usuarios, editar usuarios existentes y borrar usuarios. Lo primero que haremos, adicionalmente al código contenido será importar nuestro modelo “User.js” y asociar una variable “db” a nuestra conexión con la base de datos (para ello, importaremos también la librería **mongoose**).

```
var mongoose = require('mongoose');
var User = require('../models/User.js');
var db = mongoose.connection;
```

Con ello, ya habremos establecido conexión con la base de datos. A continuación, veremos cómo implementar una serie de servicios y su acceso mediante URIs.

3.7.1 Devolución de todos los usuarios

Para devolver todos los usuarios usaremos las sentencias de código que nos facilita `mongoose` para tratar con modelos. Gracias a ello recuperaremos los datos de la base de datos como si de objetos se tratasen, en este caso usando la función `find()`, a la cual podemos añadir filtros como por ejemplo para que se devuelvan ordenados de acuerdo a la fecha de creación (`creationdate`). Podemos implementar esta petición de la siguiente forma:

```
// GET del listado de usuarios ordenados por fecha de creación
router.get('/', function (req, res, next) {
  User.find().sort('-creationdate').exec(function(err, users) {
    if (err) res.status(500).send(err);
    else res.status(200).json(users);
  });
});
```

Por supuesto, puedes aprovechar las funciones ya implementadas en el fichero, pero ya no devolveremos datos estáticos de un JSON. En este caso, la función devolverá al cliente un archivo JSON con la información de todos los usuarios (contenido de la variable `users`), según los campos definidos en el modelo `User`. Puedes ver cómo se hacen las consultas con `mongoose` en su página web³³. En el caso de que ocurra un error se enviará dicho error al cliente con código de estado 500. En este caso estamos indicando que sea el cliente el que maneje el error (siempre que implemente el `catch` asociado). El manejo de errores no es tratado en profundidad en el libro, pero puedes obtener información al respecto en la web de Express.js³⁴.

33 <https://mongoosejs.com/docs/queries.html> [Último acceso: Julio, 2020]

34 <https://expressjs.com/es/guide/error-handling.html> [Último acceso: Julio, 2020]

3.7.2 Devolución de un usuario

Si queremos devolver un usuario identificado por su “id”, la función asociada a la petición sería algo como:

```
// GET de un único usuario por su Id
router.get('/:id', function (req, res, next) {
  User.findById(req.params.id, function (err, userinfo) {
    if (err) res.status(500).send(err);
    else res.status(200).json(userinfo);
  });
});
```

En este caso, el id es un parámetro de la URL de la petición (por ejemplo: *http://localhost:5000/users/123/*), y se usará la función `findById` que permite buscar un documento en la colección de los usuarios de la base de datos, cuyo `id` es el pasado como parámetro (y se devolverá el JSON de la variable `userinfo` que contiene el usuario).

El “id” normalmente hace referencia al identificador que usa MongoDB para identificar de forma unívoca cada documento en una colección, y no un campo que nosotros hayamos definido en el modelo. Si usamos un campo específico en lugar de ese `id`, debemos usar la función `find()` con el filtro correspondiente asociado al campo, en lugar de `findById()` que busca por el `id` definido por MongoDB.

3.7.3 Inserción de un usuario

Para añadir un nuevo usuario a la base de datos, implementaremos el método `router.post`, y que esta vez tendrá un código como el siguiente:

```
// POST de un nuevo usuario
router.post('/', function (req, res, next) {
  User.create(req.body, function (err, userinfo) {
    if (err) res.status(500).send(err);
    else res.sendStatus(200);
  });
});
```

Lo que se hace, por tanto, es crear un objeto *user* conforme al modelo del fichero “User.js” y almacenarlo en la base de datos. Obviamente, el usuario a salvar

debe estar formateado según el modelo *User*, y en el cuerpo de la solicitud irán los datos del nuevo usuario como un documento JSON que el cliente deberá crear previamente. Se podría haber devuelto al cliente el usuario almacenado nuevamente (variable `userinfo`), pero se opta tan solo por enviar el código de estado 200 con `sendStatus` para el caso de que se haya realizado la solicitud de forma satisfactoria.

3.7.4 Actualización de un usuario

Si queremos actualizar un usuario existente en la base de datos, primero debemos buscarlo atendiendo al `id` suministrado y luego actualizar su información. Para ello, tenemos la función `findByIdAndUpdate()`. De modo que el código de la solicitud (en este caso se trata de un PUT) sería como sigue:

```
.....  
// PUT de un usuario existente identificado por su Id  
router.put('/:id', function (req, res, next) {  
  User.findByIdAndUpdate(req.params.id, req.body, function (err, userinfo) {  
    if (err) res.status(500).send(err);  
    else res.sendStatus(200);  
  });  
});  
.....
```

En este caso, el `id` del usuario será un parámetro de la URL de la petición como ocurría al realizar una solicitud de devolución de un usuario determinado, mientras que la nueva información del usuario irá en el cuerpo de la solicitud, como un JSON formateado de acuerdo al modelo *User*, y que podemos recuperar con `req.body`. Al igual que en el caso anterior, se opta por devolver únicamente el código de estado 200 si todo ha ido bien. No obstante, se podría haber devuelto el JSON del usuario actualizado contenido en la variable `userinfo`.

3.7.5 Borrado de un usuario

Funciona de forma similar al anterior, pero en este caso no enviaremos nada en el cuerpo de la solicitud, tan solo el `id` del usuario a borrar. Así, usando la función proporcionada `findByIdAndDelete()`, conseguiremos encontrar el usuario y borrarlo de la base de datos. A continuación, se presenta el código asociado.

```
.....  
// DELETE de un usuario existente identificado por su Id  
router.delete('/:id', function (req, res, next) {  
  User.findByIdAndDelete(req.params.id, function (err, userinfo) {  
    if (err) res.status(500).send(err);  
    else res.sendStatus(200);  
  });  
});  
.....
```

```

    if (err) res.status(500).send(err);
    else res.sendStatus(200);
  });
});

```

Además, se nos da la posibilidad de devolver los datos del usuario que ha sido borrado, como un JSON (en la variable `userinfo`), por si queremos realizar algún procesamiento con ello. No obstante, tan solo devolveremos el código de estado de la operación como en casos anteriores.

3.7.6 Login de un usuario

Para controlar el acceso de un usuario a nuestra aplicación web, debemos comprobar que ese usuario tiene los permisos necesarios para ello. Una forma de hacer esto es creando un nuevo servicio REST que compruebe que tanto el `username` como el `password` existen y corresponden a un usuario previamente almacenado en la base de datos. Tanto si se trata de un usuario válido como inválido, devolveremos un mensaje de éxito en la petición acompañado de algún dato más para facilitar el procesamiento de la respuesta al cliente. A continuación, se muestra el código asociado a dicha petición.

```

// Comprueba si el usuario existe
router.post('/signin', function (req, res, next) {
  User.findOne({username: req.body.username}, function (err, user) {
    if (err) res.status(500).send({message: 'Error comprobando el usuario!'});
    // Si el usuario existe...
    if (user !== null){
      user.comparePassword(req.body.password, function(err, isMatch){
        if(err) return next(err);
        // Si el password es correcto...
        if (isMatch)
          res.status(200).send({message: 'ok', role: user.role, id: user._id});
        else
          res.status(200).send({message: 'ko'});
      });
    } else
      res.status(401).send({message: 'ko'});
  });
});

```

Esta nueva petición de tipo POST estará asociada a una nueva ruta en la raíz de “/users” (concretamente “/signin”). Lo primero que haremos será buscar el usuario conforme al campo único `username`, y acto seguido vamos a comparar el `password` enviado también como parte del JSON de la petición con el existente en la base de datos. Recuerda que internamente el código del modelo *User* alberga una función para la descriptación del `password` a comparar. También podríamos haber creado un token y pasarlo como respuesta a dicha petición, pero no entraremos en detalles sobre ello por no ser considerado relevante para nuestros propósitos y los de este libro. Bien es cierto que, si queremos optimizar nuestra API REST, deberemos de tener en cuenta la seguridad de sus servicios.

Podríamos implementar más solicitudes como borrar todos los usuarios, devolver los usuarios que cumplan determinadas condiciones o que hayan sido creados recientemente, entre otros. No obstante, con estos servicios REST que acabamos de implementar será suficiente por ahora.

Lo único que nos resta sería exportar esta parte del `router` como un módulo, para hacer uso de este desde otros archivos. Esto se haría con la sentencia:

```
.....  
module.exports = router;  
.....
```

Y también tenemos que incluir la siguiente sentencia, si aún no la tenemos, en el archivo “`app.js`” de nuestra aplicación, con el fin de que *Express Router* entienda las rutas de las peticiones implementadas en “`users.js`”.

```
.....  
app.use('/users', usersRouter);  
.....
```

NOTA

Una vez hemos implementado los servicios de nuestra API REST podemos sincronizar los cambios con nuestro repositorio Github, tal y como ya conocemos. Además, si tenemos el plugin correspondiente instalado en Atom, se nos marcarán los archivos a sincronizar con el repositorio, en diferentes colores.

3.7.7 Comprobación de servicios y refinamiento del “`app.js`”

Por último, conviene comprobar el funcionamiento de las solicitudes implementadas por si hubiera que refinar alguna de ellas a causa de errores. La mejor

forma de hacerlo es desplegar la API REST desde consola con `npm start` y usar un cliente como el Advanced REST Client que ya conocemos, para probar las peticiones implementadas. Tan solo recuerda que una petición POST necesita un JSON en el cuerpo del mensaje, pero es importante que esté conformado de acuerdo al esquema del modelo *User*. En cuanto a GET (para un determinado usuario), PUT y DELETE de necesitan pasar como parámetro el `id` dado por MongoDB, el cual conoceremos una vez se hayan almacenado usuarios en la colección *users* de la base de datos.

A la vez que testeamos los servicios con el cliente REST conviene visualizar la respuesta a las solicitudes también en la consola de despliegue de nuestra aplicación, así como en la base de datos MongoDB, comprobando que se hacen efectivos los cambios. Con todo ello, ya tenemos las solicitudes relacionadas con las operaciones con usuarios listas para su despliegue en un servidor.

Por otra parte, no solo debemos asegurarnos de que las solicitudes funcionan con una aplicación externa, sino que lo hacen convenientemente con nuestra futura aplicación en React.js. Para ello, en el fichero “`app.js`” vamos a realizar algunos cambios.

Primero instalamos los siguientes módulos en el proyecto de la API REST con el comando:

```
.....  
C:\microblogging-example-api>npm install --save body-parser cors  
.....
```

Gracias a `body-parser`³⁵ podremos manejar solicitudes JSON en el `req` previamente parseadas, y gracias a `cors`³⁶ obtendremos los permisos necesarios para realizar solicitudes HTTP sobre recursos que estarán en dominios y servidores distintos a la aplicación cliente.

Además, tenemos que añadir las siguientes sentencias en el “`app.js`”:

```
.....  
var bodyParser = require("body-parser");  
var cors = require('cors');  
...  
//app.use(express.json()); //Ya no es necesario  
//app.use(express.urlencoded({ extended: false })); //Ya no es necesario  
.....
```

35 <https://www.npmjs.com/package/body-parser> [Último acceso: Julio, 2020]

36 https://developer.mozilla.org/es/docs/Web/HTTP/Access_control_CORS [Último acceso: Julio, 2020]


```
app.use(cors());  
app.use(bodyParser.json({limit: '50mb'}));  
app.use(bodyParser.urlencoded({limit: '50mb', extended: true}));
```

Algunas sentencias se han eliminado (o comentado en este caso) ya que no son necesarias, al igual que podemos suprimir las sentencias relativas al motor de vistas a utilizar, porque no serviremos páginas con Express.js, tan solo lo necesitamos para manejar y enrutar solicitudes HTTP.

Y finalmente sustituimos la sentencia:

```
res.render('error');
```

por esta otra:

```
res.json({message:err.message,error:err});
```

El objetivo final será alojar nuestra API REST en un servidor de aplicaciones externo, pero no nos adelantemos aún.

NOTA

Recuerda que no estamos controlando qué hacer cuando se produce un error en las peticiones de nuestra API REST, simplemente, y de forma general, estamos generando códigos de estado que podemos devolver. Puedes conocer más sobre esto acudiendo a otras fuentes o profundizando en las lecturas recomendadas en este libro.

3.8 SERVICIOS REST PARA LA GESTIÓN DE POSTS

Al igual que hemos hecho con los usuarios en las secciones anteriores, vamos a hacer con los posts o entradas que pueden escribir y/o publicar los usuarios. Esto requiere de nuevas rutas asociadas a servicios REST para la manipulación de dichos posts.

Con esto en mente, crearemos el archivo “posts.js” en el directorio “/routes” y de forma similar a como hicimos en el caso anterior con el archivo “users.js”, importaremos las librerías necesarias y abriremos la conexión con la base de datos de la siguiente forma.

En este caso importaremos ambos modelos mongoose *User* y *Post*, ya que necesitaremos operar sobre el usuario en algunas peticiones realizadas con los posts.

```
var express = require('express');
var mongoose = require('mongoose');
var router = express.Router();

// Modelos
var Post = require('../models/Post.js');
var User = require('../models/User.js');

var db = mongoose.connection;
```

A continuación, veremos cómo se implementan las solicitudes correspondientes para la manipulación de los posts.

3.8.1 Devolución de todos los posts existentes

Para devolver los posts existentes en la base de datos al cliente mediante un GET, lo haremos de forma similar a como implementamos la devolución de todos los usuarios, pero usando el modelo *Post*, de la siguiente forma:

```
// GET del listado de posts ordenados por fecha de publicación
router.get('/', function (req, res, next) {
  Post.find().sort('-publicationdate').populate('user').exec(function(err,
posts) {
    if (err) res.status(500).send(err);
    else res.status(200).json(posts);
  });
});
```

Los posts se devolverán en un archivo JSON al cliente que lo solicitó. Conviene recordar que el modelo *Post* contiene una referencia al modelo *User*, algo que podemos ver en el fichero “Post.js”, con lo que tenemos que “poblar” ese atributo *user* con el usuario correspondiente mediante la sentencia *populate*. El propio motor de *mongoose* ya sabe cómo traerse los usuarios asociados como parte de la consulta de los posts.

3.8.2 Devolución de los posts de un usuario

Si lo que queremos es devolver todos los posts de un usuario en concreto, realizaremos una petición sobre los posts, pero pasando como parámetro el `id` del usuario, que atendiendo al modelo *Post* es un atributo que hace referencia a un “objeto” de tipo *User*. En este caso podemos modificar la URL de acceso al recurso para para hacerla más “amigable” y reconocible.

```
// GET de todos los posts de un usuario dado (identificado por su Id)
router.get('/all/:id', function (req, res, next) {
  Post.find({'user':req.params.id}).sort('-publicationdate').populate('user').
  exec(function (err, posts) {
    if (err) res.status(500).send(err);
    else res.status(200).json(posts);
  });
});
```

En este caso, la búsqueda en la colección *posts* viene determinada por el filtro “`{ 'user': req.params.id }`”, que nos proveerá de todos los posts publicados y/o escritos por ese usuario. Como vemos, no realizamos ningún procesamiento con los datos antes de devolvérselos al cliente, pero bien podríamos realizarlo si fuera necesario en este punto.

Recuerda que lo que estamos devolviendo al cliente es un documento JSON con todos los posts a partir de las peticiones realizadas.

3.8.3 Inserción de un nuevo post

Para añadir un nuevo post a la base de datos, debemos conocer qué usuario lo ha escrito y/o publicado y que ese usuario exista.

En este caso, tenemos que cumplir con las referencias establecidas en los modelos y agregar el nuevo post a la colección de *posts* y también como nuevo elemento del array `posts[]` del usuario al que está asociado el post. Recuerda que en el modelo *User*, tenemos definido un atributo `posts` que es un array de referencias al modelo *Post*. Para ello, comprobamos que el usuario existe y obtenemos su información, y con la información del `body` del mensaje JSON de la solicitud, creamos una instancia de *Post* y la añadimos tanto al array de posts del usuario en cuestión, como a la colección de posts. Esto lo podemos hacer de la siguiente forma:

```
// POST de un nuevo post o entrada
router.post('/', function (req, res, next) {
  User.findById(req.body.iduser, function (err, userinfo) {
    if (err) res.status(500).send(err);
    else{
      // crear la instancia Post
      var postInstance = new Post({
        user: req.body.iduser,
        title: req.body.title,
        description: req.body.description
      });
      // añadir postInstance al array de posts del usuario
      userinfo.posts.push(postInstance);
      // salvar el post en las colecciones users y posts
      userinfo.save(function (err) {
        if (err) res.status(500).send(err);
        else{
          postInstance.save(function (err) {
            if (err) res.status(500).send(err);
            res.sendStatus(200);
          });
        }
      });
    }
  });
});
```

Este fragmento de código presentado supone otra forma de añadir elementos a la base de datos, a través de la creación previa de instancias de modelos.

3.8.4 Actualización de un post

La actualización de un post, implica buscar ese post en la colección basándose en su `id`, que viene del cliente como parámetro en la propia URL, y actualizarlo con la información que llega en el cuerpo del mensaje con la función ya conocida `findByIdAndUpdate()`.

```
// PUT de un post existente (identificado por su Id)
router.put('/:id', function (req, res, next) {
  Post.findByIdAndUpdate(req.params.id, req.body, function (err, postinfo) {
    if (err) res.status(500).send(err);
    else res.sendStatus(200);
  });
});
```

En el caso de una actualización exitosa, el servidor únicamente devolvería el código de estado 200.

3.8.5 Borrado de un post

En este caso, dado que un post está relacionado con un usuario, cuando lo borramos de la colección *posts*, también debemos eliminarlo del array “posts” asociado al usuario. De este modo, tendremos el siguiente código:

```
.....
// DELETE de un post existente (identificado por su Id)
router.delete('/:id', function (req, res, next) {
  Post.findByIdAndDelete(req.params.id, function (err, postinfo) {
    if (err) res.status(500).send(err);
    else{
      User.findByIdAndUpdate(postinfo.user, {$pull: { posts : postinfo._id }},
function (err, userinfo) {
  if (err) res.status(500).send(err);
  else {
    res.sendStatus(200);
  }
});
}
});
});
.....
```

En nuestro caso no estamos considerando borrados en cascada, como el que puede realizarse cuando se borra un usuario, que desencadene, a su vez, el borrado de todos los posts escritos y/o publicados por dicho usuario, pero que obviamente podríamos haber tenido en cuenta.

Finalmente, exportamos esta parte del router implementada como un módulo con:

```
.....
module.exports = router;
.....
```

También debemos asegurarnos de que, tanto las rutas como servicios implementados están importados y pueden ser usados. Para ello, en el archivo “app.js” del proyecto, tendremos que añadir lo siguiente si aún no lo hemos añadido:

```
.....  
var usersRouter = require('./routes/users');  
var postsRouter = require('./routes/posts');  
.....  
app.use('/users', usersRouter);  
app.use('/posts', postsRouter);  
.....
```

De esta forma, tendremos nuestro código modularizado con las peticiones que es capaz de servir del servidor en los ficheros correspondientes y atendiendo a las rutas raíces asociadas, que en nuestro caso serán “/users/” y “/posts/”.

Además de estos servicios, pueden implementarse otros muchos asociados a diferentes URLs, como por ejemplo los siguientes:

- Devolver todos los posts publicados entre dos fechas dadas
- Editar un post previamente almacenado
- Borrar todos los posts de un determinado usuario
- Obtener la información del usuario que ha escrito y/o publicado un determinado post
- Y muchos más.

Por otra parte, la comprobación de la existencia de documentos en la base de datos antes de operar con estos, es algo que tendríamos que refinar en el código de cada servicio pero que no es una prioridad en los ejemplos propuestos. Esto podremos hacerlo considerando manejo de errores con **express** en el servidor; aunque también podemos devolver el error al cliente y que este actúe en consecuencia, como ya hemos comentado con anterioridad.

NOTA

Al igual que en el caso de los usuarios, podemos comprobar el funcionamiento de la parte de la API REST asociada a los posts utilizando un cliente REST, y sincronizar nuestro código con el repositorio de Github.

¡Enhorabuena! Ya tenemos nuestra API REST funcional. A continuación, vamos a ver cómo podemos desplegar dicha API REST en un servidor para que otros clientes puedan consumir sus recursos.

3.9 PUESTA EN PRODUCCIÓN DE LA API REST

Cuando hemos finalizado nuestra API REST y la hemos testado en desarrollo, debemos pasarla a producción para que pueda ser utilizada por cualquier cliente que lo necesite, siempre y cuando tenga acceso a consumir dichos servicios.

El despliegue o puesta en producción de cualquier aplicación web, incluida una API REST, la podemos realizar en un servidor propio, el cual debemos configurar convenientemente, o bien usando un servidor o plataforma de servicios existente en la Nube, las cuales nos facilitan multitud de tareas y nos ofrecen un amplio abanico de opciones y herramientas para la gestión de nuestras aplicaciones a desplegar.

Tanto si optamos por desplegar la aplicación en un entorno de producción propio como de terceros, es esencial tener dicha aplicación accesible en un repositorio como Github e interactuar mediante comandos `git`, los cuales, en este caso, nos facilitarán determinadas tareas para la puesta en producción de nuestras APIs y/o aplicaciones.

Si optamos por configurar un servidor propio para producción de aplicaciones Node.js, posiblemente tendremos que instalar y configurar herramientas Pm2³⁷, Nginx³⁸, y/u otras similares. Obviamente debemos disponer de un servidor web (preferiblemente con un dominio asociado). En este caso, todo el mantenimiento del servidor nos corresponde, gestionando cualquier aspecto relacionado con la escalabilidad, seguridad, optimización, etc., que afectará a nuestros servicios y/o aplicaciones web.

No obstante, la forma más sencilla y rápida de poner en producción una aplicación web y/o una API REST en este caso, es usar una plataforma en la Nube que permita el despliegue de dichas aplicaciones, sin preocuparnos de temas que afectan a la parte del mantenimiento y gestión del servidor.

En la actualidad existen numerosas alternativas como Amazon AWS³⁹, Microsoft Azure⁴⁰, Google Cloud Platform⁴¹ o Heroku⁴²; cuyo potencial se centra en ofrecer multitud de servicios relacionados con satisfacer nuestras necesidades de

37 <https://pm2.keymetrics.io/> [Último acceso: Julio, 2020]

38 <https://nginx.org/> [Último acceso: Julio, 2020]

39 <https://aws.amazon.com/es/> [Último acceso: Julio, 2020]

40 <https://azure.microsoft.com/es-es/> [Último acceso: Julio, 2020]

41 <https://cloud.google.com/?hl=es> [Último acceso: Julio, 2020]

42 <https://www.heroku.com/> [Último acceso: Julio, 2020]

desarrollo y producción de aplicaciones (son auténticos proveedores de servicios). En este caso, Heroku es la más sencilla e interesante desde el punto de vista de nuestro proyecto, realizando el despliegue de la API REST de forma rápida y accesible (a través de la conexión con repositorios de Github).

3.9.1 Introducción a Heroku

Heroku se define como un PaaS (*Platform as a Service*) en donde podemos alojar y desplegar nuestras aplicaciones sin preocuparnos en exceso por la configuración del servidor.

Para empezar, ingresaremos en Heroku²⁹ y crearemos una cuenta gratuita. Una vez creada, accederemos al *dashboard*, en el cual podemos encontrar tutoriales sencillos para los tipos de aplicaciones que puede implementar y desplegar. En esencia, lo que hará Heroku con nuestra aplicación (o librería) es empaquetarla y ponerla a disposición del usuario en una URL determinada.

En la Figura 33 podemos ver el *dashboard* de Heroku con algunas aplicaciones desplegadas.

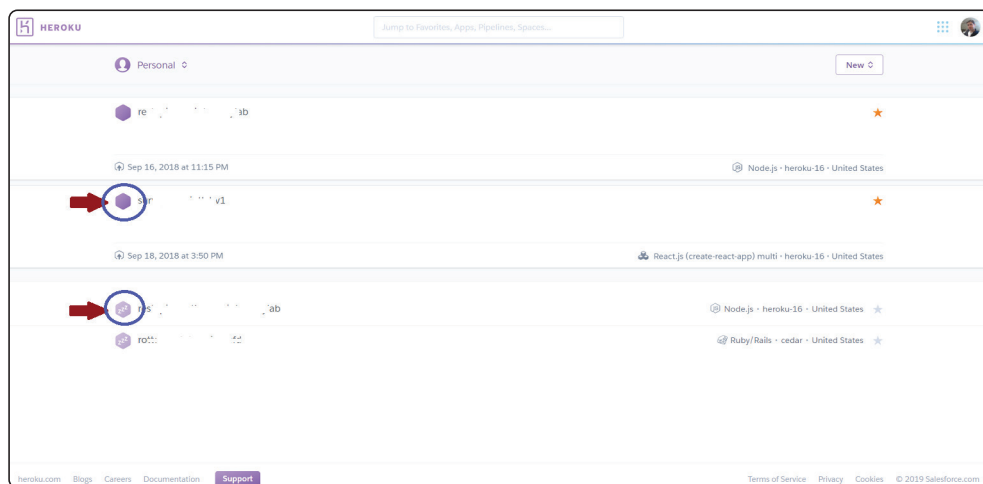


Figura 33. Ejemplo de dashboard de Heroku con diversas aplicaciones desplegadas

Podemos observar cómo en algunas de las aplicaciones aparece el símbolo “zzz”, que significa que la aplicación está dormida y que si la ejecutamos (a través de la URL asociada), necesitará un tiempo para despertar y “ponerse en marcha”.

Esto ocurre debido a la configuración de nuestra cuenta gratuita Heroku, que nos permite tener aplicaciones despiertas mientras se interacciona con ellas, en otro caso, estas se duermen (lo que ocurre por defecto, si no se ejecutan durante 30 minutos). Además, tenemos un límite de tiempo al mes en el que nuestras aplicaciones pueden estar “despiertas”. Para incrementarlo, debemos contratar uno de los planes de pago existentes.

En la parte de “Manage Account”, pestaña ”Billing”, podemos ver información al respecto del uso en horas de nuestras aplicaciones tal y como muestra la Figura 34. En Heroku, denominaremos *dynos* a las instancias de servidor.

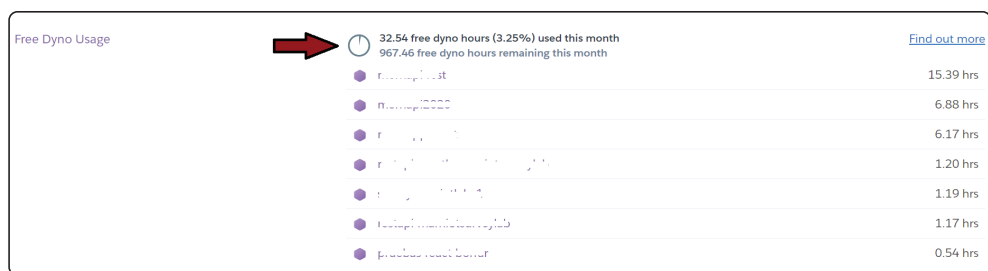


Figura 34. Dynos y horas de uso de las aplicaciones en Heroku

Volviendo al *dashboard*, si pulsamos en una aplicación ya creada, accederemos a un resumen de actividades y sucesos desencadenados para esa aplicación (ver Figura 35).

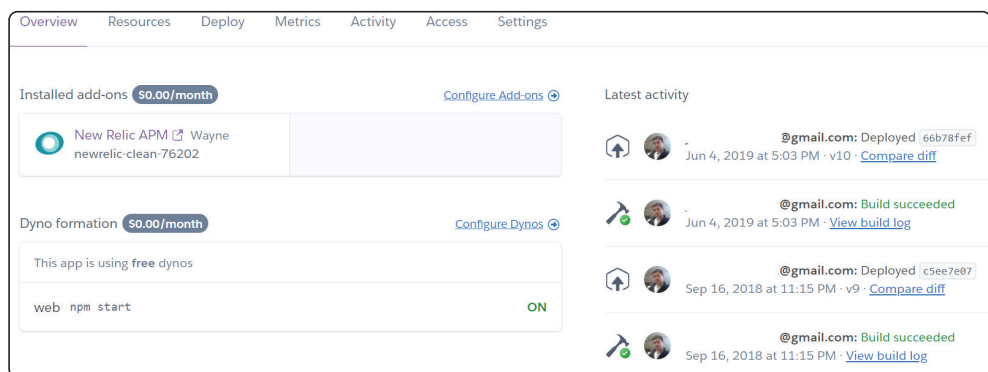


Figura 35. Descripción general de acciones sobre una aplicación desplegada en Heroku

Así, tendremos información sobre *add-ons* o *plugins* instalados (por ejemplo, *New Relic APM*⁴³ que permite monitorizar las transacciones con una aplicación o librería determinada, detectando errores, tiempo consumido en peticiones, etc.). Además de una línea de tiempo con las acciones sobre esa aplicación y otra información de utilidad. También contamos con otras pestañas para el control y configuración de nuestra aplicación desplegada, aunque no entraremos en detalle sobre las mismas.

3.9.2 Instalación y despliegue de la aplicación en Heroku

Tras crear una cuenta y conocer las características de la plataforma, es momento de desplegar nuestra API REST en Heroku. Para ello, tenemos varias opciones. Una de ellas, se describe en esta sección, y corresponde al despliegue de la aplicación desde nuestro propio PC.

Lo primero que necesitamos es tener instaladas las siguientes herramientas:

- **Node.js y npm.** Estas deberíamos tenerlas ya instaladas ya que han sido necesarias para el desarrollo de la API REST.
- **Heroku CLI (*Command Line Interface*)**⁴⁴. Nos permitirá usar Heroku a través de la consola del sistema operativo, como un comando más, con numerosas opciones.
- **Git bash.** Conforme a la sección **Instalación de Git**, deberíamos tenerlo también instalado en nuestro sistema operativo.

Una vez instaladas las herramientas anteriores, podemos iniciar sesión en Heroku desde la línea de comandos para así poder utilizar sus funcionalidades como la de enviar nuestras aplicaciones para su despliegue en la plataforma. Esto lo haremos con el siguiente comando, e ingresando las credenciales apropiadas según nuestro registro.

```
.....  
C:\microblogging-example-api>heroku login  
.....
```

43 <https://newrelic.com/products/application-monitoring>

44 <https://devcenter.heroku.com/articles/heroku-cli>

A continuación, es el momento de seguir los pasos necesarios para desplegar la aplicación en Heroku. Para ello, ingresaremos el siguiente comando desde el directorio de nuestro proyecto:

```
.....  
C:\microblogging-example-api>heroku create  
.....
```

Eso crea una aplicación vacía en Heroku (algo que también podemos hacer cómodamente desde el *dashboard* en la web de Heroku).

Una vez creada, procedemos a subir nuestra aplicación (en este caso la detectará como una aplicación Node.js) a Heroku mediante el siguiente comando:

```
.....  
C:\microblogging-example-api>git push heroku master  
.....
```

Con esto, ya tendremos nuestra aplicación en Heroku. Si bien es cierto, es posible que tengamos que configurar algunas cosas más a modo de optimización, como por ejemplo decir a Heroku que estamos en un entorno de producción y/o que vamos a usar una base de datos que se encuentra en una URL determinada (por ejemplo, en un servicio de MongoDB Atlas). Para ello, tendremos que disponer de esos datos asociados a variables de entorno (como vimos en secciones anteriores), y si lo deseamos, establecer determinadas condiciones. Recuerda que el fichero “.env” almacena los valores que queremos dar las variables de entorno.

Con el siguiente bloque de código, por ejemplo, indicamos lo siguiente: si el valor de la variable de entorno `NODE_ENV` es distinto a ‘production’ se tenga en cuenta los valores de las variables de entorno definidas en el fichero “.env”. En cualquier caso, yo podré configurar los valores de las variables de entorno desde la plataforma web de Heroku (seleccionando la aplicación recientemente creada y accediendo a la pestaña “Settings”).

```
.....  
if (process.env.NODE_ENV !== 'production') {  
  require('dotenv').config()  
}  
  
const DB_URI = process.env.DB_URI  
const PORT = process.env.PORT  
.....
```

Otra forma de asignar valores a las variables de entorno de nuestra aplicación y salvarlos para que Heroku los utilice, es haciendo uso de los comandos que el cliente de consola de Heroku nos proporciona:

```
C:\microblogging-example-api>heroku config:set NODE_ENV=production
C:\microblogging-example-api>heroku config:set DB_URI=["cadena de conexión con la
base de datos"]
```

La primera variable de entorno la usamos para definir que nuestra aplicación está en producción y que podría, por ejemplo, utilizar un puerto y una base de datos diferente a la que usamos en desarrollo. Y con el segundo comando, definimos la URL de conexión con nuestra base de datos (que podrá ser la suministrada por MongoDB Atlas u otra diferente). En caso de cambiar la ubicación de la base de datos, simplemente fijaríamos otro valor a la variable de entorno de esta manera, sin tener que modificar nada en el código.

Además, para evitar que durante el despliegue en Heroku ocurran errores relacionados con el intento del *binding* del puerto a una máquina localhost, cambiaremos la sentencia siguiente de configuración del servidor (en “/bin/www”):

```
server.listen(port);
```

Por esta otra:

```
server.listen(port, '0.0.0.0'); //Binding sobre la IP 0.0.0.0 en lugar de local-
host
```

Una vez hemos configurado y desplegado nuestra aplicación en Heroku, podremos testearla desde el propio terminal del sistema operativo, pudiendo ver la estructura de directorios que Heroku ha creado, comprobar problemas de nombres, o si todos los ficheros de la aplicación se han subido correctamente a la plataforma. Para ello, podemos ejecutar el *bash* de Heroku con el siguiente comando:

```
C:\microblogging-example-api>heroku run bash
```

Igualmente, podemos ver los *logs* lanzados por Heroku (por ejemplo, para descubrir errores de nuestra aplicación) con el siguiente comando:

```
C:\microblogging-example-api>heroku logs --app <NOMBRE_APLICACION>
```

Por último, si queremos ejecutar la aplicación, basta con abrir el navegador e ingresar la dirección donde esta se encuentra, que debería ser algo como (dependiendo del nombre que se le haya dado):

<https://mi-aplicacion.herokuapp.com>

O bien, desde el propio directorio del proyecto, con el comando:

```
C:\microblogging-example-api>heroku open
```

Además, si nuestra aplicación se encuentra en un repositorio como Github, Heroku tiene herramientas que posibilitan el despliegue de esas aplicaciones desde el propio repositorio, sin necesidad de tenerlas alojadas en nuestro PC, ni tan siquiera instalar el cliente de Heroku y hacer uso de los comandos anteriores. En la siguiente sección veremos cómo se puede desplegar la aplicación en Heroku directamente desde el repositorio de Github dónde se encuentra alojada.

3.9.3 Despliegue desde Github

Si el código de nuestra API REST está almacenado en un repositorio como Github, existe una forma mucho más simple y rápida de integrar el código del repositorio en la plataforma de Heroku y, gracias a sus herramientas, realizar el despliegue.

Recuerda que Heroku trabaja con los denominados *dynos*. Por entrar un poco más en detalle, un *dyno* es una unidad que provee la capacidad de cómputo dentro de la plataforma y que está asociada a tu cuenta Heroku. Como hemos creado una cuenta gratuita, tienes un número limitado y gratuito de horas mensuales en tu *dyno*, para ser usadas por tus aplicaciones (ver Figura 34). Y no solo eso, sino que, si Heroku detecta un periodo de inactividad en tu aplicación, la “duerme”, y volverla a lanzar costará más tiempo. Para evitar esto, no tenemos más remedio que suscribirnos a un *dyno* de pago. Pero para nuestra API REST será suficiente con la cuenta gratuita, aunque debes tener en cuenta que algunas veces esperaremos más tiempo para recibir la respuesta del servidor.

Existen scripts que pueden dejar “despiertas” las aplicaciones de Heroku indefinidamente, pero estarán consumiendo horas del *dyno*, con lo que si agotamos las horas de las que disponemos en nuestra cuenta, no podremos desplegar las

aplicaciones durante el resto del mes, a no ser que paguemos por una suscripción mensual.

Volviendo al objetivo de esta sección, para desplegar nuestra API REST (o cualquier otra aplicación) alojada en Heroku, seguiremos los siguientes pasos.

Primero, desde el *dashboard* de Heroku, creamos una nueva aplicación. Tras configurar algunos datos, elegiremos “Github” como método de despliegue, y lo conectaremos con nuestro repositorio, tal y como se muestra en la Figura 36.

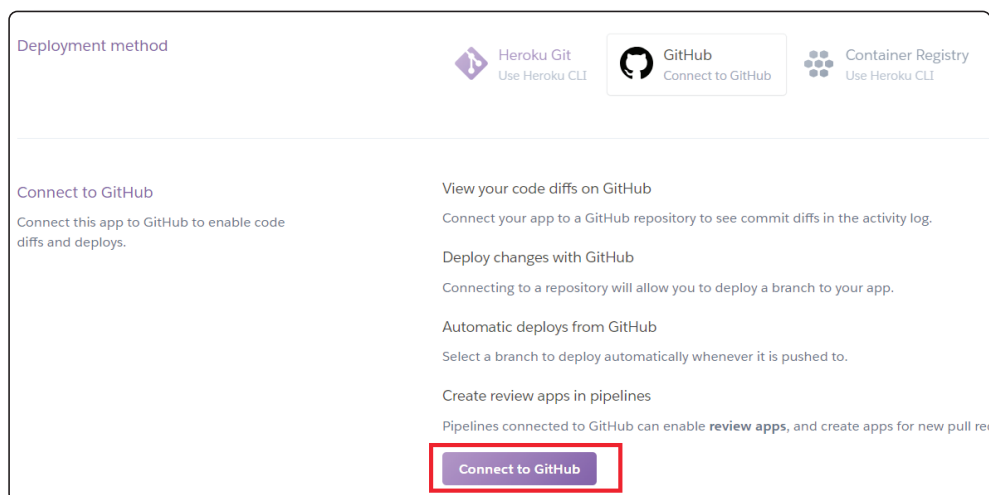


Figura 36. Conexión de la aplicación creada en Heroku a un repositorio Github

Hecho esto, en el siguiente paso, autorizaremos a Heroku el acceso a nuestra cuenta Github, indicándole posteriormente el nombre del repositorio al que queremos conectarnos, como indica la Figura 37.

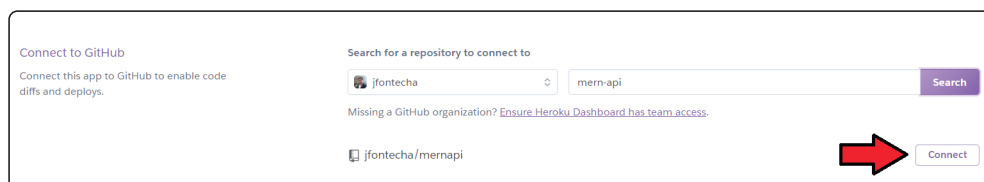


Figura 37. Elección de repositorio Github y autorización de conexión desde Heroku

Por otra parte, en el caso de que nuestra aplicación use variables de entorno, podemos asignarlas valores en la pestaña “Settings” del dashboard de Heroku. En

el caso de la Figura 38 se muestra el valor dado a la variable de entorno `DB_URI`, correspondiente a la cadena de conexión con la base de datos de MongoDB Atlas, la cual la teníamos definida como una variable de entorno en nuestro código.

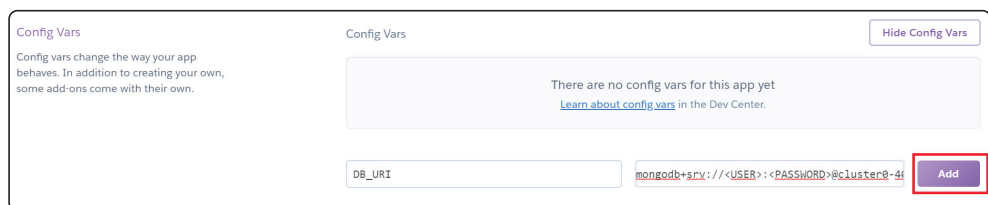


Figura 38. Configuración de variables de entorno desde Heroku

Finalmente, indicaremos cómo queremos que se nos despliegue nuestra aplicación sincronizada en Github, en la plataforma Heroku, bien manualmente o de forma automática cuando se hagan nuevos `push` a la rama indicada del repositorio (por ejemplo: “master”). Por el momento, no habilitaremos despliegues automáticos y realizaremos el despliegue manual (ver Figura 39).

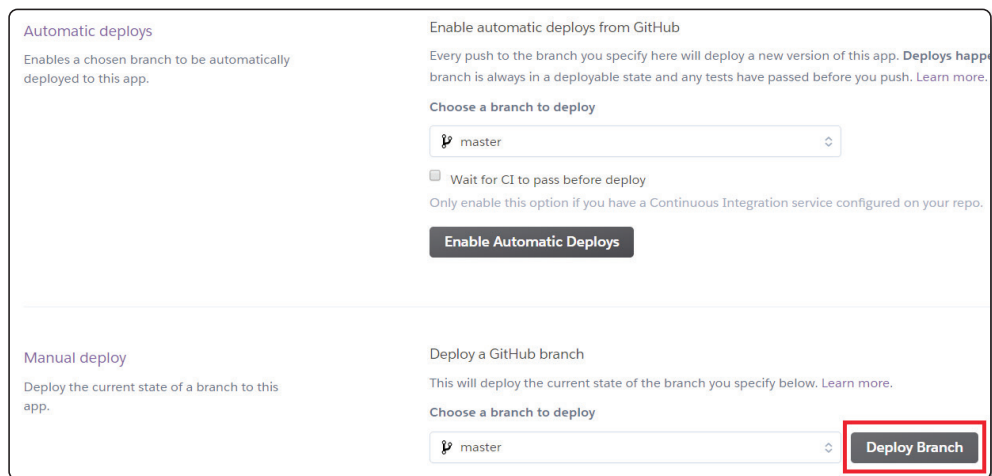


Figura 39. Configuración de las opciones de despliegue del repositorio Github en Heroku

Esperamos que acabe el proceso de configuración y despliegue y ya tenemos disponible nuestra API REST para consumir sus servicios en la URL correspondiente.

¿Qué tal si ahora probamos dichos servicios desde un cliente REST pero usando esa nueva URL?

 NOTA

Recuerda que, si actualizamos el código del repositorio, debemos volver a desplegar nuestra aplicación en Heroku siguiendo los pasos anteriores (a no ser que hayamos habilitado el despliegue automático).

3.10 CONCEPTOS AVANZADOS

En este libro estamos trabajando con los fundamentos que nos llevan a la construcción de una API REST con distintas funcionalidades, como el acceso a recursos de una base de datos, y modificación de estos. Es por ello, que no vamos a entrar en detalle en aspectos de seguridad, autenticación y producción de servicios.

Sin embargo, sí que deberíamos saber que cualquier API REST desarrollada o de la que consumimos servicios (por ejemplo, de terceros), puede requerir de una autenticación previa para el acceso a los recursos que ofrece. Así, podemos restringir el acceso a los servicios implementados, y ofrecerlos a terceros previa autenticación, o hacerlos públicos para que cualquier cliente pueda consumirlos, conociendo las URLs y URIs asociadas.

En el caso de APIs REST ofrecidas por grandes compañías de Internet, podemos ver cómo es necesaria una autenticación, por ejemplo, a través de una cuenta Google (u otras), e incluso la obtención de lo que denominamos “key” para poder llamar a los servicios correspondientes, es decir, debemos tener autorización.

Dicho esto, existen protocolos que nos aseguran la disponibilidad de nuestros servicios de forma restringida, haciéndolos más robustos. E incluso herramientas que permiten la encriptación de las solicitudes a través de la red.

En la siguiente sección veremos de forma general cómo se puede dotar de mecanismos de autenticación y autorización a través de tokens a nuestra API REST, para limitar su acceso. Aunque esto no se haya implementado en nuestra aplicación de *back-end*, conviene tener un conocimiento general al respecto, y se anima al lector a integrarlo en una versión mejorada de dicha API REST.

Por otra parte, conviene saber que cuando queremos llevar a producción una aplicación o API REST, esta es convertida y empaquetada en un “bundle.js” a través de las herramientas correspondientes. Una vez la aplicación ha sido preparada para su ejecución en producción, simplemente alojaremos ese archivo “bundle.js” (con otros directorios y ficheros de recursos) en el servidor. Esto es lo que suelen hacer de forma interna las plataformas de producción y despliegue (como Heroku).

3.10.1 Seguridad de la API REST y uso de tokens

Volviendo al tema de la seguridad y autenticación de una API REST, diremos que una de las formas más sencillas de identificarnos en un servidor para consumir los recursos alojados en él (por ejemplo, a partir de una API REST), es hacerlo mediante *tokens*.

Pero ¿Qué es un *token*?

Un token es una cadena alfanumérica con caracteres aparentemente aleatorios. A continuación, podemos ver un ejemplo de *token*:

```
.....  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIx  
MjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWV  
Rtaw4iOnRydWV9.TjVA95OrM7E2cBab30RMHr-  
HDcEfxjoYZgeFONFh7HgQ  
.....
```

Lo cierto es que estos *tokens* tienen un significado real para el servidor o institución que lo emite, el cual puede entender y de este modo, validar al usuario que intenta acceder a la información.

En un ejemplo de solicitud a un servicio que requiere autenticación del cliente, el proceso es el siguiente:

1. El cliente se autentica por primera vez para el acceso a ciertos recursos del servidor.
2. Se genera un token asociado al usuario que desea realizar peticiones al servidor.
3. El token es validado por el servidor, y comunica al usuario que se acepta la ejecución de las peticiones que se vayan a realizar.

La forma más usada y sencilla de añadir autenticación a nuestros servicios REST es a través de JWT⁴⁵ (*JSON Web Tokens*). Los JWT son un tipo de *token* con una estructura definida, que puede ser descryptado por el servidor y proceder a nuestra autenticación. En dicha estructura encontramos 3 partes:

- *Header*: almacena el tipo de *token* y algoritmo de encriptación.

45 <https://jwt.io/> [Último acceso: Julio, 2020]

- *Payload*: contiene los datos que identifican al usuario (Id, nombre de usuario, etc.).
- *Signature*: se genera con las partes anteriores y sirve para validar que el contenido no se ha alterado.

Cuando necesitamos autenticar a un usuario, lo hacemos mediante un login tradicional (usuario y password). Una vez el servidor valida al usuario, se genera un *token* que retorna al usuario cliente. Este almacenará el *token*, y a partir de ese momento, todas las peticiones realizadas al servidor deberán llevar dicho token. El *token*, por lo general, se almacena como una Cookie⁴⁶ o en el LocalStorage⁴⁷ del navegador, y cuando se necesita en una solicitud, se recupera y se envía en el *Header*.

El *token* no requiere ser almacenado en el servidor, ya que por sí solo contiene toda la información para ser validado e identificado, sabiendo de quién es el *token* a través de su *payload*. Además, todo *token* expira después de un tiempo, por lo que deja de ser válido y el usuario debe volverse a autenticar con sus credenciales para obtener un nuevo *token* válido. En la Figura 40 se muestra el ciclo de vida de un JWT:

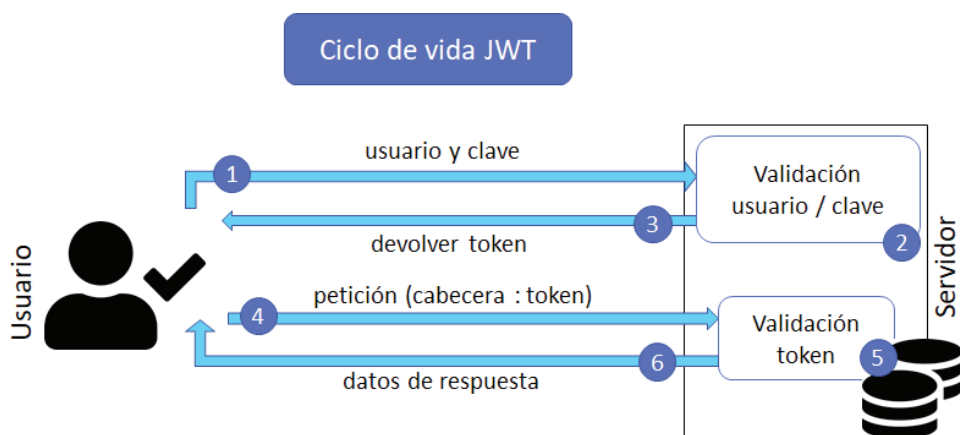


Figura 40. Ciclo de vida de un JWT

46 <https://developer.mozilla.org/es/docs/Web/HTTP/Cookies> [Último acceso: Julio, 2020]

47 <https://developer.mozilla.org/es/docs/Web/API/Window/localStorage> [Último acceso: Julio, 2020]

Aquí⁴⁸ puedes encontrar información más detallada de las etapas del ciclo de vida de un JWT, así como entrar en detalle acerca del uso de JWT para integrarlos en la API REST desarrollada.

Ten en cuenta que, si quieres usar las funcionalidades de JWT, primero tienes que instalar el paquete asociado vía `npm` en el directorio del proyecto de la siguiente forma:

```
>npm install --save jsonwebtoken
```

Tras ello, deberemos crear el *token*, además de alguna función auxiliar, y comprobar que el acceso a cada servicio puede realizarse atendiendo a la validez de dicho *token*. No es objetivo de este libro entrar en detalles acerca de esta implementación, pero es aconsejable profundizar sobre ello y tratar de “asegurar” los servicios de nuestra API REST tal y como se propuso anteriormente.

NOTA

Existen numerosas formas de implementar autenticación a servicios basada en *tokens*, cada una con sus ventajas y sus desventajas. Otra forma bastante extendida y usada en APIs complejas y con acceso a la información por multitud de clientes y arquitecturas de la que seguro has oído hablar es OAuth2.0⁴⁹.

3.11 LECTURAS RECOMENDADAS

Conforme a lo desarrollado en esta parte, el *framework* Express.js nos proporciona, entre otras características, los mecanismos necesarios para manejar solicitudes HTTP de forma sencilla y eficiente. Recuerda que este *framework* se soporta sobre Node.js.

48 <https://www.oscarblancarteblog.com/2017/06/08/autenticacion-con-json-web-tokens/> [Último acceso: Julio, 2020]

49 <https://oauth.net/2> [Último acceso: Julio, 2020]

En este sentido, te podemos recomendar las siguientes referencias como material adicional a lo explicado en esta parte (así como en las webs de referencia):

- Brown, E. *Web Development with Node and Express: Leveraging the JavaScript Stack*. O'Reilly Media, 2014.
- Mardanov, A., Chakkaev, A. *Express.js Guide: The Comprehensive Book on Express.js*. Createspace Independent Pub., 2013.
- Bojinov, V. *RESTful Web API Design with Node.js 10: Learn to create robust RESTful web services with Node.js, MongoDB, and Express.js*. Packt Publishing (3ª Edición), 2018.
- Beerbhom, M., Mohammed, M. *Express.js: The Ultimate Beginner's Guide to Learn Express.js Step by Step*. Publicación independiente (2ª Edición), 2020.
- Gourley, D., Totty, B., Sayer, M., Aggarwal, A., Reddy, S. *HTTP: The Definitive Guide*. O'Reilly Media, 2002.
- Pollard, B. *HTTP/2 in Action*. Manning Publications, 2019.

Tal y como sabemos, **mongoose** es el *Object Document Mapper* (ODM) utilizado para definir los objetos con un esquema fuertemente tipado asignado a documentos MongoDB. La principal fuente de información sobre **mongoose** es la propia web de referencia⁵⁰. No obstante, en la mayoría de libros existentes de MongoDB y/o Node.js se explica el papel de los ODM y la librería **mongoose**. Referencias como la siguiente se centran, además, en dicha librería:

- Holmes, S. *Mongoose for Application Development*. Packt Publishing, 2013.

Finalmente, existen multitud de recursos relacionados con la plataforma Heroku y el despliegue de aplicaciones en la misma. A continuación, se citan algunas referencias a las que conviene echar un vistazo si deseas profundizar en el tema:

- Hanjura, A. *Heroku Cloud Application Development*. Packt Publishing, 2014.
- Middleton, N., Shneeman, R. *Heroku: Up and Running: Effortless Application Deployment and Scaling*. O'Reilly Media, 2013.
- Coutermarsh, M. *Heroku Cookbook*. Packt Publishing, 2014.

50 <https://mongoosejs.com/> [Último acceso: Julio, 2020]

4

REACT.JS

La última parte de este libro corresponde al desarrollo del *front-end*, en el que crearemos una aplicación en React.js, apoyándonos en la librería **axios** para consumir los servicios de la API REST implementada, y en librerías como **reactstrap** para el uso de estilos en la interfaz de usuario (UI) basados en Bootstrap e implementados como componentes React.js. En el desarrollo de las vistas de la UI no haremos uso de hojas de estilo CSS. Si deseas profundizar acerca de la implementación de interfaces de usuario para aplicaciones web, te recomendamos que eches un vistazo a la sección de lecturas recomendadas. Finalmente, daremos los pasos necesarios para desplegar la aplicación *front-end* en Heroku. El área coloreada de la Figura 41 muestra el lugar ocupado por React.js en la pila MERN y los puntos más destacados que se tratarán en esta última parte.



Figura 41. Tecnologías y herramientas de desarrollo a partir de la pila MERN. React.js

4.1 INTRODUCCIÓN AL FRAMEWORK REACT.JS

El *framework* React.js constituye la tecnología de cliente o *front-end* en la pila MERN, y a la que vamos a dedicar esta última parte del libro. En este caso, también se trata de una librería JavaScript, creada por Facebook, que nos ayuda a la hora de desarrollar aplicaciones en una sola página, conocidas como SPA (*Single Page Application*). Gracias a una comunidad en constante crecimiento, actualmente se pueden encontrar recursos implementados en React.js para satisfacer casi cualquier necesidad. De este modo, los desarrolladores podrán crear aplicaciones React.js basadas en componentes propios y/o importarlos de terceros. Como vemos, aparece la palabra *componente*, elemento muy importante en React.js, el cual detallaremos más adelante.

Otra de las características de React.js que la diferencia de otras librerías, es el uso de la sintaxis JSX⁵¹ para construir componentes de interfaz de usuario. JSX crea plantillas donde se pueden anidar elementos, por lo que se pueden crear componentes tan pequeños o grandes como se necesite. Veremos esta sintaxis del lenguaje en la sección **Introducción a la sintaxis JSX**.

Por otra parte, React.js utiliza un DOM virtual o *Virtual DOM* que recarga cada componente de forma individual cuando hace falta, por lo que ofrece gran velocidad a la hora de renderizar las vistas. En este sentido, cuando una aplicación cliente pretende mostrar la información al usuario a través de su interfaz, esta se representa como un árbol de etiquetas (HTML, XML, etc.) jerarquizado. A dicha representación se le denomina DOM (*Document Object Model*). Teniendo en cuenta esto, el *Virtual DOM* es considerado una abstracción del DOM, de forma que, al renderizar las vistas, se renderizará lo estrictamente necesario de una forma mucho más optimizada. Para profundizar sobre estos conceptos es recomendable acudir a las lecturas recomendadas al final de esta parte.

Otro de los elementos interesantes de este *framework* son los componentes. Estas “piezas de software” son independientes y contienen su propio comportamiento, un estado y el contenido a renderizar (es decir, la vista a mostrar al usuario). De esta forma se pueden establecer jerarquías y relaciones entre componentes para posibilitar su comunicación. A su vez, las aplicaciones creadas con React.js tienen un estado global que repercute en el estado individual de cada componente. Por tanto, estamos tratando con un *framework* orientado a componentes.

El isomorfismo (renderización de código HTML tanto en el servidor como en cliente) es otra de las características de React.js, ofreciendo la posibilidad de entregar el HTML ya renderizado a los buscadores web (por ejemplo, Google) y así mejorar el posicionamiento. De esta manera se evita el problema de que el buscador se encuentre el cuerpo de la página vacío.

51 <https://jsx.github.io/> [Último acceso: Julio, 2020]

En la página web oficial de React.js⁵² tienes toda la información detallada de este *framework*, así como tutoriales y ejemplos prácticos.

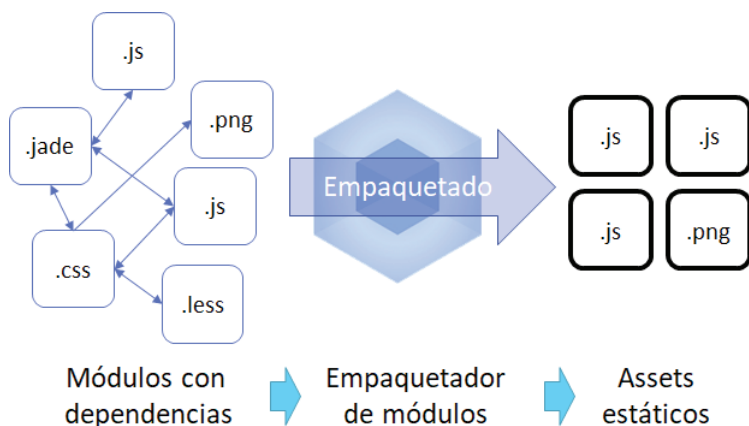
A partir de aquí, explicaremos los conceptos más importantes que debes tener en cuenta a priori, así como el desarrollo paso a paso de nuestra aplicación *front-end* en React.js.

4.1.1 Empaquetadores de módulos

A la hora de crear una aplicación con React.js conviene conocer el papel que juegan los empaquetadores de módulos o *bundler*.

Estas herramientas permiten automatizar procesos como la transpilación de código y el preprocesamiento, además de realizar otras tareas. Su finalidad es traducir todo el código implementado, junto con sus dependencias, en un conjunto reducido de *assets* en lenguajes comunes y comprensibles por cualquier navegador. En definitiva, lo que realizan es el procesamiento de unos archivos de entrada para convertirlos en otros archivos de salida (mediante cargadores o *loaders*).

Un ejemplo muy conocido de *bundler*, es *webpack*⁵³. La Figura 42 muestra una visión general de lo que *webpack* es capaz de hacer. Una vez construido ese conjunto de *assets* estáticos, la aplicación quedará embebida en estos y podrá ser desplegada fácilmente en el servidor.



Además, también hemos comentado que, junto a estos empaquetadores, se encuentran los *loaders*, los cuales pueden definirse como *plugins* para soportar las transformaciones que va a tener el código fuente de nuestra aplicación. Por ejemplo, un *loader* de un archivo “*.scss” se encargará de cargar el archivo y convertirlo en un fichero “*.css” por medio de paquetes externos. De igual forma, otro *loader* diferente (por ejemplo, **babel**) podría convertir código JavaScript ES-2017 o JSX a una especificación JavaScript que la mayoría de navegadores de Internet puedan ejecutar. A este proceso, se le denomina *transpilación* de código (concepto que ya se mencionó en secciones anteriores).

Los comandos tanto para instalar *webpack* de forma global como algunos de los *loaders* o cargadores más interesantes son los siguientes. Igualmente, la forma de instalar todos estos paquetes conlleva el uso del gestor **npm**, previamente instalado en nuestro sistema operativo.

```
>npm install -g webpack  
>npm install --save-dev babel-loader babel-core babel-preset-env
```

Podemos comprobar la instalación de dichos paquetes en el “package.json”, así como las dependencias de los mismos bajo el ítem **devDependencies** en el mismo archivo.

Además, se le puede dotar a *webpack* de diversas configuraciones, proporcionando información sobre el componente principal de nuestra aplicación (para su despliegue inicial), el nombre del archivo del *bundle* final generado, especificar un determinado *loader* para la *transpilación*, etc. Dichas configuraciones residirán en el archivo “webpack.config.js” de la aplicación. En la sección de lecturas recomendadas de esta parte, se exponen diferentes fuentes donde poder profundizar más en estos aspectos de configuración y empaquetado de aplicaciones cliente, sobre los cuales no vamos a detallar mucho más por el momento ya que forman parte de la puesta en producción y despliegue de las aplicaciones.

4.1.2 Hola Mundo en React.js

Al igual que sucedía con Express.js, existen paquetes que nos posibilitan la generación de aplicaciones React.js desde cero componiendo todo el esqueleto y estructura de directorios y ficheros necesaria para su ejecución.

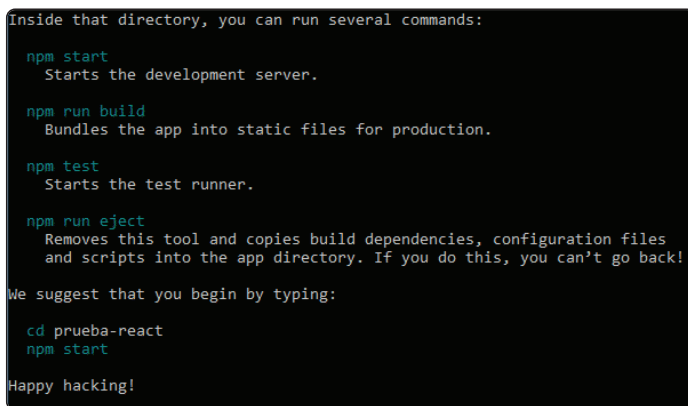
En este caso, la herramienta de *scaffolding* a instalar vía **npm** es la siguiente (la instalaremos de forma global en el sistema):


```
>npm install -g create-react-app
```

Cuando finalice la instalación ya podremos crear nuestra primera aplicación en React.js con el comando `create-react-app`. Para ello, en una carpeta del sistema o en la unidad raíz ejecutaremos el siguiente comando desde el terminal (necesitaremos abrir una consola en modo Administrador para ello):

```
C:\>create-react-app microblogging-example-react
```

Tras un breve tiempo y un mensaje de éxito, el proyecto, con el nombre `microblogging-example-react`, estará disponible en un directorio con el mismo nombre, con la ejecución de dicho comando. A partir de ahora podremos desplegarlo y testearlo con los comandos que se nos indican en consola tras la creación, tal y como se muestra en la Figura 43.



```
Inside that directory, you can run several commands:

npm start
  Starts the development server.

npm run build
  Bundles the app into static files for production.

npm test
  Starts the test runner.

npm run eject
  Removes this tool and copies build dependencies, configuration files
  and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

cd prueba-react
npm start

Happy hacking!
```

Figura 43. Comandos principales para la ejecución de la aplicación React.js

Si accedemos a la estructura de directorios del proyecto creado con el explorador de archivos del sistema operativo, tendremos algo parecido a lo presentado en la Figura 44:



node_modules	21/02/2018 18:15	Carpeta de archivos	
public	21/02/2018 18:15	Carpeta de archivos	
src	21/02/2018 18:15	Carpeta de archivos	
.gitignore	21/02/2018 18:15	Documento de texto	1 KB
package.json	21/02/2018 18:15	Archivo JSON	1 KB
package-lock.json	21/02/2018 18:15	Archivo JSON	331 KB
README.md	21/02/2018 18:15	Archivo MD	117 KB

Figura 44. Estructura de ficheros y directorios del proyecto React.js

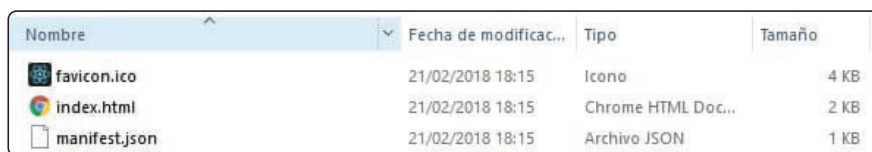
A continuación, comentaremos de forma general cada uno de estos directorios y ficheros.

/node_modules

Este directorio contendrá todos los módulos como dependencias del proyecto. Por defecto se instalarán las contenidas por defecto en el “package.json”, pero podremos instalar más a través de la herramienta `npm` y el propio “package.json”.

/public

Es la raíz del proyecto, dónde se encuentra el archivo principal “index.html” y recursos como el icono que aparecerá en la pestaña del navegador cuando se despliegue la aplicación (“favicon.ico”). En el “index.html” habrá una etiqueta `<div>` con un `id` “root” que indica en qué lugar de la página se va a introducir el componente principal de nuestra aplicación. Este estará implementado en el fichero “App.js”. La Figura 45 muestra el contenido de este directorio.



Nombre	Fecha de modificac...	Tipo	Tamaño
favicon.ico	21/02/2018 18:15	Icono	4 KB
index.html	21/02/2018 18:15	Chrome HTML Doc...	2 KB
manifest.json	21/02/2018 18:15	Archivo JSON	1 KB

Figura 45. Archivos del directorio /public generados automáticamente

/src

Es la carpeta donde se almacenan los componentes (implementados en JavaScript o JSX) y los estilos “.css”, aparte de otros ficheros. Por defecto, el componente principal del proyecto se encuentra en “App.js” (o bien “App.jsx”). También existe un fichero “index.js” que se encarga de introducir el componente *App* en el `<div>` del “index.html”. En la Figura 46, se muestran los archivos contenidos en este directorio.



Nombre	Fecha de modificac...	Tipo	Tamaño
App.css	21/02/2018 18:15	Documento de hoj...	1 KB
App.js	21/02/2018 18:15	Archivo JavaScript	1 KB
App.test.js	21/02/2018 18:15	Archivo JavaScript	1 KB
index.css	21/02/2018 18:15	Documento de hoj...	1 KB
index.js	21/02/2018 18:15	Archivo JavaScript	1 KB
logo.svg	21/02/2018 18:15	Documento SVG	3 KB
registerServiceWorker.js	21/02/2018 18:15	Archivo JavaScript	5 KB

Figura 46. Archivos del directorio /src

Podemos observar que también se encuentran las hojas de estilos “App.css” e “index.css”, una imagen en formato *svg* del logo de React.js, un archivo para realizar tests sobre el componente App y un archivo llamado “registerServiceWorker.js”. Este archivo será usado por la aplicación para intentar obtener los recursos de caché antes de obtener alguna información de la red, proporcionando experiencia de uso determinada incluso cuando se está sin conexión.

Habitualmente, esta estructura de directorios y ficheros dentro de */src* es adaptada a las necesidades del desarrollador y de la propia aplicación, teniendo una organización en subdirectorios para los componentes, modelos, estilos, y otro tipo de recursos necesarios.

README.md

Se trata d un fichero de información general del proyecto.

package.json

Al igual que en la aplicación *back-end* asociada a nuestro API REST, en el caso de la aplicación React.js también tendremos un archivo “package.json” a modo de manifiesto con información del proyecto, paquetes necesarios, dependencias y *scripts*.

.gitignore

En este fichero se describen aquellos archivos y/o directorios a ignorar cuando usamos Git (por ejemplo, es común ignorar el directorio “/node_modules”, y no almacenar todo el código de los módulos en el repositorio, ya que podemos instalarlos a la hora de desplegarlo con el comando `npm install`). Igualmente podemos decidir ignorar archivos como el “.env” si nuestra aplicación tuviera variables de entorno definidas y que no queremos sincronizar con el repositorio (cómo ya hicimos durante la implementación de la API REST).

Con ello, hemos visto como es la estructura de directorios creada por el generador de aplicaciones React.js. Es importante que el código de nuestra aplicación quede convenientemente embebido atendiendo a esta organización.

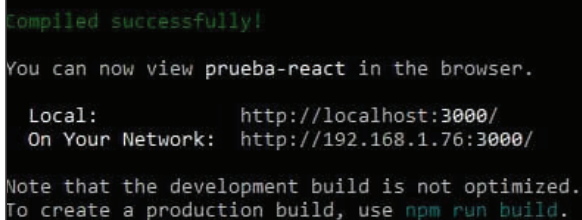
Dado que el generador de aplicaciones nos ha generado una aplicación lista para ser ejecutada, lo primero que haremos será desplegarla desde consola y visualizar el resultado en el navegador. En un entorno local y desde la carpeta del proyecto, usaremos el siguiente comando:

```
C:\microblogging-example-react>npm start
```

Hecho esto, se nos abrirá el navegador por defecto y en una pestaña nueva veremos la interfaz de usuario de nuestra aplicación React.js, la cual está accesible desde la URL:

http://localhost:3000

De la misma forma, en el terminal nos deberá salir un mensaje de éxito como el mostrado en la Figura 47.



```
Compiled successfully!  
  
You can now view prueba-react in the browser.  
  
Local:            http://localhost:3000/  
On Your Network:  http://192.168.1.76:3000/  
  
Note that the development build is not optimized.  
To create a production build, use npm run build.
```

Figura 47. Ejecución de la aplicación React.js. Mensajes del terminal

4.1.2.1 TRABAJANDO CON COMPONENTES

Como hemos indicado anteriormente, el *framework* React.js está basado en componentes. Si estás acostumbrado o conoces la programación orientada a objetos, podríamos asemejar un componente a una clase (de hecho, son clases que heredan de una clase “Component” superior). Por ello, es recomendable tener una buena organización de los componentes en directorios del proyecto, y reutilizarlos en las partes del código donde se precisen. Los componentes podrán ser independientes o estar relacionados con otros componentes. De todo ello hablaremos con algo más de detalle en posteriores secciones.

En un principio, para ahondar más en el conocimiento de los componentes y su implementación, crearemos un nuevo componente en el directorio “/src” del proyecto anteriormente generado, a la misma altura que el componente principal “App.js”, que llamaremos “Saludo.js”. A continuación, se muestra el código de este componente:

```
import React, { Component } from 'react';
class Saludo extends Component {
  render() {
    return (
      <h1>
        Hola Mundo!
      </h1>
    );
  }
}
export default Saludo;
```

Definiremos “Saludo.js” como una clase que va a heredar todo aquello que tiene un componente React.js (**Component**).

El método `render()` de la clase `Saludo`, renderizará un bloque de código HTML con un título cuyo contenido es “Hola Mundo!”. También se puede implementar una función o método constructor donde definiremos el estado del componente y donde podremos acceder y definir las propiedades y comportamiento de dicho componente.

Una vez hecho esto, para poder hacer uso de este componente en “App.js”, o en cualquier otro fichero, y renderizarlo, debemos exportarlo (con `export default`) dándole un nombre. En este caso `Saludo`.

Desde el componente principal “App.js” se orquestará el despliegue de componentes de la aplicación, ya sean propios o de terceros. En nuestro ejemplo, vamos a embeber el componente “Saludo.js” en el componente “App.js” para observar el efecto que tiene. Para ello, gracias a React.js, podemos hacer uso de la etiqueta `<Saludo />` (que representará el componente “Saludo.js”) tal y como se muestra a continuación.

```
import React, { Component } from 'react';
import Saludo from './Saludo';
import logo from './logo.svg';
import './App.css';

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
        </header>
      </div>
    );
  }
}
```

```
        <h1 className="App-title">Welcome to React</h1>
      </header>
      <Saludo />
    </div>
  );
}
}

export default App;
```

Obviamente lo tendremos que importar al inicio del fichero que lo usará, en nuestro caso el “App.js”. En el bloque de código anterior se observa cómo queda el “App.js” tras la integración del componente “Saludo.js” en el mismo.

NOTA

Es posible que al generar la aplicación React.js con el generador de aplicaciones instalado, este haya creado el componente “App.js” como una función. Eso es porque dicha versión del generador ya está adaptada a la creación de aplicaciones React.js bajo la nueva versión del *framework* liberada hace poco, y que usa los denominados React *hooks* en lugar de clases. Por el momento, implementaremos nuestros componentes como clases, por lo que es aconsejable sustituir el código generado para el componente “App.js” por el mostrado anteriormente.

Por último, ¿Qué contienen los ficheros “index.js” e “index.html”?

En el archivo “index.js” se define como incrustar nuestro componente principal “App.js”, que contendrá en su interior el componente “Saludo.js” que hemos creado anteriormente. Haciendo uso de la clase `ReactDOM`, podremos renderizar “App.js” en el fichero “index.html”, archivo base HTML de inicio de la aplicación.

A continuación, se muestra el código del “index.js” que renderizará dicho componente principal “App.js” en el elemento HTML que contenga el `id` “root”.

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import registerServiceWorker from './registerServiceWorker';

ReactDOM.render(<App />, document.getElementById('root'));
registerServiceWorker();
```

El fichero “index.html” declara la cabecera del HTML, enlaces a librerías externas o scripts en JavaScript, y demás código HTML asociado. Además, define el lugar dónde queremos embeber el componente principal “App.js” a través de la etiqueta:

```
<div id="root"></div>
```

A continuación, se muestra el contenido completo de este “index.html”.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-
to-fit=no">
    <meta name="theme-color" content="#000000">

    <link rel="manifest" href="%PUBLIC_URL%/manifest.json">
    <link rel="shortcut icon" href="%PUBLIC_URL%/favicon.ico">

    <title>React App</title>
  </head>
  <body>
    <noscript>
      You need to enable JavaScript to run this app.
    </noscript>
    <div id="root"></div>

  </body>
</html>
```

Una vez hecho esto, podemos volver a ejecutar el proyecto con el comando `npm start` y visualizar en el navegador el nuevo resultado, comprobando el renderizado del nuevo componente “Saludo.js” creado (ver Figura 48).



Figura 48. Resultado del despliegue de la aplicación React.js “Hola mundo” en el navegador

NOTA

Hasta la versión 16.8 de React.js, los componentes se implementaban como clases heredadas, pero a partir de dicha versión aparece el concepto de React *hooks*, dónde los componentes son implementados como funciones JavaScript con algunas peculiaridades, sobre todo relacionadas con el uso de estados y etapas del ciclo de vida. En la sección **Introducción a los *hooks* en React.js** presentaremos estos cambios y sus consecuencias sobre la aplicación a implementar. No obstante, ambas filosofías son perfectamente compatibles hoy en día.

4.1.3 React Developer Tools

*React Developer Tools*⁵⁴ es una extensión para los navegadores Google Chrome y Mozilla Firefox que, instalada, nos provee de un conjunto de utilidades para la inspección de los componentes de una aplicación web React.js en ejecución. De esta forma podremos observar una vista del árbol de componentes, así como el estado actual y las propiedades de cada componente seleccionado. Tan solo tenemos que hacer *click* con el botón derecho del ratón y seleccionar “inspeccionar” para acceder a la información de la web. Después, en la pestaña *React* accedemos a la información proporcionada por estas herramientas. En la Figura 49 podemos ver un ejemplo.

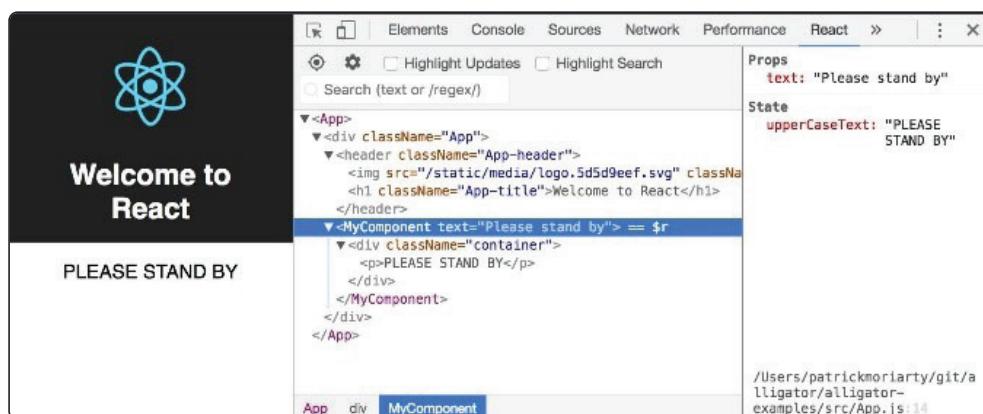


Figura 49. Uso de React DevTools y la inspección de código desde el navegador

54 <https://reactjs.org/blog/2019/08/15/new-react-devtools.html> [Último acceso: Julio, 2020]

Así mismo, gracias a estas herramientas, podemos ver si una aplicación web está desarrollada en React.js y si se está ejecutando en producción o en depuración, entre otras características.

Es muy aconsejable utilizar este tipo de herramientas para depurar tus aplicaciones web, a nivel de cliente desde el navegador. Sin embargo, no entraremos en detalle acerca de su uso por no ser objetivo de este libro.

4.1.4 Introducción a la sintaxis JSX

Como ya se mencionó en secciones anteriores, una de las ventajas de React.js es el uso del *Virtual DOM* y la rapidez de renderizado de las vistas, la cuales incluirán componentes (sencillos o complejos). Para optimizar este trabajo, Facebook crea el lenguaje JSX que se define como una extensión de la sintaxis JavaScript, y cuyos ficheros albergan la extensión “*.jsx”. Esta sintaxis es específica de React.js y permite mezclar código JavaScript y HTML (XML). De esta forma, podremos escribir un código más limpio, sin tantas repeticiones, y con pocos factores o condiciones a tener en cuenta.

JSX será usado por React.js para generar su *Virtual DOM*, sincronizando después con el DOM real, para representar la información de las vistas en el navegador.

Como hemos visto en la aplicación inicial desarrollada en la sección Hola Mundo en React.js, al basar el desarrollo de aplicaciones en componentes, necesitamos crear elementos HTML que definan nuestros componentes. De esta forma, cuando se trata de componentes creados por nosotros (por ejemplo, para su uso mediante las etiquetas asociadas: `<Image/>`, `<List/>`, `<Saludo/>`, etc.), lo podemos implementar directamente en JavaScript a partir de los métodos y funciones que nos ofrece React.js mediante `React.createElement`.

Imagina que quieres crear un componente HTML `<Icon/>`, definido por un `<div>`, `<img>` y algunas hojas de estilo CSS. Usando JavaScript, la implementación sería similar a esta:

```
var image = React.createElement('img', {
  src: 'react-icon.png',
  className: 'icon-image'
});

var container = React.createElement('div', {
  className: 'icon-container'
```

```

    }, image);

    var icon = React.createElement('Icon', {
      className: 'avatarContainer'
    }, container);

    ReactDOM.render(icon, document.getElementById('app'));

```

Este componente <Icon/> se traduciría en el siguiente código HTML:

```

<div class='icon-container'>
  <img src='icon-react.png' class='icon-image' />
</div>

```

Siendo el CSS asociado, el siguiente:

```

.icon-image {
  width: 100px
}
.icon-container {
  background-color: #222;
  width: 100px
}

```

De esta forma, si quisiéramos hacer lo mismo, pero empleando la sintaxis JSX, el código del componente <Icon/> quedaría como sigue:

```

var Icon = (
  <div className='icon-container'>
    <img
      src='icon-react.png'
      className='icon-image'
    />
  </div>)

ReactDOM.render(Icon, document.getElementById('app'))

```

Como se puede observar, este código es más fácil de comprender y más legible. Es como escribir código HTML pero realmente es JavaScript.

Obviamente debemos de tener en cuenta algunas palabras reservadas propias de la sintaxis JSX, como puede el caso del atributo `class` que pasa a ser `classname`, entre otras.

Cuando nuestra aplicación va creciendo en componentes y complejidad, usar JSX nos ayudará a agilizar nuestros desarrollos.

Así, el hecho de usar JSX para codificar los componentes de nuestra aplicación, hace que tengamos que utilizar un *bundler* como **webpack** para empaquetar, junto con un transpilador como **babel** que permita la transpilación del código JS a JSX antes de desplegarlo por ejemplo en un entorno de producción (ver sección **Empaquetadores de módulos**).

Por último, vamos a ver otro ejemplo del código de un componente utilizando la especificación ES6 con su sintaxis de clases, y el mismo ejemplo utilizando la sintaxis JSX.

En el primer caso, el código sería similar a este:

```
.....  
class ComponenteConEstado extends React.Component{  
  
  constructor(props) {  
    super(props);  
  }  
  
  render() {  
    return React.createElement(  
      'div',  
      null,  
      React.createElement(  
        'h1',  
        null,  
        'Cabecera 1'  
      ),  
      React.createElement(  
        'h2',  
        null,  
        'Cabecera 2'  
      )  
    );  
  }  
}
```

```
.....
```

Mientras que, usando JSX se podría simplificar a algo como:

```
.....  
class ComponenteConEstado extends React.Component{  
  
  constructor(props){  
    super(props);  
    this.state = { title: 'Introducción React'};  
  }  
  
  render() {  
    const { title } = this.state;  
    return ( <div> <h1>Cabecera 1 { title }</h1> <h2>Cabecera 2</h2> </div> );  
  }  
}
```

```
.....
```

Nuevamente podemos comprobar la facilidad de comprensión de dicho código usando JSX, así como las ventajas de renderización aportadas por el *framework* React.js. Por eso, lo mejor y más recomendable cuando desarrollamos una aplicación en React.js es usar la sintaxis JSX en todos nuestros ficheros, ya que, a la hora de desplegar la aplicación, será el transpilador quien se encargue de convertir esa sintaxis a JavaScript (en sus diferentes especificaciones: ES5, ES6, etc.). Aunque también podría existir un fichero JavaScript en el proyecto que será el punto de entrada (o *entry point*), como puede ser el “index.js”.

La flexibilidad que otorga React.js en combinación con la extensión JSX para el desarrollo de aplicaciones web basadas en componentes es inmensa, logrando hacer cualquier cosa de muchas formas posibles. Por tanto, en nuestra aplicación usaremos la sintaxis JSX, nombrando los archivos derivados de los componentes con extensión “*.jsx”, y el motor de React.js se encargará de su optimización durante el renderizado. No obstante, nuestros componentes podrían tener extensión “*.js” sin problema.

4.1.5 Uso de componentes. Tipos y fases del ciclo de vida

Como ya hemos mencionado en secciones anteriores, uno de los conceptos más interesantes de React.js es que se basa en componentes. Así, un componente es un elemento software visual que tiene su propio estado, recibe unas propiedades e implementa su propia lógica de renderizado.

¡Las aplicaciones React.js están formadas por componentes! y ¡las páginas de la aplicación web en el navegador (accesibles mediante rutas) podrán desplegar uno o multitud de componentes!

En esta sección, comentaremos algunas generalidades de los tipos de componentes existentes, así como el ciclo de vida general de un componente con estado. Si deseas profundizar más acerca de la teoría sobre los tipos de componentes y el flujo del ciclo de vida de los componentes con estado, echa un vistazo a las lecturas recomendadas propuestas al final de esta parte del libro.

En React.js, los tipos de componentes los podemos agrupar en dos categorías: atendiendo a su comportamiento, y a su estructura.

4.1.5.1 COMPONENTES DE COMPORTAMIENTO

En función del tipo de comportamiento que vaya a realizar el componente, tendremos componentes *Stateful*, *Stateless*, *Pure Components* y/o *HOC*.

Componente con estado (*Stateful*)

Son el tipo de componentes más utilizados, también denominados componentes con estado. Su principal característica es que utilizan la encapsulación en clases, tienen un estado que definen y actualizan, y cada cambio tanto en las propiedades **props** como en el estado **state** hace que se llame al método **render**.

A continuación, se presenta un ejemplo de este tipo de componentes.

```
.....  
class MiBoton extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state.style = { background: 'blue', color: 'white' };  
  }  
  render () {  
    return (  
      <button {...this.props} style={this.state.style} />  
    );  
  }  
}
```

```
.....
```

Componente sin estado (*Stateless*)

Son una versión simplificada del tipo de componente anterior. Estos componentes se definen como si fueran funciones, y no tienen ni trabajan con estados (son componentes sin estado). Los únicos datos con los que trabajan son las propiedades **props** recibidas, además no sobrescriben los métodos de su ciclo de vida.

La ventaja de este tipo de componentes es que son sencillos de escribir, fácilmente testeables y suelen mejorar el rendimiento. A continuación, se muestra un ejemplo de un componente *stateless*.

```
const MiBoton = props => {  
  const styles = { background: 'blue', color: 'white' }  
  
  return (  
    <button  
      {...props}  
      style={styles}  
    />  
  )  
}
```

Componente puro (*Pure Component*)

Este componente es similar a los *stateful* en cuanto a su definición. También se implementan como clases, pero en este caso heredan de `React.PureComponent`. Al igual que los componentes *stateless*, estos no definen un estado, siendo puramente solo un componente visual. Esto hace que se consiga un mayor rendimiento en el renderizado, ya que solo cambian si detectan un cambio en sus **props** y sus valores son distintos a los valores que tenían anteriormente.

La implementación, como se muestra en el ejemplo a continuación, es muy parecida a un componente con estado.

```
class MiBoton extends React.PureComponent {  
  
  const styles = { background: 'blue', color: 'white' }  
  
  render () {
```

```
    return (  
      <button {...this.props} style={styles} />  
    );  
  }  
}
```

Componente de orden superior (HOC)

Este tipo de componentes constituyen un patrón de diseño utilizado en aplicaciones React.js. Suelen ser funciones que toman como parámetro otro componente, extendiendo su funcionalidad y devolviendo un nuevo componente con funcionalidad extendida. Si las **props** del HOC cambian, este se renderizará de nuevo y actualizará el componente envuelto en él, por lo que también se denominan *wrappers*.

Este tipo de componentes se usan para implementar funcionalidades comunes en aplicaciones webs como pueden ser la paginación, interceptar y modificar la renderización, hacer llamadas a APIs y alimentar el componente envuelto, controlar las entradas en campos de formularios, etc.

Estos componentes son usados en otros paquetes de React.js como **react-redux** y **react-redux-form**. Básicamente es un buen método para desacoplar funcionalidad, extender la funcionalidad de nuestros componentes y reutilizar el código en nuestra aplicación.

Puedes conocer más acerca de este tipo de componentes en la web de React.js⁵⁵ así como en otras fuentes de interés conforme a las lecturas recomendadas.

4.1.5.2 COMPONENTES ESTRUCTURALES

Estos componentes no se corresponden técnicamente con ningún elemento de la API, clase o función en React.js, son simplemente puramente conceptuales.

El propósito por tanto de esta categoría es organizar nuestra aplicación para que sea más sencilla e intuitiva a la hora de su desarrollo. No se trata de un estándar React.js, sino más bien de la propia comunidad y permite definir una arquitectura en nuestras aplicaciones. Así, tenemos dos tipos de componentes estructurales.

55 <https://reactjs.org/docs/higher-order-components.html> [Último acceso: Julio, 2020]

Componentes visuales

Este tipo de componentes únicamente centran y enfocan sus esfuerzos en cómo debe renderizarse la interfaz de usuario (UI). Además, pueden componerse de otros elementos visuales y suelen incluir estilos y clases. Por último, todos los datos implicados en su renderización se deben recibir a través de propiedades **props**, por lo que deben ser independientes a llamadas a servicios externos y suelen ser componentes *Stateless* ya que no necesitan estado.

A continuación, se muestra un ejemplo sencillo al respecto.

```
.....  
class Item extends React.Component {  
  render () {  
    return (  
      <li><a href='#'>{ this.props.valor }</a></li>  
    );  
  }  
}
```

```
.....
```

Componentes contenedores

A diferencia de los visuales, estos componentes dejan a un lado la interfaz para encargarse de la parte funcional. Son contenedores de otros componentes encargándose de gestionar la lógica de interacción y la lógica de los datos, mediante las llamadas necesarias a servicios externos.

También como diferencia de los componentes anteriores, estos suelen gestionar su propio estado, siendo un nodo importante en la jerarquía del árbol de componentes.

A continuación, se muestra un ejemplo que utiliza un bucle para recorrer un *array* de ítems del tipo `<Item/>`, anteriormente implementado.

```
.....  
class ItemsContainer extends React.Component {  
  constructor (props) {  
    super(props);  
    this.state = {  
      temas: ['Vue', 'React', 'Angular']  
    };  
  }  
}
```



```
render () {  
  const items = this.state.temas.map(t => (  
    <Item valor={ t } />  
  ));  
  return (  
    <div>  
      <ul>{ items }</ul>  
    </div>  
  );  
}
```

Los componentes contenedores pueden representar cada una de las páginas, y en este caso, coincidir con las rutas definidas. Igualmente, puedes profundizar más en estos componentes si así lo deseas, aunque no es nuestro objetivo principal.

NOTA

La aparición de los React *hooks* simplifica en gran medida esta categorización.

4.1.5.3 FASES DEL CICLO DE VIDA DE UN COMPONENTE

Podemos definir el ciclo de vida de un componente como una serie de estados o fases por las que pasa este a lo largo de su existencia. Esos estados tienen correspondencia en diversos métodos (funciones), que podemos implementar para realizar diferentes acciones cuando se pasa por los mismos.

En React.js es fundamental el ciclo de vida, ya que hay determinadas acciones que debemos realizar necesariamente en el momento correcto de ese ciclo. Algunos de estos estados son fáciles de comprender, pero otros no son una tarea trivial.

Las fases o estados por los que pasa un componente, es algo específico de componentes con estado (*Stateful*), ya que los componentes sin estado tienen apenas un método que se usará para renderizar el componente y React.js no controlará su ciclo de vida.

La Figura 50 nos muestra el flujo del ciclo de vida de un componente React.js, junto con sus posibles estados.

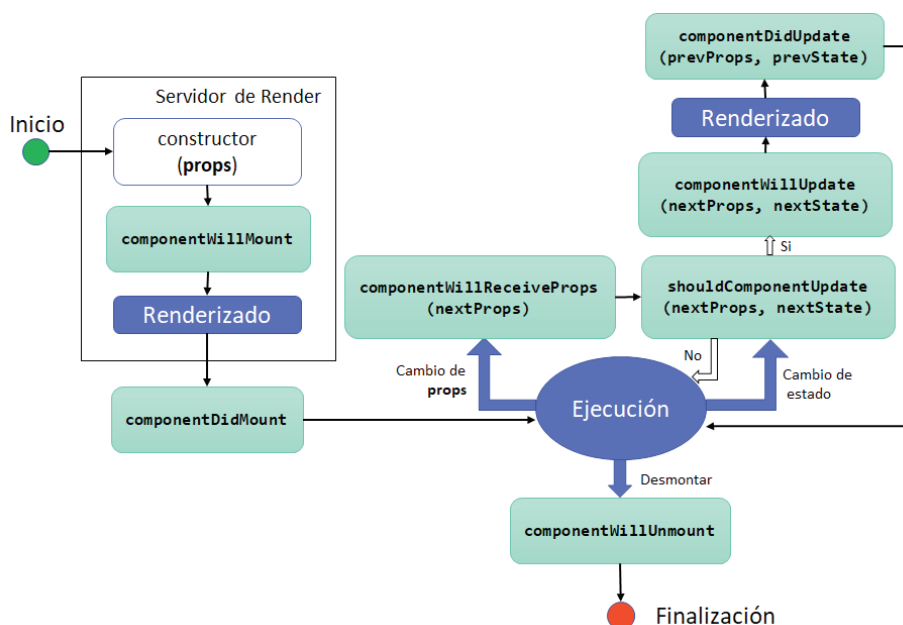


Figura 50. Diagrama de flujo. Fases del ciclo de vida de un componente React.js

NOTA

Nuevamente, las últimas versiones de React.js y la refactorización de las clases como *React hooks* simplifica las fases del ciclo de vida, y las sustituye por los denominados *hooks* de efecto. Sin embargo, estudiar y comprender todos los estados por los que puede pasar un componente, supondrá una base de conocimiento que nos ayudará a entender mejor la implementación de estos componentes bajo la filosofía de los *React hooks*.

Las funciones o métodos usados para controlar el ciclo de vida de un componente corresponden a métodos de: montaje, actualización y desmontaje (ver bloques de la Figura 50). En cuanto a esas etapas, podríamos decir que:

- El montaje se produce la primera vez que un componente va a generarse, incluyéndose en el DOM.
- La actualización se produce cuando el componente ya generado se está actualizando.
- El desmontaje ocurre cuando el componente se elimina del DOM.

Además, dependiendo del estado actual de un componente y lo que está ocurriendo en él, se producirán grupos diferentes de etapas del ciclo de vida. En la Figura 51 se puede ver de forma general lo que ocurre la primera vez que un componente es renderizado, cuando se produce un cambio en sus propiedades o en su estado, o cuando se desmonta. La secuencia de ejecución de las etapas del ciclo de vida se da de arriba hacia abajo.

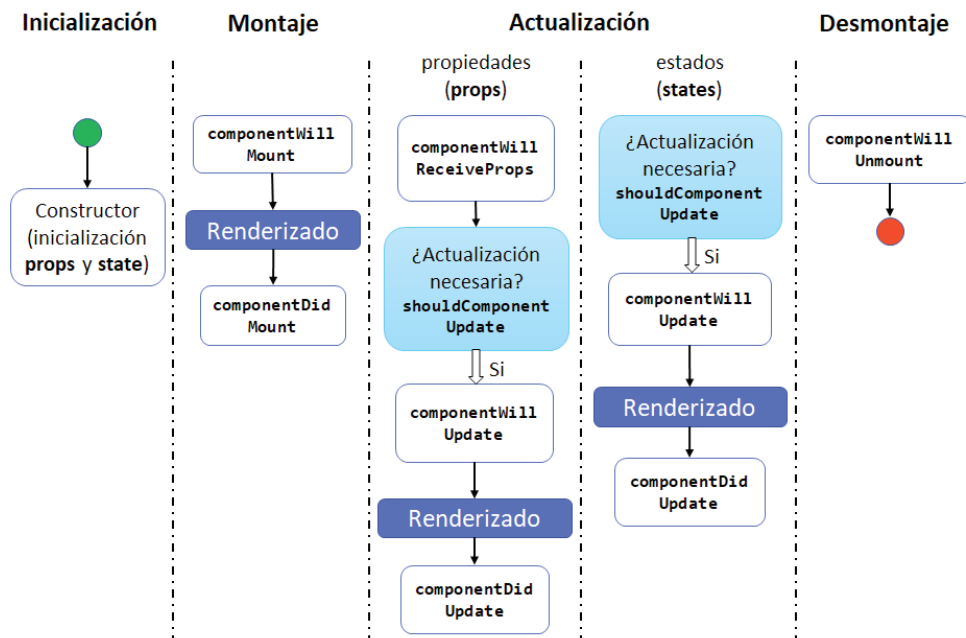


Figura 51. Etapas del ciclo de vida de un componente React.js

A continuación, describiremos de forma breve cada una de estas funciones, que permitirán ejecutar código en diferentes instantes (dependiendo de las situaciones), tal y como suelen hacer los manejadores de eventos en cualquier lenguaje de programación.

componentWillMount()

Este método, que se da en el montaje del componente, se ejecuta justo antes de su primer renderizado. Si dentro de este método se fija el estado del componente con **setState()**, el primer renderizado mostrará ya el dato actualizado y sólo se renderizará una vez el componente.

componentDidMount()

Es un método de montaje que solo se ejecuta en el lado del cliente. Se produce inmediatamente después del primer renderizado. Una vez invocado este método, ya están disponibles los elementos asociados al componente en el DOM. Si el elemento tuviera otros componentes hijos, se tiene certeza que estos se han inicializado también y han pasado por los correspondientes `componentDidMount()`.

componentWillReceiveProps(nextProps)

Método de actualización que se invoca cuando las propiedades se van a actualizar, aunque no en el primer renderizado del componente, así que no se invocará antes de inicializar las propiedades por primera vez.

Tiene como particularidad que recibe el valor futuro del objeto de propiedades que se va a tener. El valor anterior es el que está todavía en el componente, pues este método se invocará antes de que esos cambios se hayan producido.

shouldComponentUpdate(nextProps, nextState)

Es un método de actualización y tiene la particularidad de que debe devolver un valor booleano. Si devuelve *true* quiere decir que el componente se debe renderizar de nuevo y si se recibe *false* quiere decir que el componente no se debe renderizar de nuevo.

Se invocará cuando se produzcan cambios en propiedades o estados, y es una oportunidad de comprobar si esos cambios debieran producir una actualización en la vista del componente. Por defecto (si no se definen) devuelve siempre *true*, para que los componentes se actualicen ante cualquier cambio de propiedades o estado.

El motivo de su existencia es la optimización, puesto que el proceso de renderizado tiene un coste computacional y podemos evitarlo si realmente no es necesario realizarlo. Recibe dos parámetros con el conjunto de propiedades y estado futuro.

componentWillUpdate(nextProps, nextState)

Este método de actualización se invoca justo antes de que el componente vaya a actualizar su vista. Es indicado para tareas de preparación de esa inminente renderización causada por una actualización de propiedades o estado.

componentDidUpdate(prevProps, prevState)

Supone un método de actualización que se ejecuta justo después de haberse producido la actualización del componente. En este caso, los cambios ya están trasladados al DOM del navegador, así que se podría operar con el DOM para hacer nuevos cambios. Como parámetros se reciben el valor anterior de las propiedades y el estado.

componentWillUnmount()

Es el único método de desmontado y se ejecuta en el momento que el componente se va a retirar del DOM.

Es un método importante, porque es el momento en el que se debe realizar una limpieza de cualquier cosa que tuviese el componente y que no deba seguir existiendo cuando se retire de la página. Por ejemplo, temporizadores o manejadores de eventos que se hayan generado sobre partes del navegador que no dependen de este componente.

Con esto finalizamos la parte teórica sobre los componentes, su ciclo de vida y los métodos asociados. Se recomienda profundizar sobre ello y revisar las lecturas recomendadas al respecto.

4.1.6 Propiedades y estados

A lo largo de las secciones anteriores ya habían aparecido las palabras propiedades (**props**) y estados (**state**) de un compoene, y es que no se puede entender React.js sin explicar lo que significan.

Debemos saber que, tanto las **props** como el **state** son objetos JavaScript simples y los cambios en ambos desencadenan la ejecución de la función **render()** del componente.

Entonces, ¿Cuándo utilizamos **props** y cuando **state**?

La respuesta es sencilla. Si un componente va a necesitar modificar alguno de sus atributos en algún momento de su vida, entonces ese atributo debe formar parte de su estado, de lo contrario será una propiedad de dicho componente.

Cada componente puede tener uno o varios estados además de sus propiedades. Estos estados son intrínsecos al componente y no dependen de

componentes superiores o padre (si no, los pasaríamos por propiedades). Hay que tener en cuenta estas tres consideraciones:

- Es mejor tener componentes sin estado (más reusables)
- Los estados se definen en el constructor de clase mediante `this.state`
- Si queremos modificar un estado, se debe usar el método `this.setState` (no se puede modificar directamente)

A continuación, detallaremos con mayor nivel de detalle lo que suponen tanto las propiedades como los estados de un componente React.js.

4.1.6.1 PROPIEDADES

Pueden definirse como los atributos de configuración para un componente. Son recibidos desde un nivel superior (normalmente al instanciar el componente) y por definición inmutables, es decir, un componente no puede cambiar sus propias propiedades o `props`.

A continuación, se presenta un ejemplo de uso en un componente al que se le pasan las propiedades cuando se va a renderizar el DOM.

```
import React from 'react';
import ReactDOM from 'react-dom';

class MyComponent extends React.Component{
  render(){
    return <div>Me llamo {this.props.name} y tengo {this.props.anios}</div>
  }
}

export default MyComponent;
```

De esta forma, al renderizar el componente con `ReactDOM.render()` se pasan los valores de las propiedades `name` y `anios`. Si lo hacemos desde el fichero “app.js”, sería:

```
import React from 'react';
import ReactDOM from 'react-dom';
import MyComponent from "../components/ejemplo";

const app = document.getElementById('app');

ReactDOM.render(<MyComponent name = "Jesus" anios = "35" />, app);
```

También podemos definir en el componente unos valores por defecto para las propiedades utilizando de forma conveniente la variable `defaultProps` una vez creado el componente, de esta forma:

```
.....
class MyComponent extends React.Component{
  render(){
    return (<div>Me llamo {this.props.name} y tengo {this.props.anios}</div>)
  }
}

MyComponent.defaultProps = {
  name: 'Jesus',
  anios: '35'
};

ReactDOM.render(<MyComponent />, document.getElementById('app'));
```

```
.....
```

Así, si renderizamos el componente sin dar valores a las `props`, este se creará con los valores por defecto que se hayan indicado.

Por otra parte, los componentes React.js están diseñados para poder agruparlos en componentes más grandes y ser reutilizados, ya sea en el mismo proyecto, en otro proyecto, o por otros desarrolladores. De modo que, es una buena práctica definir explícitamente las propiedades que acepta un componente, cuáles serán requeridas, y los tipos de dato de cada una de ellas.

Este es el propósito de las `propTypes` de un componente, que nos ayudará a revisar rápidamente cuáles son las propiedades que debemos pasar a dicho componente (por ejemplo, cuando hay un error, React.js mostrará un mensaje en la consola indicando qué propiedades faltan y qué método generó el problema). Esto supone una herramienta útil tanto a nivel de depuración como para validar valores de propiedades a nivel de *front-end* del cliente.

El objeto `propTypes` se define como una propiedad en el constructor de la clase. En el siguiente ejemplo se puede apreciar un componente *Titulo* que recibe una propiedad `nombre` que se muestra dentro de un encabezado.

```
.....
class Titulo extends React.Component {
  render () {
    return (
      <h1>{ this.props.nombre }</h1>
    );
  }
}
```

```
    }  
  }  
  
  Titulo.propTypes = {  
    nombre: React.PropTypes.string.isRequired  
  };  
  
  ReactDOM.render(<Titulo nombre="Titulo " />, document.getElementById('root'));
```

Para especificar que esa propiedad es de tipo “string” indicamos el tipo `PropTypes.string.isRequired`. La parte `isRequired` (opcional) sirve para especificar que una propiedad es obligatoria.

Esto nos da una idea de cómo la implementación de validaciones de determinados valores de variables en React.js puede ser una tarea bastante sencilla.

Una vez explicado, de forma general, lo que son las propiedades y algunas características importantes al respecto, pasaremos a comentar la parte de los estados de un componente React.js.

4.1.6.2 ESTADOS

Un estado se podría definir como una representación del componente en un instante determinado. El estado o `state` de un componente se iniciará con un valor por defecto, pero a diferencia de las propiedades ese valor sí podrá cambiar durante la vida del componente.

Al contrario que ocurre con las propiedades, un componente puede gestionar su propio `state`. Debemos ser cuidadosos al utilizar estados ya que estos podrían aumentar la complejidad y reducir la previsibilidad en el código.

La forma usual de informar a un componente React.js que sus datos han cambiado, es a través del método `setState()` (ver sentencia de código más abajo). Este método recibe un objeto JavaScript plano, combina los nuevos datos en el `state` y renderiza el componente. Cuando el renderizado finaliza, se ejecuta un `callback`⁵⁶ si fue especificado.

```
this.setState( { key: 'value' } );
```

56 https://developer.mozilla.org/es/docs/Glossary/Callback_function [Último acceso: Julio, 2020]

El estado del componente se almacena en la propiedad **state** y está formado por aquella información que sí es gestionada y modificada directamente por el componente. Cada vez que se realiza un cambio en el estado del componente como respuesta a una acción del usuario (o debido a algún otro factor externo), el *framework* se encargará de renderizar nuevamente el componente por completo.

Cabe destacar que, como un componente puede contener a otros componentes, en el proceso de renderizado se volverán a crear estos componentes con la nueva información. Es posible, y bastante habitual, que el estado cambiante de un componente sean propiedades inmutables de sus componentes hijos.

A continuación, se muestra un ejemplo que inicializa el estado de un componente con unos valores determinados. Si antes esos valores se definían en las **props**, ahora vamos a ver qué pasa si forman parte del **state**.

```
.....  
class MyComponent extends React.Component{  
  constructor(props){  
    super(props);  
    this.state = {name:'Jesus',  
                  edad: '35'};  
  }  
  render(){  
    return <div>Me llamo {this.state.name} y tengo {this.state.edad}</div>;  
  }  
};  
  
export default MyComponent;  
.....
```

Los valores de los estados pueden ser modificados a través de eventos disparados desde los propios componentes (por ejemplo, de un componente hijo hacia uno padre). En el caso de las propiedades, estas suelen pasar de padres a hijos (se heredan).

De esta forma, si nosotros quisiéramos cambiar el valor de **name**, del estado del componente, lo podríamos hacer a través de un evento, cuyo método asociado lleve consigo el **setState** sobre la propia variable **name**, tal y como se muestra en el bloque de código siguiente.

```
.....  
render() {  
  return (  
    <MyComponent  
      name={this.state.name}  
    >  
.....
```

```
      onChangeName={this.handleChangeName.bind(this)}  
    />  
  )  
}
```

Ahora, el componente `<MyComponent />` tiene una nueva propiedad llamada `onChangeName`. Esta propiedad va a llamar a la función `handleChangeName` que estará implementada en el componente padre, y modificará su estado, tal y como se presenta más abajo.

```
handleChangeName () {  
  this.setState({  
    name: this.state.name  
  })  
}
```

Este será el mecanismo que utilizaremos en nuestra aplicación *front-end*, y que permitirá volver a renderizar los componentes ante cambios en determinadas variables que forman parte de su estado.

Si deseas conocer más acerca de los estados y las propiedades de los componentes en React.js, te aconsejo que eches un vistazo a las lecturas recomendadas.

NOTA

La forma de manejar las propiedades y los estados a partir de la versión de React.js 16.8 también ha experimentado algunos cambios. En este caso se han introducido los *hooks* de estado, así como otras particularidades. Para más información, nuevamente te emplazamos a la sección del libro **Introducción a los *hooks* en React.js**.

4.1.7 Enrutado con React Router

Una aplicación React.js, como toda aplicación web, se basa en rutas que determinan qué páginas y componentes se deben mostrar al usuario dependiendo donde este se encuentre dentro de la aplicación.

Recuerda que en nuestra API REST, el *framework* Express.js se encarga de manejar las rutas a los recursos del *back-end* que el cliente puede solicitar haciendo

uso de peticiones HTTP. Mientras que, en este caso, las rutas estarán asociadas a componentes y páginas que nuestra aplicación web cliente podrá renderizar.

La librería usada para manejar las rutas en una aplicación React.js se llama **react-router**, que cuenta con dos *add-ons*, uno más enfocado a web (**react-router-dom**) y otro para desarrollo móvil (**react-router-native**).

La última versión de dicha librería corresponde a *React Router v4*, que difiere en algunas cosas con respecto a versiones anteriores, haciendo distinción entre el enrutamiento estático que se usaba en anteriores versiones y el enrutamiento dinámico actual.

Para instalar la librería correspondiente en nuestro proyecto (si aún no la hemos instalado), ejecutaremos el siguiente comando:

```
.....  
C:\microblogging-example-react>npm install react-router-dom --save  
.....
```

Recuerda que, en secciones posteriores del libro implementaremos una aplicación cliente completa y abordaremos estos conceptos de forma práctica, pero mientras, podemos trabajar en el directorio del proyecto inicialmente creado para tal fin.

4.1.7.1 TIPOS DE ENRUTAMIENTO Y COMPONENTES DEL ROUTER

Como se ha mencionado anteriormente, una de las mejoras de la última versión de **react-router** radica en la distinción y uso del enrutamiento dinámico con respecto al estático.

El enrutamiento estático es el enrutamiento más común. En este caso, las rutas son definidas al momento en que nuestra aplicación es inicializada. Es decir, antes de que nuestra aplicación se renderice.

El enrutamiento dinámico es el usado por **react-router**. A diferencia del estático, este se da en el momento en que la aplicación se está renderizando, debido a que también hace uso de componentes para definir sus rutas.

Dado que **react-router** es una librería implementada en React.js, está basada en componentes. A continuación, vamos a describir los componentes del Router más utilizados en las aplicaciones web. Ten en cuenta que, los componentes encargados de mostrar las diferentes rutas siempre renderizan, o bien un componente o un *null*, dependiendo de la localización.

BrowserRouter

Podríamos decir que es una envoltura de nuestra aplicación, la cual nos da acceso al *History API*⁵⁷ de HTML5 para mantener nuestra interfaz gráfica en sincronía con la localización o URL actual. Esta envoltura solo puede tener un hijo, que por lo general es el componente *Switch*.

Switch

Este componente causa que solo se renderice el primer hijo *Route* o *Redirect* que coincida con la localización o URL actual. En el caso de no usar *Switch*, todas las rutas que cumplan con la condición se renderizarán.

Route

Este componente se usa para definir las diferentes rutas de nuestra aplicación. La función de *Route* es elegir qué renderizar según la localización actual. Al igual que se expuso anteriormente, todos los componentes *Route* siempre renderizan, pero algunas veces renderizan un componente y otras veces retornan *null*.

Teniendo en cuenta el siguiente ejemplo:

```
import React, { Component } from 'react';
import { BrowserRouter, Switch, Route } from 'react-router-dom';

export default class App extends Component {
  render() {
    return (
      <BrowserRouter>
        <Switch>
          <Route path="/" exact />
          <Route path="/index" exact />
          <Route path="/post/:id" />
        </Switch>
      </BrowserRouter>
    );
  }
}
```

57 https://developer.mozilla.org/es/docs/DOM/Manipulando_el_historial_del_navegador [Último acceso: Julio, 2020]

En caso de que la URL actual sea similar a *www.midominio.com/index*, se mostrará la segunda ruta, aunque en este caso no tiene asignada ningún componente a renderizar cuando esto ocurra.

El componente *Route* puede recibir las siguientes propiedades:

- **path**: define la ruta en la que se renderizará el componente.
- **exact**: un booleano que define si queremos que la ruta sea o no exacta para renderizar un componente.
 - Ejemplo: `"/index i== /index/all"`
- **strict**: un booleano que define si queremos que la última barra `"/` se tenga en cuenta para renderizar un componente.
 - Ejemplo: `/index i== /index/`
- **sensitive**: booleano para distinguir entre mayúsculas y minúsculas, y tener eso en cuenta a la hora de renderizar un componente.
 - Ejemplo: `/index i== /Index`
- **component**: recibe un componente a renderizar. Crea un nuevo elemento `React.js` cada vez, y causa que el componente se monte y desmonte cada vez (no actualiza).
- **render**: recibe un método que retorna un componente. A diferencia de **component** no remonta el componente.
- **children**: es similar a **render**, pero con algunos añadidos.

Los tres métodos para renderizar el componente (propiedades de *Route*), nos permiten acceder a tres **props** adicionales. Estos van a contener los datos (**data**) del *History API* de HTML5 y son: *match*, *location*, *history*.

En el caso de querer acceder a estas propiedades desde un componente que no fue renderizado, de ninguna de estas maneras, **react-router** nos permite el uso de un componente HOC para darnos este acceso. Este componente se llama *withRouter*.

A continuación, se muestra un ejemplo de componentes del **react-router** y las propiedades que pueden incluir:

```

import React, { Component } from 'react';
import { BrowserRouter, Route, Switch, Redirect } from 'react-router-dom';
import NavBar from './NavBar/NavBar';
import Home from './Home/Home';
import Page1 from './Page1/Page1';
import Page2 from './Page2/Page2';
import PageError from './PageError/PageError';
import './App.css';

class App extends Component {
  render() {
    return (
      <BrowserRouter>
        <div>
          <NavBar />
          <Redirect from="/" to="/home" />
          <Switch>
            <Route path="/home" component={Home} />
            <Route
              exact
              path="/page1"
              render={() => <Page1 name="React Example" /> } />
            <Route
              exact
              path="/page2"
              render={() => <Page2 /> } />
            <Route component={PageError} />
          </Switch>
        </div>
      </BrowserRouter>
    );
  }
}
export default App;

```

En dicho ejemplo se renderizan los componentes de la siguiente forma:

- Si la ruta es “/home”, renderizará el componente *Home*.
- Si la ruta es exactamente “/page1”, renderizará el componente *Page1*.
- Si la ruta es exactamente “/page2”, renderizará el componente *Page2*.
- Y si no es ninguna de las anteriores, se renderizará el componente *PageError*, para mostrar, por ejemplo, que hemos ingresado a una ruta no válida.

Redirect

Este componente, el cual aparece en el ejemplo anterior, causa un redireccionamiento a una ruta diferente a la actual. La ruta a la que nos redirecciona reemplaza la localización actual en la historia del navegador.

Redirect a su vez puede recibir las siguientes propiedades:

- **from**: localización desde la cual se va a hacer el redireccionamiento. Esta propiedad funciona de forma similar a **path** del componente *Route*, y cuando la localización coincide con **from**, se ejecuta el *Redirect*.
- **to**: localización a la cual se realizará el redireccionamiento.
- **push**: define un booleano que permite modificar el comportamiento de *Redirect*. Cuando es verdadero, en lugar de reemplazar la localización actual en la historia, este agrega una nueva localización.
- Al igual que en *Route*, también podemos usar las propiedades **strict** y **exact**.

Dicho esto, en el caso del ejemplo anterior, si navegamos al **index** de la aplicación (“/”), el componente *Redirect* cambiará esa localización por “/home”. Y además renderizará el componente *Route* que coincida con esta ruta, en este caso *Home*.

Link

Para navegar entre las rutas configuradas, lo que puede suponer cambiar de un componente a otro, habría que cambiar la URL “de forma manual”.

Con el componente *Link* crea un hipervínculo que nos permite navegar por nuestra aplicación. Al contrario que *Redirect*, este componente agrega una nueva localización a la historia. *Link* puede recibir las siguientes propiedades:

- **to**: será la localización a la que navegaremos cuando hacemos *click* sobre un hipervínculo.
- **replace**: booleano que nos permite modificar el comportamiento de *Link*. Cuando es verdadero en lugar de agregar una nueva localización a la historia, este reemplaza la localización actual.

A continuación, se muestra un ejemplo de componente que utiliza *Link*. La propiedad `className` indica que se usará el estilo CSS denominado “link” (en el componente *Link*) a la hora de mostrar dicho enlace.

```
import React, { Component } from 'react';
import { Link } from 'react-router-dom';
import './NavBar.css';

class NavBar extends Component {
  render() {
    return (
      <div className="NavBar">
        <div className="link-container">
          <Link to="/page1" className="link">Página 1</Link>
        </div>
        <div className="link-container">
          <Link to="/page2" className="link">Página 2</Link>
        </div>
      </div>
    );
  }
}

export default NavBar;
```

4.2 DEFINIENDO EL ESTILO DE LA INTERFAZ CON REACTSTRAP

Dar estilo a nuestra interfaz, a través de un diseño atractivo y *responsive*, puede llegar a ser sumamente importante en el éxito de nuestra aplicación, facilitando su uso por el usuario, desde cualquier navegador y dispositivo.

En la actualidad, existen *frameworks* como Bootstrap⁵⁸ cuyo objetivo es facilitar el diseño de cualquier aplicación y sitio web. Así, permite crear de forma sencilla webs de diseño adaptable, que se ajustan a cualquier dispositivo y tamaño de pantalla, siendo igualmente usables y atractivas al usuario, mejorando su experiencia. Bootstrap fue creado y liberado por *Twitter*, siendo uno de los *frameworks* de código abierto más utilizados. Actualmente la última versión es *Bootstrap 4* y se trata de un *framework* completo que combina CSS, HTML y JavaScript.

58 <https://getbootstrap.com/> [Último acceso: Julio, 2020]

Algunas ventajas de Bootstrap son las siguientes:

- Permite simplificar el proceso de maquetación, sirviendo de guía para aplicar buenas prácticas y estándares.
- La curva de aprendizaje es sencilla y rápida.
- Permite el uso de multitud de elementos, desde iconos a desplegables (combinando, como ya se ha mencionado, CSS, HTML5 y JavaScript).
- Facilita el diseño *responsive* de cara al dispositivo, escala y resolución de pantalla.
- Se maneja el concepto de *grid*, dividiendo la página en columnas facilitando el trabajo por zonas.
- Se integra fácilmente con numerosas librerías JavaScript.
- Posee una gran comunidad de desarrollo con constantes actualizaciones y mejoras del *framework*.

Este *framework* puede complementarse con otras librerías y lenguajes de diseño de interfaces web, así como disponer de implementaciones del mismo para ser usadas en diversas tecnologías del *front-end*, como puede ser React.js.

En este punto, existen numerosas opciones para instalar *Bootstrap* en una aplicación React.js, e incluso hay librerías que, implementándose por medio de componentes React.js, ofrecen todos los elementos de *Bootstrap* de una forma sencilla para ser usados dentro de nuestra aplicación. Una de estas librerías es *Reactstrap*⁵⁹.

Reactstrap permite el uso de los elementos derivados de *Bootstrap 4* como componentes de React.js. *Reactstrap* no incluye *Bootstrap* CSS, con lo que debemos instalarlo también. Ambos (y sus dependencias) pueden ser instalados en el proyecto vía *npm* con el siguiente comando:

```
.....  
C:\microblogging-example-react>npm install bootstrap reactstrap react react-dom  
--save  
.....
```

Recuerda que instalaremos estos paquetes y sus dependencias tras crear la estructura de directorios y ficheros de nuestro proyecto con la herramienta de *scaffolding* **create-react-app**, tal y como se vio en la sección **Hola Mundo en React.js** y volveremos a comentar en **Estructura de la aplicación React.js**.

Tras la instalación, debemos importar Bootstrap CSS en el fichero “/src/index.js” de la aplicación de esta forma:

59 <https://reactstrap.github.io/> [Último acceso: Julio, 2020]

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Una vez hemos instalado e importado la librería, podemos hacer uso de sus componentes (*Layout, Alert, Button, Card, Badge, Form, Navbar, Table, etc.*), importándolos directamente en el código de nuestra aplicación, en las páginas correspondientes, tal y como se indica en el siguiente ejemplo, en este caso para el uso de un componente *Button*:

```
import React from 'react';
import { Button } from 'reactstrap';

export default (props) => {
  return (
    <Button color="danger">Danger!</Button>
  );
};
```

También podemos incluir *Reactstrap* directamente en nuestra aplicación de forma global a través de los CDNs (*Content Delivery Network*) incluidos en su web de referencia.

Para ver la apariencia de cada uno de los componentes implementados, así como el código asociado podemos dirigirnos a la página web de *Reactstrap*. Cada componente utilizado tiene una serie de propiedades que podemos cambiar o configurar, conforme a la especificación de la librería. La Figura 52 muestra una captura de pantalla de la sección de componentes de tipo *Button* en la web de *Reactstrap*.

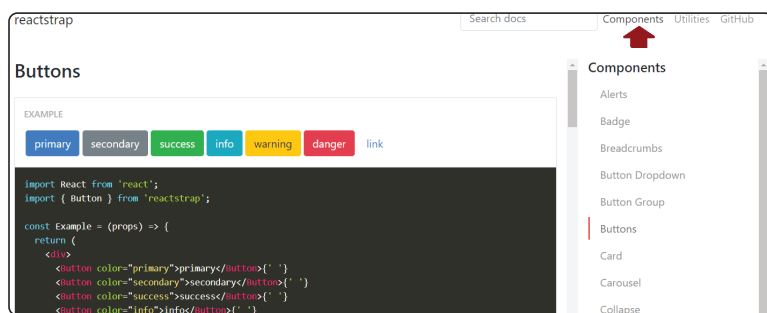


Figura 52. Ejemplo y propiedades de componentes Button en Reactstrap⁶⁰

De esta forma, nuestra aplicación React.js tendrá un aspecto profesional, atractivo y *responsive* acorde al estilo *Bootstrap*.

60 <https://reactstrap.github.io/components/buttons/> [Último acceso: Julio, 2020]

4.3 ESTRUCTURA DE LA APLICACIÓN REACT.JS

A partir de esta sección del libro, comenzaremos el desarrollo de la parte *front-end* de nuestro proyecto. En este caso, se ha optado por desarrollar la aplicación React.js de forma independiente al API REST que constituye nuestro *back-end*, el cual ya habíamos desarrollado y desplegado en Heroku.

Para crear la aplicación React.js, lo haremos con la herramienta **create-react-app** en el caso de que queramos crearla de nuevo. De otro modo, podremos trabajar sobre el proyecto creado con anterioridad, eliminando los componentes de ejemplo que fueron implementados.

Con el proyecto React.js en el directorio correspondiente, podemos abrir el editor Atom y añadirlo para trabajar con él, igual que hicimos con el API REST ubicado en “C:\microblogging-example-api”.

Además, también podemos crear un repositorio en Github para alojar la aplicación React.js. Para ello, basta con seguir nuevamente los pasos definidos en Control de versiones con Git y sincronizar nuestro proyecto React.js con Github.

Conforme a los servicios REST desarrollados en el *back-end*, nuestra aplicación *front-end* consistirá en un cliente de microblogging con las siguientes funcionalidades:

- Registrar nuevos usuarios y permitir el acceso (mediante *login*) a un panel personalizado en el caso de los usuarios registrados.
- Mostrar todos los posts publicados hasta la fecha por todos los usuarios registrados.
- Mostrar todos los posts asociados al usuario previamente *logueado* (identificado).
- Permitir la edición y borrado de posts a un usuario identificado.
- Mostrar el perfil del usuario identificado.
- Permitir la desconexión del panel personalizado.

Partiendo de los directorios creados por el generador con **create-react-app**, el contenido de la carpeta “/src” deberá corresponder al de la Figura 53, al final de nuestro desarrollo.

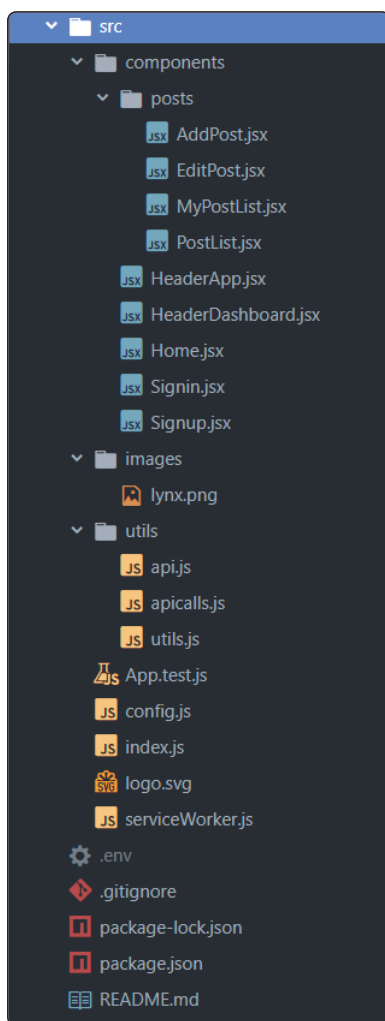


Figura 53. Estructura del directorio “/src” de la aplicación React.js

Es por ello, que a partir de este momento implementaremos paso a paso cada uno de los componentes mostrados, así como los ficheros auxiliares.

Como podemos observar, todos los ficheros asociados a componentes se almacenarán en el directorio “src/components” y tendrán extensión JSX, para aprovechar las ventajas de esta sintaxis del lenguaje JavaScript en React.js (ver **Introducción a la sintaxis JSX**). Gracias al *transpilador* `babel`, los archivos JSX serán convenientemente *transpilados* a archivos JavaScript.

Atendiendo a la figura anterior, nuestros componentes se distribuyen como sigue (acompañados de una breve descripción de la función que desempeñan):

- `components/HeaderApp.jsx`: Componente de cabecera de nuestra aplicación
- `components/Signup.jsx`: Componente de registro de nuevo usuario
- `components/Signin.jsx`: Componente de la página principal con posibilidad de *login* de usuarios
- `components/HeaderDashboard.jsx`: Componente de cabecera del panel personalizado (únicamente visible tras el *login* o identificación del usuario).
- `components/Home.jsx`: Componente principal del panel personalizado (visible tras el *login* o identificación del usuario).

Para el manejo de los posts o entradas, implementaremos diversos componentes que alojaremos en el subdirectorio “/components/posts”:

- `posts/PostList.jsx`: Componente que muestra todos los posts existentes de todos los usuarios.
- `posts/MyPostList.jsx`: Componente que muestra únicamente los posts del usuario identificado (componente visible tras *login* del usuario)
- `posts/AddPost.jsx`: Componente para permitir añadir un nuevo post (visible tras *login* del usuario).
- `posts/EditPost.jsx`: Componente que permite editar un post (visible tras *login* del usuario).

En la carpeta “src/images” alojaremos recursos gráficos como la imagen de cabecera de nuestra aplicación.

Mientras, en “src/utills”, implementaremos los siguientes archivos de utilidades (esta vez serán ficheros “*.js” ya que no implementarán ningún componente React.js):

- `utills/utills.js`: Utilidades varias (por ejemplo: funciones para formatear fechas)
- `utills/api.js`: Definición de la URL, como instancia base, de acceso a los servicios de la API REST, y configuración de la librería **axios** utilizada para la conexión con estos.
- `utills/apicalls.js`: Fichero que agrupa las funciones para la realización de peticiones a nuestra API REST y devolución de resultados al resto de componentes de la aplicación React.js.

4.3.1 Fundamentos y primeros pasos

Una vez conocemos la estructura que tendrá nuestra aplicación, vamos a instalar algunos módulos necesarios y modificar algunos de los archivos base.

Lo primero será instalar las siguientes dependencias (si aún no las tenemos instaladas):

```
.....  
C:\microblogging-example-react>npm install --save react-router react-router-dom  
bootstrap reactstrap  
.....
```

Adicionalmente, vamos a instalar la librería **react-icons**⁶¹, que nos permitirá utilizar la iconografía de diferentes conjuntos o librerías de iconos como la famosa *Fontawesome*⁶². Con **react-icons**, tendremos acceso a cientos de iconos como si fueran componentes React.js para usarlos en nuestros proyectos.

```
.....  
C:\microblogging-example-react>npm install --save react-icons  
.....
```

Debemos asegurarnos que las librerías **react** y **react-dom** son dependencias en el “package.json”, sino deberíamos instalarlas también mediante npm.

En el directorio “/public/index.html” añadimos la librería **babel** (para permitir la *transpilación* del código JSX a JavaScript), y si optamos por no instalar alguna librería como **reactstrap** mediante npm, debemos añadir también el acceso global a dicha librería, tal y como se muestra a continuación (el número de versión de estas librerías podría variar). Con lo que tenemos dos formas de incluir ciertas librerías a nuestros proyectos (siempre que estas lo permitan).

```
.....  
<script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>  
<script src="https://cdnjs.cloudflare.com/ajax/libs/reactstrap/6.0.1/reactstrap.  
min.js"></script>  
.....
```

61 <https://react-icons.github.io/react-icons/> [Último acceso: Julio, 2020]

62 <https://fontawesome.com/> [Último acceso: Julio, 2020]

Para mostrar los elementos iniciales asociados a componentes principales de la aplicación, debemos contenerlos en una etiqueta `<div>` como atributos. En nuestro caso, tendremos los siguientes:

- **header**: Renderizará el componente del fichero “HeaderApp.jsx”
- **root**: Renderizará uno u otro componente en función de la ruta ingresada en el navegador. El manejo de rutas lo realizaremos a través del tipo de enrutamiento manejado por el componente *HashRouter*, que tiene un comportamiento similar a *BrowserRouter* mencionado en la sección Tipos de enrutamiento y componentes del router pero para páginas estáticas).

La renderización de los elementos anteriores se realiza gracias al DOM de React.js. Por tanto, incluiremos estas sentencias en el `<body>` del fichero “index.html”.

```
<div id="header"></div>
<div id="root"></div>
```

Así mismo, en el fichero “index.js” debemos posibilitar la renderización en base al `id` determinado anteriormente en el “index.html”, de esta forma:

```
ReactDOM.render(<HeaderApp />, document.getElementById('header'));
ReactDOM.render(<HashRouter>
  <div>
    <Route exact path="/" component={Signin} />
    <Route path="/signup" component={Signup} />
    <Route path="/home" component={Home} />
  </div>
</HashRouter >, document.getElementById('root'));
```

Por supuesto, debemos importar los componentes de los que haremos uso, y que deberán estar implementados (esto lo haremos poco a poco durante las siguientes secciones). En este momento, añadiremos todas las sentencias `import` necesarias, pero recuerda que dichos componentes importados que no provienen de librerías de `react` deberán existir si no queremos obtener errores durante los posibles despliegues parciales que hagamos de la aplicación:

```
import React from 'react';
import ReactDOM from 'react-dom';
import { HashRouter, Route } from 'react-router-dom';
```

```
import * as serviceWorker from './serviceWorker';

import Signin from './components/Signin';
import Signup from './components/Signup';
import Home from './components/Home';
import HeaderApp from './components/HeaderApp';
```

Además, tenemos que importar los estilos CSS de *Bootstrap* (que usaremos mediante *Reactstrap*), también en el “index.js”:

```
import 'bootstrap/dist/css/bootstrap.css';
```

Ahora crearemos un directorio “src/components”, donde alojaremos todos los componentes de nuestra aplicación si aún no lo hemos hecho. También un “src/utills” para alojar los ficheros relativos a utilidades y toda la lógica de transacciones con nuestro API REST; y finalmente un “src/images” para alojar las imágenes de nuestra aplicación.

Dado que la cabecera de nuestra aplicación se mostrará siempre, vamos a ver cómo se implementaría el componente asociado a esta en primer lugar.

Primero creamos un fichero “HeaderApp.jsx” en el directorio “/components” con el siguiente código:

```
import React, { Component } from 'react';
import { Navbar, NavbarBrand, Nav, NavItem, NavLink, Media, Row, Col, Container }
  from 'reactstrap';

import { AiOutlineCopyright } from 'react-icons/ai';

import MyPlaceholderPicture from '../images/lynx.png';
var imgStyle = {
  maxWidth: "85px",
};

class HeaderApp extends Component {
  render() {
    return (
      <Container>
        <Row>
          <Col>
            <Navbar color="dark" light expand="md">
```



```

        <NavbarBrand><h4 className="text-warning">Mi Aplicación de Mi-
croblogging</h4></NavbarBrand>
        <Nav className="ml-auto" navbar>
          <NavItem>
            <NavLink href="http://jesusfontecha.name/mern" target="_
blank">
              <span className="text-white">Desarrollo web full-
stack con MERN <AiOutlineCopyright /></span><Media style={imgStyle} object
src={MyPlaceholderPicture} alt="Desarrollo web MERN" /></NavLink>
            </NavItem>
          </Nav>
        </Navbar>
      </Col>
    </Row>
  </Container>
);
}
}
export default HeaderApp;

```

El componente anterior hace uso de una imagen de “src/images”, que representará el logotipo de nuestra aplicación. Además, importaremos algunos componentes de la librería *reactstrap* necesarios para construir la vista de la cabecera; y algún icono, también como componente de la librería *react-icons*; ambas instaladas previamente vía npm.

Con la sentencia:

```
export default HeaderApp;
```

haremos el componente *HeaderApp* exportable, para poder ser usado en otros lugares de la aplicación si fuera necesario.

Los componentes *Container*, *Row*, *Col*, *Navbar*, etc. proceden de la librería *reactstrap*. Las etiquetas que embeben a dichos componentes serán elementos del *layout* de nuestra aplicación. En cualquier aplicación web actual, el *layout* tiene una anchura determinada por la resolución y el tamaño de la pantalla donde se ejecuta dicha aplicación. Aunque no es objetivo de este libro entrar en los detalles acerca de la distribución del *layout* para hacer la aplicación *responsive*, es interesante conocer que este se divide en 12 “franjas” de igual tamaño y que cada componente podrá utilizar un tamaño de 1 a 12. Por ello, observaremos como algunos de los

componentes implementados harán uso del atributo correspondiente, ajustándose al tamaño asociado en “franja”.

Finalmente, todo el código (por ejemplo: variables, llamadas a funciones, etc.) que no forme parte de etiquetas JSX y HTML pero que deseamos usar en el interior de estas, debemos embeberlo entre llaves {...}.

Cuando queramos probar y desplegar nuestra aplicación durante el desarrollo, ejecutaremos el siguiente comando desde el terminal:

```
.....  
C:\microblogging-example-react>npm start  
.....
```

Además, con la aplicación en ejecución, si realizamos cualquier cambio en el código y guardamos, se desplegará esta nuevamente, de forma automática.

4.4 CONSUMO DE UNA API REST DESDE REACT.JS CON AXIOS

.....

En esta sección describiremos los fundamentos para la conexión de nuestra API REST ya desplegada en Heroku con la aplicación React.js que estamos desarrollando en esta parte.

Actualmente existen multitud de formas y librerías que nos permiten establecer conexión con una API REST y consumir sus servicios desde el *front-end*. Estas librerías actúan como clientes HTTP con diversas características que las hacen potentes y flexibles en sus funciones. Todas ellas suelen implementar funcionalidades para el manejo de peticiones HTTP con soporte a autenticación, manipulación de datos y transformación en documentos JSON, intercepción y cancelación de solicitudes y respuestas, compatibilidad con la mayoría de navegadores web, etc.

Es importante volver a destacar el agnosticismo existente entre el *back-end* y el *front-end* de una aplicación web. Habitualmente, para consumir servicios de una API REST, desde el punto de vista del cliente REST, solamente es importante conocer el nombre del servicio o función a llamar, los argumentos necesarios para realizar la solicitud y la respuesta en el formato adecuado. Gracias a ello, tanto la API REST como la parte del cliente pueden estar implementadas usando diferentes tecnologías y lenguajes, y accesibles desde distintos dispositivos, con el nexo común del protocolo HTTP y otras consideraciones adicionales. Todo esto suele venir en la especificación del API, o bien debemos conocerlo si se trata de servicios propios implementados y que aún no contienen una especificación formal.

En nuestro caso, utilizaremos la librería **axios**⁶³ para consumir los servicios de la API REST desde React.js.

Axios es una librería JavaScript que, entre otras características, implementa un cliente HTTP ligero similar a otras librerías como Fetch⁶⁴. Está basado en promesas, de forma que podemos aprovechar las ventajas de las funciones asíncronas **async/await** (sobre las que no entraremos en detalle en este libro), y es realmente potente y sencilla de utilizar.

Para utilizar **axios** en nuestro proyecto React.js, debemos instalarlo a través del siguiente comando:

```
.....  
C:\microblogging-example-react>npm install axios --save  
.....
```

A continuación, vamos a ver cómo usar la librería en peticiones GET, POST y DELETE sobre la JSON Placeholder API⁶⁵ (una API REST gratuita con documentos JSON ficticios para la realización de pruebas de consumo de servicios), con la finalidad de asegurar su comprensión antes de implementar dichas peticiones en nuestra aplicación cliente.

Aunque en los ejemplos siguientes se mostrarán las llamadas mediante el uso de **axios** dentro de los componentes React.js, es buena práctica aislar estas funcionalidades en archivos JavaScript independientes (por ejemplo: utilidades, llamadas al API o similares), para desacoplar aún más la parte lógica de la interfaz (así es como lo llevaremos a cabo en nuestro proyecto).

4.4.1 GET con **axios**

Imagina que creamos un nuevo componente React.js denominado *PersonList* que mostrará los nombres de una lista de personas. En este caso, necesitamos hacer un *hook* (o “enganche”) en la función **componentDidMount()** del ciclo de vida del componente, para que cuando el componente se renderice una vez montado, se pueda navegar por la lista de personas (que se almacenarán en una variable o array de estado denominada **persons**) y mostrar sus elementos. A continuación, se presenta el código relacionado:

63 <https://github.com/axios/axios> [Último acceso: Julio, 2020]

64 https://developer.mozilla.org/es/docs/Web/API/Fetch_API [Último acceso: Julio, 2020]

65 <https://jsonplaceholder.typicode.com/> [Último acceso: Julio, 2020]

```
import React from 'react';
import axios from 'axios';

export default class PersonList extends React.Component {
  state = {
    persons: []
  }
  componentDidMount() {
    axios.get(`https://jsonplaceholder.typicode.com/users`)
      .then(res => {
        const persons = res.data;
        this.setState({ persons });
      })
  }
  render() {
    return (
      <ul>
        { this.state.persons.map(person => <li>{person.name}</li>)}
      </ul>
    )
  }
}
```

Como se puede apreciar, tenemos que importar el paquete **axios** previamente instalado, y mediante **axios.get(url)** obtenemos una promesa que devuelve un objeto **response**. Para acceder a los datos de esa respuesta (en nuestro caso las personas), lo haremos con **res.data** (o el nombre que le hayamos asignado a la respuesta). También podremos recuperar otra información de la solicitud como el código de estado con **res.status** o más información con **res.request**.

La función **map()** de JavaScript permite iterar sobre los elementos del array **persons**, y para cada elemento “person”, en este caso, mostrar su nombre (con **person.name**), que corresponderá con un atributo válido del documento JSON devuelto en la respuesta.

4.4.2 POST con axios

El manejo de las solicitudes POST y PUT es muy similar. En el siguiente ejemplo, se presenta la creación de un formulario a renderizar, que cuando se pulsa el botón “Add” (de tipo submit), se dispara el evento **handleSubmit** que hace el POST del contenido, en este caso el usuario (**user**), cuyos datos se mostrarán en pantalla. Igualmente tenemos la función manejadora **handleChange** asociada a los cambios que se produzcan en el campo de texto del formulario.

```
import React from 'react';
import axios from 'axios';

export default class PersonList extends React.Component {
  state = {
    name: '',
  }
  handleChange = event => {
    this.setState({ name: event.target.value });
  }
  handleSubmit = event => {
    event.preventDefault();
    const user = {
      name: this.state.name
    };
    axios.post(`https://jsonplaceholder.typicode.com/users`, { user })
      .then(res => {
        console.log(res);
        console.log(res.data);
      })
  }
  render() {
    return (
      <div>
        <form onSubmit={this.handleSubmit}>
          <label>
            Person Name:
            <input type="text" name="name" onChange={this.handleChange} />
          </label>
          <button type="submit">Add</button>
        </form>
      </div>
    )
  }
}
```

Aunque no entraremos en detalle sobre el lenguaje JavaScript y sus especificaciones, en este punto es interesante comentar que existen diversas formas de implementar funciones, principalmente asociadas a eventos dentro de los componentes. La forma en la que se implementan y la necesidad de ligarlas con eventos JavaScript repercuten en su sintaxis.

Nótese que, si estamos usando una especificación JavaScript diferente (por ejemplo, ES5 vs. ES6), las funciones del manejo del evento tendrán una sintaxis

algo diferente (sin hablar de las “funciones *bindeadas*” que comentaremos más adelante). A continuación, se presenta el mismo código con sintaxis procedente de la especificación ES5.

```
.....
handleChange (event){
  this.setState({ name: event.target.value });
}
handleSubmit (event){
  event.preventDefault();
  const user = {
    name: this.state.name
  };
  axios.post(`https://jsonplaceholder.typicode.com/users`, { user })
    .then(res => {
      console.log(res);
      console.log(res.data);
    })
  }
}
.....
```

Aunque ambas sintaxis son compatibles, muchas veces se aconseja homogeneizar el desarrollo y utilizar solamente un tipo de especificación si fuera posible.

No es objetivo de este libro profundizar sobre las características del lenguaje JavaScript y sus especificaciones, aunque puedes conocer más detalles de esto en las lecturas recomendadas correspondientes.

4.4.3 DELETE con axios

Mediante `axios.delete`, podremos borrar elementos pasando la URL correcta como parámetro. En el ejemplo siguiente, a través de un formulario, obtendremos el ID de una persona y accederemos mediante `axios` a dicha persona para borrar sus datos. Los datos borrados, son devueltos como respuesta y mostrados en pantalla.

```
.....
import React from 'react';
import axios from 'axios';
export default class PersonList extends React.Component {
  state = {
    id: '',
  }
}
.....
```

```

    handleChange (event) {
      this.setState({ id: event.target.value });
    }
    handleSubmit (event) {
      event.preventDefault();
      axios.delete(`https://jsonplaceholder.typicode.com/users/${this.state.id}`)
        .then(res => {
          console.log(res);
          console.log(res.data);
        })
    }
    render() {
      return (
        <div>
          <form onSubmit={this.handleSubmit}>
            <label>
              Person ID:
              <input type="text" name="id" onChange={this.handleChange} />
            </label>
            <button type="submit">Delete</button>
          </form>
        </div>
      )
    }
  }
}

```

4.4.4 Definición de una instancia base

La librería **axios** nos permite definir una “*instancia base*” en la cual podemos definir una URL base de acceso a los recursos y otras configuraciones (como por ejemplo el uso de autenticación, etc.). Lo más común es crear un fichero JavaScript con dicha configuración, exportarlo y usarlo en el resto de archivos que puedan usar **axios**.

A continuación, se muestra un ejemplo de la creación de una instancia base.

```

import axios from 'axios';
export default axios.create({
  baseURL: `http://jsonplaceholder.typicode.com/`
});

```

Esta instancia podrá ser utilizada en componentes React.js u otros ficheros de la siguiente forma (en el caso de haber creado dicha instancia en un fichero con el nombre “api.js”, por ejemplo):

```
import API from '../api';

export default class PersonList extends React.Component {
  handleSubmit (event){
    event.preventDefault();
    API.delete(`users/${this.state.id}`)
      .then(res => {
        console.log(res);
        console.log(res.data);
      })
  }
  ...
}
```

Como vemos, ya no es necesario importar la librería **axios** en cada archivo o componente que requiera utilizarla, basta con importarla al crear la instancia base, y únicamente tendremos que importar el fichero que la define, allí donde lo necesitemos.

Teniendo en cuenta estos fundamentos, ya podremos realizar cualquier petición para consumir recursos de una API REST usando la librería **axios**.

4.5 DESARROLLO DEL CLIENTE. PÁGINAS, COMPONENTES Y COMUNICACIÓN CON EL API REST

En esta sección pondremos en práctica los conceptos explicados en las secciones anteriores con la finalidad de implementar paso a paso nuestra aplicación *front-end* con React.js, y establecer una comunicación con el API REST en base a las distintas solicitudes HTTP con el uso de la librería **axios**.

Cada uno de los apartados siguientes implementará una funcionalidad o funcionalidades de la aplicación, normalmente encapsuladas en componentes. Durante la explicación de estos, también se irán creando las funciones de envoltura o *wrappers* correspondientes a la comunicación con la API REST, de modo que aislaremos las funciones de comunicación de las visuales o que tienen que ver más con las vistas de la aplicación web y el propio componente.

4.5.1 Login de un usuario

Considerando la estructura y el enrutado de nuestra aplicación, así como la instalación de los paquetes iniciales con `npm`, explicado en la sección Fundamentos y primeros pasos, y en base al `HashRouter` definido para nuestra página inicial de la aplicación “/”, se renderizará el componente `Signin.jsx`, el cual crearemos con el editor Atom. La renderización de este componente incluye la renderización del componente `PostList.jsx` y un formulario de login; aparte de contar con un enlace a la ruta “/signup”, que renderizará el componente `Signup.jsx` para el registro de nuevos usuarios.

En la Figura 54 se muestra la apariencia de esta página principal conforme a los componentes renderizados.



Figura 54. Página principal de la aplicación web. Componentes y otros elementos

Vamos a centrarnos en el *login* del usuario y ver qué ocurre atendiendo al siguiente código del componente `Signin.jsx`:

```
import React, { Component } from 'react';
import { Link } from 'react-router-dom';
import { Button, Form, FormGroup, Label, Input, Container, Row, Col, Card, Card-
Title, Alert } from 'reactstrap';

import { login } from "../../utils/apicalls.js";
import PostList from "../../posts/PostList";
```

```
class Signin extends Component {
  constructor(props) {
    super(props);
    this.state = {
      username: '',
      password: '',
      loginMessage: null
    }
    this.handleUsernameChange = this.handleUsernameChange.bind(this);
    this.handlePasswordChange = this.handlePasswordChange.bind(this);
    this.onSignin = this.onSignin.bind(this);
  }

  handleUsernameChange(e){
    this.setState({username:e.target.value})
  }

  handlePasswordChange(e){
    this.setState({password:e.target.value})
  }

  onSignin(e) {
    e.preventDefault();
    const {
      username,
      password
    } = this.state;
    // Comprueba si es un usuario válido de la base de datos
    login(username, password).then((res) => this.checkLogin(res, username));
  }

  checkLogin(res, username) {
    //Si el usuario es válido...
    if (res.message === 'ok') {

      sessionStorage.setItem('role', res.role);
      sessionStorage.setItem('iduser', res.id);
      sessionStorage.setItem('username', username);

      this.props.history.push("/home");
    }
    //Si no, tratar el error (mostrarlo en un 'Alert')...
    else {
      this.setState({
```

```

        loginMessage: (<Alert color="danger">{res.message}</Alert>)
      });
    }
  }

  render() {
    return (
      <Container>
        <Row>
          <Col xs= "9">
            <PostList/>
          </Col>
          <Col xs="3">
            <Card body>
              <CardTitle tag="h4">Login</CardTitle>
              {this.state.loginMessage}
              <Form>
                <FormGroup>
                  <Label for="aUsername">Username</Label>
                  <Input type="text" name="username" id="aUsername"
placeholder="Introduce tu username" onChange={this.handleUsernameChange} requi-
red/>

                  </FormGroup>
                  <FormGroup>
                    <Label for="aPassword">Password</Label>
                    <Input type="password" name="password" id="aPassword"
placeholder="Introduce tu password" onChange={this.handlePasswordChange} requi-
red/>

                    </FormGroup>
                    <Button onClick={this.onSignin}>Entrar</Button>
                  </Form>
                </Card>
              <Row>
                <Col tag="center">
                  <Link to="/signup"><strong className="text-muted">Registrarse</
strong></Link>
                </Col>
              </Row>
            </Col>
          </Row>
        </Container>
      );
    }
  }
  export default Signin;

```

A partir del código anterior, conviene destacar las siguientes partes:

- Importación de componentes y librerías.** Estas sentencias muestran los componentes importados tanto de las librerías del *framework* como de la librería *reactstrap*. Así mismo, necesitaremos importar la función *login*, implementada en el fichero “apicalls.js”, y el componente *PostList.jsx* que estará implementado en el subdirectorio “/posts”.

```
import React, { Component } from 'react';
import { Link } from 'react-router-dom';
import { Button, Form, FormGroup, Label, Input, Container, Row, Col,
Card, CardTitle, Alert } from 'reactstrap';
import { login } from '../utils/apicalls.js';
import PostList from './posts/PostList';
```

- Definición de estados y bindings de los campos del formulario,** lo que hace referencia tanto a los estados que cambiarán el comportamiento del componente como a las funciones *bindeadas* sobre dichos estados y relacionados con diferentes campos del formulario de *login* presentado. Esto se realiza en el constructor del componente conforme al código siguiente:

```
this.state = {
  username: '',
  password: '',
  loginMessage: null
}
this.handleUsernameChange = this.handleUsernameChange.bind(this);
this.handlePasswordChange = this.handlePasswordChange.bind(this);
this.onSignin = this.onSignin.bind(this);
```

Teniendo en cuenta esto, el código de las funciones *bindeadas* se presenta a continuación. Estas funciones determinan lo que sucederá cuando el usuario está escribiendo su nombre, *password* y/o pulsa el botón “Entrar”; y están asociadas a los eventos de JavaScript *onChange* y *onClick* respectivamente.

```
handleUsernameChange(e){
  this.setState({username:e.target.value})
}
handlePasswordChange(e){
  this.setState({password:e.target.value})
}
```

```
}
onSignin(e) {
  e.preventDefault();
  const {
    username,
    password
  } = this.state;
  // Comprueba si es un usuario válido de la base de datos
  login(username, password).then((res) => this.checkLogin(res, username));
}
```

Pero ¿qué significa que una función está *bindeada*? Para que una función ligada a un evento que se encuentra dentro de un componente React.js funcione adecuadamente, es decir, que lleve los datos que tiene que llevar cuando se precisa su ejecución, se necesita que esa función esté *bindeada* en la clase que implementa el componente, de otra forma su ejecución producirá un error de dato o propiedad indefinida o “undefined”. Y para que esto no ocurra, necesitamos que dicha función se llame añadiendo `.bind(this)`. El hecho de pasarle el argumento `this` tiene que ver con la necesidad de conocer el contexto de la clase del componente para poder ejecutar dicha función, que podrá estar implementada en dicho componente o en un componente padre. Además, para evitar tener que hacer las llamadas a dichas funciones con el `.bind(this)`, podemos declararlas en el constructor de la clase (ver código de nuestro componente), y llamarlas en el resto de la misma como si fueran funciones regulares. Por supuesto, existen otras formas de implementar funciones *bindeadas* y ligadas a eventos dentro de un componente React.js, también relacionadas con las distintas sintaxis de JavaScript. En la sección de lecturas recomendadas de esta parte se presenta alguna referencia interesante al respecto.

Por otra parte, la renderización de la vista implica la construcción del *layout* asociado al componente, en el que se diferencian dos partes: la encargada de mostrar la lista de *posts*, y la que representa el formulario de *login*. La distribución y ubicación de dichas partes viene determinada por el atributo “*xs*” asociado al componente `<Col />`. Teniendo en cuenta el tamaño total de 12 “*franj*as” de nuestro *layout*, la primera parte ocupará 9 “*franj*as” (`<Col xs=”9”/>`) y la segunda 3 (`<Col xs=”3”/>`).

NOTA

En las últimas versiones del *framework* que trabajan con React *hooks*, tal y como se explica en Introducción a los *hooks* en React.js, ya no es necesario realizar el *binding* de funciones ni recuperar el contexto con `this`.

Volviendo al código de nuestro componente “Signin.jsx”, observamos como en este componente existen tres atributos que cambiarán su estado. Dos de ellos corresponden a los campos `username` y `password` del formulario de *login* y otro al mensaje a mostrar cuando se recibe una respuesta inválida del servidor. En este caso, el *binding* de las funciones asociadas al evento `onChange` del campo, ejecutará la función correspondiente y actualizará el estado de dicho campo “en tiempo real” (con `setState`).

También cabe destacar la ejecución del método o función `onSignin` cuando se pulsa el botón “Entrar”. Dicho método realizará una petición al servicio de correspondiente de la API REST (la cual deberá estar desplegada previamente, bien en producción o en desarrollo en otro terminal y puerto diferente al de la aplicación web) y el resultado del servicio será comprobado por el método `checklogin` que recibe la respuesta del servidor (en la variable `res`), tal y como se muestra a continuación.

```
.....
checkLogin(res, username) {
  // Si el usuario es válido...
  if (res.message === 'ok') {
    sessionStorage.setItem('role', res.role);
    sessionStorage.setItem('iduser', res.id);
    sessionStorage.setItem('username', username);
    this.props.history.push("/home");
  }
  // Si no, tratar el error (mostrarlo en un 'Alert')...
  else {
    this.setState({
      loginMessage: (<Alert color="danger">{res.message}</Alert>)
    });
  }
}
}
.....
```

Aquí, conviene revisar la función `router.post('/signin')` en nuestra API REST, y veremos de forma clara cuáles son esos mensajes y parámetros que devuelve el servidor al cliente. Así, si el parámetro `message` de la respuesta contiene el valor “ok”, crearemos una sesión con el `username` y el `rol` del usuario que intenta identificarse (o *loguearse*), y accederemos a la página “/home” de nuestro *front-end*. En caso contrario, mostraremos el mensaje de error usando un componente `Alert` de `reactstrap` y cambiando el estado de la variable `loginMessage` en el componente renderizado.

Con `this.props.history` accedemos a la historia de navegación de nuestra aplicación, conformada por las rutas correspondientes. La variable `history` suele ser parte de las propiedades de los componentes, con lo que puedo servirme de ella para manejar dicha historia. Por supuesto, existen otras formas de transitar entre páginas de una aplicación web ante determinados eventos.

Por otra parte, ante cualquier error conviene desarrollar una estructura páginas y componentes asociados que favorezca su traza y mantenimiento, pero por el momento lo trataremos como un simple mensaje al nivel de este componente.

Como ya se ha explicado en la sección **Consumo de una API REST desde React.js con axios**, para conectar con nuestros servicios REST usaremos la librería `axios`. De modo que, si aún no la tenemos instalada en nuestro proyecto, es el momento de hacerlo.

Y en el fichero “api.js” (ver estructura del proyecto en la sección **Estructura de la aplicación React.js**) importamos la librería `axios` y declaramos la URL base del servidor que contiene desplegados nuestros servicios de esta forma:

```
.....  
import axios from 'axios';  
import config from '../config.js';  
  
export default axios.create({  
  baseURL: config.baseURL_API  
});  
.....
```

Realmente, la URL de nuestra API REST podría asignarse a una variable de entorno (ver **Introducción a las variables de entorno**) de la aplicación `React.js`, de forma que podamos modificarla durante el despliegue sin tener que editar nuevamente el código. Así mismo, el valor de dicha variable podría encontrarse en un fichero “.env” que podremos ignorar durante el proceso de sincronización del repositorio, igual que hicimos con algunas variables en el desarrollo de nuestra API REST con `Express.js`.

El uso de variables de entorno en aplicaciones `React.js`, es muy similar a cómo se hace en `Node.js` y `Express.js`, pero con alguna pequeña diferencia. En este caso no es necesario instalar ningún paquete adicional vía `npm`.

Lo primero que haremos será crear un fichero “.env” en la raíz del proyecto con la siguiente variable de entorno:

```
.....  
REACT_APP_APIURI = <URL DE CONEXIÓN CON LA API REST>  
.....
```

Como norma, toda variable de entorno en React.js debe comenzar con “REACT_APP_” para evitar conflictos con otras variables internas definidas. En este caso, el valor de dicha variable de entorno será la URL completa donde hemos desplegado nuestro API, bien la de Heroku, por ejemplo, o la URL local con el puerto.

Adicionalmente lo que haremos en la aplicación React.js, será crear un fichero de configuración “config.js” en la raíz del proyecto, que contendrá en una variable, y en formato JSON, todas las variables de entorno definidas. Esto es una buena práctica de cara a mejorar el mantenimiento de nuestras aplicaciones. De este modo, el fichero “config.js” contendrá el siguiente código:

```
.....  
const config = {  
  baseUrl_API: process.env.REACT_APP_APIURI  
}  
export default config;  
.....
```

Así cuando queramos utilizar la variable de entorno REACT_APP_APIURI, lo haremos como `config.baseUrl_API`, importando convenientemente el fichero “config.js” como hemos hecho en “api.js”.

Si finalmente no queremos sincronizar el fichero “.env” con el repositorio, tendremos que indicarlo en el fichero “.gitignore” como hicimos en el desarrollo de la API REST, añadiendo la siguiente línea:

```
.....  
.env  
.....
```

NOTA

Si tienes desplegada la API REST, desarrollada anteriormente, en local, deberás utilizar la URL *localhost* seguida del puerto correspondiente como variable de entorno. Además, tienes que indicar a React.js que tu API REST se encuentra desplegada en local, añadiendo la variable proxy con el valor de la URL donde se encuentran desplegados los servicios en el fichero “package.json” de la aplicación React.js, como indica el siguiente ejemplo:

```
.....  
...  
"proxy": "http://localhost:5000"  
...  
.....
```


Ahora crearemos un fichero “/utils/apicalls.js” en el que agruparemos todas las llamadas a la API REST con la configuración del fichero “api.js”, en un único fichero, con el fin de conseguir un fácil mantenimiento. Añadiremos al fichero “apicalls.js”, por tanto, la llamada al servicio del *back-end* relativo al *login* que devolverá el resultado a quien lo solicitó, tal y como se muestra en el código siguiente (haremos la función exportable para usarla en otro lugar de la aplicación):

```
import API from './api';

export {
  login
};

function login(username, password) {
  return API.post('/users/signin', {
    username,
    password
  }).then(result => result.data)
  .catch(function(error){
    //TODO Puedes determinar qué hacer cuando un estado de error es enviado por
    el servidor (también para el resto de llamadas)
  });
}
```

El resultado del servicio lo tenemos en `result.data` que será lo que necesita nuestra función `login` del *front-end*. Opcionalmente, y si no lo realizamos en el servidor, podemos capturar a nivel de cliente los errores producidos en las peticiones al API REST (con códigos de estado no exitosos), utilizando un `catch` y el código correspondiente. En este libro no entraremos en detalle al respecto, pero es recomendable echar un vistazo a las lecturas recomendadas.

Siempre que queramos comprobar el funcionamiento de la aplicación y corregir posibles errores, recuerda que podemos desplegarla en local con el siguiente comando y visualizar el resultado en el navegador:

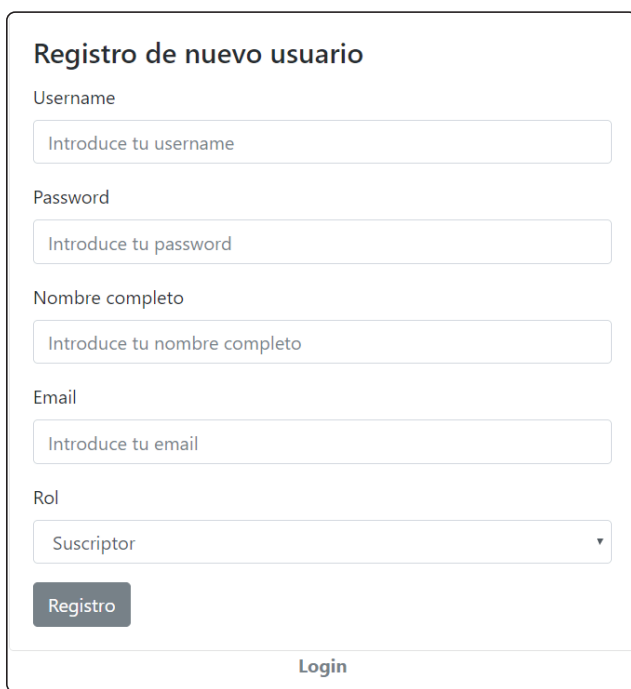
```
C:\microblogging-example-react>npm start
```

NOTA

Si estás haciendo uso de Git para controlar las versiones de tu proyecto, este sería buen momento de realizar un `commit` y sincronizarlo con tu repositorio de Github.

4.5.2 Registro de un nuevo usuario

Tal y como vimos en el código de la sección **Fundamentos y primeros pasos**, crearemos una página asociada a la ruta “/signup” que mostrará el componente *Signup.jsx*, que contendrá un formulario de registro y se comunicará, mediante **axios**, con el servicio REST correspondiente para el almacenamiento de un nuevo usuario en la base de datos. En la Figura 55, se muestra la apariencia de este componente:

El formulario tiene un título "Registro de nuevo usuario" en la parte superior. A continuación, hay cinco campos de entrada: "Username" con el placeholder "Introduce tu username", "Password" con el placeholder "Introduce tu password", "Nombre completo" con el placeholder "Introduce tu nombre completo", "Email" con el placeholder "Introduce tu email", y "Rol" con un menú desplegable que muestra "Suscriptor". Debajo de los campos hay un botón "Registro" de color gris oscuro. En la parte inferior del formulario, centrado, hay un enlace "Login".

Registro de nuevo usuario

Username

Introduce tu username

Password

Introduce tu password

Nombre completo

Introduce tu nombre completo

Email

Introduce tu email

Rol

Suscriptor

Registro

Login

Figura 55. Interfaz gráfica (vista) del componente de registro

Para acceder a esta nueva página, lo podremos hacer desde la página principal, con la siguiente sentencia:

```
<Link to="/signup"><strong className="text-muted">Registrarse</strong></Link>
```

Link es un componente de **react-router** que permite crear un hipervínculo a rutas definidas (ver sección **Enrutado con React Router**). Igualmente, desde el registro, podremos dirigirnos a la página principal mediante:

```
<Link to="/"><strong className="text-muted">Login</strong></Link>
```

Con todo ello, el componente *Signup.jsx* tendrá el siguiente código:

```
import React, { Component } from 'react';
import { Button, Form, FormGroup, Label, Input, Container, Row, Col, Card, Card-
Title } from 'reactstrap';
import { Link } from 'react-router-dom';
import { postNewUser } from "../../utils/apicalls.js";

class Signup extends Component {
  constructor(props) {
    super(props);
    this.state = {
      username: '',
      password: '',
      fullname: '',
      email: '',
      role: 'subscriber'
    };
    this.handleUsernameChange = this.handleUsernameChange.bind(this);
    this.handlePasswordChange = this.handlePasswordChange.bind(this);
    this.handleFullnameChange = this.handleFullnameChange.bind(this);
    this.handleEmailChange = this.handleEmailChange.bind(this);
    this.handleRoleChange = this.handleRoleChange.bind(this);
    this.onSignup = this.onSignup.bind(this);
  }
  handleUsernameChange(e){
    this.setState({username:e.target.value})
  }
  handlePasswordChange(e){
    this.setState({password:e.target.value})
  }
  handleFullnameChange(e){
    this.setState({fullname:e.target.value})
  }
  handleEmailChange(e){
    this.setState({email:e.target.value})
  }
  handleRoleChange(e){
    this.setState({role:e.target.value})
  }
  onSignup(e) {
```

```

    const {
      username,
      password,
      fullname,
      email,
      role
    } = this.state;
    // Crea e inserta un nuevo usuario en la base de datos
    postNewUser(username, password, fullname, email, role)
      .then((res) => this.checkPOSTNewUser(res));
  }
  // Comprueba la respuesta del servidor
  checkPOSTNewUser(res) {
    // si la respuesta es OK: crea un nuevo usuario y redirecciona a la página
    de login
    if (res === "OK")
      this.props.history.push("/");
    else{
      //TODO Por ejemplo mostrar un "Modal" avisando del error al añadir el
    usuario
    }
  }
  render() {
    return (
      <Container>
        <Row>
          <Col sm="12" md={{ size: 6, offset: 3 }}>
            <Card body>
              <CardTitle tag="h4">Registro de nuevo usuario</CardTitle>
              <Form>
                <FormGroup>
                  <Label for="aUsername">Username</Label>
                  <Input type="text" name="username" id="aUsername"
placeholder="Introduce tu username" onChange={this.handleUsernameChange} requi-
red/>
                </FormGroup>
                <FormGroup>
                  <Label for="aPassword">Password</Label>
                  <Input type="password" name="password" id="aPassword"
placeholder="Introduce tu password" onChange={this.handlePasswordChange} requi-
red/>
                </FormGroup>
                <FormGroup>
                  <Label for="aFullname">Nombre completo</Label>
                  <Input type="text" name="fullname" id="aFullname"

```

```

placeholder="Introduce tu nombre completo" onChange={this.handleFullnameChange}
/>

    </FormGroup>
    <FormGroup>
      <Label for="aEmail">Email</Label>
      <Input type="email" name="email" id="aEmail"
placeholder="Introduce tu email" onChange={this.handleEmailChange} required/>
    </FormGroup>
    <FormGroup>
      <Label for="aRole">Rol</Label>
      <Input type="select" name="role" id="aRole" onChange={this.
handleRoleChange}>
        <option value="subscriber">Suscriptor</option>
        <option value="admin">Administrador</option>
      </Input>
    </FormGroup>
    <Button onClick={this.onSignup}>Registro</Button>
  </Form>
</Card>
<Row>
  <Col tag="center">
    <Link to="/"><strong className="text-muted">Login</strong></
Link>
  </Col>
</Row>
</Col>
</Row>
</Container>
);
}
}
export default Signup;

```

Al igual que en el componente *Signin.jsx*, estableceremos como variables del estado los campos del formulario de registro que se renderizará, e implementaremos las funciones *bindeadas* (*handleXChange* y *onSignup*) correspondientes a efectuar cambios en dichos campos, que traerán consigo cambios en las variables modificadas del estado cuando se lance el evento *onChange* asociado a cada *Input* del formulario; así como el envío del formulario.

En este caso, cuando pulsamos en el botón “Registro” llamaremos a la función *postNewUser* tomando como argumentos los valores de los campos del formulario

(que estarán recogidos en el estado del componente) y después comprobaremos la respuesta de esa solicitud procedente del servidor, tal y como sigue:

```

onSignup(e) {
  const {
    username,
    password,
    fullname,
    email,
    role
  } = this.state;
  // Añade un nuevo usuario a la base de datos
  postNewUser(username, password, fullname, email, role)
    .then((res) => this.checkPOSTNewUser(res));
}

// Comprueba la respuesta del servidor
checkPOSTNewUser(res) {
  // si la respuesta es OK: crea el nuevo usuario y redirecciona a la página
  de login
  if (res === "OK")
    this.props.history.push("/");
  else{
    //TODO Por ejemplo mostrar un "Modal" avisando del error al añadir el
    usuario
  }
}

```

Así cuando la respuesta del servidor corresponde a un código de estado 200 ("OK"), redirigiremos al usuario a la página principal para que pueda identificarse o *loguearse*; en caso contrario, no haremos nada, aunque podremos utilizar un componente `Modal` de *Reactstrap* que muestre un mensaje de error en pantalla.

Por otra parte, la función encargada de llamar al servicio REST del API mediante `axios` es importada en el componente *Signup.jsx* con:

```

import { postNewUser } from "../utils/apicalls.js";

```

La implementación de dicha función se encontrará en `"/utils/apicalls.js"` con el siguiente código (sin olvidar el `export` de la misma al inicio del archivo):

```
function postNewUser(username, password, fullname, email, role) {  
  return API.post('/users', {  
    username,  
    password,  
    fullname,  
    email,  
    role }).then(result => result.data);  
}
```

Tanto el tipo de solicitud (en este caso *POST*) como la ruta ('/users') deberá coincidir con el *endpoint* definido en nuestro API REST.

Debemos recordar que las llamadas al API REST desde nuestra aplicación React.js pueden monitorizarse desde el terminal en el caso de tener desplegado el API REST de modo local.

4.5.3 Visualización de todas las entradas

También como parte de la página principal y embebido en el componente *Signin.jsx*, tendremos el componente *PostList.jsx*. La forma de llamar a un componente dentro de otro (previa importación de este) es usando su nombre entre etiquetas típicas de JSX y React.js, en este caso de la siguiente forma:

```
<PostList/>
```

Pudiendo embeber el componente en determinada parte de la página o entre otras etiquetas que definan nuestro *layout* o *grid*, por ejemplo, como parte de una columna de una fila:

```
<Col xs= "9">  
  <PostList/>  
</Col>
```

El componente *PostList.jsx* se encargará de mostrar todas las entradas o posts publicados/escritos que se encuentren almacenados en la base de datos, teniendo una apariencia final como la mostrada en la Figura 56:



Figura 56. Apariencia del componente `PostList.jsx`

Primero crearemos el directorio “components/posts” con la finalidad de alojar todos los componentes relacionados con la gestión de posts en una misma carpeta.

El código asociado a `PostList.jsx` será el siguiente:

```
.....
import React, { Component } from 'react';
import { Row, Col, Card, CardTitle, Badge, CardBody, Table, Alert } from 're-
actstrap';
import { FaFeather } from 'react-icons/fa';

import { getAllPosts } from "../../utils/apicalls.js";
import { getDateInStrFormat } from "../../utils/utils.js";

class PostList extends Component {
  state = {
    posts: []
  };
  getPosts() {
    getAllPosts().then((posts) => {
      this.setState({
        posts
      });
    });
  }
}
```

```

    }
    componentDidMount() {
      this.getPosts();
    }
    render() {
      return (
        <div>
          <CardTitle tag="center"><Alert color="info"><strong>Posts publicados
</strong><Badge pill={this.state.posts.length}</Badge></Alert></CardTitle>
          <Table>
            <tbody>
              { this.state.posts.map((post, index) => {
                return(
                  <div>
                    <Alert color="dark">
                      <Row>
                        <Col>
                          <CardTitle tag="h5"><FaFeather /> {post.title}</Card-
Title>
                          <Card>
                            <CardBody>
                              <Row>
                                <Col>
                                  {post.description}
                                </Col>
                              </Row>
                              <Row>
                                <Col align="right">
                                  <small>{getDateInStrFormat(new Date(post.pu-
blicationdate))} - {post.user.username}</small>
                                </Col>
                              </Row>
                            </CardBody>
                          </Card>
                        </Col>
                      </Row>
                    </Alert>
                  </div>
                )}
              )}
            </tbody>
          </Table>
        </div>
      );
    }
  }
  export default PostList;

```

En este caso, el estado del componente incluye un array de posts que en principio estará vacía, pero que se llenará con todos los posts existentes en cuanto el componente se monte y antes de renderizarlo con:

```
componentDidMount() {  
  this.getPosts();  
}
```

La función `getPosts()` llamará a `getAllPosts()` que estará implementada en “apicalls.js” y que se encargará de obtener todos los posts del servidor, haciendo uso del servicio correspondiente de la API REST. El código de la función `getAllPosts()` es el siguiente:

```
function getAllPosts() {  
  return API.get('/posts').then(res => res.data);  
}
```

No debemos olvidar de realizar el `export` de dicha función. También, podemos implementar en este punto un `catch` si se desea capturar cualquier excepción y problema con la petición realizada al servidor, aunque no nos detendremos en ello.

Una vez tenemos los posts en el array `posts[]`, actualizaremos el estado del componente con los nuevos valores de la matriz, y podemos iteracionar sobre dicho array y renderizar cada post en el render del componente principal. De cada post, mostraremos su título, descripción, fecha de publicación y nombre de usuario (`username`) del autor. Para mostrar la fecha de cada post en el formato adecuado, usaremos la función implementada `getDateInStrFormat(date)`, que estará en “utils/utils.js”. Este fichero, lo usaremos para implementar diversas funciones útiles de nuestra aplicación. En nuestro caso, su contenido hasta ahora será el siguiente:

```
export {  
  getDateInStrFormat  
};  
  
// Recursos para el formateado de fechas  
const monthNames = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",  
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];  
const dayNames = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];  
  
function getDateInStrFormat(date){
```

```

var strformatted = dayNames[date.getDay()+], "+
    date.getDate()+ " +
    monthNames[date.getMonth()+] "+
    date.getFullYear()+, "+
    ("0" + date.getHours()).slice(-2)+":"+
    ("0" + date.getMinutes()).slice(-2)+":"+
    ("0" + date.getSeconds()).slice(-2);

return strformatted;
}

```

Para iterar y mostrar los datos de cada post, usaremos la función `map`. Puedes conocer más acerca de iteración sobre arrays en React.js echando un vistazo a las lecturas recomendadas. De esta forma, cada post del array será formateado entre etiquetas `<div>` de la siguiente forma:

```

return(
  <div>
    <Alert color="dark">
      <Row>
        <Col>
          <CardTitle tag="h5"><FaFeather /> {post.title}</CardTitle>
          <Card>
            <CardBody>
              <Row>
                <Col>
                  {post.description}
                </Col>
              </Row>
              <Row>
                <Col align="right">
                  <small>{getDateInStrFormat(new Date(post.publica-
tiondate))} - {post.user.username}</small>
                </Col>
              </Row>
            </CardBody>
          </Card>
        </Col>
      </Row>
    </Alert>
  </div>)

```

Con esto, conseguimos la siguiente apariencia (ver la Figura 57) para cada post individual:

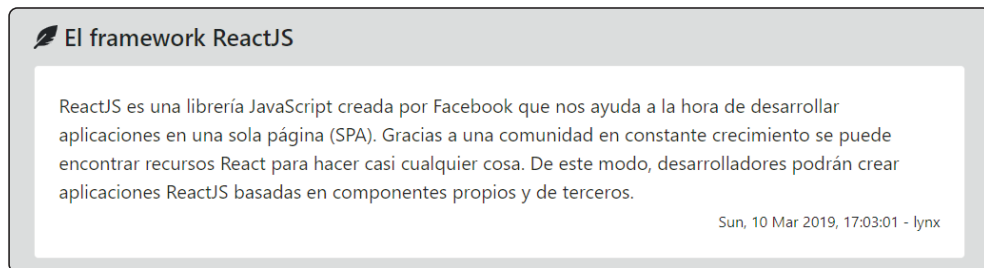


Figura 57. Apariencia de un post en PostList.jsx

También, podemos dar más detalles acerca del número de posts existentes (determinado por la longitud del array `posts`), por ejemplo, usando el componente `Badge` de *Reactstrap* con el número total de posts, previo a la iteración sobre los posts, de esta forma:

```
<Badge pill>{this.state.posts.length}</Badge>
```

Hecho esto, observaremos como al integrar el componente `<PostList />` con su sintaxis JSX en el fichero del componente *Signin.jsx*, se sustituirá esa etiqueta por nuestro componente, con todos sus estados, propiedades (si las tuviera) y comportamiento.

4.5.4 Panel de control o Dashboard

Ahora vamos a desarrollar un panel de control accesible solamente a los usuarios registrados. De modo que, cuando hacemos login en el componente *Signin.jsx*, nos llevará a la ruta `‘/home’`, la cual renderizará los componentes de esta parte “privada” que están embebidos en un componente denominado *Home.jsx*. Por supuesto, hay multitud de formas de desarrollar jerárquicamente los componentes de nuestra aplicación y en este libro se explica una posibilidad. La apariencia final de esta página se muestra en la Figura 58:



Figura 58. Apariencia del Dashboard de la aplicación

Por tanto, el componente *Home.jsx* renderizará los componentes *HeaderDashboard.jsx* y *PostList.jsx* (inicialmente), solo en el caso de que el usuario exista en la base de datos. En caso contrario (por ejemplo, un usuario no registrado previamente que intenta acceder a `/home` directamente), redirigiremos al usuario a la página principal `/` (conforme a lo definido en el *React Router*) y no renderizaremos el *Home.jsx*.

El código de este componente *Home.jsx* es el siguiente:

```
import React, { Component } from 'react';
import { Container, Row, Col } from 'reactstrap';

import HeaderDashboard from './HeaderDashboard';
import MyPostList from './posts/MyPostList';
import PostList from './posts/PostList';

class Home extends Component {
  constructor(props) {
    super(props);
    this.state = {
      show: (<PostList />)
    };
    this.handleLogout = this.handleLogout.bind(this);
    this.handleOnShow = this.handleOnShow.bind(this);
  }
  handleLogout(){
```

```

    sessionStorage.clear();
    this.props.history.push("/");
  }
  handleOnShow(option){
    if (option === 1){
      this.setState({
        show: (<PostList />)
      });
    }else if (option === 2){
      this.setState({
        show: (<MyPostList />)
      });
    }else if (option === 3){
      //TODO Mostrar por ejemplo un componente "UserEdit" a implementar
      alert('Usuario: '+sessionStorage.getItem('username')+''\nRol: "+sessionStorage.getItem('role')));
    }
  }
  render() {
    if (sessionStorage.getItem("username") === null){
      this.props.history.push("/");
    }
    else{
      return (
        <Container>
          <Row>
            <Col><HeaderDashboard onLogout = {this.handleLogout} onShow= {this.handleOnShow} /></Col>
          </Row>
          <Row>
            <Col xs="12">
              {this.state.show}
            </Col>
          </Row>
        </Container>
      );
    }
  }
}
export default Home;

```

Este componente tiene un estado con una variable que inicializa otro componente, en este caso:

```

    this.state = {
      show: (<PostList />)
    };

```

Para que desde un componente se pueda cambiar el estado de otro componente (usualmente de un componente hijo que hereda las propiedades o **props** de un padre), debemos pasar como propiedades las funciones definidas en el componente padre. En el método o función padre implementaremos el código de las funciones asociadas a los estados y que estarán *bindeadas* en el constructor. Finalmente, desde el componente hijo, llamaremos a dichas funciones del padre con los parámetros procedentes de este componente hijo.

En nuestro caso, este proceso está determinado por la opción elegida en el componente *HeaderDashboard.jsx*, que realizará cambios en los estados del componente *Home.jsx*, para mostrar o no otros componentes y permitir realizar el *logout* del panel de administración o *dashboard*.

El código del componente *HeaderDashboard.jsx* es el siguiente:

```

import React, { Component } from 'react';
import { Collapse, Navbar, NavbarBrand, Nav, NavItem, NavLink, Tooltip } from
'reactstrap';
import { FaSignOutAlt, FaCogs } from 'react-icons/fa';

class HeaderDashboard extends Component {
  constructor(props) {
    super(props);
    this.state = {
      tooltipOpen: false
    }
    this.toggle = this.toggle.bind(this);
  }
  toggle() {
    this.setState({
      tooltipOpen: !this.state.tooltipOpen
    });
  }
  render() {
    return (
      <Navbar color="warning" light expand="md">
        <NavbarBrand><FaCogs /> Dashboard</NavbarBrand>
        <Collapse navbar>
          <Nav className="ml-auto" navbar>

```

```

        <NavItem>
          <NavLink href="#" onClick={this.props.onShow.bind(this, 1)}>To-
dos los Posts</NavLink>
        </NavItem>
        <NavItem>
          <NavLink href="#" onClick={this.props.onShow.bind(this, 2)}>Mis
Posts</NavLink>
        </NavItem>
        <NavItem>
          <NavLink href="#" onClick={this.props.onShow.bind(this,
3)}>{sessionStorage.getItem('username')}</NavLink>
        </NavItem>
        <NavItem>
          <NavLink href="#" id="tooltip_logout" onClick={ this.props.onLo-
gout }><FaSignOutAlt />
            <Tooltip placement="bottom" isOpen={this.state.tooltipOpen}
target="tooltip_logout" toggle={this.toggle}>
              Salir
            </Tooltip>
          </NavLink>
        </NavItem>
      </Nav>
    </Collapse>
  </Navbar>
);
}
}
export default HeaderDashboard;

```

Este componente muestra la barra de navegación del *dashboard* con varias opciones y un *Tooltip* de *Reactstrap* asociado al botón de salir (representado por un icono de *react-icons*) que se mostrará cuando pasemos el cursor por encima. En función de la opción escogida en este *HeaderDashboard.jsx*, cambiará el estado definido en el componente *Home.jsx* tal y como hemos comentado, más concretamente lo sustituirá por los componentes *<PostList.jsx/>* o *<MyPostList.jsx/>* dependiendo de la opción pulsada (a través de la ejecución del método *handleOnShow(option)* del *Home.jsx*).

En la Figura 59 y en la Figura 60, se muestran sendos ejemplos de lo que ocurre en la interfaz de la aplicación cuando se pulsa sobre los botones de navegación del *dashboard*.

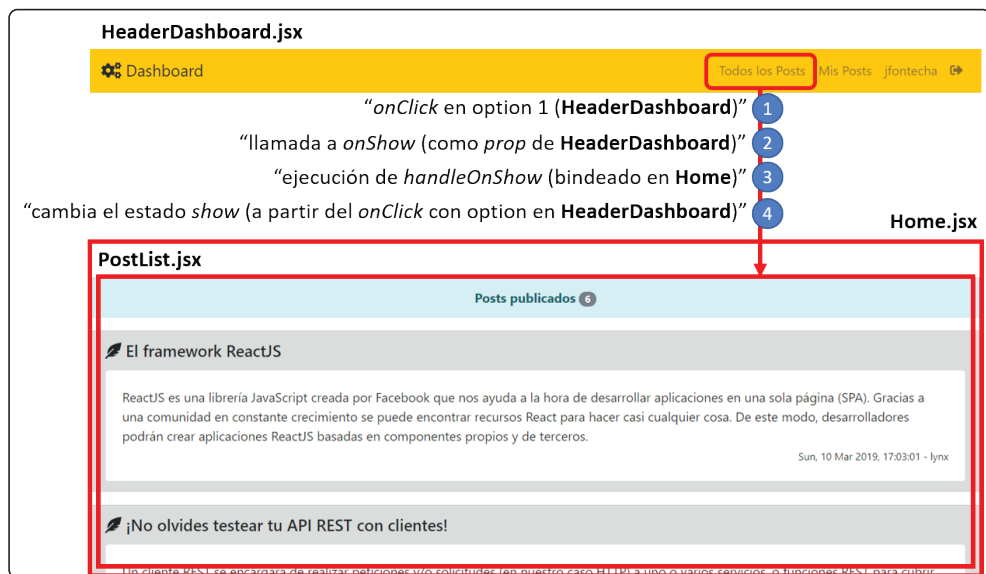


Figura 59. Jerarquía de componentes y desencadenamiento de eventos (para "todos los posts")

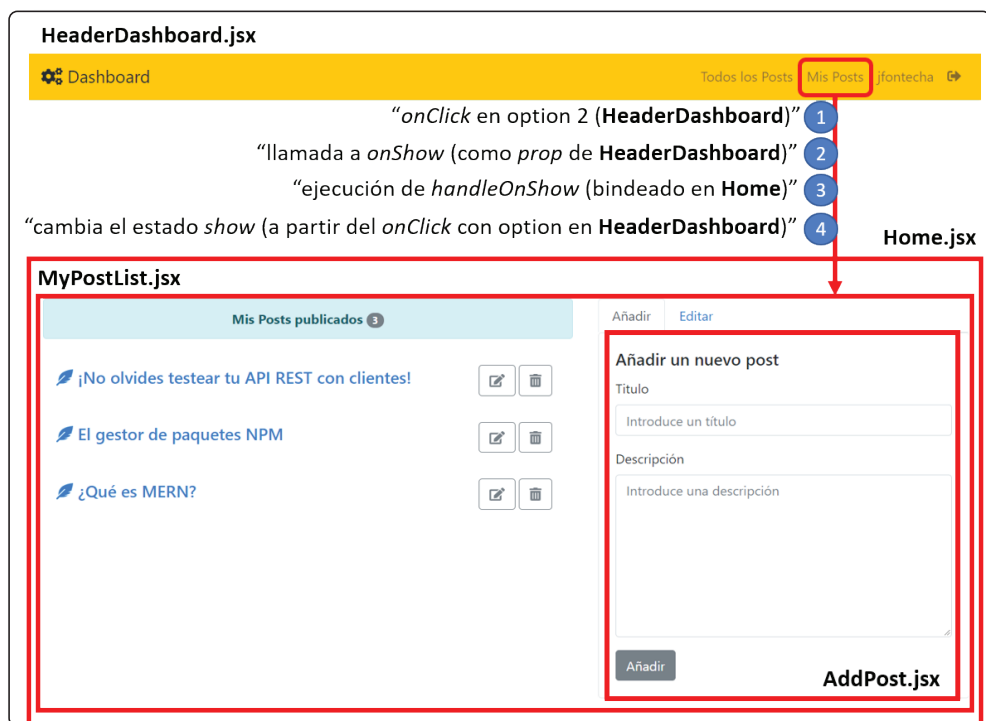


Figura 60. Jerarquía de componentes y desencadenamiento de eventos (para "mis posts")

Observamos que cuando cambiamos la variable de estado `show` para visualizar el componente `MyPostList.jsx`, este tiene a su vez embebidos los componentes `AddPost.jsx` y `EditPost.jsx`, que veremos en secciones posteriores.

4.5.5 Visualización de las entradas del usuario

Cuando pulsamos sobre “Mis Posts” en el *dashboard*, se nos mostrará el componente `MyPostList.jsx`, el cual tendrá toda la lógica asociada a la gestión de los posts del usuario *logueado*, incluyendo: mostrar sus posts, añadir un nuevo post, editar y borrar un post existente.

El código del componente `MyPostList.jsx`, alojado en “components/posts/MyPostList.jsx”, es el siguiente:

```
import React, { Component } from 'react';
import classNames from 'classnames';
import { Row, Col, Card, CardTitle, Badge, UncontrolledCollapse, CardBody,
Table, Alert, Button, Nav, NavItem, NavLink, Navbar, NavbarBrand, TabContent,
TabPane, Modal, ModalHeader, ModalBody, ModalFooter } from 'reactstrap';
import { FaEdit, FaFeatherAlt, FaTrashAlt } from 'react-icons';

import { getMyPosts, deletePost } from "../../utils/apicalls.js";
import { getDateInStrFormat } from "../../utils/utils.js";

import AddPost from './AddPost';
import EditPost from './EditPost';

class MyPostList extends Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: [],
      edit: (<Alert color="warning">Selecione editar un post de la lista</
Alert>),
      activeTab: '1',
      showDeleteModal: null
    };
    this.handleUpdateMyPosts = this.handleUpdateMyPosts.bind(this);
    this.deletePost = this.deletePost.bind(this);
  }
  getPosts() {
    getMyPosts(sessionStorage.getItem('idUser')).then((posts) => {
      this.setState({
```

```

        posts
      });
    });
  }
  toggleTab(tab) {
    if (this.state.activeTab !== tab) {
      this.setState({
        activeTab: tab
      });
    }
  }
}
handleUpdateMyPosts(){
  this.getPosts();
}
askForDelete(post){
  this.setState({
    showDeleteModal: (
      <Modal isOpen="true" className={this.props.className}>
        <ModalHeader>Eliminar post</ModalHeader>
        <ModalBody>
          ¿Está seguro que desea eliminar el post <strong>{post.title}</
strong>?
        </ModalBody>
        <ModalFooter>
          <Button color="warning" onClick={() => this.
deletePost(post)}>Eliminar</Button>{ ' ' }
          <Button color="secondary" onClick={() => this.setState({ showDelete-
Modal: null })}>Cancelar</Button>
        </ModalFooter>
      </Modal>)
    });
  }
  // Borrado de un post
  deletePost(post){
    deletePost(post._id)
      .then((res) => this.checkDELETEPost(res));
  }
  // Comprueba la respuesta del servidor
  checkDELETEPost(res) {
    // Si es satisfactoria (ok), quita el "Modal" y reinicia edit con el compo-
nente correspondiente
    if (res === "OK"){
      this.setState({
        showDeleteModal: null,
        edit: (<Alert color="warning">Seleccione editar un post de la lista</

```

```

Alert>
    });
    this.handleUpdateMyPosts();
  }else{
    //TODO Muestra un "Modal" con el error del servidor
  }
}
handleShowEdit(post){
  this.setState({
    edit: (<EditPost post= {post} updateMyPosts = {this.handleUpdateMyPosts}
/>)
  });
}
componentDidMount() {
  this.getPosts();
}
render() {
  return (
    <div>
      {this.state.showDeleteModal}
      <Row>
        <Col xs="7">
          <CardTitle tag="center"><Alert color="info"><strong>Mis Posts
publicados </strong><Badge pill>{this.state.posts.length}</Badge></Alert></Card-
Title>
          <Table>
            <tbody>
              { this.state.posts.map((post, index) => {
                return(<div>
                  <Row>
                    <Col>
                      <Navbar expand="md">
                        <NavbarBrand href="#" id={"toggler"+index}><h5><FaFe
atherAlt /> {post.title}</h5></NavbarBrand>
                        <Nav className="ml-auto" navbar>
                          <NavItem>
                            <NavLink>
                              <Button outline onClick={this.handleShowEdit.
bind(this, post)}><FaEdit /></Button>
                              { ' ' }
                              <Button outline onClick={this.askForDelete.
bind(this, post)}><FaTrashAlt /></Button>
                            </NavLink>
                          </NavItem>
                        </Nav>

```

```

        </Navbar>
      </Col>
    </Row>
    <Row>
      <Col>
        <UncontrolledCollapse toggler={`#toggler"+index}>
          <Card>
            <CardBody>
              <Row><Col>{post.description}</Col></Row>
              <Row><Col align="right"><small>{getDateInStrForm
at(new Date(post.publicationdate))} - {post.user.username}</small></Col></Row>
            </CardBody>
          </Card>
        </UncontrolledCollapse>
      </Col>
    </Row>
  </div>)
  }}}
</tbody>
</Table>
</Col>
<Col xs="5">
  <Nav tabs>
    <NavItem>
      <NavLink href="#" className={classnames({ active: this.state.
activeTab === '1' })} onClick={() => { this.toggleTab('1'); }}>
        Añadir
      </NavLink>
    </NavItem>
    <NavItem>
      <NavLink href="#" className={classnames({ active: this.state.
activeTab === '2' })} onClick={() => { this.toggleTab('2'); }}>
        Editar
      </NavLink>
    </NavItem>
  </Nav>
  <TabContent activeTab={this.state.activeTab}>
    <TabPane tabId="1">
      <Row>
        <Col sm="12">
          <AddPost updateMyPosts = {this.handleUpdateMyPosts}/>
        </Col>
      </Row>
    </TabPane>
    <TabPane tabId="2">

```

```

        <Row>
          <Col sm="12">
            {this.state.edit}
          </Col>
        </Row>
      </TabPane>
    </TabContent>
  </Col>
</Row>
</div>;
}
}

export default MyPostList;

```

En este componente, definiremos un estado con las siguientes variables:

```

this.state = {
  posts: [],
  edit: (<Alert color="warning">Seleccione editar un post de la lista</
Alert>),
  activeTab: '1',
  showDeleteModal: null
};

```

El array `posts` contendrá todos los posts asociados al usuario identificado o *logueado*, los cuales obtendremos cuando se monte el componente, gracias al método del ciclo de vida `componentDidMount()`, de forma similar a como se hizo en `PostList.jsx`. En este caso, se recuperarán los posts del usuario cuyo *id* coincida con el *id* del usuario *logueado*. Dicho valor de *id* lo podemos conocer debido a que, desde `Signin.jsx` se almacenó este y otros valores clave devueltos tras la solicitud de *login* en una variable de sesión del navegador.

Para obtener los posts del usuario, se implementará la llamada correspondiente al servicio REST en “`apicalls.js`”, que importaremos después en el propio `MyPostList.jsx` para que pueda ser usada y que corresponde al código siguiente:

```

function getMyPosts(iduser) {
  return API.get('/posts/all/' + iduser).then(res => res.data);
}

```

Con todos los posts obtenidos, renderizaremos una tabla, iterando en cada post y construyendo el contenido de cada fila, la cual contendrá el título del post, la descripción con la fecha formateada y el autor (embebida para ser desplegada gracias a un componente `UncontrolledCollapse` de *Reactstrap*), un botón de edición y otro de borrado del post.

La variable `edit` del estado del componente está inicializada a un componente `Alert` de *Reactstrap* con un mensaje inicial, que se muestra cuando no se ha pulsado el botón editar sobre uno de los posts. Si nos fijamos en la parte del render donde se implementa dicho botón tenemos el siguiente código:

```
.....  
<Button outline onClick={this.handleShowEdit.bind(this, post)}>  
  <FaEdit />  
</Button>  
.....
```

El evento `onClick` del botón está asociado a la función *bindeada* `handleShowEdit`, a la que pasamos el post de la matriz de posts que ha sido seleccionado.

NOTA

En este caso hemos querido añadir a propósito el `.bind(this, post)` a continuación de la llamada a la función `handleShowEdit` al ligar con el evento `onClick` para que conozcas otra de las formas de hacer el *binding*. Con esto, no es necesario declarar dicha función en el constructor (tiene el mismo efecto) ya que añadimos el `.bind` a la llamada, y además un argumento con el post.

Finalmente, esta función cambiará el valor de la variable de estado `edit` a un componente *EditPost.jsx* que implementaremos después y que tiene dos propiedades: el post seleccionado y el resultado de la actualización de mis posts (o de los posts del usuario concreto). De esta forma, cuando actualicemos un post (desde el componente *EditPost.jsx*) haremos una llamada a `updateMyPosts` que estará conectado con la función *bindeada* `handleUpdateMyPosts()` del componente *MyPostList.jsx*. Esto lo veremos en detalle en la siguiente sección del libro.

Tanto los componentes *AddPost.jsx* como *EditPost.jsx* están embebidos en las pestañas de un componente `Nav` de *Reactstrap*, y usaremos el método `toggleTab` para cambiar el valor de la variable de estado `activeTab`, y mostrar, por tanto, un componente u otro dependiendo de la pestaña pulsada.

Además, tenemos la posibilidad de borrar un post si pulsamos sobre el botón implementado atendiendo al siguiente código:

```

<Button outline onClick={this.askForDelete.bind(this, post)}>
  <FaTrashAlt />
</Button>

```

En este caso, el evento `onClick` llamará a la función *bindeada* `askForDelete` (con el binding implementado de la misma forma que antes), que preguntará si de verdad queremos borrar el post pasado como argumento, antes de llamar a la función del API REST que se encarga del borrado del post en la base de datos. El método en cuestión es como sigue:

```

askForDelete(post){
  this.setState({
    showDeleteModal: (
      <Modal isOpen="true" className={this.props.className}>
        <ModalHeader>Eliminar post</ModalHeader>
        <ModalBody>
          ¿Está seguro que desea eliminar el post <strong>{post.title}</strong>?
        </ModalBody>
        <ModalFooter>
          <Button color="warning" onClick={() => this.deletePost(post)}>Eliminar</Button>{ ' '}
          <Button color="secondary" onClick={() => this.setState({ showDeleteModal: null })}>Cancelar</Button>
        </ModalFooter>
      </Modal>)
    });
}

```

El comportamiento descrito anteriormente se hará a través de un componente `Modal` de *Reactstrap* que estará asociado a la variable de estado `showDeleteModal` que inicializamos a `null` y que definimos al inicio del render del componente. Como a priori estará a `null`, no mostrará nada.

Sin embargo, en el momento en que pulsamos sobre el botón de borrado se muestra el `Modal` anterior que tendrá dos botones, uno para cancelar, volviendo a poner la variable de estado `showDeleteModal` a `null` y otro para confirmar

la eliminación del post que estará *bindeado* o “enganchado” con la función `deletePost(post)` (ver Figura 61).

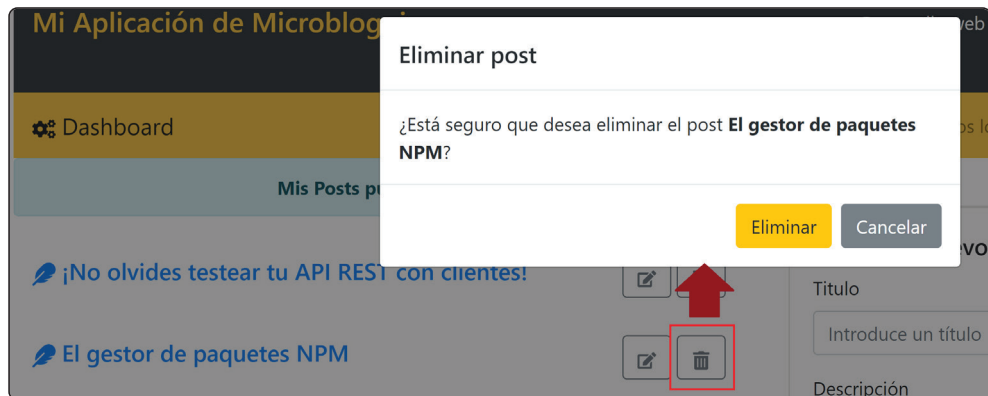


Figura 61. Modal asociado al botón de eliminar Post

NOTA

Como se puede observar en este caso, el *binding* de las funciones asociadas a estos eventos `onClick` se implementa de una tercera forma. Nuevamente, elegimos implementar el *binding* de esta otra forma (usando una función flecha de ES6 y el contexto `this`) a propósito, para que conozcas que también puede realizarse así. Aunque en este componente te presentamos varias formas de *binding* de funciones ligadas a eventos JavaScript, es conveniente ser homogéneo e implementarlas usando una de ellas. Igualmente, recuerda que a partir de la versión de React.js 16.8 y la aparición de los React *hooks* no es necesario el uso de *bindings*.

Prosiguiendo con la función anterior, esta llamará a la función correspondiente de “`apicalls.js`”, que realizará la solicitud de borrado y devolverá el resultado al cliente, tal y como se muestra a continuación.

```
function deletePost(idpost) {  
  return API.delete(`/posts/${idpost}`).then(result => result.data);  
}
```

Si el resultado es satisfactorio, inicializaremos las variables de estado `showDeleteModal` a `null` (para dejar de mostrar el Modal), y `edit` al componente `Alert` inicial con el mensaje de selección de un nuevo post. Así mismo, volveremos

a actualizar la lista de los posts del usuario para que quede reflejado el cambio en la página, a través de la solicitud de obtención de dichos posts. El siguiente código ilustra lo comentado en este párrafo.

```
.....
// Comprueba la respuesta del servidor
checkDELETEPost(res) {
  // Si es satisfactoria (ok), quita el "Modal" y reinicia edit con el compo-
  nente correspondiente
  if (res === "OK"){
    this.setState({
      showDeleteModal: null,
      edit: (<Alert color="warning">Selecione editar un post de la lista</
Alert>)
    });
    this.handleUpdateMyPosts();
  }else{
    //TODO Muestra un "Modal" con el error del servidor
  }
}
}
.....
```

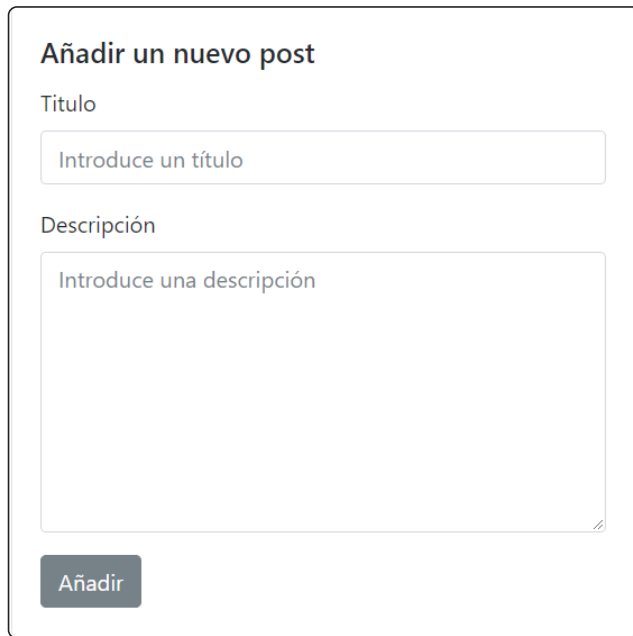
Por supuesto, podemos completar la implementación para mostrar algún `Modal` posterior en el caso de no haberse realizado el borrado correctamente, entre otras cosas.

4.5.6 Añadir y editar entradas

Como ya hemos visto, en el componente *MyPostList.jsx*, se integran los componentes *AddPost.jsx* y *EditPost.jsx*, encargados de añadir y editar posts, teniendo en cuenta el usuario *logueado*. A continuación, detallaremos la implementación y el comportamiento de estos componentes.

4.5.6.1 AÑADIR NUEVAS ENTRADAS

El componente *AddPost.jsx* renderizará un formulario con los campos título y descripción, y un botón que permitirá añadir el nuevo post a la base de datos mediante la petición al servicio correspondiente de nuestra API REST. En la Figura 62 se muestra la apariencia del componente, que estará embebido en una pestaña de un componente *Nav* de *Reactstrap* y a su vez integrado en el componente *MyPostList.jsx*.



Añadir un nuevo post

Titulo

Introduce un título

Descripción

Introduce una descripción

Añadir

Figura 62. Apariencia del componente `AddPost.jsx`

Siendo el código del componente *AddPost.jsx* el siguiente:

```
import React, { Component } from 'react';
import { Card, CardTitle, Label, Button, Form, FormGroup, Input } from 'reacts-
trap';

import { postNewPost } from "../../utils/apicalls.js";

class AddPost extends Component {
  constructor(props) {
    super(props);
    this.state = {
      title: '',
      description: ''
    }
    this.handleTitleChange = this.handleTitleChange.bind(this);
    this.handleDescriptionChange = this.handleDescriptionChange.bind(this);
    this.addPost = this.addPost.bind(this);
  }
  handleTitleChange(e){
    this.setState({title:e.target.value})
```

```

    }
    handleDescriptionChange(e){
      this.setState({description:e.target.value})
    }
    addPost(e){
      e.preventDefault();
      const {
        title,
        description
      } = this.state;
      // Guarda el post en la base de datos llamando a la función del api
      postNewPost(sessionStorage.getItem('iduser'), title, description)
        .then((res) => this.checkPOSTNewPost(res));
    }
    // Comprueba la respuesta del servidor
    checkPOSTNewPost(res) {
      if (res === "OK"){
        //TODO Muestra un "Modal" cuando el post es añadido
        this.setState(
          {
            title: "",
            description: ""
          }
        );
        this.props.updateMyPosts();
      }else{
        //TODO Muestra un "Modal" cuando ocurre un error al añadir el post
      }
    }
    render() {
      return (
        <div>
          <Card body>
            <CardTitle tag="h5">Añadir un nuevo post</CardTitle>
            <Form>
              <FormGroup>
                <Label for="aTitulo">Titulo</Label>
                <Input type="text" name="title" value={this.state.title}
id="aTitulo" placeholder="Introduce un título" onChange={this.handleTitleChange}
required/>
              </FormGroup>
              <FormGroup>
                <Label for="aDescripcion">Descripción</Label>
                <Input style={{height: '200px'}} type="textarea"
name="description" value={this.state.description} id="aDescripcion"

```

```

placeholder="Introduce una descripción" onChange={this.handleDescriptionChange}/>
      </FormGroup>
      <Button onClick={this.addPost}>Añadir</Button>
    </Form>
  </Card>
</div>
);
}
}
export default AddPost;

```

Como variables de estado tendremos `title` y `description`, correspondientes a los valores de los campos del formulario. Estas variables tendrán asociadas las siguientes funciones *bindeadas* (esta vez declaradas en el constructor del componente):

```

this.handleTitleChange = this.handleTitleChange.bind(this);
this.handleDescriptionChange = this.handleDescriptionChange.bind(this);

```

Que posibilitan actualizar el estado de la variable según el usuario modifica los valores en el campo correspondiente del formulario.

La implementación del botón Añadir corresponde al código:

```

<Button onClick={this.addPost}>Añadir</Button>

```

La ejecución del evento `onClick` hace que se llame a la función *bindeada* `addPost` que llamará a la función `postNewPost` de “utils/apicalls.js”, y que solicitará al servidor la adición de un nuevo post a la base de datos, asociado a un usuario (recuerda hacer exportables estas funciones en el archivo que las implementa):

```

function postNewPost(iduser, title, description) {
  return API.post('/posts', {
    iduser,
    title,
    description}).then(result => result.data);
}

```

La comprobación de la respuesta del servidor se realiza en la función:

```

// Comprueba la respuesta del servidor
checkPOSTNewPost(res) {
  if (res === "OK"){
    //TODO Muestra un "Modal" cuando el post es añadido
    this.setState(
      {
        title: "",
        description: ""
      }
    );
    this.props.updateMyPosts();
  }else{
    //TODO Muestra un "Modal" cuando ocurre un error al añadir el post
  }
}

```

Si la respuesta del servidor es satisfactoria, actualizaremos las variables de estado `title` y `description` de `AddPost.jsx` a cadenas vacías y, provocaremos la actualización del array de `posts` de `MyPostList.jsx` conteniendo el nuevo post, gracias a la función `updateMyPosts` que se presenta como una propiedad que pasamos a `AddPost.jsx` de la siguiente forma:

```

<AddPost updateMyPosts = {this.handleUpdateMyPosts}/>

```

y que está asociada a la función *bindeada* `handleUpdateMyPosts`, esta vez implementada en `MyPostList.jsx`. De esta forma observamos cómo podemos llamar a funciones de otros componentes padre siempre que estas estén *bindeadas* convenientemente en dicho componente.

4.5.6.2 EDITAR ENTRADAS EXISTENTES

El componente `EditPost.jsx` será el encargado de mostrar los datos de un post seleccionado de la lista de mis posts en un formulario editable, tal y como muestra la Figura 63.

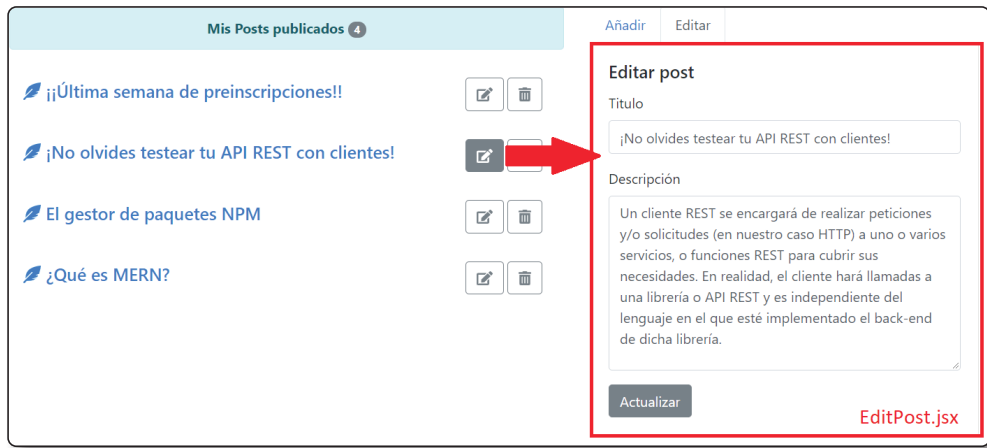


Figura 63. Apariencia del componente EditPost.jsx tras la selección de un post

El código, presentado a continuación, será similar al del componente *AddPost.jsx*:

```
import React, { Component } from 'react';
import { Card, CardTitle, Label, Button, Form, FormGroup, Input } from 'reactstrap';

import { putExistingPost } from "../../utils/apicalls.js";

class EditPost extends Component {
  constructor(props) {
    super(props);
    this.state = {
      title: this.props.post.title,
      description: this.props.post.description
    }
    this.handleTitleChange = this.handleTitleChange.bind(this);
    this.handleDescriptionChange = this.handleDescriptionChange.bind(this);
    this.editPost = this.editPost.bind(this);
  }
  handleTitleChange(e){
    this.setState({title:e.target.value})
  }
  handleDescriptionChange(e){
    this.setState({description:e.target.value})
  }
  editPost(e){
```

```

    e.preventDefault();
    const {
      title,
      description
    } = this.state;
    // Actualiza el post en la base de datos con la llamada a la función put del
api
    putExistingPost(this.props.post._id, title, description)
      .then((res) => this.checkPUTPost(res));
  }
  // Comprueba la respuesta del servidor
  checkPUTPost(res) {
    if (res === "OK"){
      //TODO Muestra un "Modal" cuando el post es actualizado
      this.props.updateMyPosts();
    }else{
      //TODO Muestra un "Modal" cuando ocurre un error al actualizar el post
    }
  }
  componentWillReceiveProps(nextProps){
    this.setState({
      title: nextProps.post.title,
      description: nextProps.post.description});
  }
  render() {
    return (
      <div>
        <Card body>
          <CardTitle tag="h5">Editar post</CardTitle>
          <Form>
            <FormGroup>
              <Label for="aTitulo">Titulo</Label>
              <Input type="text" name="title" id="aTitulo"
placeholder="Introduce un título" value={this.state.title} onChange={this.hand-
leTitleChange} required/>
            </FormGroup>
            <FormGroup>
              <Label for="aDescripcion">Descripción</Label>
              <Input style={{height: '200px'}} type="textarea"
name="description" id="aDescripcion" placeholder="Introduce una descripción"
value={this.state.description} onChange={this.handleDescriptionChange}/>
            </FormGroup>
            <Button onClick={this.editPost}>Actualizar</Button>
          </Form>
        </Card>
      </div>
    );
  }
}

```



```

        </div>
      );
    }
  }

  export default EditPost;

```

Esta vez los valores de título y descripción deberán estar rellenos con los valores `title` y `description` del post seleccionado en la lista procedente de *MyPostList.jsx*. Gracias a la función *bindeada*:

```
<EditPost post= {post} updateMyPosts = {this.handleUpdateMyPosts} />
```

implementada en *MyPostList.jsx*, puedo acceder a los datos del post, ya que lo enviaré a *EditPost.jsx* como propiedad, y también actualizar la lista de posts una vez realizados los cambios, tal y como ocurría al añadir un post gracias a `updateMyPosts` y mediante el *binding* de funciones.

En este caso, cabe destacar la implementación de la función `componentWillReceiveProps(nextProps)` del ciclo de vida del componente *EditPost.jsx*. Gracias a dicha función, se modificarán los valores de las variables del estado del componente cuando se detecte la recepción de nuevas propiedades, es decir, cuando seleccionamos un post diferente de la lista, asignando el valor de las nuevas variables al estado y mostrando estos en el formulario al renderizar nuevamente el componente.

NOTA

Tanto la función `componentDidMount` como la función `componentWillReceiveProps`, y el resto de funciones del ciclo de vida de un componente, han sido sustituidas por los *hooks* de efecto (ver más adelante), en las últimas versiones de React.js.

Al igual que ocurría en el caso de añadir un post, tendremos la implementación del botón editar:

```
<Button onClick={this.editPost}>Actualizar</Button>
```

que ante el evento `onClick`, ejecutará la función *bindeada* `editPost`, cuyo código es el siguiente:

```

editPost(e){
  e.preventDefault();
  const {
    title,
    description
  } = this.state;
  // Actualiza el post en la base de datos con la llamada a la función put del
  api
  putExistingPost(this.props.post._id, title, description)
    .then((res) => this.checkPUTPost(res));
}

```

La función `putExistingPost` está implementada en “utils/apicalls.js” y se encarga de realizar la solicitud de actualización del post al API REST:

```

function putExistingPost(idpost, title, description) {
  return API.put('/posts/'+idpost, {
    title,
    description}).then(result => result.data);
}

```

En este caso, debemos pasar el *id* del post para actualizar el post correspondiente en la base de datos.

Cuando el resultado de la petición es satisfactorio, actualizaremos los posts de la lista tal y como se hace al añadir un post, llamando a `updateMyPosts` que estará asociado a la propiedad correspondiente y a la función del componente padre *MyPostList.jsx*, que se encarga de obtener la lista de posts actualizada.

```

// Comprueba la respuesta del servidor
checkPUTPost(res) {
  if (res === "OK"){
    //TODO Muestra un "Modal" cuando el post es actualizado
    this.props.updateMyPosts();
  }else{
    //TODO Muestra un "Modal" cuando ocurre un error al actualizar el post
  }
}

```

Igualmente, podemos implementar qué ocurriría en caso de obtener una respuesta no satisfactoria.

Con todo ello, tenemos nuestra aplicación de microblogging funcional, la cual podemos desplegar en Heroku, tal y como hicimos con la API REST desarrollada.

NOTA

Al igual que se realizó durante el desarrollo de la API REST, es conveniente sincronizar nuestra aplicación de *front-end* mediante Git y Github a medida que la estamos desarrollando.

4.6 PUESTA EN PRODUCCIÓN DE LA APLICACIÓN REACT.JS

La realización de la puesta en producción y despliegue de la aplicación React.js en un servidor se puede llevar a cabo de la misma forma que se hizo con la API REST. Por ejemplo, gracias a la conexión de Github con la plataforma Heroku, dicha aplicación puede desplegarse en cuestión de minutos. Basta con crear una nueva aplicación en Heroku y conectarla con el repositorio de Github que contiene el código de la aplicación, y asignar las variables de entorno correspondientes si fuera preciso, nuevamente tal y como hicimos en el despliegue de la API REST.

En este caso es recomendable desplegar la aplicación cliente utilizando un *buildpack* correspondiente a aplicaciones React.js, optimizando así su despliegue en un *dyno* de Heroku. Podemos añadir *buildpacks* desde el cliente Heroku por consola de la siguiente forma:

```
>heroku create --buildpack <URL_BUILDPACK>
```

o bien desde la propia web de Heroku. En la Figura 64 se incluye una captura de pantalla, en la cual aparece el *buildpack* `mars/create-react-app`, así como la posibilidad de añadir otros. Podemos gestionar los *buildpacks* desde la aplicación creada en Heroku, en la pestaña “*Settings*”. Puedes conocer más sobre los *buildpacks* en la web de referencia⁶⁶.

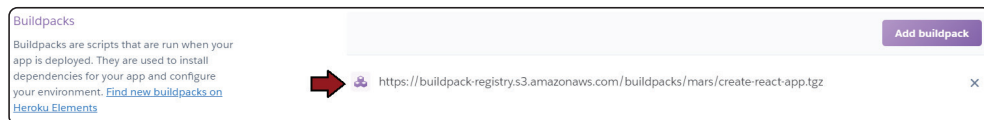


Figura 64. Configuración de buildpacks desde Heroku

66 <https://devcenter.heroku.com/articles/buildpacks> [Último acceso: Julio, 2020]

i NOTA

Pueden existir numerosas fuentes para un mismo *buildpack*. Utiliza cualquier URL válida que apunte al que necesitas y añádelo a tu aplicación para asegurar una puesta en producción con garantías.

A lo largo de este libro, hemos realizado un desarrollo independiente tanto de la API REST como de la aplicación del *front-end*, constituyendo aplicaciones independientes, teniendo que desplegarlas también por separado. Aunque nuestra aplicación es sencilla y toda ella podría haber formado parte de un único proyecto, en realidad, tener la API REST “separada” del *front-end* ofrece diferentes ventajas tanto en el mantenimiento como en la escalabilidad, que podrán ser debatidas fuera del ámbito de este libro.

4.7 CONCEPTOS AVANZADOS

Aunque React.js tiene una curva de aprendizaje asequible en sus inicios, bien es cierto que puede resultar un *framework* bastante denso si se quieren conocer y sacar provecho a todas sus características. En esta parte hemos explicado, de forma general, aquellos conceptos esenciales y necesarios para llevar a cabo el desarrollo de una aplicación *front-end* sencilla pero completa, con el fin de asentar un conocimiento base tanto del propio *framework* como de las aplicaciones web en la parte del cliente.

Sin embargo, cuando nuestra aplicación web tiene demasiados componentes y estos cuentan con estados que cambian constantemente, puede hacerse complejo gestionar todo eso de una forma eficiente, y es ahí donde entran en juego los denominados *reducers* y la librería *redux*.

4.7.1 Reducers y Redux

Redux es una librería que se encarga de cierta manera de desacoplar el estado global de una aplicación web de la parte visual, es decir, de los componentes. Así podremos controlar el estado de nuestras aplicaciones de forma fácil y consistente entre cliente y servidor, y puedan ser a su vez fácilmente “testeables”.

Redux está influenciado por un patrón o arquitectura denominada Flux⁶⁷, que está muy pensada para React.js aunque puede ser usado por otros *frameworks* de desarrollo web.

67 <https://facebook.github.io/flux/> [Último acceso: Julio, 2020]

Redux se basa en 3 principios fundamentales:

- **Una sola fuente de verdad.** Todo el estado de la aplicación está contenido en un único *store*. Todo el estado se almacenará en una estructura arbórea.
- **El estado es de solo lectura.** La única forma de modificar el estado es emitir una acción que indique que cambió.

Así, ninguna parte de la aplicación como pueden ser eventos, *callbacks* o *sockets*, alterará directamente el estado de la aplicación, y en lugar de eso emite una intención de modificarlo.

- **Los cambios se hacen mediante funciones puras.** Para controlar como el *store* es modificado por las acciones se usan *reducers* puros. Los *reducers* son funciones puras que reciben el estado actual de la aplicación y la acción a realizar y devuelven un nuevo estado, sin modificar directamente el estado actual. Podemos tener un solo *reducer* encargado de toda nuestra aplicación o si esta crece dividirlo en múltiples funciones las cuales podemos controlar en qué orden se ejecutan.

De esta forma, evitamos que un componente cambie estados interaccionando con el resto de componentes, algo que en aplicaciones complejas puede suponer no poder seguir el flujo entre componentes a nivel de depuración. En la Figura 65 se muestra el efecto que tiene el uso de Redux en una aplicación basada en componentes.

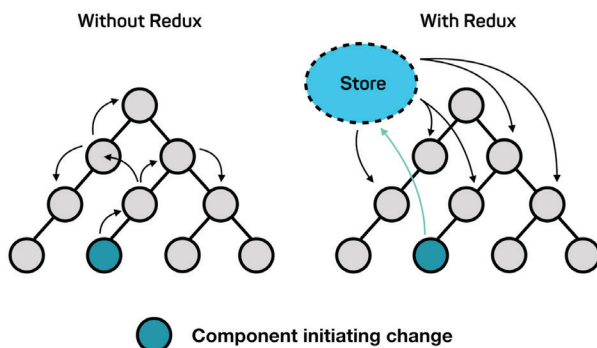


Figura 65. Ejemplo de interacciones entre componentes. Uso de Redux y Árbol de componentes⁶⁸

⁶⁸ Imagen obtenida de: <https://blog.codecentric.de/files/2017/12/Bildschirmfoto-2017-12-01-um-08.53.32.png> [Último acceso: Julio, 2020]

Para usar Redux, lo podemos instalar como un paquete del proyecto vía `npm` con:

```
>npm install --save redux
```

Aunque en este libro no hemos entrado en detalle acerca de la implementación de `reducers` mediante `redux`, sí conviene conocer su existencia y profundizar en ello, sobre todo cuando nuestros proyectos comienzan a albergar un elevado número de componentes. No obstante, en la sección de lecturas recomendadas puedes encontrar diverso material que lo explica en detalle.

Por otra parte, y al hilo del siguiente apartado del libro, las nuevas versiones del *framework* React.js han tratado de paliar también este problema con la inclusión de los React *hooks*. De modo que, hoy en día podemos plantearnos la pregunta de si realmente el uso de librerías como `redux` es necesario, ya que la introducción de los React *hooks* solucionan en gran medida las carencias que tenían los componentes heredados como clases, el comportamiento de estos y la iteración con otros componentes de nuestras aplicaciones.

4.8 INTRODUCCIÓN A LOS HOOKS EN REACT.JS

Ya que conoces los fundamentos de React.js es hora de hablar de uno de los mayores y más profundos cambios que el *framework* ha experimentado hasta el momento. ¿Qué pasaría si en este punto te dijéramos que los componentes React.js ya no se implementan con clases sino con funciones, y que existen unas funciones especiales denominadas *hooks* para manipular los estados y las fases del ciclo de vida de dichos componentes?

Esto nos lleva a cambiar ligeramente la forma en la que implementamos nuestros componentes en React.js, haciéndolos más legibles, simplificados (requerirán menos líneas de código), y optimizados. No obstante, no te preocupes por este cambio, porque todo el conocimiento adquirido en esta parte del libro, y fundamentado en la programación de componentes como clases, te ayudará a comprender, adaptar e implementar cualquier aplicación React.js bajo esta nueva filosofía. Pero ¿qué son realmente los *hooks*?

4.8.1 ¿Qué son los hooks?

Los *hooks* se incorporaron al *framework* React.js a partir de su versión 16.8, y esencialmente son funciones que nos permiten “enganchar” (de ahí la palabra

hook) el estado de React.js y el ciclo de vida desde componentes funcionales. De este modo, los componentes no funcionan dentro de las clases, sino que te permiten usar React.js sin clases. Dada la compatibilidad de esta última versión del *framework* con las anteriores, podrás tener unos componentes React.js implementados con clases y otros usando funciones, de forma que la aplicación mantenga su funcionalidad. No obstante, es aconsejable que, desde este momento, empieces a programar los nuevos componentes como funciones con *hooks* en lugar de hacerlo como hemos hecho hasta ahora, con clases que heredaban de “*React.Component*”. El hecho de usar React *hooks* hace que librerías como *redux* y otras muchas encargadas de gestionar el flujo de estados y propiedades entre clases, cuando teníamos un gran número de estas, tenga menos sentido.

Para ilustrar estos cambios, procedentes de la versión 16.8 de React.js, utilizaremos algunas de las clases implementadas en nuestra aplicación de microblogging.

4.8.2 Clases como funciones

Si recordamos como estaba implementado el componente que representaba la cabecera de nuestra aplicación, este tenía el siguiente código en la clase asociada:

```
.....
class HeaderApp extends Component {
  render() {
    return (
      <Container>
        <Row>
          <Col>
            <Navbar color="dark" light expand="md">
              <NavbarBrand><h4 className="text-warning">Mi Aplicación de Mi-
croblogging</h4></NavbarBrand>
              <Nav className="ml-auto" navbar>
                <NavItem>
                  <NavLink href="http://jesusfontecha.name/mern" target="_
blank">
                    <span className="text-white">Curso Desarrollo web full-
stack con MERN <AiOutlineCopyright /></span><Media style={imgStyle} object
src={MyPlaceholderPicture} alt="Desarrollo web MERN" /></NavLink>
                  </NavItem>
                </Nav>
              </Navbar>
            </Col>
          </Row>
```

```

        </Container>
      );
    }
  }
  export default HeaderApp;

```

Como se puede apreciar, es una clase que hereda de “Component”, definido en la librería **react** y que debíamos importar adecuadamente. Y, por otra parte, teníamos la función **render**, que renderiza la vista del propio componente. Pues bien, en el caso de reescribir esa clase utilizando funciones, esta tendrá el siguiente aspecto:

```

export default function HeaderApp(){
  return(
    <Container>
      <Row>
        <Col>
          <Navbar color="dark" light expand="md">
            <NavbarBrand><h4 className="text-warning">Mi Aplicación de Micro-
blogging</h4></NavbarBrand>
            <Nav className="ml-auto" navbar>
              <NavItem>
                <NavLink href="http://jesusfontecha.name/mern" target="_blank">
                  <span className="text-white">Curso Desarrollo web full-
stack con MERN <AiOutlineCopyright /></span><Media style={imgStyle} object
src={MyPlaceholderPicture} alt="Desarrollo web MERN" /></NavLink>
                </NavItem>
              </Nav>
            </Navbar>
          </Col>
        </Row>
      </Container>
    );
  }

```

Como se puede apreciar, ya no es necesario importar “Component” puesto que no vamos a heredar de dicha clase, y tampoco incluir el renderizado de la vista en una función **render** de la clase (¡ya no hay clase!). Ahora nuestro componente es una función (exportable) con el mismo código de vista que en el caso anterior. Ambos componentes son compatibles siempre que utilicemos la versión del *framework* React.js 16.8 o superior.

Por otra parte, recuerda que, en nuestra aplicación, teníamos componentes con variables de estado las cuales podían cambiar el comportamiento del componente cuando este se renderizaba. Eso ahora se va a manejar con los denominados *hooks* de estado.

4.8.3 Hooks de estado

Son funciones especiales incorporadas a partir de React.js 16.8 para manipular el estado de un componente. Concretamente este *hook* se debe importar dónde vayamos a hacer uso de él y recibe el nombre de “useState”:

```
.....  
import React, { useState } from 'react';  
.....
```

React.js mantendrá este estado entre re-renderizados del componente. Habitualmente `useState` devuelve un par: el valor de estado actual y una función que le permite actualizarlo. De esta forma, podemos llamar a esta función desde un controlador de eventos o desde otro lugar. Es similar a `this.state` en una clase, excepto que no combina el estado antiguo y el nuevo. Además, el único argumento para `useState` es el estado inicial, que solo se usará durante el primer renderizado del componente.

A continuación, podemos ver cómo se declaraban (en el constructor de la clase) las variables de estado usando componentes como clases (este es el caso del componente de nuestra aplicación “Signup.jsx”):

```
.....  
this.state = {  
  username: '',  
  password: '',  
  fullname: '',  
  email: '',  
  role: 'subscriber'  
};  
.....
```

Además, teníamos que “bindear” dicho estado con las funciones correspondientes para permitir la actuación ante eventos ocurridos en la vista. En el caso de `useState` esto ya no es necesario (ni siquiera usar la palabra reservada “this”), y solo tenemos que preocuparnos de declarar las variables de estado de esta forma (recordad el par “valor, función”), dentro de la función que representa nuestro componente:

```

const [username, setUsername] = useState('');
const [password, setPassword] = useState('');
const [fullname, setFullname] = useState('');
const [email, setEmail] = useState('');
const [role, setRole] = useState('subscriber');

```

Así, cuando necesitemos cambiar el valor de una variable de estado haremos uso de la función definida (por convenio con el formato “setEstado”). A continuación, se muestra el código completo del componente “Signup.jsx” de nuestra aplicación, empleando estos *hooks* y la sintaxis ES6 de *JavaScript*.

```

import React, { useState } from 'react';
import { Button, Form, FormGroup, Label, Input, Container, Row, Col, Card, Card-
Title } from 'reactstrap';

import { Link } from 'react-router-dom';
import { postNewUser } from "../../utils/apicalls.js";

export default function HeaderApp(props){

  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [fullname, setFullname] = useState('');
  const [email, setEmail] = useState('');
  const [role, setRole] = useState('subscriber');

  const onSignup = (e) => {
    //Post a new user in the database
    postNewUser(username, password, fullname, email, role)
      .then((res) => checkPOSTNewUser(res));
  }

  //Check the response from the server
  const checkPOSTNewUser = (res) => {
    //if response is OK: create a new user and redirect to login page
    if (res === "OK")
      props.history.push("/");
    else{
      //TODO Show Modal error adding a new user
    }
  }

  return (

```

```

<Container>
  <Row>
    <Col sm="12" md={{ size: 6, offset: 3 }}>
      <Card body>
        <CardTitle tag="h4">Registro de nuevo usuario</CardTitle>
        <Form>
          <FormGroup>
            <Label for="aUsername">Username</Label>
            <Input type="text" name="username" id="aUsername"
placeholder="Introduce tu username" onChange={(e) => setUsername(e.target.va-
lue)} required/>
          </FormGroup>
          <FormGroup>
            <Label for="aPassword">Password</Label>
            <Input type="password" name="password" id="aPassword"
placeholder="Introduce tu password" onChange={(e) => setPassword(e.target.va-
lue)} required/>
          </FormGroup>
          <FormGroup>
            <Label for="aFullname">Nombre completo</Label>
            <Input type="text" name="fullname" id="aFullname"
placeholder="Introduce tu nombre completo" onChange={(e) => setFullname(e.tar-
get.value)} />
          </FormGroup>
          <FormGroup>
            <Label for="aEmail">Email</Label>
            <Input type="email" name="email" id="aEmail"
placeholder="Introduce tu email" onChange={(e) => setEmail(e.target.value)} re-
quired/>
          </FormGroup>
          <FormGroup>
            <Label for="aRole">Rol</Label>
            <Input type="select" name="role" id="aRole" onChange={(e) =>
setRole(e.target.value)}>
              <option value="subscriber">Suscriptor</option>
              <option value="admin">Administrador</option>
            </Input>
          </FormGroup>
          <Button onClick={onSignup}>Registro</Button>
        </Form>
      </Card>
    <Row>
      <Col tag="center">
        <Link to="/"><strong className="text-muted">Login</strong></Link>
      </Col>

```

```
        </Row>
      </Col>
    </Row>
  </Container>
);
}
```

Como podemos observar, el código del componente se simplifica y mejora su comprensión.

Por último, y tomando como base este componente presentado, observamos cómo podemos hacer uso de las propiedades (**props**) sin necesidad de tomarlas del componente padre. En este caso, basta con pasarlas como argumento de la función principal de nuestro componente.

4.8.4 Hooks de efecto

Al igual que ocurre con los *hooks* de estado, a partir de React.js 16.8 también tenemos los denominados *hooks* de efecto, caracterizados por la función “*useEffect*”, la cual debemos importar de igual forma que antes:

```
import React, { useState, useEffect } from 'react';
```

Estos *hooks* de efecto agregan la capacidad de realizar efectos secundarios desde un componente funcional, que pueden afectar a otros componentes y no se pueden hacer durante el renderizado. De hecho, esta operación tiene el mismo propósito que las funciones del ciclo de vida “*componentDidMount*”, “*componentDidUpdate*” y “*componentWillUnmount*” entre otras, pero unificadas en una única función.

Un ejemplo de uso de estos *hooks* de efecto tendría lugar en el caso de la recuperación de datos, para la que anteriormente nos servíamos de la función “*componentDidMount*”, que se ejecutaba previa renderización del componente. En nuestra aplicación, lo que la función anterior implementaba era la llamada al servicio correspondiente, que mediante solicitudes HTTP a un API REST y la librería **axios**, devolvía, por ejemplo, una serie de datos que se inicializaban en una variable de estado. A continuación, se muestran partes de código correspondientes al componente “PostList.jsx” original:

```
import React, { Component } from 'react';
...
```

```
import { getAllPosts } from "../../utils/apicalls.js";
...
class PostList extends Component {
  state = {
    posts: []
  };

  getPosts() {
    getAllPosts().then((posts) => {
      this.setState({
        posts
      });
    });
  }

  componentDidMount() {
    this.getPosts();
  }

  render() {
    return (...);
  }
}
export default PostList;
```

En el caso de reescribir dicho componente, y esa función “*componentDidMount*”, utilizando React *hooks*, tendríamos algo como lo siguiente:

```
import React, { useState, useEffect } from 'react';
...
import { getAllPosts } from "../../utils/apicalls.js";
...
export default function PostList(){

  const [posts, setPosts] = useState([]);

  const getPosts = () => {
    getAllPosts().then((posts) => {
      setPosts(posts);
    });
  }

  useEffect(() =>{
```

```
    getPosts();  
  },[]);  
  
  return (...);  
}
```

Para la variable de estado, utilizamos el *hook* de estado `useState` visto anteriormente, inicializando la variable de estado `posts` a un array vacío. Y usando el *hook* de efecto `useEffect`, nos traeremos los datos JSON de los `posts` gracias al servicio REST correspondiente, y los asignaremos a la variable de estado `posts` (con la función definida “`setPosts()`” como parte del estado). Para evitar crear un bucle infinito de renderizado y hacer el renderizado únicamente cuando se precisa, incluiremos como segundo argumento “`[]`”.

Por otra parte, puede existir la posibilidad de usar un efecto únicamente cuando se observa una actualización de algunas variables de estado o propiedades (que pueden proceder de otro componente). En este caso, el segundo argumento de la función `useEffect` se encargará de controlar esa circunstancia, de forma que solo se ejecute el efecto cuando ocurra una actualización en los estados y/o propiedades que le indiquemos en ese segundo argumento.

Para ejemplificar esto último, recurriremos al código original de la función “`componentWillReceiveProps(nextProps)`” del componente “`EditPost.jsx`”:

```
componentWillReceiveProps(nextProps){  
  this.setState({  
    title: nextProps.post.title,  
    description: nextProps.post.description});  
}
```

En este caso, cada vez que seleccionábamos un nuevo post en el componente padre, el componente `EditPost` asociaba las nuevas propiedades a estados de dicho componente. Para hacer esto usando React *hooks*, lo que haremos será lo siguiente:

```
useEffect(() =>{  
  setTitle(props.post.title);  
  setDescription(props.post.description);  
},[props.post]);
```

Como vemos en el efecto, asignamos el título del post que nos llega en las propiedades y como nuevo valor de la variable de estado correspondiente, y lo mismo ocurre con la descripción. También vemos como el segundo argumento del efecto es `[props.post]`, lo que nos indica que solo ejecutará dicho efecto cuando observe un cambio en dicha variable de las propiedades. Nuevamente esto es un ejemplo de optimización de React *hooks* con respecto a trabajar con clases.

NOTA

En el Apéndice A. Código fuente se encuentran disponibles las URLs a los repositorios Github con el código fuente de las aplicaciones desarrolladas en el presente libro, incluyendo la aplicación React.js refactorizada utilizando React *hooks*.

4.8.5 Reglas y hooks personalizados

Como ya sabemos, los *hooks* son funciones JavaScript, pero en este caso se imponen dos reglas adicionales:

- Únicamente deben **llamarse en el nivel superior**; es decir, no se pueden realizar llamadas a *hooks* dentro de bucles, condiciones o funciones anidadas.
- Deben **llamarse desde componentes funcionales de React.js**; es decir, no llames a *hooks* desde las funciones regulares de JavaScript. Tan solo hay otro lugar válido para llamar *hooks*: tus propios *hooks* personalizados.

Actualmente existen *plugins* que podemos instalar con el gestor de paquetes npm y que posibilitan forzar el uso de estas reglas automáticamente⁶⁹.

Por otra parte, implementando los componentes como clases, cuando queremos reutilizar alguna lógica de estado entre componentes tenemos dos soluciones: crear un componente de orden superior (HOC) y las “*render props*” (que permiten compartir código entre componentes utilizando una propiedad cuyo valor es una función). Mediante el uso de *hooks* personalizados podemos hacer esto mismo, pero sin agregar más componentes al árbol de componentes de nuestra aplicación.

69 <https://www.npmjs.com/package/eslint-plugin-react-hooks> [Último acceso: Julio, 2020]

Imagina que tenemos dos componentes que entre otras cosas implementan una misma funcionalidad en su interior, como por ejemplo averiguar si un usuario está online dentro de una plataforma web. En lugar de duplicar la lógica de dicha funcionalidad en ambos componentes, podríamos crear un *hook* personalizado que implemente esta lógica, el cual podría embeber tanto *hooks* de estado como de efecto. En la sección de lecturas recomendadas puedes encontrar más información al respecto.

No obstante, existen otros *hooks* menos comunes, incorporados en las últimas versiones del *framework* React.js, pero que en determinados casos pueden resultar útiles, como por ejemplo “useContext”, “useReducer”, etc. Puedes conocer más acerca de estas y otras funciones en la documentación oficial de React.js.

NOTA

Debido a la constante evolución de las tecnologías de desarrollo web, es recomendable estar al tanto de las novedades sobre distintas versiones y/o especificaciones de lenguajes, *frameworks*, entornos y buenas prácticas que la comunidad de desarrollo pone a disposición de los programadores, a través de los canales y webs oficiales.

4.9 LECTURAS RECOMENDADAS

En la actualidad, existen multitud de recursos muy interesante para el desarrollo, tanto básico como avanzado, de aplicaciones de *front-end* con React.js. Habitualmente estas fuentes incluyen todo lo que rodea a dicho *framework*, como la sintaxis JSX, la interacción con el *back-end*, el manejo de la autenticación, el *binding* de funciones, el uso de Redux, etcétera. A continuación, te presentamos algunas de las referencias, adicionalmente a lo contenido en la web oficial de React.js, que conviene revisar si se desea profundizar en el desarrollo de aplicaciones con React.js:

- Accomazzo, A., Lerner, A., Allsopp, C., Guttman, D., McGinnis, T., Murray, N. *FullStack React: The Complete Guide to ReactJS and Friends*. Fullstack.io, 2020.
- Wieruch, R. *The Road to React: Your journey to master plain yet pragmatic React.js*. Publicación independiente, 2018.
- Altadill, P. X. *Desarrollo web con React*. Anaya Multimedia, 2019.
- Freeman, A. *Pro React 16*. Apress, 2019.

- Fernández, G. *¿Por qué es necesario usar bind en los evento listener de los components?*, 2018. Disponible en: <https://medium.com/@ger86/por-qu%C3%A9-es-necesario-usar-bind-en-los-event-listener-de-los-components-d2f3a5944f4c> [Último acceso: Julio, 2020]

También nos gustaría recomendarte algunas referencias sobre la creación de aplicaciones web usando toda la pila de tecnologías MERN, como complemento a lo presentado en este libro:

- Subramanian, V. *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node*. Apress, 2017.
- Iriarte, E. W. *MERN Quick Start Guide: Build web applications with MongoDB, Express.js, React, and Node*. Packt Publishing, 2018.
- Hoque, S. *Full-Stack React Projects: Modern web development using React 16, Node, Express, and MongoDB*. Packt Publishing, 2018.

Finalmente, la tendencia a la programación de componentes React.js mediante el uso de React *hooks*, supone una mejora también de la simplicidad y optimización de las aplicaciones web React.js. En este punto, te recomendamos echar un vistazo a los siguientes recursos:

- ReactJS en Español. *Un vistazo a los hooks*. Disponible en: <https://es.reactjs.org/docs/hooks-overview.html> [Último acceso: Julio, 2020].
- Bugl, D. *Learn React Hooks: Build and refactor modern React.js applications using Hooks*. Packt Publishing, 2019.
- Larsen, J. *React Hooks in Action: With Suspense and Concurrent Mode*. Manning Publications, 2020.

Apéndice A

CÓDIGO FUENTE

El código fuente, asociado tanto a la API REST como a la aplicación React.js de microblogging, desarrolladas en este libro, se encuentra disponible en repositorios públicos de uno de los autores. A continuación, tienes disponibles los enlaces a los mismos:

- **Código de la API REST**

<https://github.com/jfontecha/microblogging-example-api>

- **Código de la aplicación React.js (desarrollada con clases)**

<https://github.com/jfontecha/microblogging-example-react-classes>

- **Código de la aplicación React.js (desarrollada con React *hooks*)**

<https://github.com/jfontecha/microblogging-example-react-hooks>

Apéndice B

RECETARIOS

Este apéndice incluye diversos recetarios, para que el lector tenga siempre a mano las utilidades, comandos, sentencias y expresiones más comunes, relacionadas con lo presentado en este libro. Los recetarios aquí expuestos, son versiones en español inspiradas de los “*cheat sheets*” originales, procedentes de:

- ▼ <https://cheatography.com/>
- ▼ <https://services.github.com/>
- ▼ <https://devhints.io/react>

JAVASCRIPT. EXPRESIONES Y OPERACIONES BÁSICAS

Sintaxis de las Expresiones Regulares

[^]	Comienzo de una cadena/string
^{\$}	Final de una cadena/string
[.]	Cualquier carácter simple
(a b)	a o b
(...)	Agrupamiento
[abc]	Cualquier carácter de (a, b o c)
[^abc]	Cualquier carácter no (a, b o c)
\s	Espacio o blanco
a?	Ninguna o una a
a*	Ninguna o más aes
a+	Una o más aes
a{3}	Exactamente 3 aes
a{3,}	Tres o más aes
a{,6}	Hasta seis aes
a{3,6}	Entre tres y seis aes
\	Carácter de escape
[punct.]	Cualquier símbolo de puntuación
[space]	Cualquier carácter de espacio
[blank]	Espacio o tabulador
Probar expresiones regulares:: http://regexpal.com/	

Modificadores

g	Búsqueda global de coincidencias
i *	Búsqueda ignorando mayúsculas
m *	Búsqueda en múltiples líneas
s *	Tratar string como línea única
x *	Permite comentarios y espacios en el patrón
e *	Evaluar el reemplazo

Cadenas en JavaScript

charAt()	slice()
charCodeAt()	split() x
concat()	substr()
fromCharCode()	substring()
indexOf()	toLowerCase()
lastIndexOf()	toUpperCase()
length	toLocaleLowerCase()
localeCompare()	toLocaleUpperCase()
match() x	toSource()
replace() x	valueOf()
search() x	
Los métodos para cadenas con una x soportan expresiones regulares	

Funciones JavaScript

decodeURI()	isNaN()
decodeURIComponent()	Number()
encodeURI()	parseFloat()
encodeURIComponent()	parseInt()
escape()	String()
eval()	unescape()
isFinite()	

Objetos JavaScript para Expr. Regulares

compile()	lastParen
exec()	leftContext
global	multiline
ignoreCase	rightContext
input	source
lastIndex	test()
lastMatch	

Manejadores de eventos JavaScript

onabort	onmousedown
onblur	onmousemove
onchange	onmouseout
onclick	onmouseover
ondblclick	onmouseup
ondragdrop	onmove
onerror	onreset
onfocus	onresize
onkeydown	onselect
onkeypress	onsubmit
onkeyup	onunload
onload	

Booleanos en JavaScript

toSource()	valueOf()
toString()	

Fecha y Hora en JavaScript

getUTCHours()	toString()
getUTCMinutes()	toGMTString()
getUTCSeconds()	toUTCString()
getUTCMilliseconds()	toLocaleString()
parse()	UTC()
setDate()	valueOf()

Utilidades para Arrays JavaScript

concat()	slice()
join()	sort()
length	splice()
pop()	toSource()
push()	toString()
reverse()	unshift()
shift()	valueOf()

Números y Matemáticas en Javascript

abs()	min()
acos()	NEGATIVE_INFINITY
asin()	PI
atan()	POSITIVE_INFINITY
atan2()	pow()
ceil()	random()
cos()	round()
E	sin()
exp()	sqrt()
floor()	SQRT1_2
LN10	SQRT2
LN2	tan()
log()	toSource()
LOG10E	toExponential()
LOG2E	toFixed()
max()	toPrecision()
MAX_VALUE	toString()
MIN_VALUE	valueOf()
NaN	

Fecha y Hora en JavaScript (cont.)

Date()	setMonth()
getDate()	setFullYear()
getDay()	setHours()
getMonth	setMinutes()
getFullYear	setSeconds()
getYear	setMilliseconds()
getHours	setTime()
getMinutes	setUTCDate()
getSeconds	setUTCDay()
getMilliseconds	setUTCMonth()
getTime	setUTCFullYear()
getTimezoneOffset()	setUTCHours()
getUTCDate()	setUTCMinutes()
getUTCDay()	setUTCSeconds()
getUTCMonth()	setUTCMilliseconds()
getUTCFullYear()	toSource()

MONGODB. OPERACIONES CRUD Y FILTROS

Básico	
db	
mongod	
mongo	
show dbs	
use db	
show collections	

Creación	
insert(data)	Insertar documento/s, devuelve el resultado de la escritura
insertOne (data, options)	Inserta un documento
insertMany(data, options)	Inserta varios documentos
insertMany([{}], {}, {})	Requiere corchetes

Lectura	
db.collection.find()	Muestra documentos de collection
find(filter, options)	Encuentra todos los documentos coincidentes
findOne(filter, options)	Encuentra el primer documento coincidente

Borrado	
deleteOne(filter, options)	Borra un documento
deleteMany(filter, options)	Borra varios documentos

Filtros	
{ "key": "value" }	Se utiliza para los argumentos que filtran la colección
{key: { \$operator: value }}	Operadores para la consulta de datos
{key: { \$exists: true }}	Coincide con todos los documentos que contienen la clave key

Filtros (cont)	
\$eq	Coincide con los valores iguales a uno dado
\$gt	Coincide con los valores mayores que uno dado
\$gte	Coincide con los valores mayores o iguales que uno dado
\$in	Coincide con cualquiera de los valores especificados en un array
syntax:	{key: { \$in: [array of values] } }
\$lt	Coincide con los valores menores que uno dado
\$lte	Coincide con los valores menores o iguales que uno dado
\$ne	Coincide con los valores no iguales que uno dado
\$nin	No coincide con ninguno de los valores especificados en un array.
\$and	Realiza una operación AND
syntax:	{ \$and: [{ }, { }] }
{key: { \$op: filter }, {filter} }	el operador \$and es necesario cuando el mismo campo u operador tiene que ser especificado en múltiples expresiones
find({doc.subdoc:value})	Filtrar subdocuments

Funciones	
.count()	Contabiliza el número de resultados
.sort(filter)	Ordena ascendente: 1 descendente: -1

Actualización	
updateOne(filter, data, options)	Cambia un documento
updateMany(filter, data, options)	Cambia varios documentos
replaceOne(filter, data, options)	Reemplaza un documento entero

NODE PACKAGE MANAGER. COMANDOS BÁSICOS

Links

Instalar node y NPM	https://nodejs.org/en/download/
Configurar Proxy	http://stackoverflow.com/questions/7559648/is-there-a-way-to-make-npm-install-the-command-to-work-behind-proxy
Documentación NPM	http://docs.npmjs.com

Instalar Paquetes

Instalar un paquete y actualizar package.json con el nombre del paquete y la versión	<code>npm install <package name> --save</code>
Instalar la última versión del paquete – no actualizar package.json	<code>npm install <package name></code>
Instalar una versión específica del paquete	<code>npm install <package name>@<version-number></code>
Instalar un paquete de tipo "scoped"	<code>npm install @<namespace>/<package name></code>
Instalar en dependencias dev	<code>npm install <package name> --save-dev</code>
Instalar paquete de manera global	<code>npm install <package name> -g</code>
Instalar desde GitHub	<code>npm install <GitHub-URL></code>
Instalar desde carpeta (para paquetes privados)	<code>npm install <path></code>
Restaurar los paquetes definidos en package.json	<code>npm install</code>
Encontrar el Directorio Global de Instalación	<code>npm config get prefix</code>

Desinstalar Paquetes

Desinstalar paquete y quitar de package.json	<code>npm uninstall <package name> --save</code>
Desinstalar paquete	<code>npm uninstall <package name></code>
Desinstalar paquete de manera global	<code>npm uninstall -g <package name></code>
Quitar paquetes no usados	<code>npm prune</code>
Limpiar la NPM Cache	<code>npm cache clean</code>

Listado de Paquetes

Listar paquetes usados por tu aplicación (sin dependencias)	<code>npm ls --depth 0</code>
Listar paquetes usados por tu aplicación (con dependencias)	<code>npm ls</code>
Listar paquetes instalados globalmente (sin dependencias)	<code>npm ls -g --depth 0</code>
Listar paquetes y dependencias instalados globalmente	<code>npm ls -g true</code>
Listar dependencias dev	<code>npm ls --dev true</code>
Listar dependencias prod	<code>npm ls --prod true</code>
Mostrar la última versión de un paquete en npm	<code>npm view <package name> version</code>
Mostrar la versión actual de un paquete npm local	<code>npm show <package name> version</code>

Ayuda

Obtener ayuda general	<code>npm -h</code>
Obtener ayuda sobre un comando	<code>npm <command-name> -h</code>
Ir a la página de ayuda sobre un comando	<code>npm help <command-name></code>
Ir al repositorio	<code>npm repo <package name></code>

CÓDIGOS DE ESTADO HTTP

1xx: HTTP Códigos de Información	
100	Continuar
101	Cambio de Protocolos
102	Procesando petición (WebDAV)
103	Checkpoint (PUT, POST)
122	Solicitud-URI demasiado larga

2xx: HTTP Códigos de Éxito	
200	OK
201	Creado
202	Aceptado
203	Información no autorizada
204	No contenido
205	Resetear contenido
206	Contenido parcial
207	Respuesta Multi-Status (WebDAV 4918)
208	Notificado previamente (WebDAV 5842)
226	IM Usado 3229 GET

3xx: HTTP Códigos de Redirección	
300	Opciones múltiples para el URI
301	Movido permanentemente
302	Encontrado
303	Ver otro
304	No modificado
305	Acceso a través de Proxy
306	Seleccionar Proxy
307	Redirección temporal
308	Redirección permanente

307 y 308 son similares a 302 y 301, pero el nuevo método HTTP usado en la solicitud después de la redirección debe ser el mismo que en la solicitud inicial.

4xx: HTTP Códigos de Error (Cliente)	
400	Solicitud errónea
401	No autorizado
402	Pago requerido
403	Prohibido
404	No encontrado
405	Método no permitido
406	No aceptable
407	Se requiere Autenticación Proxy
408	Tiempo de Espera Superado
409	Conflicto
410	Ya no disponible
411	Longitud requerida
412	La condición previa falló
413	Entidad de solicitud demasiado grande
414	Solicitud-URI demasiado larga
415	Tipo de medio no soportado
416	Rango solicitado no satisfactorio
417	Requerimientos de la cabecera Expect no se cumplen
418	"Soy una tetera"
422	No procesable por errores semánticos (WebDAV 4918)
423	Recurso bloqueado (WebDAV 4918)
424	Dependencia fallida (WebDAV 4918)
425	Demasiado temprano
426	Actualización requerida
428	Se requiere petición condicional
429	Demasiadas solicitudes/conexiones
431	Campos de la cabecera de la solicitud demasiado largos
444	Sin respuesta
449	Reintentar con
450	Bloqueado por control parental de Windows
451	No disponible por motivos legales
499	Conexión de cliente cerrada, durante el procesamiento

5xx: HTTP Códigos de Error (Servidor)	
500	Error interno del servidor
501	No implementado
502	Error de Gateway
503	Servicio no disponible
504	Límite de tiempo del Gateway alcanzado
505	Versión HTTP no soportada
506	Petición con referencias circulares
507	Almacenamiento insuficiente (WebDAV 4918)
508	Bucle detectado (WebDAV 5842)
509	Excedido límite de Ancho de Banda
510	Extensiones adicionales requeridas
511	Autenticación de red requerida
598	Error de tiempo de espera bajo proxy
599	Error de límite de tiempo de conexión a red

HTTP Comentarios sobre el código	
WebDAV	Extensión WebDAV
1.1	HTTP/1.1
GET, POST, PUT, POST	Sólo para estos métodos HTTP
IE	Extensión IE
MS	Extensión MS
nginx	Extensión nginx
2518, 2817, 2295, 2774, 3229, 4918, 5842	Número RFC
draft	Proposición de borrador
nostd	Extensión no estándar
res	Reservado para uso futuro
unused	No más en uso (deprecated)

EXPRESS.JS. CONFIGURACIONES, SOLICITUDES Y RESPUESTAS

Configuración

```
app.set('x', 'yyy')
app.get('x') //=> 'yyy'
app.enable('trust proxy')
app.disable('trust proxy')
app.enabled('trust proxy') //=> true
```

Respuesta

```
res.redirect('/')
res.redirect(301, '/')
res.set('Content-Type', 'text/html')
res.send('hi')
res.send(200, 'hi')
res.json({ a: 2 })
```

Petición

```
// GET /user/tj
req.path //=>
"/user/tj" req.url //=>
"/user/tj" req.xhr //=>
true|false req.method
//=> "GET" req.params
req.params.name //=> "tj"
req.params[0]
// GET /search?q=tobi+ferret
req.query.q // => "tobi ferret"
req.cookies
req.accepted
[ { value: 'application/json', quality: 1, type:
  'application', subtype: 'json' },
  { value: 'text/html', quality: 0.5, type:
    'text', subtype: 'html' } ]
req.is('html')
req.is('text/html')
req.headers
req.headers['host']
req.headers['user-agent']
req.headers['accept-encoding']
req.headers['accept-language']
```

Env

```
app.get('env')
```

Configuración

```
app.configure('production', function() {
  app.set...
});
```

Wares

```
app.use(express.static(_dirname + '/public'));
app.use(express.logger());
```

Helpers

```
app.locals({
  title: "MyApp",
});
```

GIT Y GITHUB. COMANDOS BÁSICOS

Instalación

- GitHub en Windows**
<https://windows.github.com>
- GitHub en Mac**
<https://mac.github.com>
- Git for All Platforms⁷⁰**
<http://git-scm.com>

⁷⁰ Las distribuciones para Linux y sistemas POSIX están disponibles en el sitio Web oficial de Git SCM.

Configuración de Git

Configurar la información del usuario para todos los repositorios locales.

```
$ git config --global user.name "[name]"
```

Indica el nombre (“name”) que quieres adjuntar a tus transacciones de commits

```
$ git config --global user.email "[email address]"
```

Indica el “email” que quieres adjuntar a tus transacciones de commits

```
$ git config --global color.ui auto
```

Habilita el uso color de ayuda en la línea de comandos

Branches/ramas

Las ramas son parte importante del trabajo con Git. Cualquier **commit** que hagas se realizará en la rama que tienes seleccionada. Se puede usar **git status** para ver cuál es la rama seleccionada.

```
$ git branch [branch-name]
```

Crea una nueva rama

```
$ git checkout [branch-name]
```

Cambia a una rama específica y actualiza el directorio de trabajo

```
$ git merge [branch]
```

Combina el historial de una rama específica “branch” con la rama actual

```
$ git branch -d [branch-name]
```

Borra una rama específica

Hacer cambios

Navegar e inspeccionar la evolución de archivos de proyecto.

```
$ git log
```

Lista versiones del historial para la rama actual

```
$ git log -follow [archivo]
```

Lista versiones del historial para un archivo, incluyendo sus cambios de nombre

```
$ git diff [a-branch]...[b-branch]
```

Muestra las diferencias de contenido entre dos ramas

```
$ git show [commit]
```

Muestra los metadatos y cambios en el contenido de un commit específico

```
$ git add [archivo]
```

Capturas del archivo en preparación para control de versión

```
$ git commit -m "[Mensaje descriptivo]"
```

Almacena capturas/snapshot del archivo permanentemente

El archivo .gitignore

Algunas veces es una buena idea excluir archivos del tracking de Git. Esto es típicamente hecho en el archivo específico **.gitignore**

Puedes encontrar plantillas de ejemplo de **.gitignore** en github.com/github/gitignore.

Sincronizar cambios

Sincroniza tu repositorio local con un repositorio remoto de GitHub.com

\$ git fetch

Descarga todo el historial de ramas remotas en *tracking*/seguimiento

\$ git merge

Combina una rama remota en seguimiento en la rama local actual

\$ git push

Sube todos los commits de la rama local a GitHub

\$ git pull

Actualiza la rama local actual de trabajo con todos los nuevos commits de la correspondiente rama remota en GitHub. **git pull** es una combinación de **git fetch** y **git merge**

Rehacer commits

Borra errores y realiza reemplazos en el historial de versiones

\$ git reset [commit]

Deshace todos los commits después de “commit”, preservando los cambios localmente

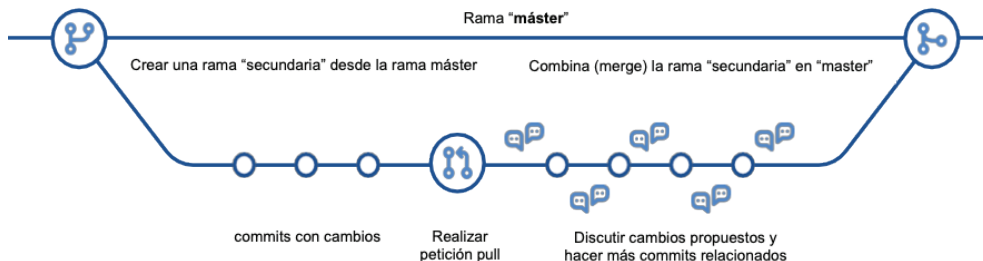
\$ git reset --hard [commit]

Descarta todo el historial y vuelve al “commit” específico

CUIDADO

Cambiar el historial puede tener efectos no deseados. Si se necesita cambiar commits que existen en GitHub (repos remotos), se debe proceder con precaución. Si necesitas ayuda ponte en contacto con [github.community](https://github.com/community)

Flujo en GitHub



Glosario

- **Git:** sistema de control de versiones distribuido y open source
- **GitHub:** una plataforma para el almacenamiento y trabajo colaborativo sobre repositorios Git
- **commit:** se trata de un objeto Git, una captura/snapshot del repositorio completo, comprimida en SHA
- **branch/rama:** un puntero ligero y desplazable hacia un commit
- **clone:** una versión local de un repositorio, incluyendo todos sus commits y ramas
- **remoto:** un repositorio común en GitHub que todos los miembros de un equipo usan para compartir su trabajo
- **fork:** una copia de un repositorio en GitHub, cuyo propietario es un usuario diferente
- **pull request:** un lugar en el que comparar y discutir sobre las diferencias introducidas en una rama/branch, mediante comentarios, reviews, tests de integración, etc.
- **HEAD:** directorio de trabajo actual, el puntero HEAD puede moverse a diferentes ramas, etiquetas/tags o commits cuando se usa.

REACT.JS. FUNDAMENTOS Y COMPONENTES

Componentes

```
import React from 'react'
import ReactDOM from 'react-dom'

class Hello extends React.Component {
  render () {
    return <div className='message-box'>
      Hello {this.props.name}
    </div>
  }
}

const el = document.body
ReactDOM.render(<Hello name='John' />, el)
```

- *. Usa el jsfiddle de React.js para probar código (o jsbin)

Estados / States

```
constructor(props) {
  super(props)
  this.state = { username: undefined }
}

this.setState({ username: 'rstacruz' })

render () {
  this.state.username
  const { username } = this.state
  ...
}
```

- *. Usa estados (this.state) para gestionar datos dinámicos. Con Babel a su vez puedes usar “campos de propuesta” y no implementar el constructor

```
class Hello extends Component {
  state = { username: undefined };
  ...
}
```

Hijos

```
<AlertBox>
  <h1>You have pending notifications</h1>
</AlertBox>

class AlertBox extends Component {
  render () {
    return <div className='alert-box'>
      {this.props.children}
    </div>
  }
}
```

*. Los hijos se pasan a través de la propiedad children.

Fijar propiedades por defecto

```
Hello.defaultProps = {
  color: 'blue'
}
```

Componentes funcionales

```
function MyComponent ({ name }) {
  return <div className='message-box'>
    Hello {name}
  </div>
}
```

*. Los componentes funcionales no tienen estado. Además, sus propiedades se pasan como el primer parámetro de la función

Importación múltiple

```
import React, {Component} from 'react'
import ReactDOM from 'react-dom'

class Hello extends Component {
  ...
}
```


Propiedades

```
<Video fullscreen={true} autoplay={false} />

render () {
  this.props.fullscreen
  const { fullscreen, autoplay } = this.props
  ...
}
```

- *. Usa `this.props` para acceder a propiedades que han sido pasadas al componente.

Anidado

```
class Info extends Component {
  render () {
    const { avatar, username } = this.props

    return <div>
      <UserAvatar src={avatar} />
      <UserProfile username={username} />
    </div>
  }
}
```

- *. Como los fragmentos en React v16.2.0 pueden ser usados para múltiple devolver múltiples hijos sin necesidad de añadir nodos extra al DOM.

```
import React, {
  Component,
  Fragment
} from 'react'

class Info extends Component {
  render () {
    const { avatar, username } = this.props

    return (
      <Fragment>
        <UserAvatar src={avatar} />
        <UserProfile username={username} />
      </Fragment>
    )
  }
}
```

Fijar estado por defecto

```
class Hello extends Component {
  constructor (props) {
    super(props)
    this.state = { visible: true }
  }
}
```

- *. Fijar el estado por defecto en el constructor(). Sin constructor usando Babel con “campos de propuesta”:

```
class Hello extends Component {
  state = { visible: true }
}
```

API para Component

```
this.forceUpdate()

this.setState({ ... })
this.setState(state => { ... })

this.state
this.props
```

- *. Estos métodos y propiedades están disponibles para las instancias de Component.

Mounting

constructor (props)	Antes del renderizado
componentWillMount()	<i>No usar esto</i>
render()	Renderizar
componentDidMount()	Después de renderizar (DOM disponible)
componentWillUnmount()	Antes de eliminar del DOM
componentDidCatch()	Catch errores (16+)

- *. Poner el estado/state inicial en el constructor(). Añadir los manejadores de eventos del DOM, temporizadores/timers (etc) en componentDidMount(), luego eliminarlos en componentWillUnmount().

Componentes puros

```
import React, {PureComponent} from 'react'

class MessageBox extends PureComponent {
  ...
}
```

- *. Versión de rendimiento optimizado del React Component. No se re-renderiza si el estado o las props no han cambiado

Actualizando

<code>componentDidUpdate (prevProps, prevState, snapshot)</code>	Usar <code>setState()</code> aquí, pero recorder comparar props
<code>shouldComponentUpdate (newProps, newState)</code>	Salta <code>render()</code> si devuelve false
<code>render()</code>	renderizar
<code>componentDidUpdate (prevProps, prevState)</code>	Opera sobre el DOM aquí

- *. Llamado cuando los padres cambian propiedades y `.setState()`. No son llamados para los renders iniciales.

REACT.JS. HOOKS

Hooks: Estado

```
import React, {useState} from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

- *. Los Hooks han sido añadidos en React 16.8

Hooks: Effect

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    // Update the document title using the browser API
    document.title = `You clicked ${count} times`;
  }, [count]);
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

- *. Si estás familiarizado con los métodos de ciclo de vida de la clase React, puedes pensar en usar el Hook `useEffect` como el `componentDidMount`, el `componentDidUpdate` y el `componentWillUnmount` combinados. Por defecto, React ejecuta los efectos/effects después de cada renderizado, incluido el primero de ellos.

Hooks: Declarando múltiples variables de estado

```
function ExampleWithManyStates() {
  // Declare multiple state variables!
  const [age, setAge] = useState(42);
  const [fruit, setFruit] = useState('banana');
  const [todos, setTodos] = useState([{ text: 'Learn Hooks' }]);
  // ...
}
```

Construyendo tus propios Hooks

```
import React, { useState, useEffect } from 'react';

function FriendStatus(props) {
  const [isOnline, setIsOnline] = useState(null);

  useEffect(() => {
    function handleStatusChange(status) {
      setIsOnline(status.isOnline);
    }

    ChatAPI.subscribeToFriendStatus(props.friend.id, handleStatusChange);
    return () => {
      ChatAPI.unsubscribeFromFriendStatus(props.friend.id, handleStatusChange);
    };
  }, [props.friend.id]);

  if (isOnline === null) {
    return 'Loading...';
  }
  return isOnline ? 'Online' : 'Offline';
}
```

- *. Effects puede también especificar opcionalmente cómo “limpiar” después de ellos, devolviendo una función.

```
Usa FriendStatus:
function FriendStatus(props) {
  const isOnline = useFriendStatus(props.friend.id);
  if (isOnline === null) return 'Loading...';
  return isOnline ? 'Online' : 'Offline';
}
```

